



Proiectarea cu Micro-Procesoare

Lector: Mihai Negru

An 3 – Calculatoare și Tehnologia Informației

Seria B

Curs 1: Introducere

<http://users.utcluj.ro/~negrum/>



- **Obiective**
 - Cunoașterea, înțelegerea și utilizarea conceptelor: microprocessor, magistrală, memorie, sistem de intrare-ieșire, metode de transfer a datelor, interfețe.
 - Analiza și proiectarea sistemelor cu microprocessor.
- **Cunoștințe preliminare necesare**
 - Proiectare Logică, Proiectarea sistemelor numerice, Arhitectura Calculatoarelor, Programare în Limbaj de Asamblare, Programarea Calculatoarelor (C/C++)
- **Structura disciplinei**
 - 2C + 1L + 1P / săptămână
- **Structura cursului**
 - Partea 1 – ATMEL (ATmega2560, Arduino) și aplicații
 - Partea 2 – aspecte ale proiectării sistemelor cu microprocesor (exemplificate folosind familia x86)
- **Tematica laboratorului**
 - Lucrări practice folosind plăci Arduino (ATmega2560 (MEGA2560), ATmega328P(UNO)) și multiple module periferice (modules)



- **Slide-urile de curs**, disponibile pe site:
 - <http://users.utcluj.ro/~negrum/src/html/dmp.html>
- **Microcontrolere**
 - G. Grindling, B. Weiss, Introduction to Microcontrollers, Vienna Univ. of Technology, 2007: <https://ti.tuwien.ac.at/ecs/teaching/courses/mclu/theory-material/Microcontroller.pdf>
- **Atmel AVR, Arduino**
 - M. A. Mazidi, S. Naimi, S. Naimi, The AVR Microcontroller and Embedded Systems Using Assembly And C, 1-st Edition, Prentice Hall, 2009.
 - Michael Margolis, Arduino Cookbook, 2-nd Edition, O'Reilly, 2012.
- **Familia 8086**
 - Barry B. Brey, The Intel Microprocessors: 8086/8088, 80186,80286, 80386 and 80486. Architecture, Programming, and Interfacing, 4-rd edition, Prentice Hall, 1997
 - S. Nedevschi, L. Todoran, „Microprocesoare”, editura UTC-N, 1995, Biblioteca UTCN
- **Documente suplimentare**
 - Data sheets Atmel, Intel etc.
 - Tutoriale Arduino: <https://www.arduino.cc/en/Tutorial/HomePage>



- **Evaluare:** nota examen (E) + nota laborator / proiect (LP)

```
if (LP >= 5) AND (E >= 4.5)
    Final_mark = 0.5 * LP + 0.5 * E
else
    Final_mark = 4 OR Absent
```

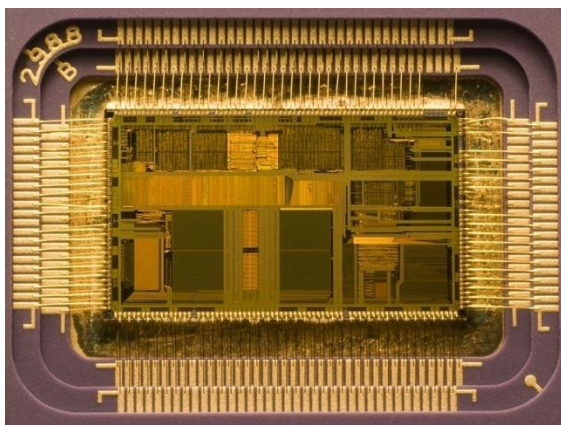
- **Bonus** – se poate acorda pentru activitate deosebită la curs / laborator, sau pentru participarea la concursuri studențești de hardware



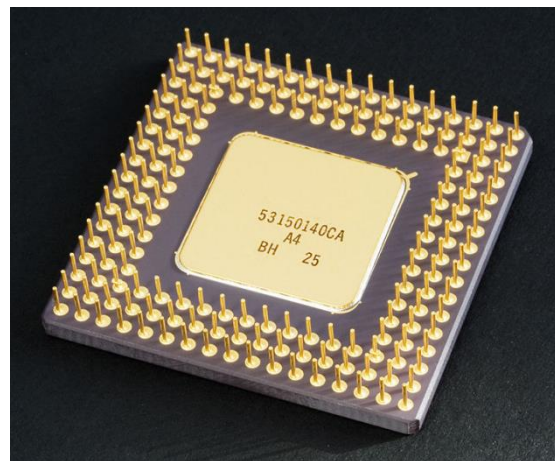
Ce este un microprocessor?



- Un **microprocesor** încorporează toate sau majoritatea funcțiilor unei **unități centrale de procesare** într-un singur circuit integrat.
- O unitate centrala de procesare (**Central Processing Unit, CPU**) este o mașina logică ce poate executa programe de calculator.
- Funcția fundamentală a oricărui CPU, indiferent de forma fizica pe care o are, este sa execute o **secvență de instrucțiuni (programul)**, stocate într-o memorie. Execuția instrucțiunilor se face de obicei în patru pași: citire instrucțiune (**fetch**), decodificare (**decode**), execuție (**execute**) si scriere rezultate (**write back**).



Intel 80486DX2 – interior



Intel 80486DX2 – vedere exterioara

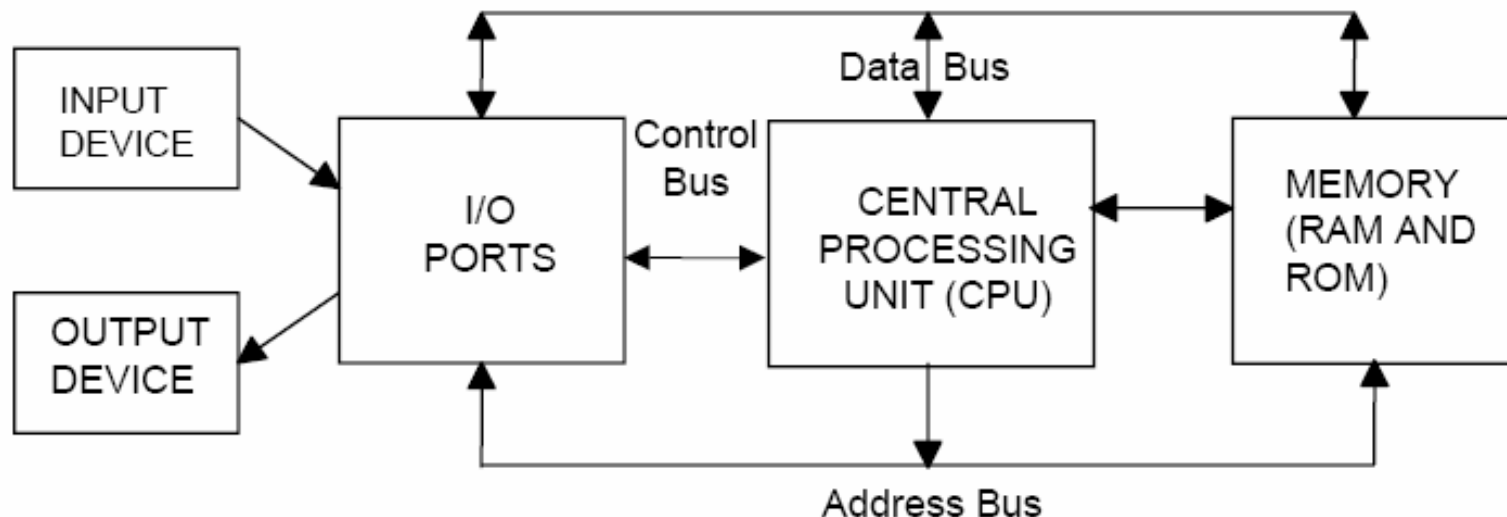


- Microprocesoare

- **4 bit**: Intel's 4004 (1971), Texas Instruments (TI) TMS 1000, si Garrett AiResearch's Central Air Data Computer (CADC).
- **8 bit**: 8008 (1972), primul procesor pe 8 biți. A fost urmat de Intel 8080 (1974), Zilog Z80 (1976), si alte procesoare derivate pe 8 biți de la Intel. Competitorul Motorola a lansat Motorola 6800 in August 1974. Arhitectura acestuia a fost clonată si îmbunătățita in MOS Technology 6502 in 1975, cu popularitate similara lui Z80 in anii 1980.
- **16 bit** (Intel 8086, 80186, 80286, 8086 SX, TMS 9900)
- **32 bit** (MC68000, Intel 80386DX, 80486DX, Pentium, MIPS R2000 (1984) si R3000 (1989) etc.)
- **64 bit** (majoritatea procesoarelor moderne)

- Tipuri:

- **RISC**: MIPS (R2000, R3000, R4000), Motorola 88000, AVR
- **CISC**: VAX, PDP-11, Motorola 68000, Intel x86

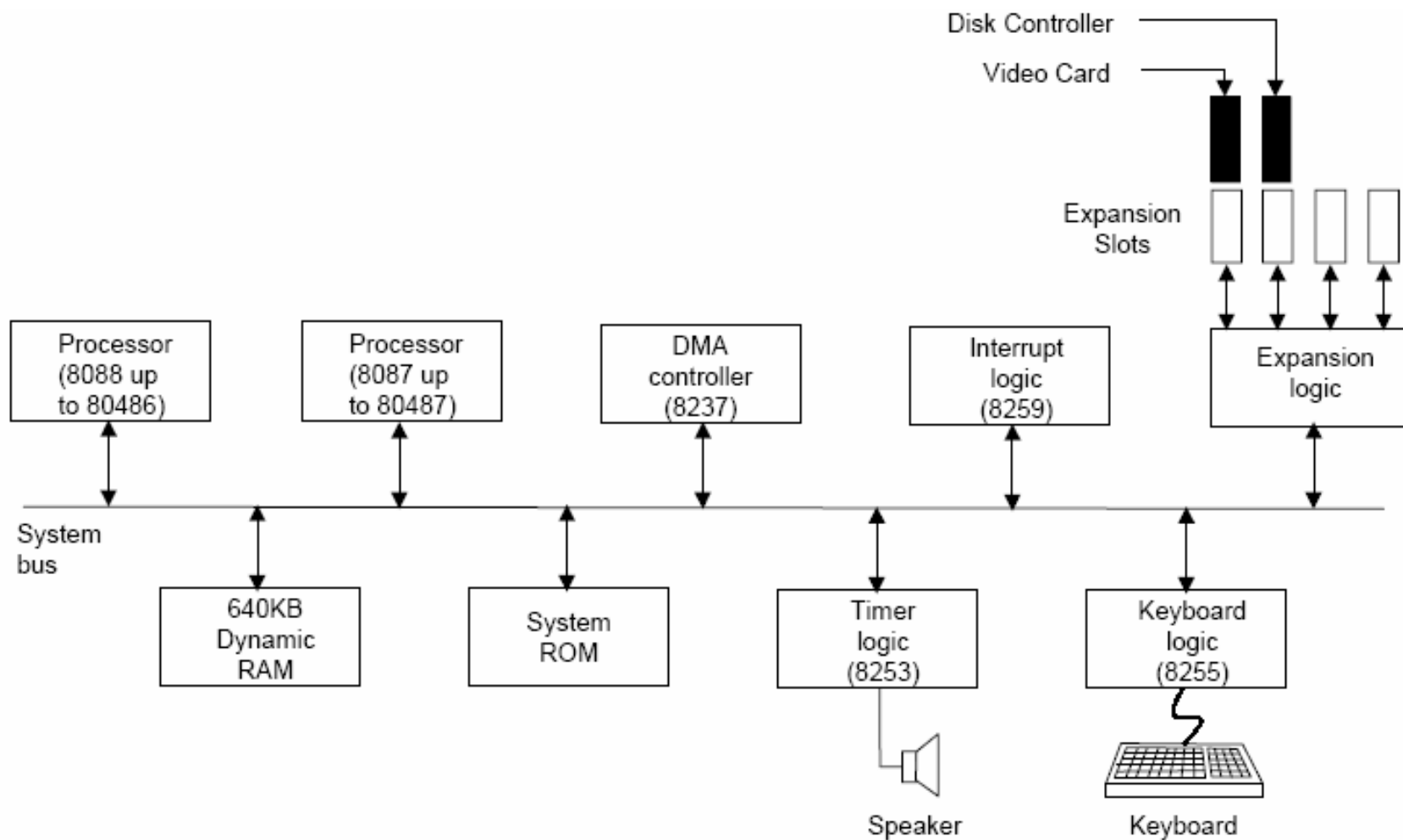


Dispozitive esențiale: CPU, Memorie, I/O

Dispozitive adiționale: Controller întreruperi, DMA, coprocesor, etc

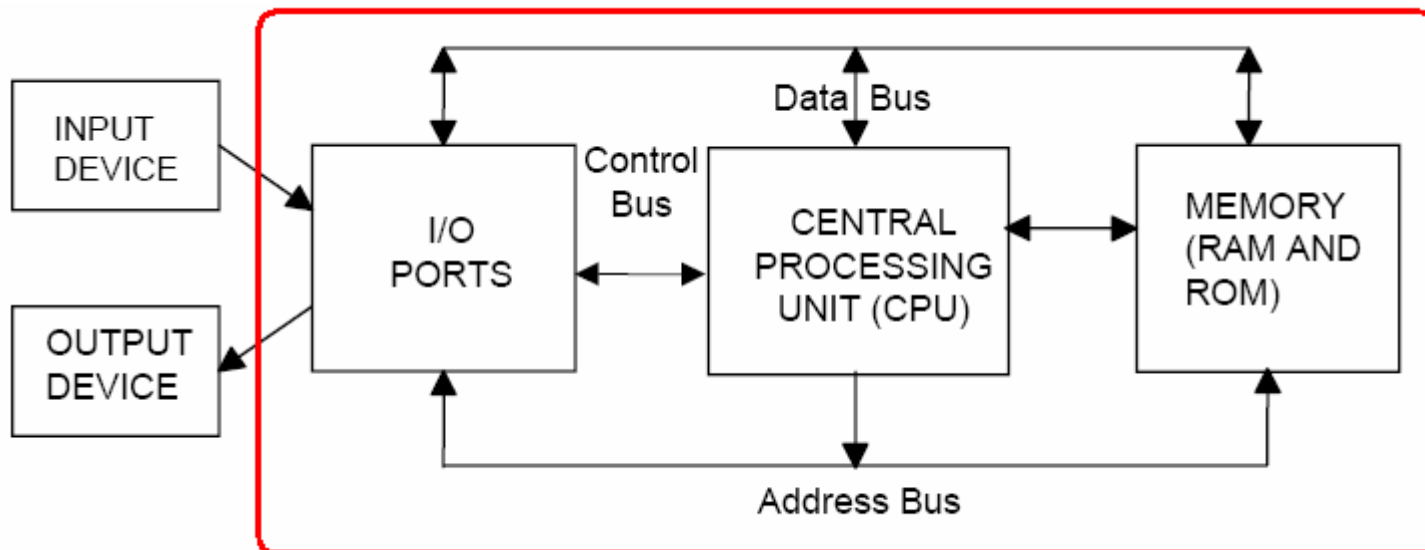


Exemplu: placă de bază PC





Microcontroller (MCU)



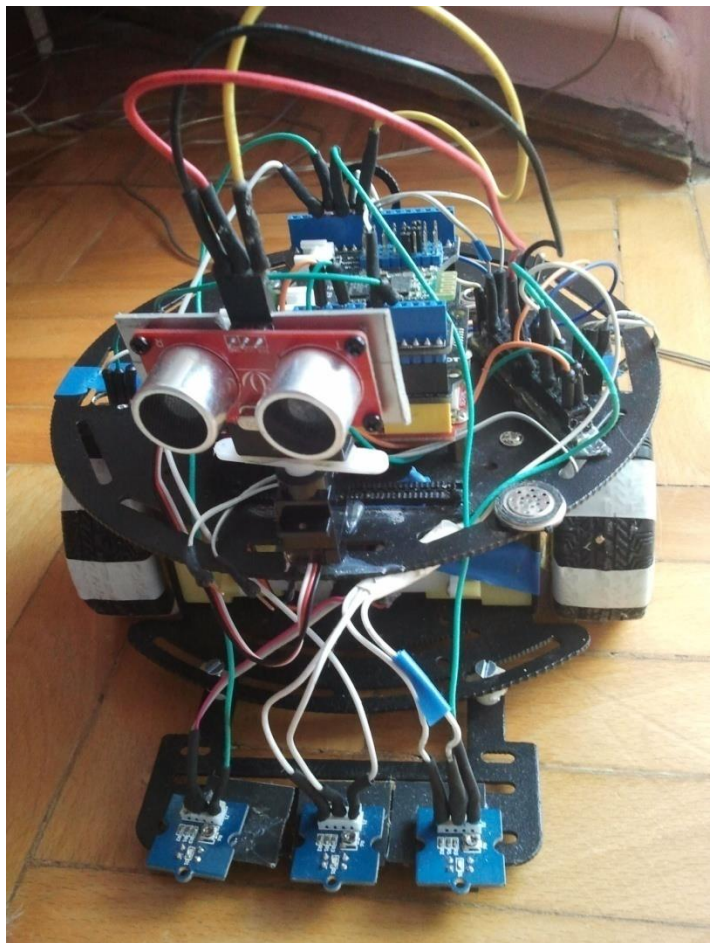
Multiple componente ale unui sistem cu microprocesor sunt incluse in același circuit integrat – Microcontroller

- Memorie RAM si ROM (Flash), pentru program si date
- Unele dispozitive periferice (Timer, Numărător, Controllere pentru comunicații seriale / paralele, etc.)



- **Obiectiv general:** utilizarea microprocesoarelor (microcontrolerelor) în dezvoltarea de sisteme electronice adaptate unor probleme specifice.
- **Exemple de aplicații:** roboți autonomi, senzori inteligenți, senzori mobili, procesare de semnal audio, procesare de imagini, controlul automat al unor procese, etc.
- **Pașii pentru îndeplinirea acestui obiectiv:**
 - Studiul capabilităților microcontrolerului, familiarizarea cu programarea acestuia
 - Studiul resurselor integrate în microcontroler și resurselor disponibile pe placă cu microcontroler
 - Studiul dispozitivelor externe necesare pentru rezolvarea unor probleme specifice
 - Studiul interfețelor de comunicare, a formatului datelor, și a diagramelor de timp, necesare pentru conectarea microcontrolerului la dispozitivele externe.

- **Exemplu:** proiectarea unui robot capabil să se deplaseze autonom, evitând obstacolele, sau sub controlul unui operator uman, sau ghidat de o bandă de culoare închisă.



Microcontroller: AVR ATmega328, placă Arduino, programare în C/C++

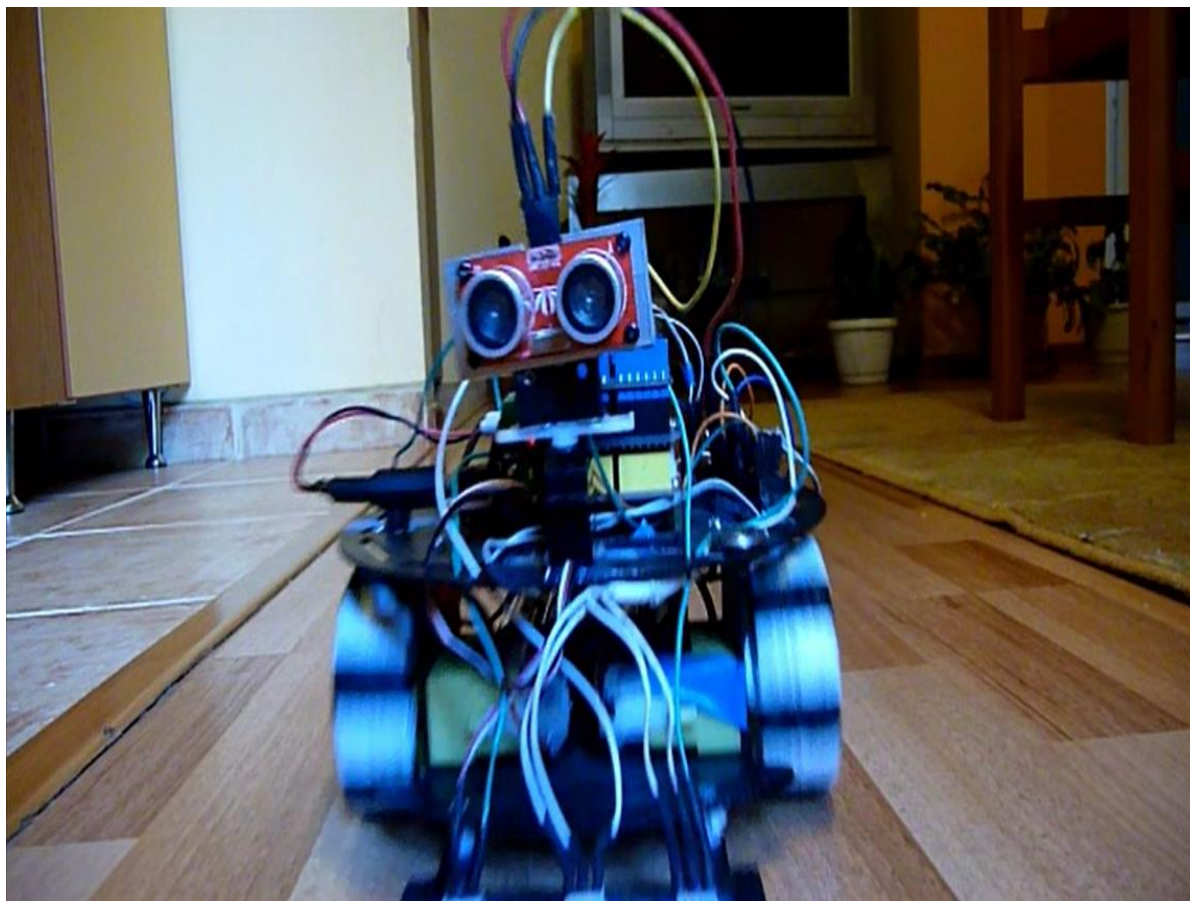
Componente interne: porturi I/O, întreruperi, interfață de comunicare serială, generator PWM

Componente externe: motoare DC, 1 motor servo, senzori de reflectivitate, punte H, senzor de distanță sonar, modul de comunicare Bluetooth.

Comunicare: serială, tip UART, între MCU și modulul Bluetooth, PWM între MCU și servo, și între MCU și puntea H, semnal analogic de la senzorii de reflectivitate, puls digital între sonar și MCU.

Algoritmi: scanare mediu și detecția obstacolelor, urmărirea liniei, controlul roților pentru mersul în linie dreaptă, etc.

- **Exemplu:** proiectarea unui robot capabil să se deplaseze autonom, evitând obstacolele, sau sub controlul unui operator uman, sau ghidat de o bandă de culoare închisă.



- **Exemplu:** proiectarea unui robot capabil să se deplaseze autonom, evitând obstacolele, sau sub controlul unui operator uman, sau ghidat de o bandă de culoare închisă.

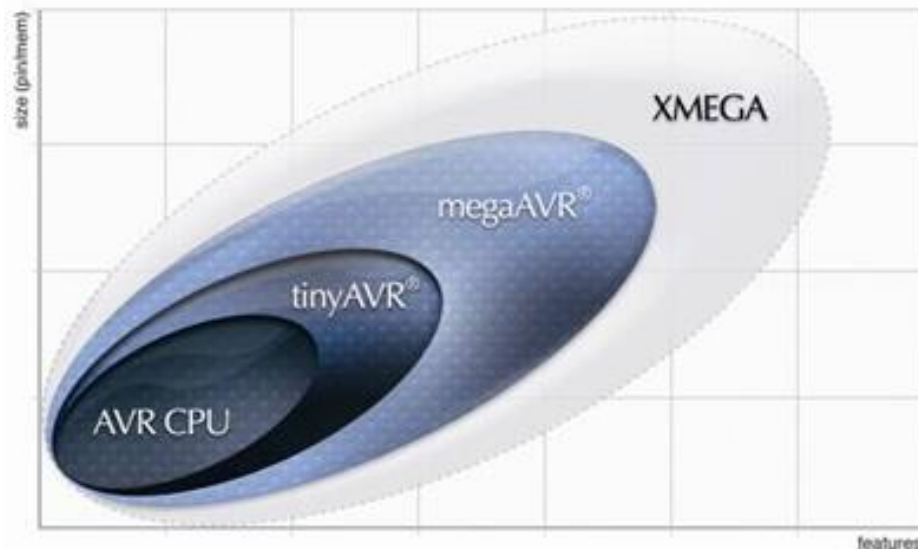




Familia de microcontrolere Atmel AVR 8 biți



- Arhitectura **RISC**
- Execuție 1 instrucțiune / ciclu
- 32 regiștri de uz general
- Arhitectura Harvard
- Tensiune de alimentare 1.8 - 5.5V
- Frecvență controlată software
- Mare densitate a codului
- Gama largă de dispozitive
- Număr de pini variat
- Compatibilitatea integrală a codului
- Familii compatibile între pini și capabilități
- Un singur set de unelte de dezvoltare



tinyAVR

1–8 kB memorie program

megaAVR

4–256 kB memorie program

Set extins de instrucțiuni (înmulțire)

XMEGA

16–384 kB memorie program

Extra: DMA, suport pentru criptografie

AVR specific pentru aplicații

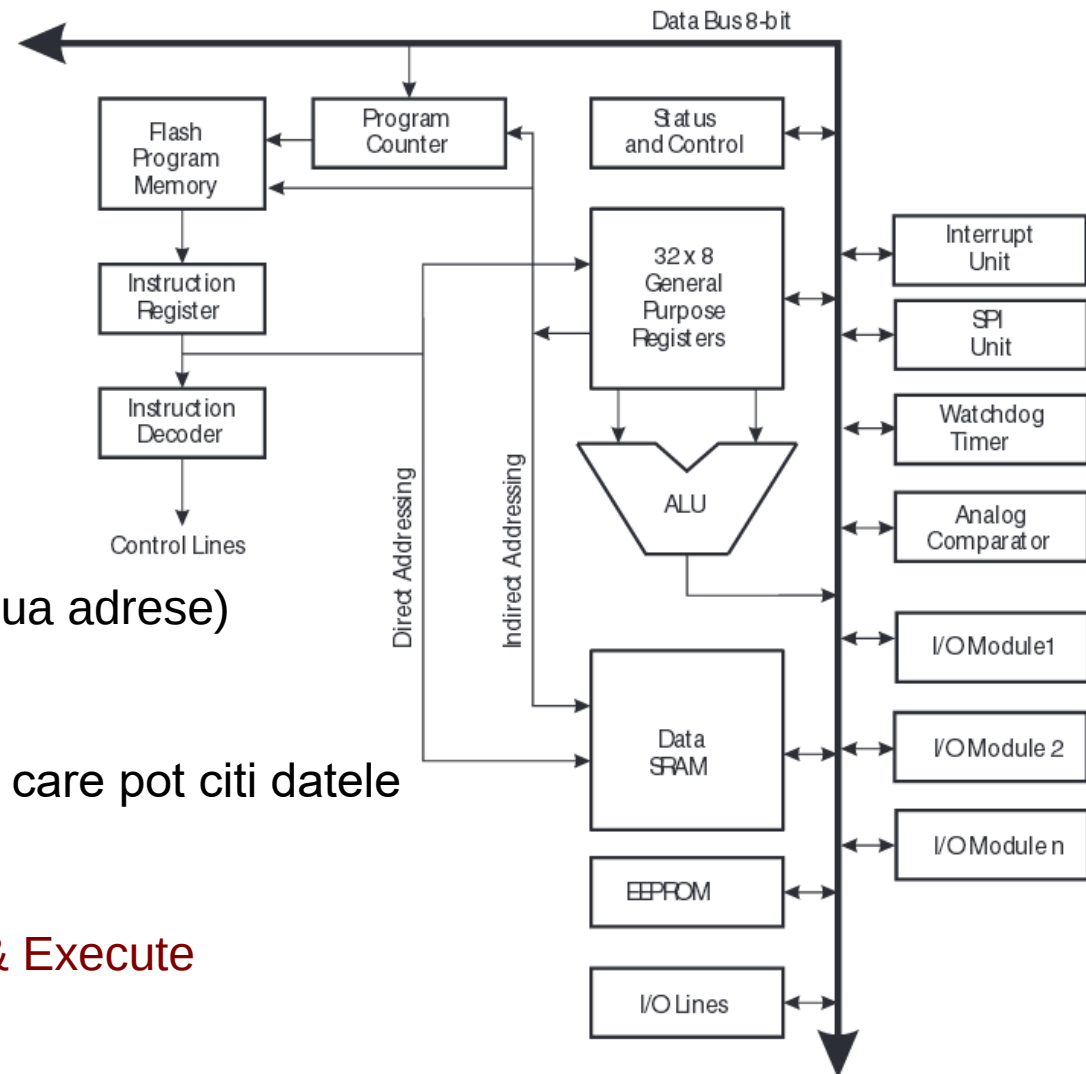
megaAVR cu interfețe particulare: LCD, USB, CAN etc.



Arhitectura generală a unui microcontroler AVR



- Mașină RISC (Load-Store cu doua adrese)
- Arhitectura Harvard modificată
 - exista instrucțiuni speciale care pot citi datele din memoria program
- Pipeline pe doua nivele: Fetch & Execute

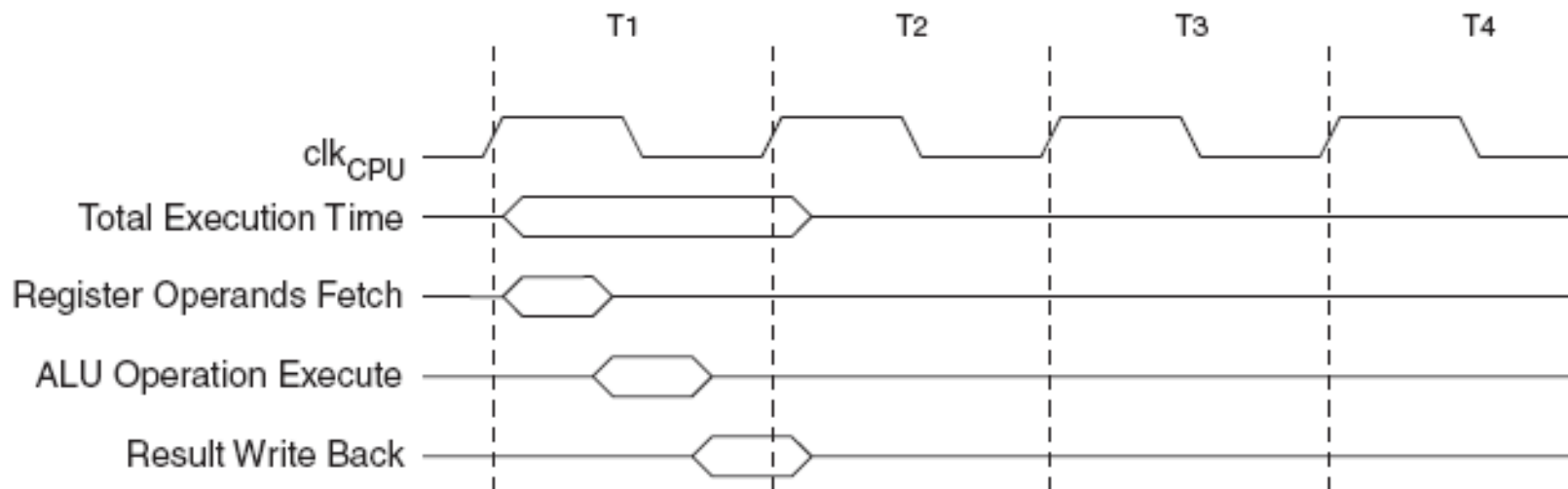




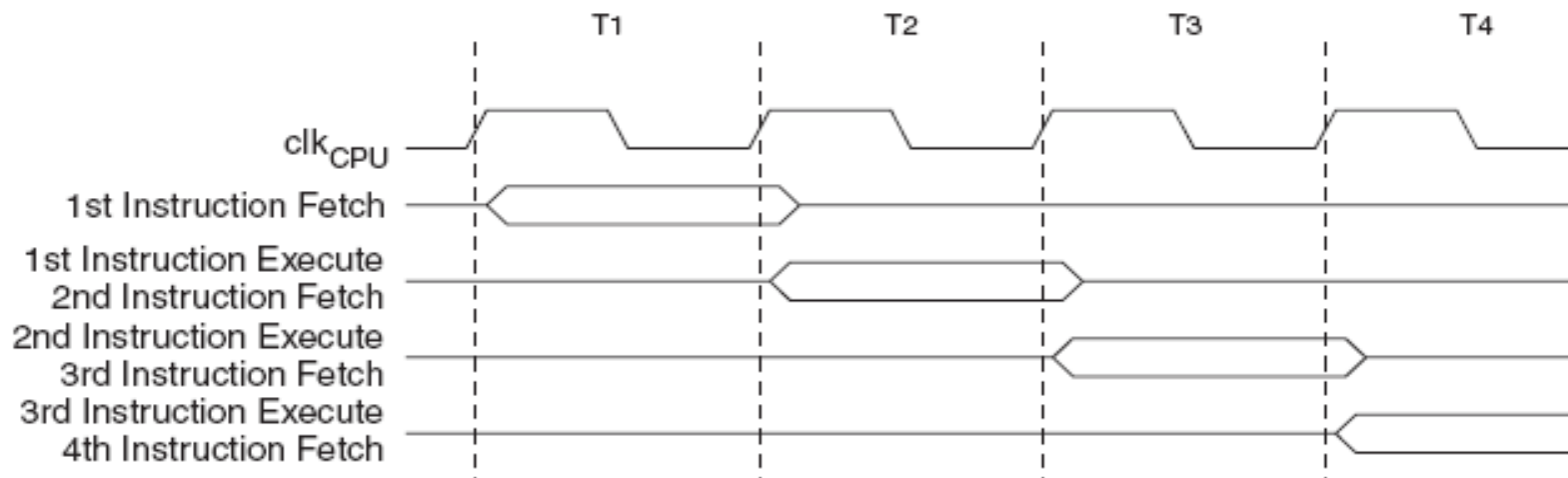
Diagrame de timp AVR



Execuția instrucțiunilor aritmetico-logice



Pipeline asigură suprapunerea citirii următoarei instrucțiuni cu execuția celei curente

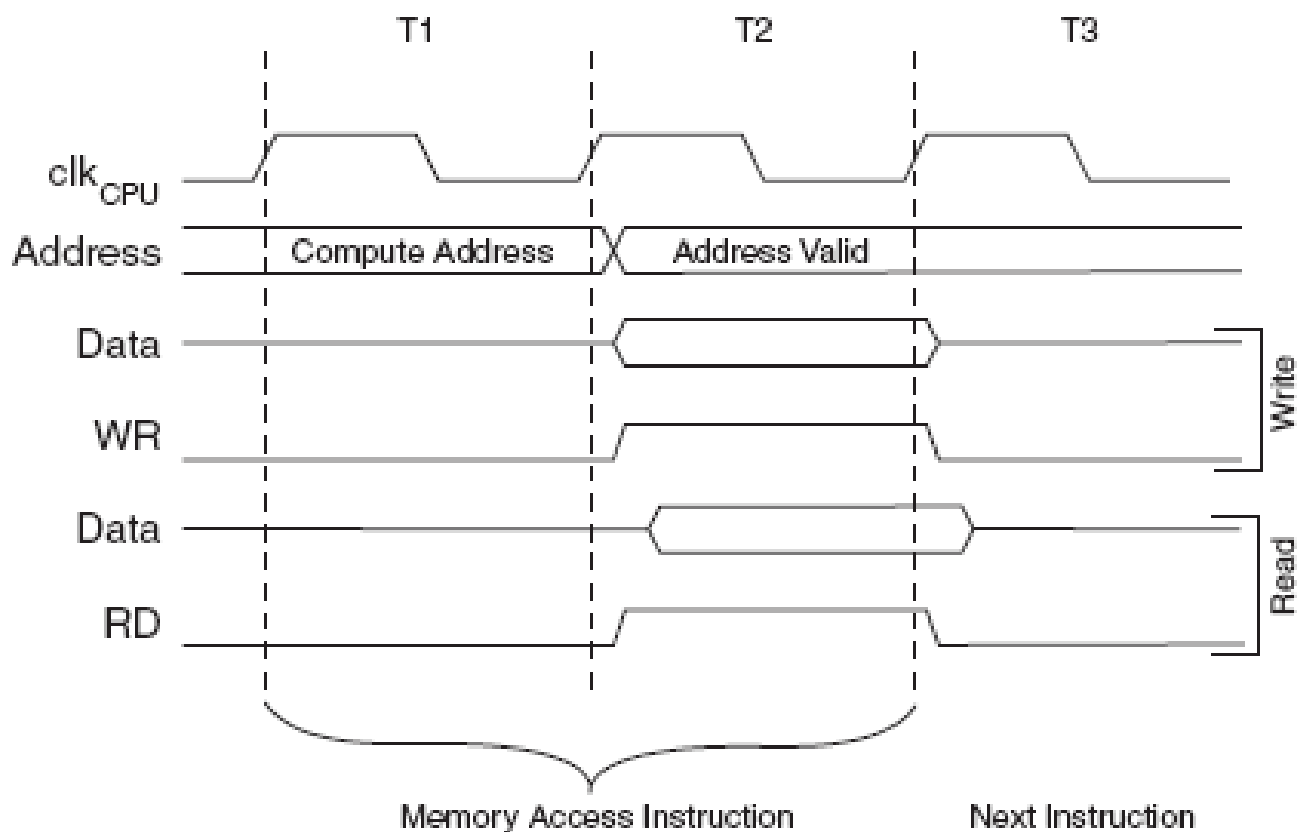




Diagrame de timp AVR



Instrucțiunile care accesează memoria internă SRAM
2 cicluri de ceas / instrucțiune





Registri de uz general

(General Purpose Registers – GPR)



- Valori imediate se pot încarcă doar în regiștrii R16-R31
- Regiștrii R26 – R31 sunt folosiți în perechi ca și pointeri
- Fiecare registru are și o adresă în spațiul memoriei de date – adresare uniformă

General
Purpose
Working
Registers

7	0	Addr.	
	R0	0x00	
	R1	0x01	
	R2	0x02	
	...		
	R13	0x0D	
	R14	0x0E	
	R15	0x0F	
	R16	0x10	
	R17	0x11	
	...		
	R26	0x1A	X-register Low Byte
	R27	0x1B	X-register High Byte
	R28	0x1C	Y-register Low Byte
	R29	0x1D	Y-register High Byte
	R30	0x1E	Z-register Low Byte
	R31	0x1F	Z-register High Byte



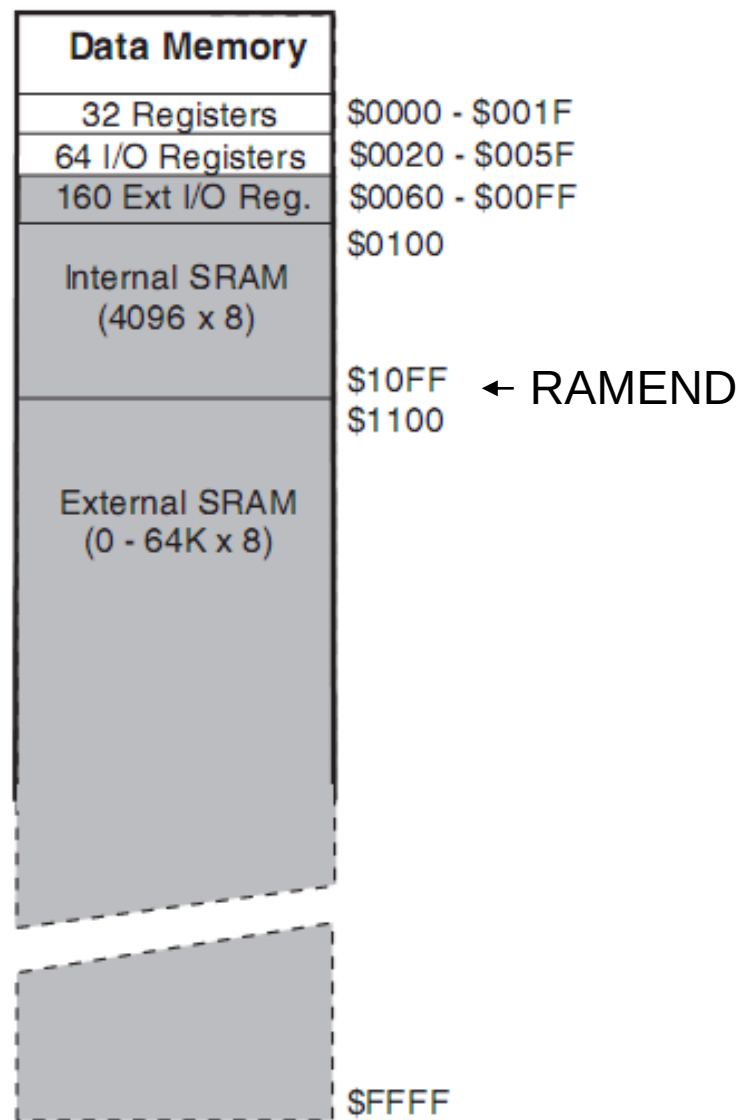
- Copiere date
`mov r4, r7`
- Lucrul cu valori imediate – posibil doar cu regiștrii r16 – r31
`ldi r16, 5`
`ori r16, 0xF0`
`andi r16, 0x80`
`subi r20, 1`
- Operații aritmetice și logice între regiștri
`add r1, r2`
`or r3, r4`
`lsl r5`
`mul r5, r18` – $r1:r0 = r5 * r18$
`rol r7`
`ror r9`
`inc r19`
`dec r17`



Memoria de date



- Primele 32 de adrese – blocul de regiștri
- 64 de adrese – regiștri I/O accesabili prin instrucțiuni speciale
- 160 adrese – spațiu I/O extins, accesabil prin instrucțiuni standard de acces la memorie
- SRAM, de ordinul Kbytes (2, 4, 8 ...)
- Posibilitate de extensie pana la 64 KB
- Constanta predefinita RAMEND marchează sfârșitul memoriei de date interne





- Accesarea directă a memoriei de date

lds r3, 0x10FE

lsl r3

sts 0x10FE, r3

- Accesarea indirectă a memoriei de date, prin intermediul regiștrilor X, Y, Z

ldi r27, 0x10

octetul superior al lui X este r27

ldi r26, 0xFE

octetul inferior al lui X este r26

ld r0, X

lsl r0

st X, r0

- Accesarea cu autoincrementare/decrementare a adresei

ld r0, X+

se accesează locația X, apoi se incrementează X

ld r0, +X

se incrementează X, apoi se accesează locația X

ld r0, X-

ld r0, -X



Memoria program



- Memorie flash, pentru programarea aplicațiilor
- Organizată în cuvinte de 16 biți
- Două secțiuni: Boot și Aplicație
- Cel puțin 10.000 cicluri scriere/ștergere
- Constante pot fi declarate în segmentul de cod, ele vor fi stocate în memoria program

- Accesarea memoriei program:

- **Citirea** – accesul este la nivel de BYTE, adresarea se face doar prin pointerul Z

LPM r5, Z

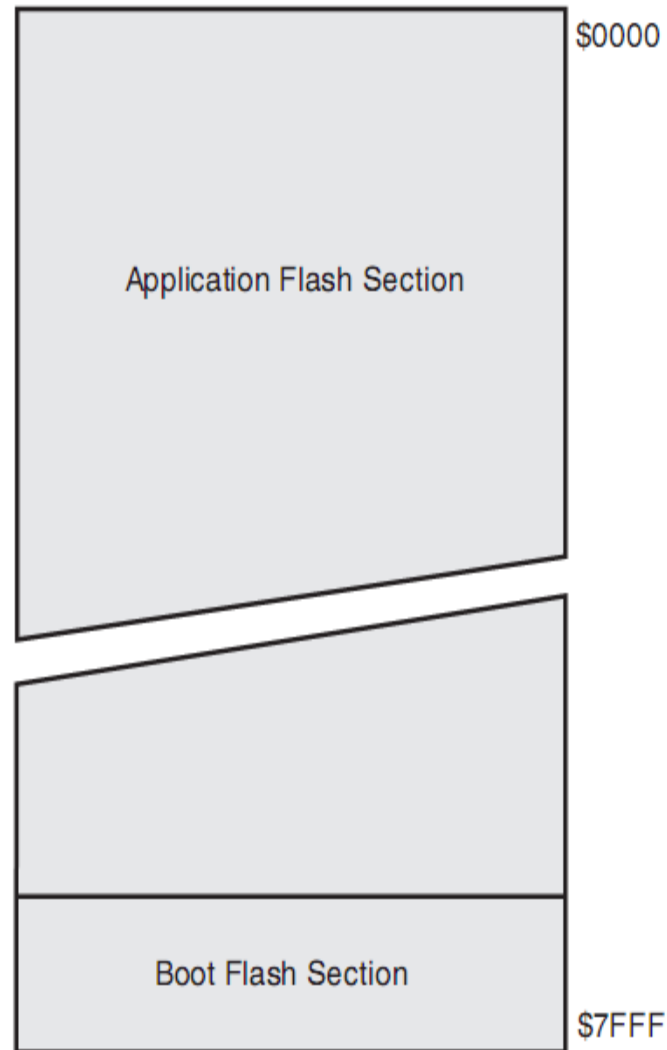
LPM r5, Z+

LPM r0 este destinație, Z adresa

- **Scrierea** – doar pe cuvânt

SPM

PM(Z) <= R1:R0





Registrul de stare SREG



- Registrul SREG (8 biți) conține informații despre starea sistemului și rezultatul unor operații
- Folosit pentru modificarea comportamentului programului sau pentru salturi condiționate
- **Nu este salvat automat la apelul procedurilor sau la execuția întreruperilor!**

Bit	7	6	5	4	3	2	1	0	
0x3F (0x5F)	I	T	H	S	V	N	Z	C	SREG
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- I – activarea globala a întreruperilor
- T – bit de transfer, poate fi copiat prin instrucțiunile BLD și BST din alt registru
- H – carry între jumătăți de octet, folositor pentru operații BCD
- S – $N \text{ xor } V$, pentru teste între numere cu semn
- V – indicator de overflow la operații în complement fata de 2
- N – indicator de rezultat negativ
- Z – indicator al unui rezultat nul
- C – carry



- Salturi necondiționate

RJMP – salt relativ, PC $\pm$ 2KB

JMP – salt absolut

IJMP – salt indirect, la adresa indicata de pointerul Z

- Salturi condiționate

CP, CPI – compară doua numere

BREQ – salt daca flagul Z este setat (numerele comparate sunt egale)

BRNE – salt daca numerele nu sunt egale

BRCS – salt daca flagul C este setat

SBRS – sare peste următoarea instrucțiune daca un bit într-un registru e setat

SBRS r5, 2 – daca bitul 2 din reg. 5 este setat, execută saltul

SBRC, SBIS, SBIC (I/O register set/clear)

- Apeluri de procedură

RCALL, CALL, ICALL – se salvează adresa de revenire în stiva, nu se salvează nimic altceva

- Revenire din procedură

RET – extrage adresa de revenire din stivă și execută salt la această adresă



Exemple



C

```
char a, b;
```

```
...
```

```
a = b;
```



AVR ASM

```
lds r24, b
```

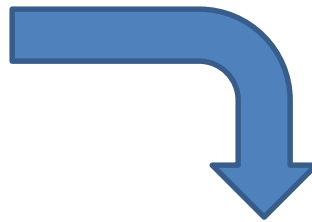
```
sts a, r24
```

C

```
char a;
```

```
...
```

```
a = 0x10;
```



AVR ASM

```
ldi r24, 0x10 ; Load imm 10
```

```
sts a, r24 ; Store to a
```



Exemple



C

```
int a = *pInt;
```

AVR ASM

```
; Use the Z register (R31:R30)
```

```
lds R30, pInt      ; Load from pInt
```

```
lds R31, pInt+1    ;
```

```
ld  r24, Z         ; load from (*pInt)
```

```
ldd r25, Z+1       ;
```

```
sts a, r24         ; store to a
```

```
sts a+1, r25       ;
```



Exemple



C

```
void strcpy (char *dst, char *src)
{
    char ch;

    do {
        ch = *src++;
        *dst++ = ch;
    } while (ch);
}
```

AVR ASM

```
; dst in R25:R24, src in R23:R22
strcpy:
    movw r30, r24      ; Z<=dst
    movw r26, r22      ; X<=src
loop:
    ld    r20, X+      ; ch=*src++
    st    Z+, r20      ; *dst++=ch
    tst   r20           ; ch==0?
    brne  loop         ; loop if not
    ret
```

MOVW	Rd, Rr	Copy Register Word	Rd+1:Rd ← Rr+1:Rr
------	--------	--------------------	-------------------



Exemple



C

AVR ASM

```
int a, b;          lds    r18, a      ; load a
...               lds    r19, a+1    ;
a = a + b;        lds    r24, b      ; load b
                  lds    r25, b+1    ;
                  add    r24, r18    ; add lower half
                  adc    r25, r19    ; add higher half
                  sts    a+1, r25    ; store a.byte1
                  sts    a, r24      ; store a.byte0
```



Exemple



C

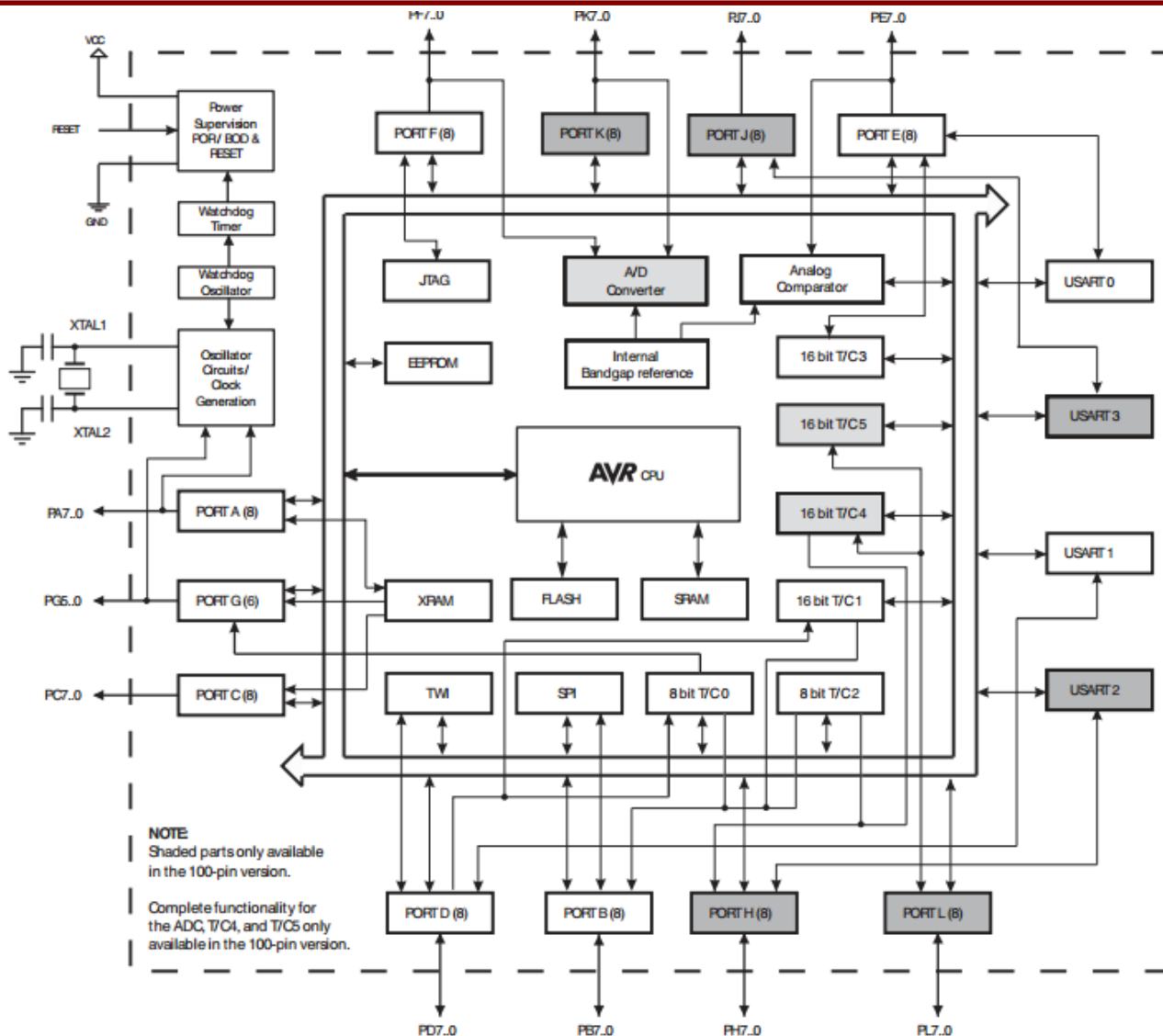
```
char sum, n;  
...  
for (n = 0; n < 10;  
    n++)  
    sum += n;
```

AVR ASM

```
; assume r16=n, r3=sum  
        clr     r16      ; n = 0  
        rjmp    check  
loop:    add     r3, r16   ; sum+= n  
        inc     r16      ; n++  
check:   cmpi    r16, 10   ; comp n and 10  
        brlt    loop     ; br if n<10
```



Microcontrolerul AVR Atmega 2560





Date tehnice Atmega 2560



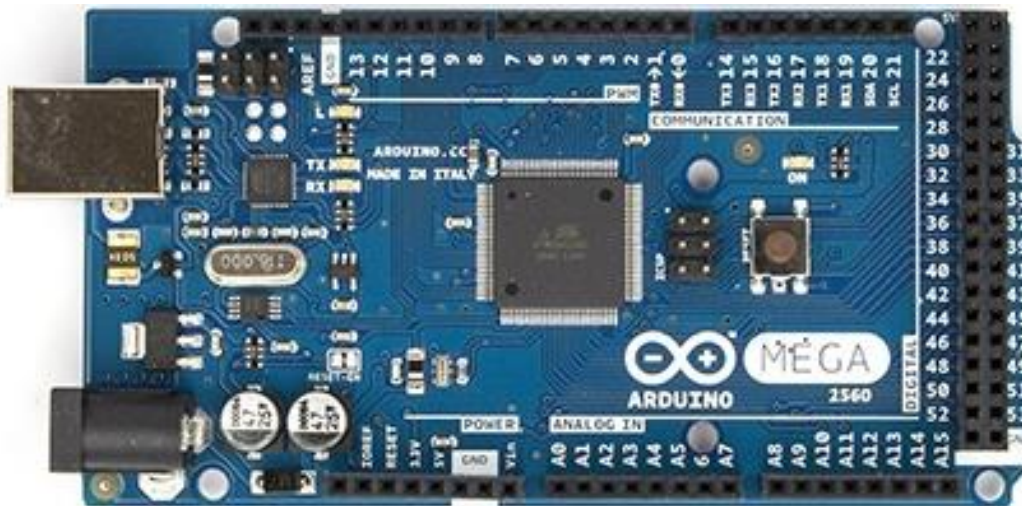
- 135 Instrucțiuni, majoritatea executate pe 1 ciclu de ceas
- 32 regiștri pe 8 biți
- Memorie program Flash reprogramabilă – 256 K Bytes
- Memorie EEPROM – 4 K Bytes
- Memorie SRAM internă – 8 K Bytes
- Cicluri de citire/scriere posibile: 10,000 Flash / 100,000 EEPROM
- Pană la 64 KB spații de adresă pentru memoria externă

- **Periferice integrate pe chip**
 - Două temporizatoare / numărătoare pe 8-biti
 - Patru temporizatoare / numărătoare pe 16 biți
 - 4 canale PWM pe 8 biți, 12 canale PWM pe 16 biți
 - 16 canale de conversie Analog / Digital pe 10 biți
 - 4 interfețe programabile USART
 - Interfață SPI
 - Interfață two-wire (TWI), similară cu I2C
 - Generare de întreruperi prin schimbarea stării pinilor



- Plăci cu microcontroler, și unelte de dezvoltare software open source
- Ascunde detaliile specifice diferitelor microcontrolere, folosind o abordare unificată
- Este disponibilă o largă mulțime de placi, shield-uri și accesorii
- O cantitate impresionantă de documentație gratuită sau contra cost
- O cantitate impresionantă de exemple pentru orice problemă
- Site web: www.arduino.cc
- Distribuitori in Romania: www.robofun.ro

Arduino Mega 2560



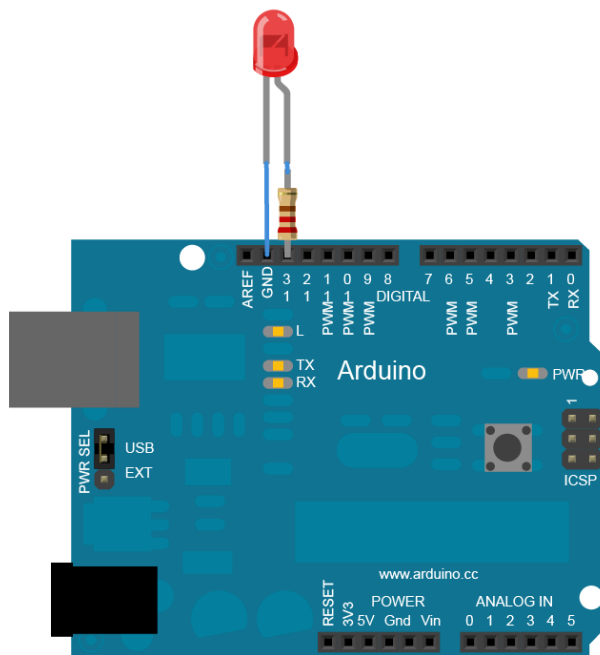
- Bazată pe microcontrolerul ATMega2560, pe 8 biți
- 54 pini de I/O digitali
- 16 pini de intrare pentru semnale analogice
- 4 porturi de comunicare serială UART
- Frecvența procesorului: 16 MHz
- Alimentare și programare prin cablu USB



Un exemplu de program Arduino



- Aprindere intermitentă a unui LED, conectat la un pin digital de ieșire (digital output)





Un exemplu de program Arduino



```
/*  
  Blink  
  Turns on an LED on for one second, then off for one second, repeatedly.  
  
  This example code is in the public domain.  
*/  
  
// Pin 13 has an LED connected on most Arduino boards.  
// give it a name:  
int led = 13;  
  
// the setup routine runs once when you press reset:  
void setup() {  
  // initialize the digital pin as an output.  
  pinMode(led, OUTPUT);  
}  
  
// the loop routine runs over and over again forever:  
void loop() {  
  digitalWrite(led, HIGH);   // turn the LED on (HIGH is the voltage level)  
  delay(1000);               // wait for a second  
  digitalWrite(led, LOW);    // turn the LED off by making the voltage LOW  
  delay(1000);               // wait for a second  
}
```




Proiectarea cu Micro-Procesoare

Lector: Mihai Negru

An 3 – Calculatoare și Tehnologia Informației

Seria B

Curs 2: Intrare / ieșire

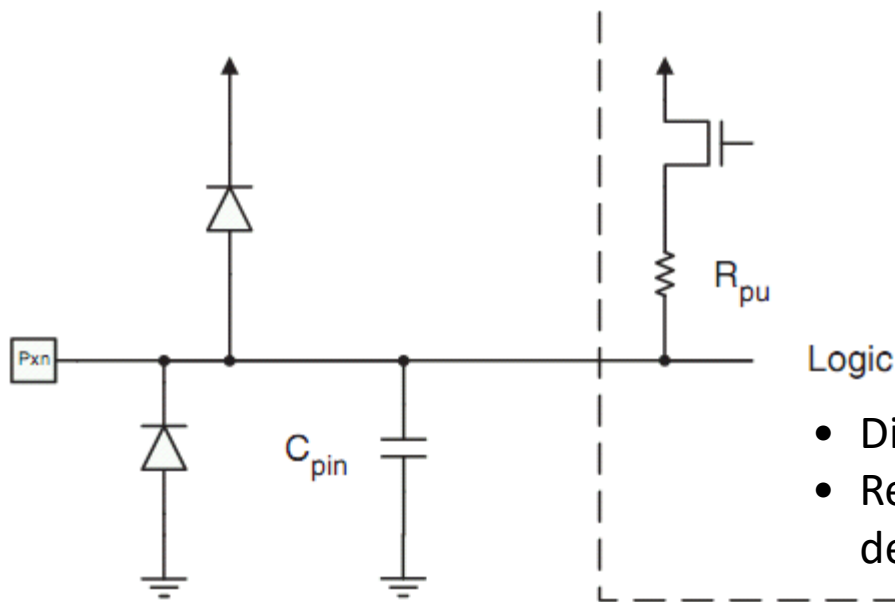
<http://users.utcluj.ro/~negrum/>



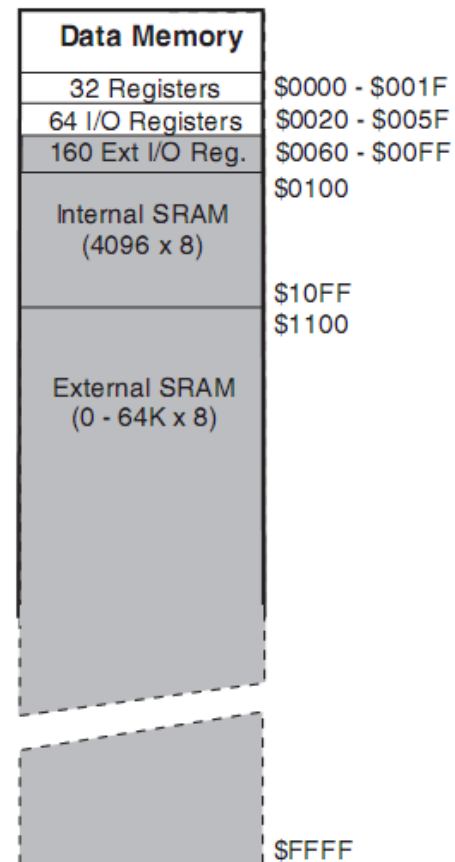
Intrare / ieșire



- Porturile de intrare / ieșire: PORTA ... PORTG
- PORTA – PORTE: accesate prin instrucțiuni speciale **in**, **out**
- PORTF – PORTG: doar prin **ld**, **st** (spațiul de adrese I/O extins)
- Registrul de direcție **DDRx** – fiecare bit din fiecare port poate fi configurat ca intrare / ieșire
- Scrierea portului se face prin registrul **PORTx**
- Citirea stării pinilor se face prin **PINx**



- Diode de protecție, împotriva electricității statice
- Rezistența “pull-up”, care poate fi activată / dezactivată prin logica

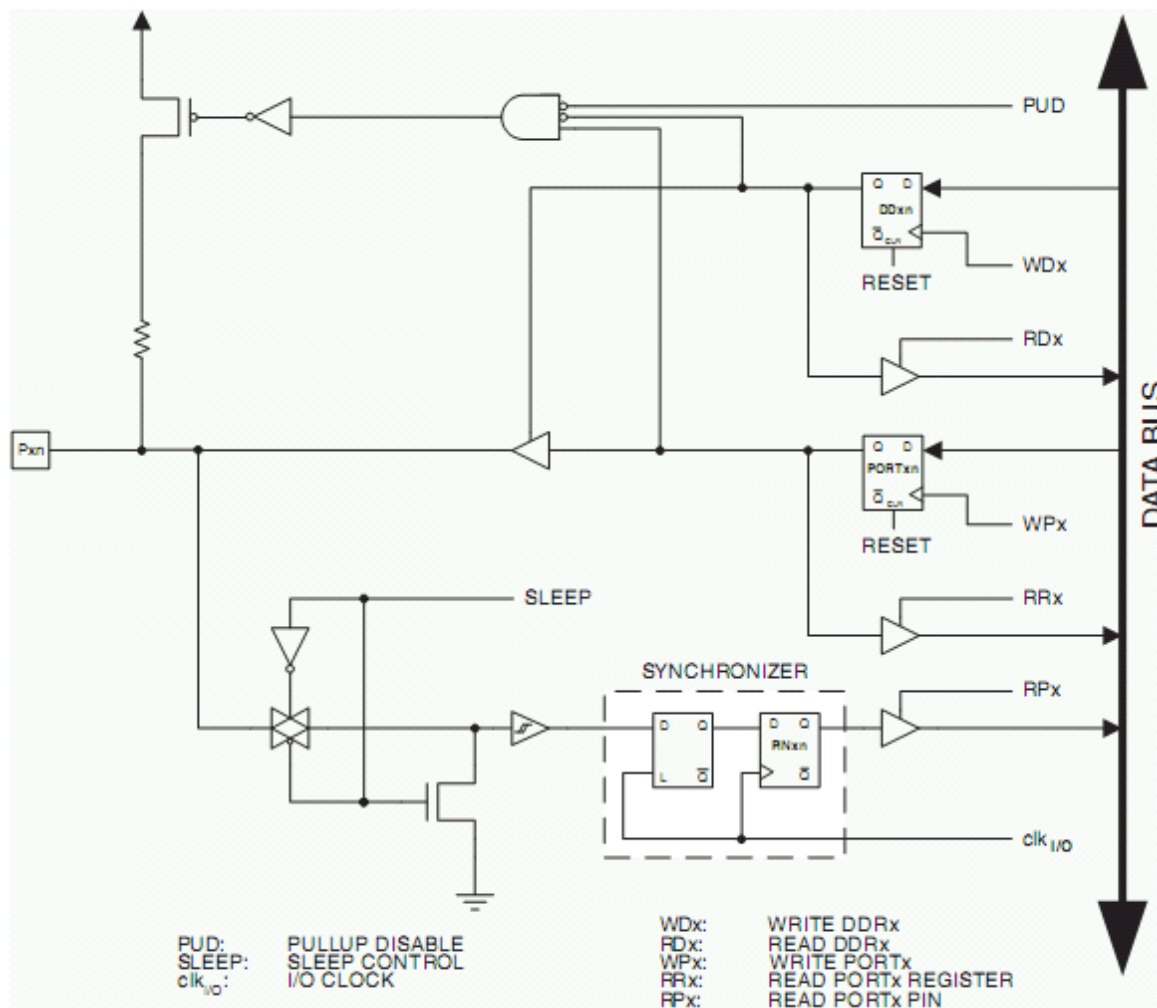




Intrare / Ieșire



- Schema generală pentru 1 bit dintr-un port I/O



Control direcție

Datele ce vor fi trimise la ieșire

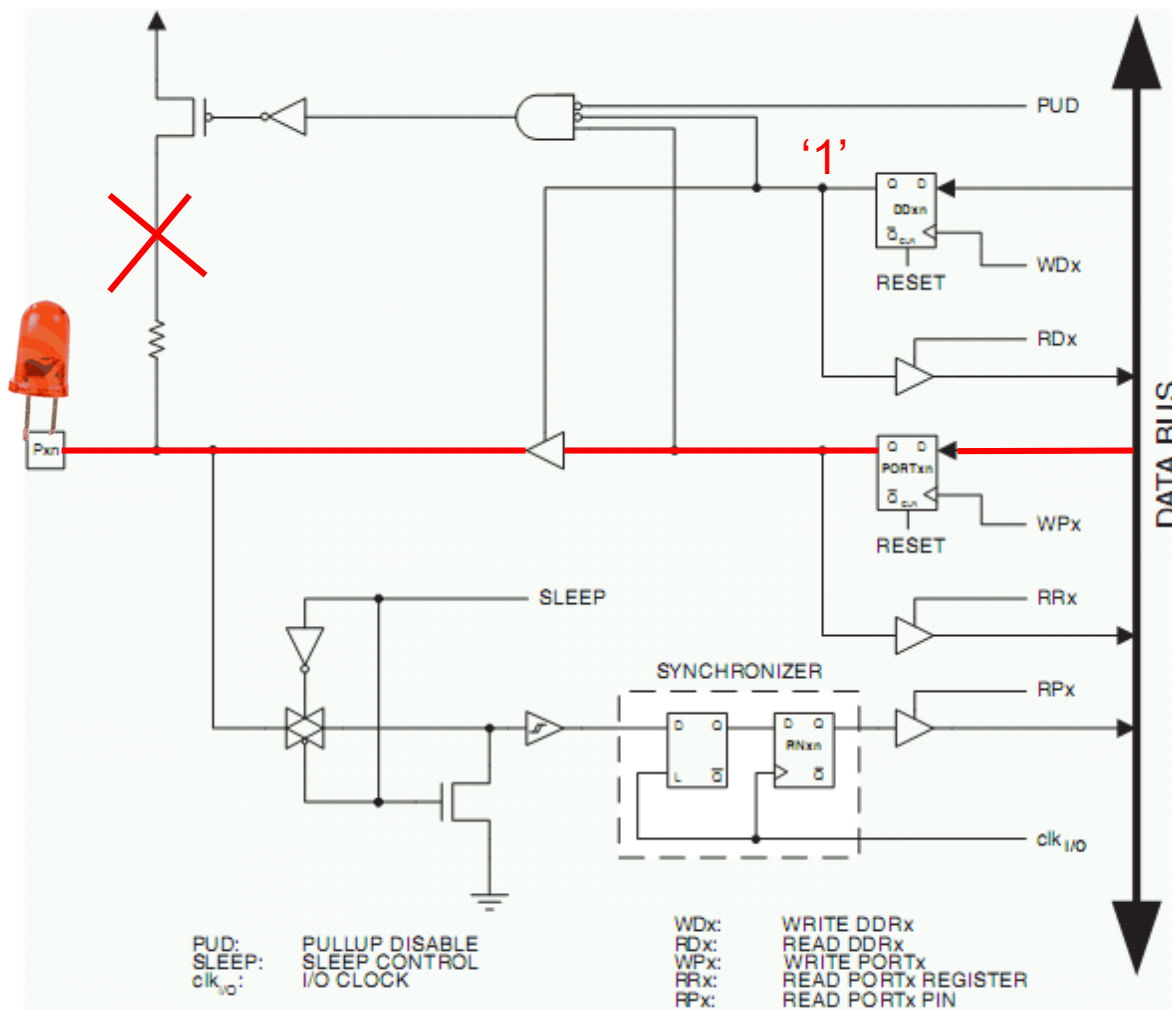
Datele citite de pe intrare



Intrare / Ieșire



- Configurația pentru ieșire



Direcție = 1

Datele scrise
în PORTx sunt
trimise la ieșire



'1' scris in PORTx
activează rezistența
pull-up

Datele devin
disponibile la PINx



- Stări posibile ale pinilor I/O

DDxn	PORTxn	PUD (in SFIOR)	I/O	Pull-up	Comment
0	0	X	Input	No	Tri-state (Hi-Z)
0	1	0	Input	Yes	Pxn will source current if ext. pulled low.
0	1	1	Input	No	Tri-state (Hi-Z)
1	0	X	Output	No	Output Low (Sink)
1	1	X	Output	No	Output High (Source)

- PUD – Pull Up Disable: GLOBAL
 - Valoarea '1' a bitului 2 din SFIOR dezactivează toate rezistentele pull-up

Bit	7	6	5	4	3	2	1	0	
0x20 (0x40)	TSM	–	–	–	ACME	PUD	PSR0	PSR321	SFIOR
Read/Write	R/W	R	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

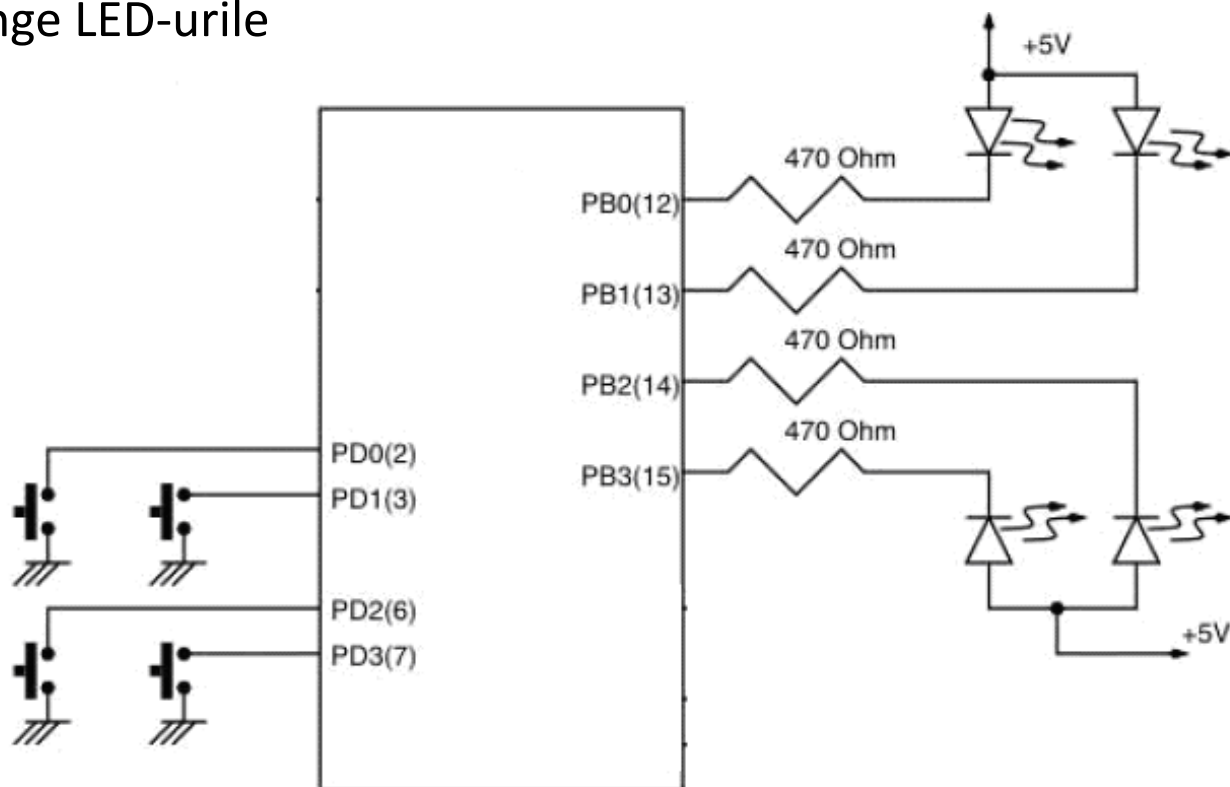
in r17, SFIOR
ori r17, 0b00000100
out SFIOR, r17



Intrare / ieșire – Butoane si LED-uri



- Rezistentele pull-up asigură nivelul '1' pe pin când butonul este in repaus
- Când butonul este apăsat, nivelul pinului este '0' prin legare la GND
- Un nivel '0' pe pinii de ieșire (B) cauzează diferența de potențial pe LED-uri, provocând aprinderea lor
- Nivelul '1' pe pinii B stinge LED-urile





- **Scrierea programului**

ldi r16, 0x00

out DDRD, r16 Direcția portului D - intrare

ldi r16, 0xFF

out PORTD, r16 '1' in PORTD – rezistente pull-up activate

ldi r16, 0xFF

out DDRB, r16 Direcția portului B - ieșire

loop:

in r16, PIND Citire port D

out PORTB, r16 Scriere port B

rjmp loop

- **Atenție!!!**

in r16, PIND Citește starea pinilor exteriori,
modificată de activitate exterioară

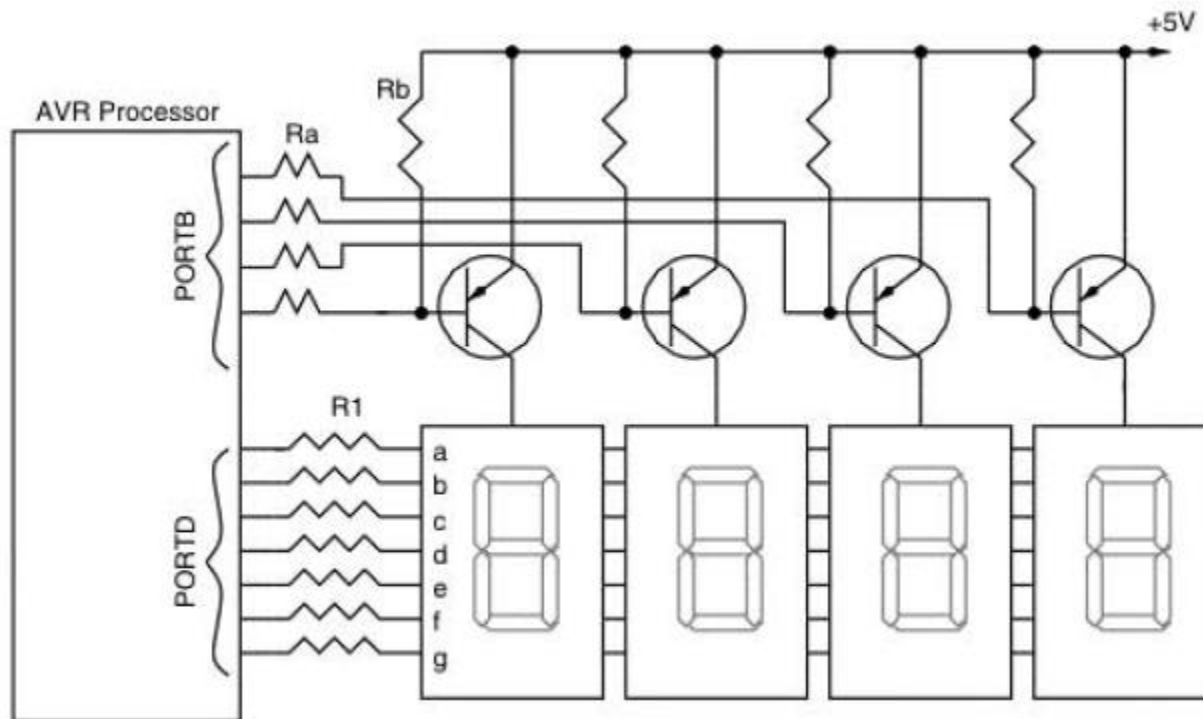
in r16, PORTD Citește starea registrului PORTD,
setat din interiorul micro-controllerului prin program



Intrare / Ieșire – Bloc 4x7 segmente

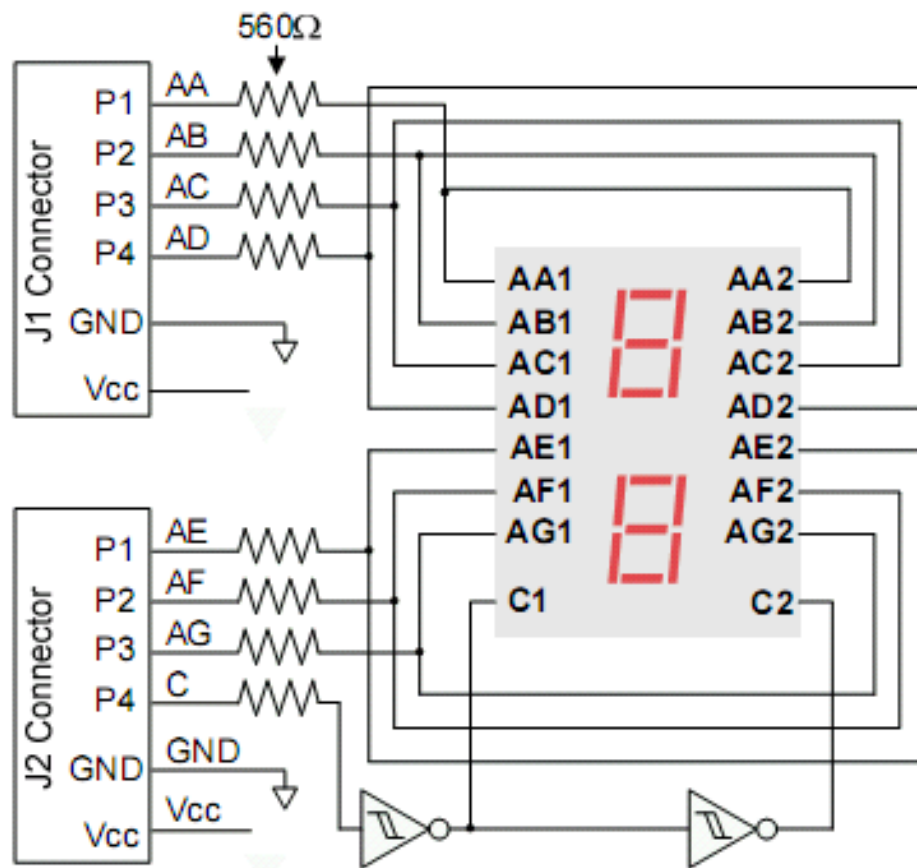
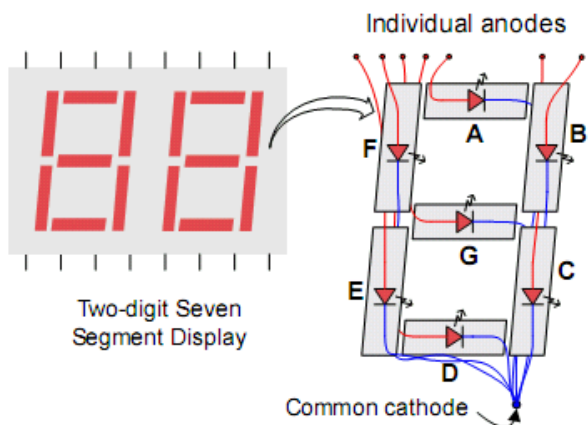
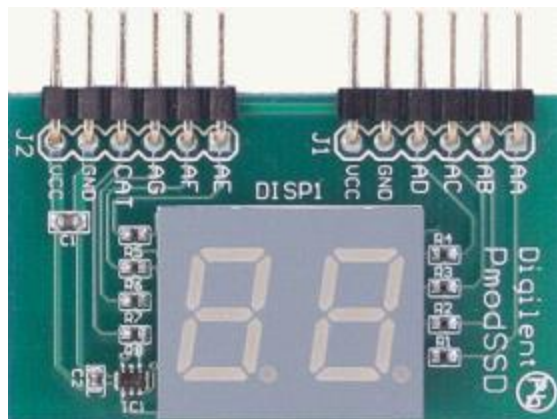


- Fiecare cifra este alcătuită din 7 led-uri, cu anod comun
- Nivelul '1' pe anod activează cifra – una singură activă la un moment dat
- Valori selective de '0' pe fiecare catod realizează modelul cifrei
- Baleiere pentru utilizarea întregului dispozitiv



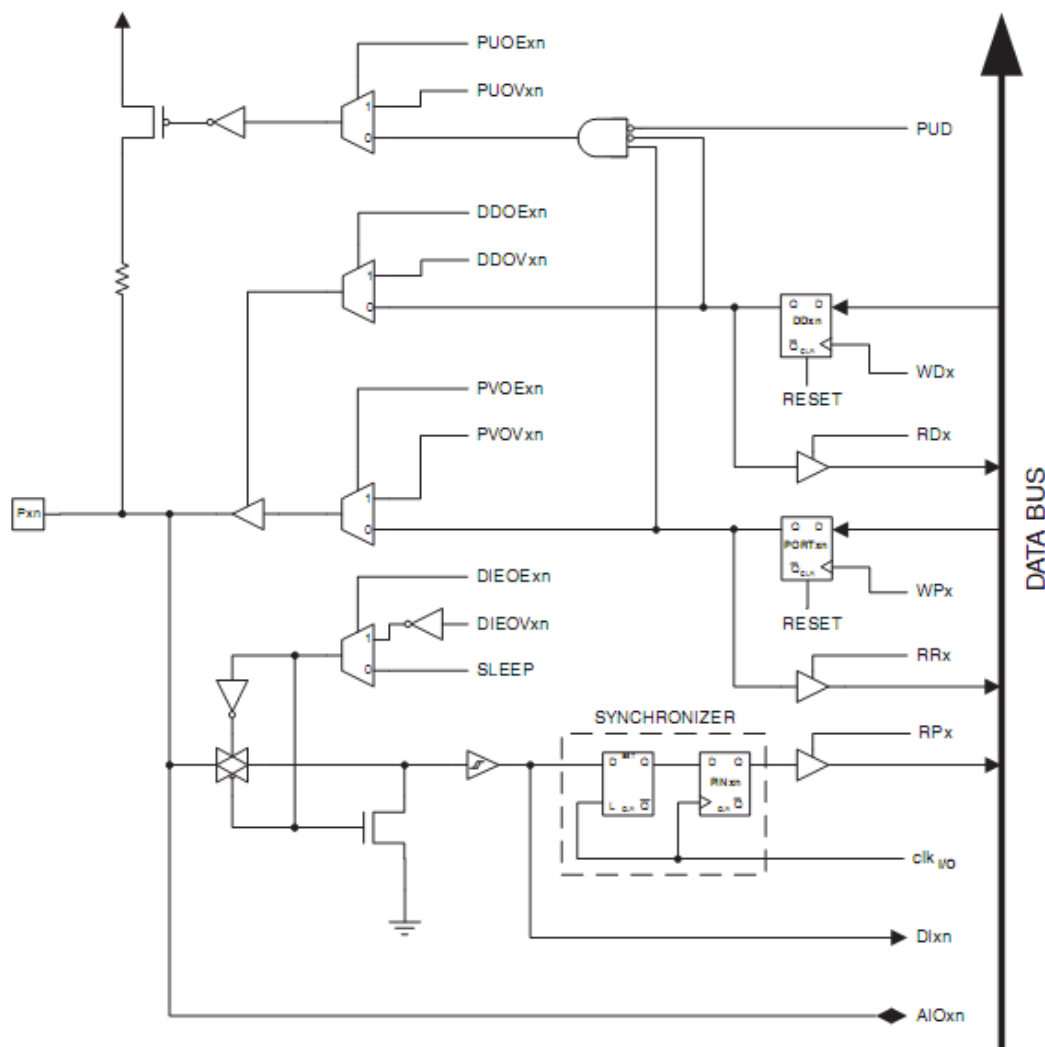


Intrare / Ieșire – Digilent PMOD SSD (2x7 segmente)





- Funcții alternative ale pinilor I/O la ATmega64



PUExn: Pxn PULL-UP OVERRIDE ENABLE
 PUEVxn: Pxn PULL-UP OVERRIDE VALUE
 DUExn: Pxn DATA DIRECTION OVERRIDE ENABLE
 DUEVxn: Pxn DATA DIRECTION OVERRIDE VALUE
 PVExn: Pxn PORT VALUE OVERRIDE ENABLE
 PVEVxn: Pxn PORT VALUE OVERRIDE VALUE
 DIEOExn: Pxn DIGITAL INPUT-ENABLE OVERRIDE ENABLE
 DIEOVxn: Pxn DIGITAL INPUT-ENABLE OVERRIDE VALUE
 SLEEP: SLEEP CONTROL

PUD: PULLUP DISABLE
 WDx: WRITE DDRx
 RDx: READ DDRx
 RRx: READ PORTx REGISTER
 WPx: WRITE PORTx
 RPx: READ PORTx PIN
 clk_{I/O}: I/O CLOCK
 DInx: DIGITAL INPUT PIN n ON PORTx
 AIOxn: ANALOG INPUT/OUTPUT PIN n ON PORTx



- Funcții alternative ale pinilor I/O la ATmega64
 - Exemplu – funcțiile alternative ale portului B

Port Pin	Alternate Functions
PB7	OC2/OC1C ⁽¹⁾ (Output Compare and PWM Output for Timer/Counter2 or Output Compare and PWM Output C for Timer/Counter1)
PB6	OC1B (Output Compare and PWM Output B for Timer/Counter1)
PB5	OC1A (Output Compare and PWM Output A for Timer/Counter1)
PB4	OC0 (Output Compare and PWM Output for Timer/Counter0)
PB3	MISO (SPI Bus Master Input/Slave Output)
PB2	MOSI (SPI Bus Master Output/Slave Input)
PB1	SCK (SPI Bus Serial Clock)
PB0	$\overline{SS}$ (SPI Slave Select input)

Generare de semnale prin intermediul timer-elor interne

Semnalele pentru comunicația SPI – controller intern



- Funcții alternative ale pinilor I/O la ATmega64
 - Exemplu – funcțiile alternative ale portului D

Port Pin	Alternate Function
PD7	T2 (Timer/Counter2 Clock Input)
PD6	T1 (Timer/Counter1 Clock Input)
PD5	XCK1 ⁽¹⁾ (USART1 External Clock Input/Output)
PD4	ICP1 (Timer/Counter1 Input Capture Pin)
PD3	INT3/TXD1 ⁽¹⁾ (External Interrupt3 Input or UART1 Transmit Pin)
PD2	INT2/RXD1 ⁽¹⁾ (External Interrupt2 Input or UART1 Receive Pin)
PD1	INT1/SDA ⁽¹⁾ (External Interrupt1 Input or TWI Serial DATA)
PD0	INT0/SCL ⁽¹⁾ (External Interrupt0 Input or TWI Serial CLock)

Întreruperi
externe, semnale
UART și TWI



Operații cu stiva



- Stack pointer (16 biti) – indică adresa vârfului stivei
- Accesabil prin cele două jumătăți de 8 biți, *SPL* si *SPH*, prin instrucțiuni de I/O
- Trebuie inițializat la începutul oricărui program care folosește operații explicite cu stiva, apeluri de procedura, sau întreruperi
- Trebuie sa fie o adresa din memoria SRAM, mai mare decât 0x60
- Deoarece prin introducerea de date pe stiva SP este decrementat, este bine ca el sa fie inițializat cu cea mai mare adresa disponibila din SRAM – *RAMEND*

Bit	15	14	13	12	11	10	9	8	
0x3E (0x5E)	SP15	SP14	SP13	SP12	SP11	SP10	SP9	SP8	SPH
0x3D (0x5D)	SP7	SP6	SP5	SP4	SP3	SP2	SP1	SP0	SPL
	7	6	5	4	3	2	1	0	
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	
	0	0	0	0	0	0	0	0	

- Exemplu inițializare SP

```
ldi R16, high(RAMEND)
out SPH, R16
ldi R16, low(RAMEND)
out SPL, R16
```

Valoarea inițială este total nepotrivită pentru utilizare!



- Instrucțiuni care operează cu stiva

push Rx

$\text{MEM}(\text{SP}) = \text{Rx}$

$\text{SP} = \text{SP} - 1$

pop Rx

$\text{SP} = \text{SP} + 1$

$\text{Rx} = \text{MEM}(\text{SP})$

rcall adresa

$\text{MEM}(\text{SP}:\text{SP}-1) = \text{PC} + 1$

adresa instrucțiunii următoare, 16 biti

$\text{SP} = \text{SP} - 2$

$\text{PC} = \text{adresa}^*$

*de fapt se modifică PC cu un offset relativ la poziția curentă

ret

$\text{SP} = \text{SP} + 2$

$\text{PC} = \text{MEM}(\text{SP}:\text{SP}-1)$



Operații cu stiva – Afișare pe modul 7 segmente



; conversia pentru afisare pe 7-segmente
; input r16, packed BCD number
; output r17, r18 - sevseg number
convssg:

```
push r20  
push r30  
push r31  
in r17, sreg  
push r17  
push r16
```

```
ldi zl, low (sevsegtable*2)  
ldi zh, high (sevsegtable*2)  
ldi r20, 0
```

```
andi r16, 0x0F  
add zl, r16  
adc zh, r20  
lpm r17, Z ; r17, cifra unitatilor
```

```
ldi zl, low (sevsegtable*2)  
ldi zh, high (sevsegtable*2)
```

sevsegtable: ; tabela pentru activarea led-urilor pentru fiecare cifră

```
.db 0b00111111, 0b00000110, 0b01011011, 0b01001111, 0b01100110, 0b01101101, 0b01111101,  
0b00000111, 0b01111111, 0b01101111
```

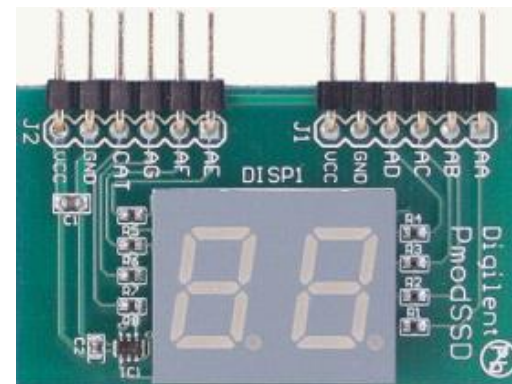
```
pop r16 ; reface r16, salveaza din nou  
push r16
```

```
lsl r16  
lsl r16  
lsl r16  
lsl r16
```

```
add zl, r16  
adc zh, r20  
lpm r18, Z ; r18, cifra zecilor
```

```
pop r16  
pop r20  
out sreg, r20  
pop r31  
pop r30  
pop r20
```

ret





Apelul funcției

main:

```
ldi r16, 0x35      ; numarul de afisat

rcall convssg      ; conversie, rezultatul in r17 și r18

out PORTA, r17     ; SSG este atașat la portul A

; cod de asteptare

ori r18, 0x80      ; aceasta operatie pune r18(7) pe '1',
                  ; pentru a activa cifra zecilor

out PORTA, r18

; cod de asteptare

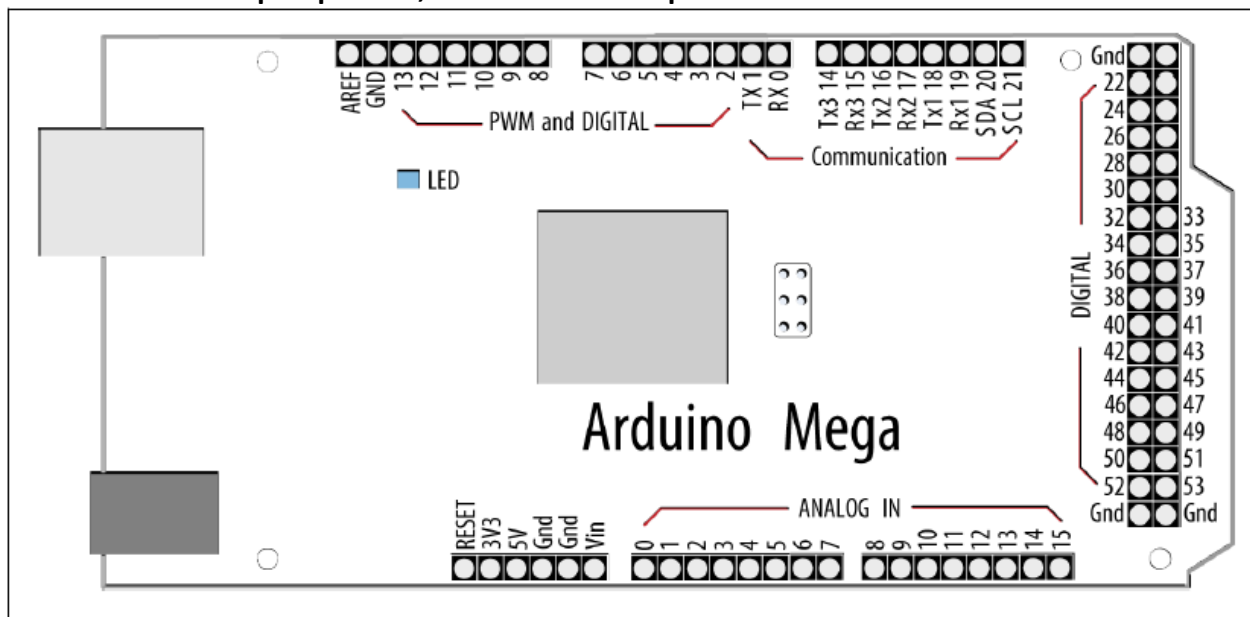
rjmp main
```



Intrare / ieșire la sistemele Arduino



- Pini de intrare/ieșire digitali, conectați la porturile microcontrollerului
- Mediul de dezvoltare se ocupa de problema corespondentei
- Logica de programare este orientată pe numărul pin-ului
- O parte din pini au funcții speciale (comunicație serială UART sau I2C, generator de undă PWM, sau semnale analogice)
- Pinii care au funcția RX și TX trebuie evitați – rezervați pentru comunicarea serială prin USB, **care include programarea plăcii**
- De obicei există un LED pe placă, conectat la pin-ul 13





Intrare / ieșire la sistemele Arduino



- Corespondență pinilor cu porturile microcontrollerului ATmega2560
- <http://arduino.cc/en/Hacking/PinMapping2560>
- Selecție:

43	PDO (SCL/INT0)	Digital pin 21 (SCL)
44	PD1 (SDA/INT1)	Digital pin 20 (SDA)
45	PD2 (RXDI/INT2)	Digital pin 19 (RX1)
46	PD3 (TXD1/INT3)	Digital pin 18 (TX1)
47	PD4 (ICP1)	
48	PD5 (XCK1)	
49	PD6 (T1)	
50	PD7 (T0)	Digital pin 38

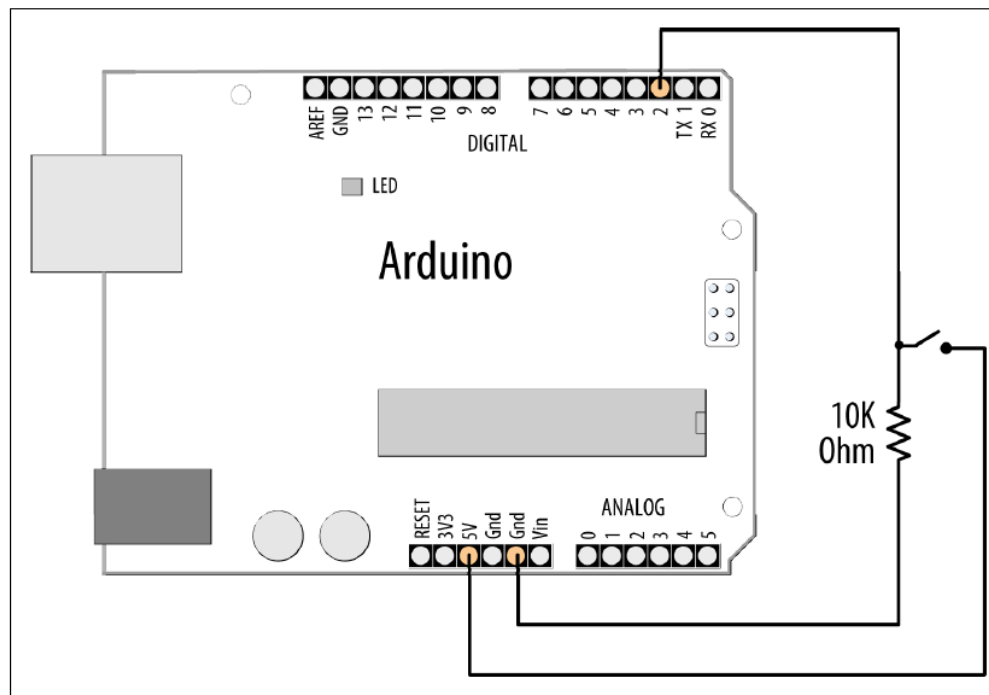
71	PA7 (AD7)	Digital pin 29
72	PA6 (AD6)	Digital pin 28
73	PA5 (AD5)	Digital pin 27
74	PA4 (AD4)	Digital pin 26
75	PA3 (AD3)	Digital pin 25
76	PA2 (AD2)	Digital pin 24
77	PA1 (AD1)	Digital pin 23
78	PA0 (AD0)	Digital pin 22



Intrare / ieșire la sistemele Arduino



- Sursa elementară de semnal: un buton conectat la un pin de intrare digital
- Se folosește o rezistență “pull down”, pentru ca atunci când butonul nu este apăsat, semnalul de intrare să fie nivel logic ‘0’
- Pentru ieșire, se folosește led-ul de pe placă





Intrare / Ieșire la sistemele Arduino



- Exemplu:

```
const int ledPin = 13;           // Constante pentru numarul pinilor implicati
const int inputPin = 2;          // se pot folosi direct numerele, dar solutia aceasta e mai
                                  // flexibila

void setup() {
    pinMode(ledPin, OUTPUT);      // configurarea directiei pinilor
    pinMode(inputPin, INPUT);     // se declara pin-ul legat la LED ca iesire
    // si cel legat la buton ca intrare
}

void loop(){
    int val = digitalRead(inputPin); // citire stare buton
    if (val == HIGH)                // daca este apasat, se scrie '1' pe pinul led-ului
    {
        digitalWrite(ledPin, HIGH);
    }
    else
    {
        digitalWrite(ledPin, LOW); // altfel se scrie '0'
    }
    // Evident, se poate si transfera direct starea butonului
    // catre LED:
    void loop()
    {
        digitalWrite(ledPin, digitalRead(inputPin));
    }
```



Intrare / ieșire la sistemele Arduino

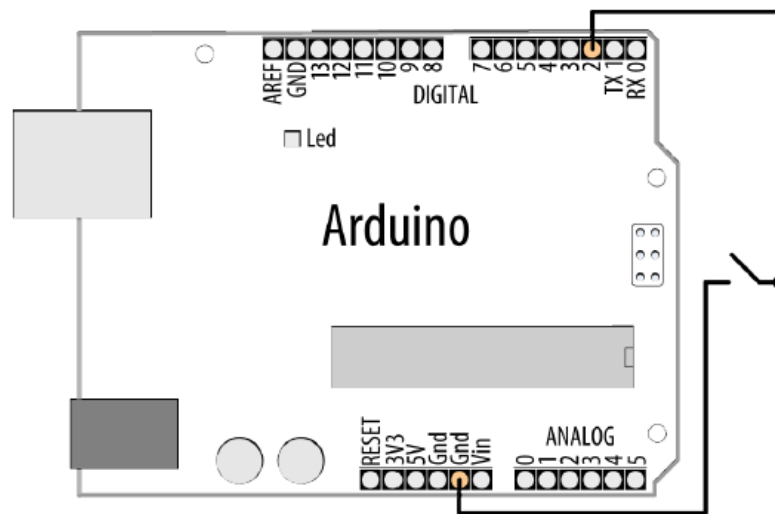


- Folosirea unui switch fără rezistențe externe
- Se folosesc rezistențele 'Pull Up' atașate fiecărui pin

```
const int ledPin = 13;           // Aceleasi constante, aceiasi pini
const int inputPin = 2;

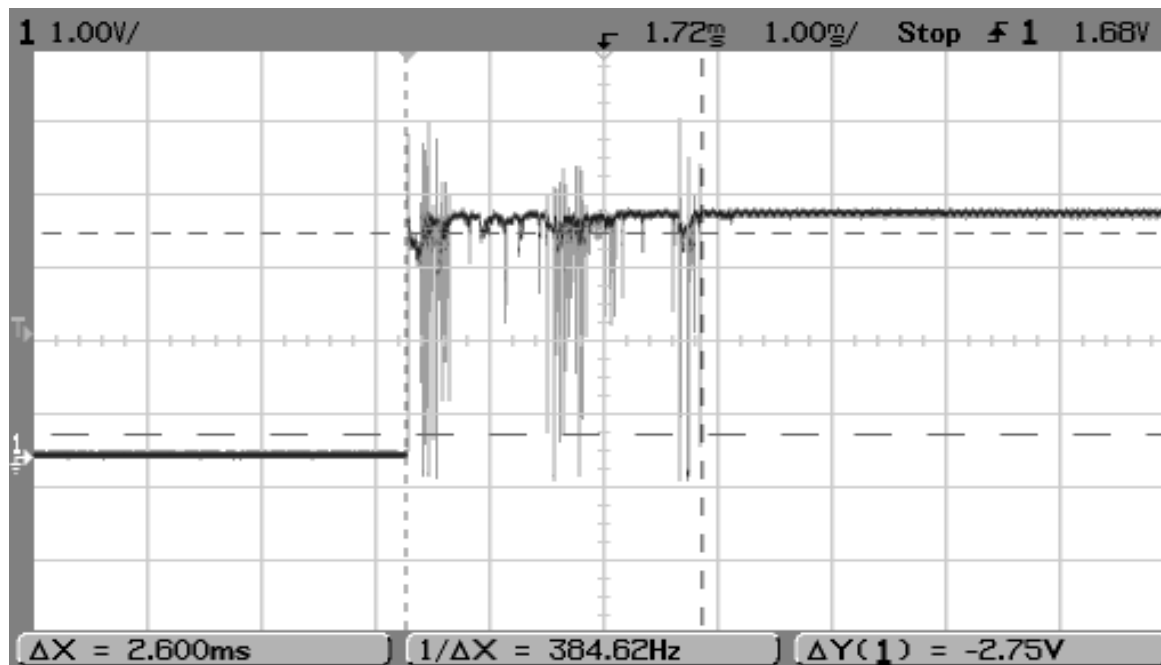
void setup() {
  pinMode(ledPin, OUTPUT);       // configurarea directiei pinilor
  pinMode(inputPin, INPUT);      // se declara pin-ul legat la LED ca iesire
  digitalWrite(inputPin,HIGH);  // activare rezistente pull up prin scrierea unei valori 'HIGH'
                                // pe pin-ul de intrare!
}

void loop(){
  int val = digitalRead(inputPin); // acelasi cod ca inainte
  if (val == HIGH)
  {
    digitalWrite(ledPin, HIGH);
  }
  else
  {
    digitalWrite(ledPin, LOW);
  }
}
```



- **Citirea unor date de intrare instabile**

- Un contact mecanic poate oscila între poziția “închis” și “deschis” de mai multe ori până la stabilizare.
- Un microcontroller poate fi suficient de rapid pentru a sesiza unele dintre aceste oscilații, percependu-le ca apăsări multiple pe buton.
- Unele dispozitive, precum Pmod BTN, au circuite speciale pentru eliminarea acestor oscilații.
- Dacă aceste circuite nu există, problema trebuie rezolvată prin software.





- **Citirea unor date de intrare instabile**

- Principiul filtrării oscilațiilor prin software: se verifică starea intrării de mai multe ori, până când aceasta nu se mai modifică.
- Efectul: ignorarea perioadei de instabilitate, validând intrarea doar atunci când aceasta e stabilă.

```
const int inputPin = 2;  
const int ledPin = 13;  
const int debounceDelay = 10; // Intervalul de timp (ms) in care semnalul trebuie sa fie stabil
```

```
boolean debounce(int pin) // Functia returneaza starea intrarii, dupa stabilizare  
{  
    boolean state; // Stare curenta, stare anterioara  
    boolean previousState;  
  
    previousState = digitalRead(pin); // Prima stare  
    for(int counter=0; counter < debounceDelay; counter++) // Se parcurge intervalul de timp  
    {  
        delay(1); // Se asteapta 1 ms  
        state = digitalRead(pin); // Citire stare prezenta  
        if( state != previousState) // Daca starile difera, o luam de la inceput  
        {  
            counter = 0; // Numaratorul primeste din nou valoarea zero  
            previousState = state; // Starea curenta devine starea initiala pentru noul ciclu  
        }  
    }  
    // // Daca s-a ajuns aici, inseamna ca semnalul a fost stabil tot intervalul de timp  
    return state; // Se returneaza starea curenta, stabila  
}
```




- Citirea unor date de intrare instabile

```
void setup()
{
  pinMode(inputPin, INPUT);
  pinMode(ledPin, OUTPUT);
}

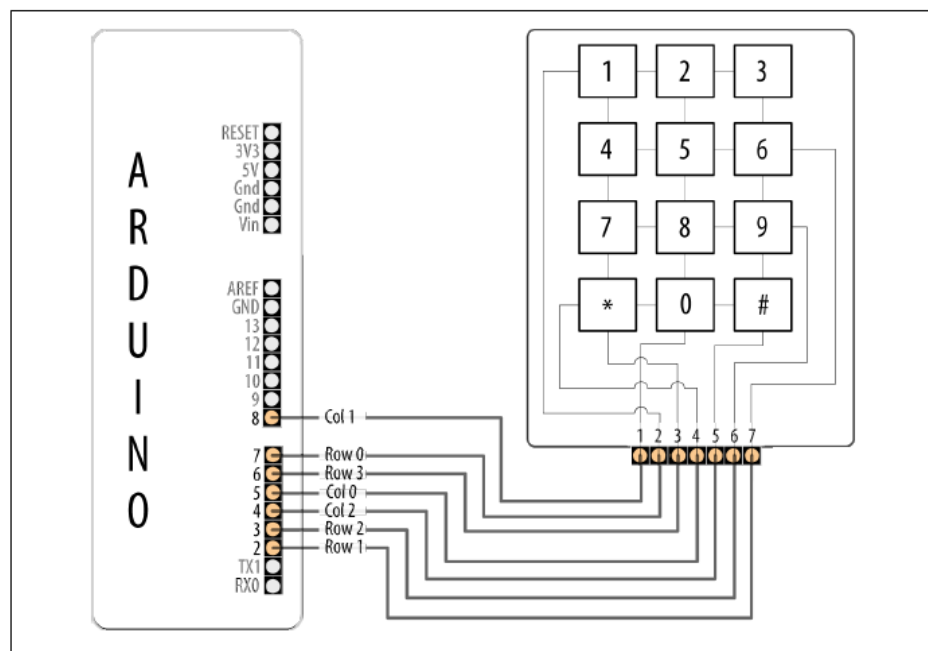
void loop()
{
  if (debounce(inputPin))           // Se foloseste functia debounce() in loc de digitalRead()
  {
    digitalWrite(ledPin, HIGH);
  }
}
```



Intrare / Ieșire la sistemele Arduino



- **I/O pe mai mulți pini. Utilizarea unei tastaturi**
 - Apăsarea unei taste face contact între coloană și rând
 - Starea rândurilor este implicit '1', prin folosirea unor rezistențe "pull-up"
 - Dacă coloana pe care se află o tastă este '0', și tasta este apăsată, rândul tastei devine '0'. Dacă coloana pe care se află o tastă este '1', nu se întâmplă nimic la apăsarea tastei.
 - Principiu: activarea pe rând a coloanelor (punerea lor pe rând la '0'), și citirea stării rândurilor
 - Coloanele trebuie legate la pini configurați ca ieșire, rândurile la pini configurați ca intrare



Arduino pin	Keypad connector	Keypad row/column
2	7	Row 1
3	6	Row 2
4	5	Column 2
5	4	Column 0
6	3	Row 3
7	2	Row 0
8	1	Column 1



- **I/O pe mai mulți pini. Utilizarea unei tastaturi**

```
const int numRows = 4;      // number of rows in the keypad
const int numCols = 3;      // number of columns
const int debounceTime = 20; // number of milliseconds for switch to be stable

// Se definesc pinii atasati randurilor si coloanelor, aranjati in ordinea logica
const int rowPins[numRows] = { 7, 2, 3, 6 }; // Rows 0 through 3
const int colPins[numCols] = { 5, 8, 4 };    // Columns 0 through 2

// LUT pentru identificarea tastei de la intersectia unui rand cu o coloana
const char keymap[numRows][numCols] = {
  { '1', '2', '3' },
  { '4', '5', '6' },
  { '7', '8', '9' },
  { '*', '0', '#' }
};

void setup() // Initializarea sistemului
{
  Serial.begin(9600); // Initializarea interfetei seriale via USB, folosita pentru comunicarea cu calculatorul
  for (int row = 0; row < numRows; row++)
  {
    pinMode(rowPins[row], INPUT);      // Pinii randurilor sunt intrare
    digitalWrite(rowPins[row], HIGH);  // Se activeaza rezistentele pull-up
  }
  for (int column = 0; column < numCols; column++)
  {
    pinMode(colPins[column], OUTPUT);   // Pinii coloanelor sunt iesire

    digitalWrite(colPins[column], HIGH); // Initial toate sunt '1', inactive
  }
}
```



- **I/O pe mai mulți pini. Utilizarea unei tastaturi**

```
void loop()
{
    char key = getKey();    // Se apeleaza functia de citire a unei taste (mai jos)
    if( key != 0) {          // Daca functia returneaza '0', nicio tasta nu este apasata
                            // daca rezultatul e diferit de zero, este apasata o tasta, si functia returneaza codul acesteia
        Serial.print("Got key ") // Folosirea interfetei seriale pentru a afisa in consola mesajul tasta apasata
        Serial.println(key);     // si codul acestei taste
    }
}

// functia principala: returneaza codul tastei, sau 0 daca nicio tasta nu e apasata.
char getKey()
{
    char key = 0;            // codul implicit zero, nicio tasta apasata

    for(int column = 0; column < numCols; column++) // baleierea coloanelor
    {
        digitalWrite(colPins[column],LOW);          // se activeaza coloana curenta
        for(int row = 0; row < numRows; row++)        // se verifica randurile unul cate unul

        {
            if(digitalRead(rowPins[row]) == LOW)      // daca randul e '0', avem tasta apasata pe acel rand
            {
                delay(debounceTime);                  // intarziere pentru filtrare intrare
                while(digitalRead(rowPins[row]) == LOW) // asteptare eliberare tasta
                ;

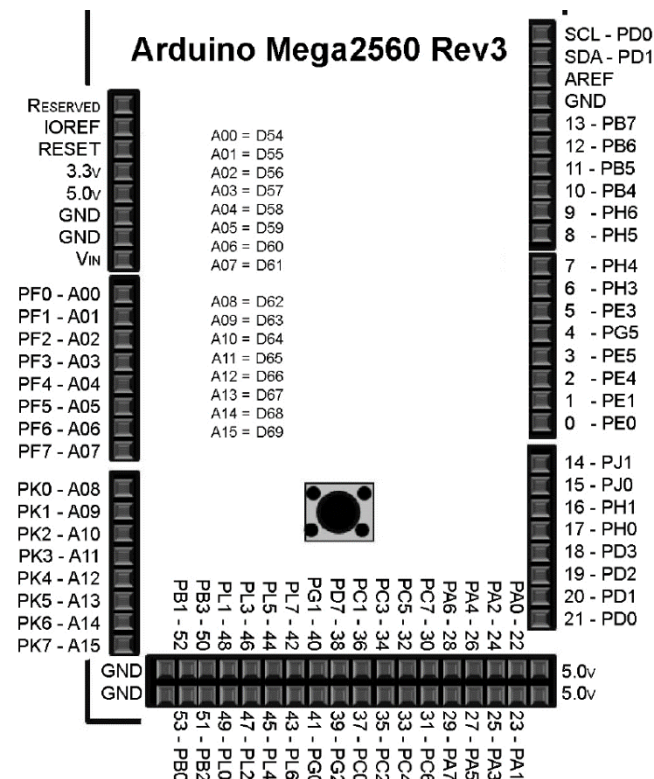
                key = keymap[row][column];              // se cunoaste coloana si randul tastei apasate
                                                        // se foloseste LUT pentru determinarea codului ASCII al tastei
            }
        }
        digitalWrite(colPins[column],HIGH);          // dezactivare coloana
    }
    return key; // returneaza codul tastei, sau 0
}
```



Intrare / ieșire la sistemele Arduino



- I/O folosind porturile microcontrollerului
- Dezavantaje
 - Abordare dependentă de hardware, nu este portabilă între plăci diferite
 - Trebuie cunoscută relația dintre pin și portul/bitul corespunzător
 - Unele porturi sunt rezervate, și modificarea stării lor nu este recomandabilă
- Avantaje
 - Viteza ridicată. Scrierea și citirea unui port sunt de aproximativ 10 ori mai rapide decât `digitalWrite()` și `digitalRead()`
 - Posibilitatea de a citi mai mulți pini simultan, sau de a scrie mai mulți pini simultan (`digitalRead` și `digitalWrite` lucrează doar la nivel de pin)





- **Exemplu:** se conectează 8 led-uri la pinii 22...29 ai Arduino Mega (la PortA). Se dorește aprinderea alternativă a led-urilor pare și impare, cu o întârziere de 1 secundă între comutații
- **Cod sursa, abordarea clasică Arduino:**

```
const int PortAPins[8]={22, 23, 24, 25, 26, 27, 28, 29}

void setup()
{
    for (int b=0; b<8; b++)
        pinMode(PortAPins[b], OUTPUT);           // toti pinii sunt configurati ca iesire
}
void loop()
{
    for (int b=0; b<8; b+=2)                       // b=0, 2, 4, 6
    {
        digitalWrite(PortAPins[b], HIGH);          // scriem '1' pe pinii pari
        digitalWrite(PortAPins[b+1], LOW);          // scriem '0' pe pinii impari
    }
    delay(1000);                                     // asteptare de 1 secunda (1000 ms)
    for (int b=0; b<8; b+=2)
    {
        digitalWrite(PortAPins[b], LOW);           // scriem '0' pe pinii pari
        digitalWrite(PortAPins[b+1], HIGH);         // scriem '1' pe pinii pari
    }
    delay(1000);                                     // asteptare de 1 secunda (1000 ms)
}
```



- **Exemplu:** se conectează 8 led-uri la pinii 22...29 ai Arduino Mega (la PortA). Se dorește aprinderea alternativă a led-urilor pare și impare, cu o întârziere de 1 secundă între comutații
- **Cod sursa, abordarea folosind portul A al ATmega2560 :**

```
void setup()
{
    DDRA = 0B11111111;           // toti pinii atasati portului A sunt configurati ca iesire
}

void loop()
{
    PORTA = 0B01010101;          // 1 pe pinii pari, 0 pe pinii impari
    delay(1000);                 // asteptare de 1 secunda (1000 ms)
    PORTA = 0B10101010;          // 0 pe pinii pari, 1 pe pinii impari
    delay(1000);                 // asteptare de 1 secunda (1000 ms)
}
```



1. **Atmel ATmega640/V-1280/V-1281/V-2560/V-2561/V datasheet**
2. **Atmel Atmega64 datasheet**



Proiectarea cu Micro-Procesoare

Lector: Mihai Negru

An 3 – Calculatoare și Tehnologia Informației
Seria B

Curs 3: Întreruperi Externe

<http://users.utcluj.ro/~negrum/>



Întreruperi



- Mecanismul de întreruperi permite micro-controlerului să răspundă la evenimente externe, sau la evenimente produse de perifericele integrate pe chip.
- În lipsa evenimentelor, procesorul poate executa programul principal, sau poate intra în stare de inactivitate (SLEEP) pentru a conserva energie.
- Tratarea întreruperilor este activată sau dezactivată prin bit-ul 7 din registrul SREG

Bit	7	6	5	4	3	2	1	0	
0x3F (0x5F)	I	T	H	S	V	N	Z	C	SREG
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- Instrucțiuni
 - **SEI** – activează sistemul de întreruperi ($SREG(7) = 1$)
 - **CLI** – dezactivează sistemul de întreruperi ($SREG(7)=0$)



- Tratarea unei întreruperi – cazul ATmega64
 1. Dispozitivul periferic generează cererea de întrerupere
 2. Se finalizează execuția instrucțiunii curente
 3. PC se salvează pe stivă
 - **TOS = PC**
 - **SP = SP – 2**
 4. Accesarea vectorului specific tipului de întrerupere
 5. Execuția saltului la Procedura de Tratare a Întreruperii (*Interrupt Service Routine, ISR*)
 6. Blocare flag întreruperi (**CLI**)
 7. Execuție ISR
 8. Revenire din ISR (**reti**)
 - **SP = SP + 2**
 - **PC = TOS**
 - Activare flag întreruperi (**SEI**)
- *Daca in ISR se activează întreruperile prin apel SEI, se pot executa întreruperi imbricate (nested).*

reti este echivalent cu **sei + ret**



Întreruperi – Surse si vectori de întrerupere (1)



Adrese absolute in
memoria program
(flash)

Vector No.	Program Address ⁽²⁾	Source	Interrupt Definition
1	0x0000 ⁽¹⁾	RESET	External Pin, Power-on Reset, Brown-out Reset, Watchdog Reset, and JTAG AVR Reset
2	0x0002	INT0	External Interrupt Request 0
3	0x0004	INT1	External Interrupt Request 1
4	0x0006	INT2	External Interrupt Request 2
5	0x0008	INT3	External Interrupt Request 3
6	0x000A	INT4	External Interrupt Request 4
7	0x000C	INT5	External Interrupt Request 5
8	0x000E	INT6	External Interrupt Request 6
9	0x0010	INT7	External Interrupt Request 7
10	0x0012	TIMER2 COMP	Timer/Counter2 Compare Match
11	0x0014	TIMER2 OVF	Timer/Counter2 Overflow
12	0x0016	TIMER1 CAPT	Timer/Counter1 Capture Event
13	0x0018	TIMER1 COMPA	Timer/Counter1 Compare Match A
14	0x001A	TIMER1 COMPB	Timer/Counter1 Compare Match B
15	0x001C	TIMER1 OVF	Timer/Counter1 Overflow
16	0x001E	TIMER0 COMP	Timer/Counter0 Compare Match
17	0x0020	TIMER0 OVF	Timer/Counter0 Overflow
18	0x0022	SPI, STC	SPI Serial Transfer Complete



Întreruperi – Surse si vectori de întrerupere (2)



Vector No.	Program Address ⁽²⁾	Source	Interrupt Definition
19	0x0024	USART0, RX	USART0, Rx Complete
20	0x0026	USART0, UDRE	USART0 Data Register Empty
21	0x0028	USART0, TX	USART0, Tx Complete
22	0x002A	ADC	ADC Conversion Complete
23	0x002C	EE READY	EEPROM Ready
24	0x002E	ANALOG COMP	Analog Comparator
25	0x0030 ⁽³⁾	TIMER1 COMPC	Timer/Counter1 Compare Match C
26	0x0032 ⁽³⁾	TIMER3 CAPT	Timer/Counter3 Capture Event
27	0x0034 ⁽³⁾	TIMER3 COMPA	Timer/Counter3 Compare Match A
28	0x0036 ⁽³⁾	TIMER3 COMPB	Timer/Counter3 Compare Match B
29	0x0038 ⁽³⁾	TIMER3 COMPC	Timer/Counter3 Compare Match C
30	0x003A ⁽³⁾	TIMER3 OVF	Timer/Counter3 Overflow
31	0x003C ⁽³⁾	USART1, RX	USART1, Rx Complete
32	0x003E ⁽³⁾	USART1, UDRE	USART1 Data Register Empty
33	0x0040 ⁽³⁾	USART1, TX	USART1, Tx Complete
34	0x0042 ⁽³⁾	TWI	Two-wire Serial Interface
35	0x0044 ⁽³⁾	SPM READY	Store Program Memory Ready



Întreruperi Externe



- Întreruperi externe – cauzate de activitate pe pinii externi INT7...INT0
- Pinii INT7:INT0 sunt comuni cu pinii porturilor **D** si **E** – daca porturile sunt configurate ca ieşire, se pot declanşa întreruperi software prin scrierea acestor porturi.
- Configurarea modului de sesizare a întreruperilor externe – regiştrii **EICRA** si **EICRB** – in total 16 biţi, 2 biţi / întrerupere

Bit	7	6	5	4	3	2	1	0	
(0x6A)	ISC31	ISC30	ISC21	ISC20	ISC11	ISC10	ISC01	ISC00	EICRA
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	
Bit	7	6	5	4	3	2	1	0	
0x3A (0x5A)	ISC71	ISC70	ISC61	ISC60	ISC51	ISC50	ISC41	ISC40	EICRB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

ISCn1	ISCn0
0	0
0	1
1	0
1	1

Nivel '0' pe INTn generează cerere de întrerupere

Rezervat

Front descrescător pe INTn generează cerere de întrerupere

Front crescător pe INTn generează cerere de întrerupere



Întreruperi Externe



- Scrierea/citirea regiștrilor de control a întreruperilor externe
 - **EICRB** se poate scrie/citi cu **in, out**
 - **EICRA** se poate scrie/citi cu **lds, sts**
- Activarea punctuală a întreruperilor externe – folosirea registrului **EIMSK**
- Fiecare întrerupere este controlată de un bit al acestui registru
- Setarea la '1' a bitului corespunzător activează întreruperea
- Registrul **EIMSK** se poate citi/scrie cu **in, out**

Bit	7	6	5	4	3	2	1	0	
0x39 (0x59)	INT7	INT6	INT5	INT4	INT3	INT2	INT1	INT0	EIMSK
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	



Întreruperi Externe – Exemplu



- Incrementarea unui numărător prin apăsarea unui buton, folosind mecanismul întreruperilor externe – butonul este conectat la întreruperea externa INT0

```
.org 0x0000                ; adresa vectorului pentru întreruperea reset
    rjmp main
.org 0x0002                ; adresa vectorului pentru întreruperea externa INT0
    rjmp isr_INT0

main:
    ldi r16, high(RAMEND)   ; inițializare stiva – necesara când folosim întreruperi!
    out SPH, R16
    ldi r16, low(RAMEND)
    out SPL, R16

    ldi r16, 0b00000011    ; configurare mod de tratare INT0 – front crescător
    sts EICRA, r16
    ldi r16, 0b00000001    ; activare întrerupere INT0
    out EIMSK, r16

    ldi r16, 0xFF          ; setăm direcția portului E – ieșire pentru numărător
    out DDRE, r16

    ldi r17, 0             ; valoarea inițială a numărătorului
    sei                   ; activare globală a sistemului de întreruperi
```




Întreruperi Externe – Exemplu



- Incrementarea unui numărător prin apăsarea unui buton, folosind mecanismul întreruperilor externe – butonul este conectat la întreruperea externă INT0

```
loop:
    out PORTE, r17          ; scriem numărătorul pe portul E (LED-uri)
    rjmp loop

isr_INT0:                  ; începutul rutinei de tratare a întreruperii INT0
    inc r17                 ; doar incrementăm numărătorul
    reti                   ; revenire din întrerupere
```

- Exercițiu: cum se rezolvă problema incrementării unui numărător prin apăsarea unui buton, fără folosirea sistemului de întreruperi?
- Folosiți două butoane, unul pentru incrementare și unul pentru decrementare.



- Detectarea unor evenimente pe pini, fără a verifica în permanență starea acestora prin `digitalRead`
- Pentru utilizarea unei întreruperi, trebuie să îi atașăm o procedură de tratare a întreruperii (Interrupt Service Routine – ISR). În acest scop, se folosește funcția **`attachInterrupt()`**, cu sintaxa:

`attachInterrupt(interrupt, ISR, mode)`

<i>interrupt</i>	– numărul întreruperii externe (0, 1, 2, ...)
ISR	– numele procedurii de tratare a întreruperii (o procedură din program)
mode	– modul de declanșare:
LOW	– declanșare pe nivel '0'
CHANGE	– declanșare când se schimbă nivelul pinului
RISING	– declanșare pe front crescător
FALLING	– declanșare pe front descrescător



- Dezactivarea tratării unei întreruperi se face prin apelul funcției **detachInterrupt()**, cu sintaxa:

`detachInterrupt(interrupt)`

interrupt

– numărul întreruperii

- Dacă se dorește dezactivarea temporară a tuturor întreruperilor, se apelează funcția **noInterrupts()**, fără parametri. Pentru re-activarea întreruperilor, se apelează funcția **interrupts()**.
- Întreruperile sunt active implicit! Dezactivarea lor trebuie făcută doar pentru perioade scurte de timp, în caz contrar alte funcții arduino pot fi afectate.



- **Exemplu:** măsurarea lăţimii pulsurilor unui semnal (de exemplu, semnalul recepţionat de un senzor IR de la o telecomandă, unde lăţimea pulsului face diferenţa dintre un '0' şi un '1')

```
const int irReceiverPin = 2;           // Pinul 2. unde se gaseste intreruperea externa 0
const int numberOfEntries = 64;        // Numarul de tranzitii de analizat

volatile unsigned long microseconds; //Variabila care tine numarul de microsecunde trecute de la pornirea programului
volatile byte index = 0;               //Pozitia in sirul de tranzitii
volatile unsigned long results[numberOfEntries]; //Sirul cu duratele tranzitiilor analizate (rezultat)

void setup()
{
  pinMode(irReceiverPin, INPUT);        //Se declara pinul intreruperii ca intrare
  Serial.begin(9600);                   //Se activeaza comunicarea prin USB, pentru afisare date
  attachInterrupt(0, analyze, CHANGE);  //Se ataseaza ISR-ul analyze la intreruperea 0, declansata la schimbarea
  results[0]=0;                         semnalului
}

void loop()
{
  if(index >= numberOfEntries)           // Se verifica daca numarul de tranzitii maxim a fost analizat
  {
    Serial.println("Durations in Microseconds are:") ; // Daca da, se afiseaza toate intervalele masurate
    for( byte i=0; i < numberOfEntries; i++)
    {
      Serial.println(results[i]);
    }
    index = 0; // Dupa afisare, se re-initializeaza contorul de tranzitii la 0, si procesul se reia
  }
  delay(1000);
}
```



- **Exemplu:** măsurarea lăţimii pulsurilor unui semnal (de exemplu, semnalul recepţionat de un senzor IR de la o telecomandă, unde lăţimea pulsului face diferenţa dintre un '0' şi un '1')

```
void analyze()           // Procedura de tratare a intreruperii
{
    if(index < numberOfEntries )    // Daca nu s-a ajuns la capatul sirului
    {
        if(index > 0)               // Dar nu este prima tranzitie detectata
        {
            results[index] = micros() - microseconds; // Se masoara timpul trecut de la ultima tranzitie
        }
        index = index + 1;          // Se incrementeaza indexul
    }
    microseconds = micros();        // Se retine timpul curent, pentru a fi etalon pentru tranzitia urmatoare
}
```

- Funcţia **micros()** returnează numărul de microsecunde de la pornirea programului.
- Pentru măsurarea unor intervale de timp mai mari, cu precizie mai mică, se poate folosi funcţia **millis()**, care returnează numărul de milisecunde de la pornirea programului.



- **Atenție:**
- Toate variabilele care se pot modifica într-o funcție de tip ISR trebuie să fie declarate ca “**volatile**”. Astfel, compilatorul va ști că aceste variabile se pot modifica în orice moment, și nu va optimiza codul prin atașarea acestora la regiștri, ci le va stoca întotdeauna în RAM
- Doar o procedură ISR poate rula la un moment dat, celelalte întreruperi fiind dezactivate în acest timp
- Deoarece **delay()** și **millis()** folosesc la rândul lor întreruperi, ele nu vor funcționa în timpul execuției unei proceduri ISR.
- Pentru întârzieri scurte într-o procedură ISR, se poate folosi funcția **delayMicroseconds()**, care nu folosește întreruperi.
- Nu se recomandă scrierea datelor prin interfața serială într-o procedură ISR



- Numărul specificat ca parametru nu este același cu numărul întreruperii externe al microcontrollerului AVR:

attachInterrupt	Name	Pin on chip (TQFP)	Pin on board
0	INT4	6	D2
1	INT5	7	D3
2	INT0	43	D21
3	INT1	44	D20
4	INT2	45	D19
5	INT3	46	D18

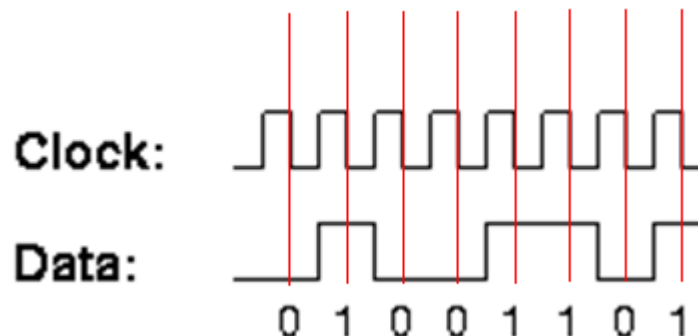
- Soluția: utilizarea funcției `digitalPinToInterrupt(pin)`
 - Exemplu:
 - `attachInterrupt(digitalPinToInterrupt(21), isr, FALLING)` va atașa întreruperii Arduino **2**, corespunzătoare întreruperii externe **INT0** de la ATmega2560, accesibilă prin pinul digital 21, procedura **isr**, care se va apela când pe pin se va efectua o tranziție din HIGH în LOW.
 - Dacă un pin digital nu are și funcție de întrerupere, funcția `digitalPinToInterrupt` va returna valoarea -1.



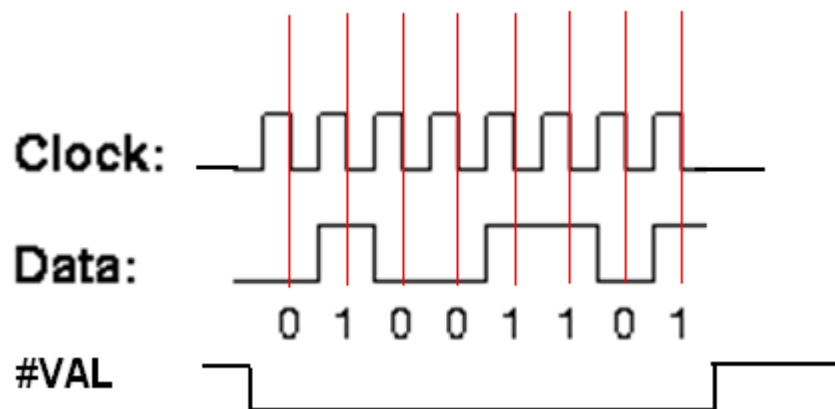
Exerciții – Arduino



- Afișați, prin interfața serială, numărul pinilor care au și funcție de întrerupere.
- Scrieți un program capabil să preia date seriale, sincronizate pe un semnal de ceas, ca în figura de mai jos:



- Modificați programul pentru a utiliza un semnal suplimentar, care marchează începutul și sfârșitul octetului:





1. **Atmel ATmega640/V-1280/V-1281/V-2560/V-2561/V datasheet**
2. <http://arduino.cc/en/Hacking/PinMapping2560>
3. **Atmel Atmega64 datasheet**



Proiectarea cu Micro-Procesoare

Lector: Mihai Negru

An 3 – Calculatoare și Tehnologia Informației

Seria B

Curs 4: Temporizatoare

<http://users.utcluj.ro/~negrum/>



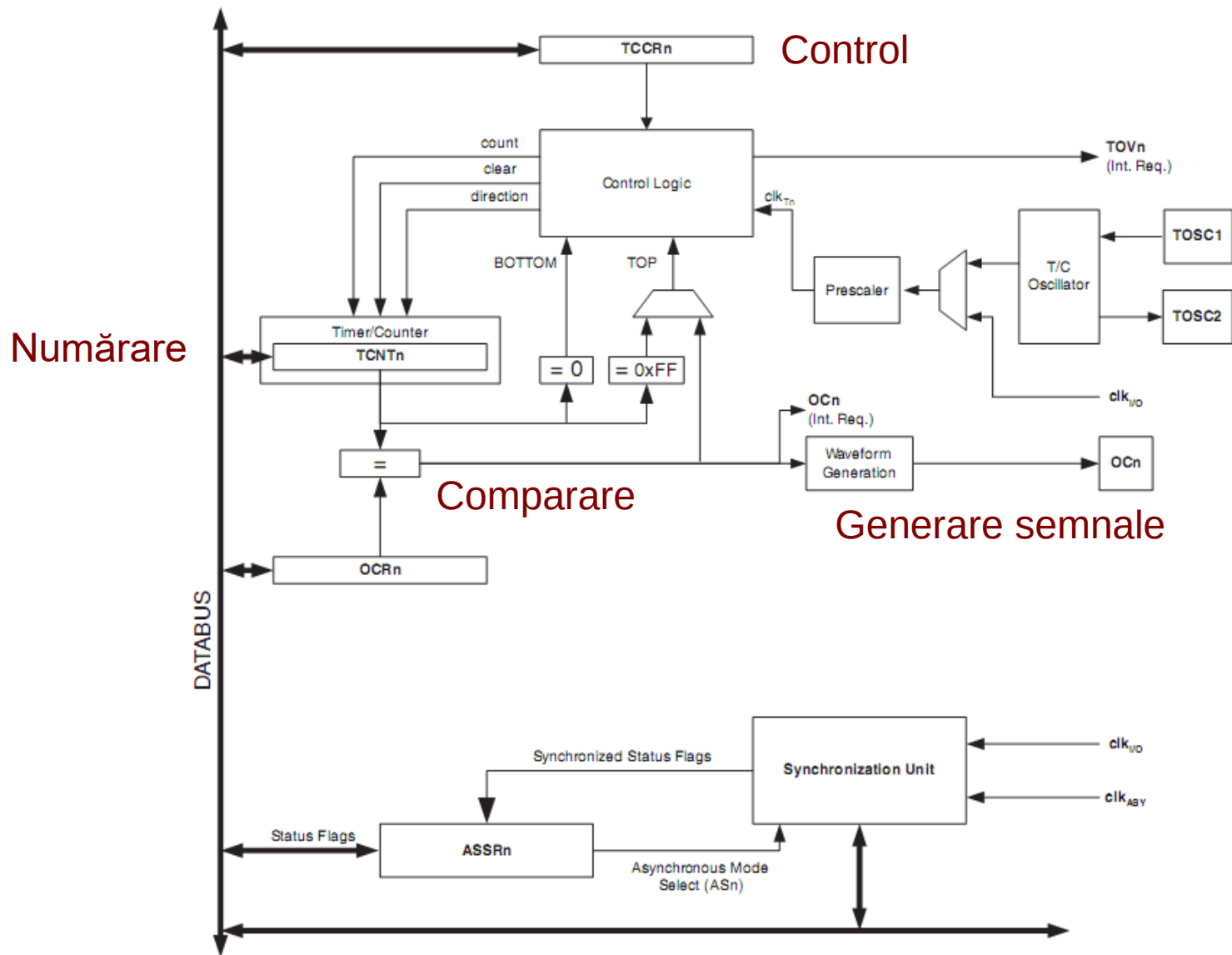
Temporizatoare (Timers)



- **Resurse ATmega 2560**
 - Două numărătoare/ temporizatoare pe 8 biți (Timer 0, Timer 2)
 - Patru numărătoare / temporizatoare pe 16 biți (Timer 1, 3, 4, 5)
- **Caracteristici**
 - Alegerea frecvenței ceasului la intrarea temporizatoarelor (prescaler)
 - Citire / scriere stare numărator
 - Generare de forme de undă prin folosirea unui registru de comparare
 - Reglare frecvență și factor de umplere – PWM (pulse width modulation)
 - Generare de cereri de întrerupere la intervale regulate
 - Declanșare la un eveniment extern (capture)
- **Utilizări**
 - Generarea diferitelor forme de undă
 - Sincronizarea programului cu intervale de timp regulate
 - Măsurare intervale de timp



Structura temporizator 8 biți

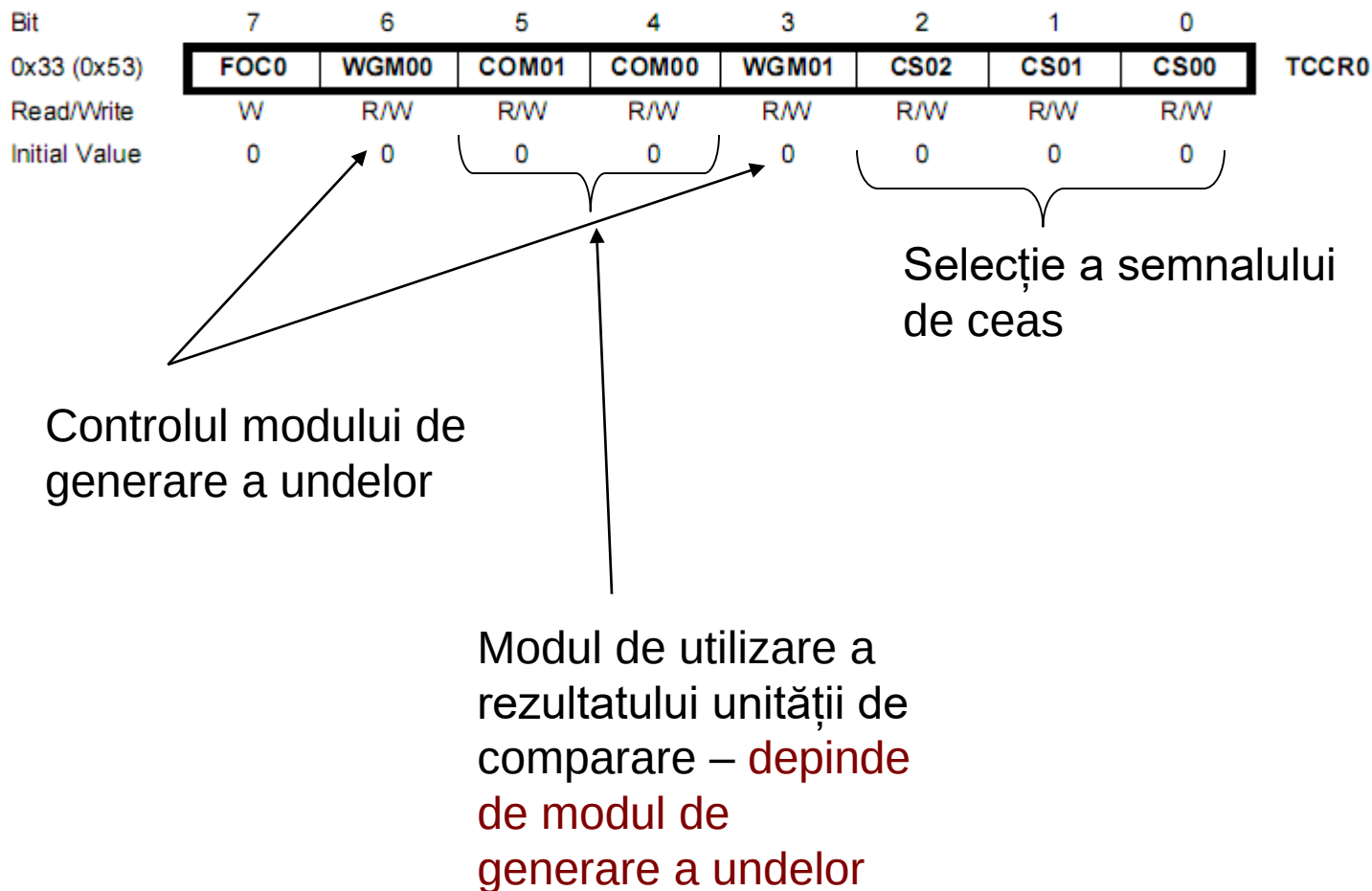




Configurare temporizator 8 biți



- Registrul TCCRn – controlează comportamentul temporizatorului



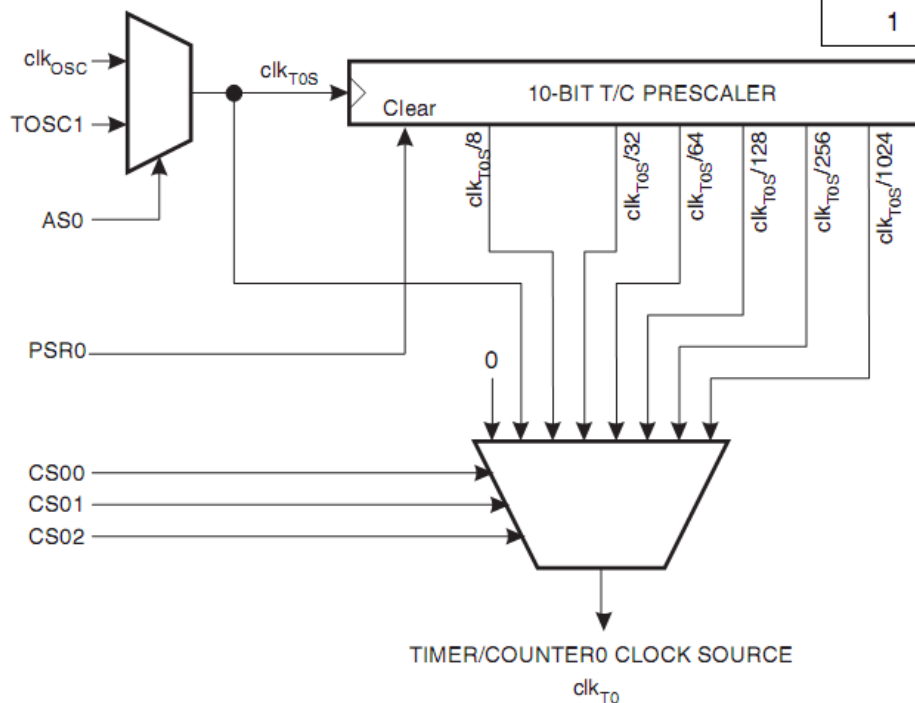


Selecția semnalului de ceas



- Biții CS02:CS00 selectează divizarea frecvenței ceasului la intrarea în numărator
- Reglarea vitezei de numărare

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/counter stopped)
0	0	1	clk _{TOS} /(No prescaling)
0	1	0	clk _{TOS} /8 (From prescaler)
0	1	1	clk _{TOS} /32 (From prescaler)
1	0	0	clk _{TOS} /64 (From prescaler)
1	0	1	clk _{TOS} /128 (From prescaler)
1	1	0	clk _{TOS} /256 (From prescaler)
1	1	1	clk _{TOS} /1024 (From prescaler)

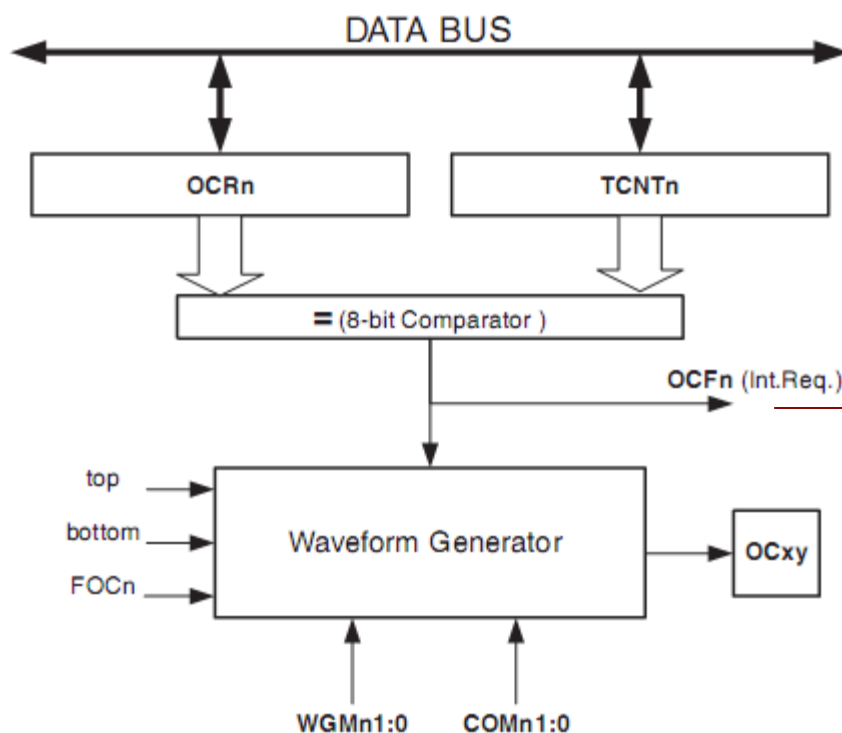




Unitatea de comparare



- Comparatia dintre registrul care conține valoarea numărătorului, **TCNT0**, si registrul valorii de comparat, **OCR0**, este folosită pentru a genera diferite forme de undă



Output compare flag – la producerea egalității acest semnal cere întrerupere

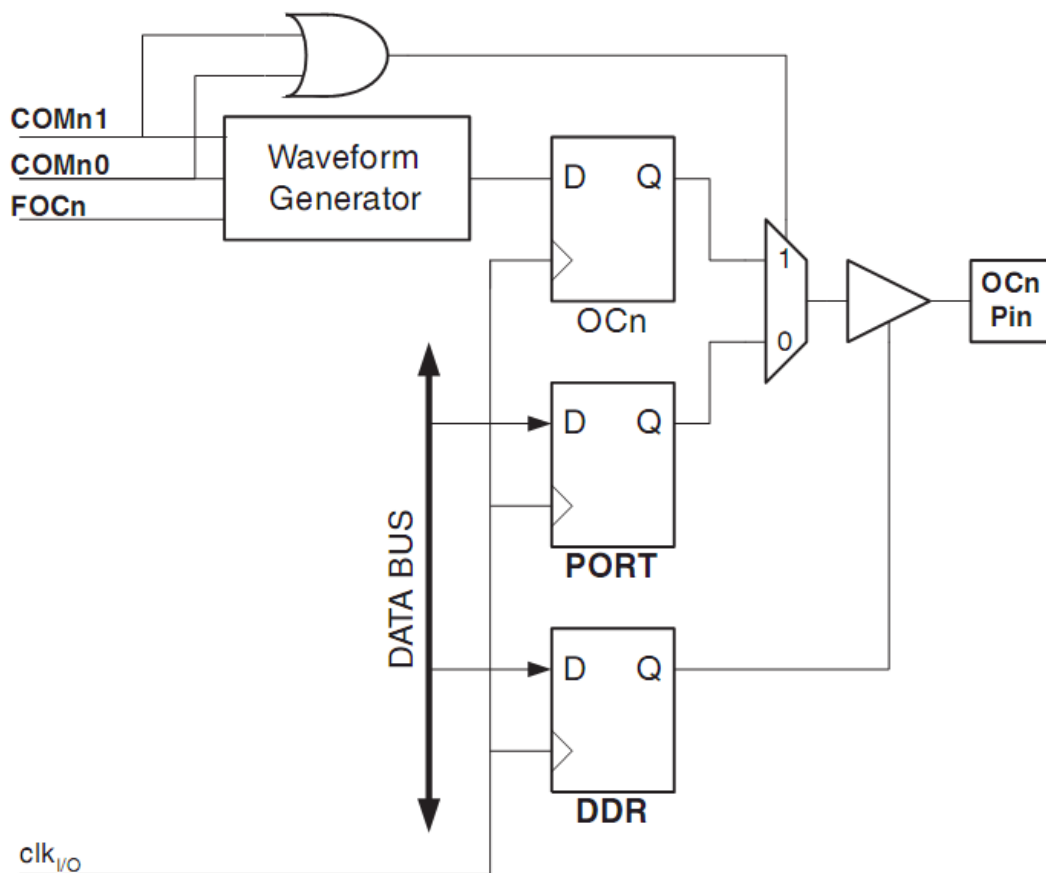
Bit de output compare – aici va fi generată unda



Unitatea de comparare



- Undele generate sunt vizibile la exterior prin pinii porturilor I/O
- Bitul din portul I/O corespunzator bitului OCn trebuie configurat ca ieșire



Semnalul OC0 este comun cu bitul 4 din portul B



Tipuri de unde (moduri de functionare)



Mode	WGM00:01	Mode
0	00	Normal
1	10	PWM, Phase Correct
2	01	CTC
3	11	Fast PWM

Biții WGM00:01 în combinație cu biții COM1:0 definesc comportamentul temporizatorului!

Normal, CTC

COM0[1:0]	Description
00	Normal, OC0 disconnected
01	Toggle OC0 on compare match
10	Clear OC0 on compare match
11	Set OC0 on compare match

PWM, Phase Correct

COM0[1:0]	Description
00	Normal, OC0 disconnected
01	Reserved
10	Clear OC0 on compare match when up-counting. Set OC0 on compare match when down counting
11	Set OC0 on compare match when up-counting. Clear OC0 on compare match when down counting.

Fast PWM

COM0[1:0]	Description
00	Normal, OC0 disconnected
01	Reserved
10	Clear OC0 on compare match, set OC0 at TOP
11	Set OC0 on compare match, clear OC0 at TOP



Tipuri de unde (moduri de functionare)

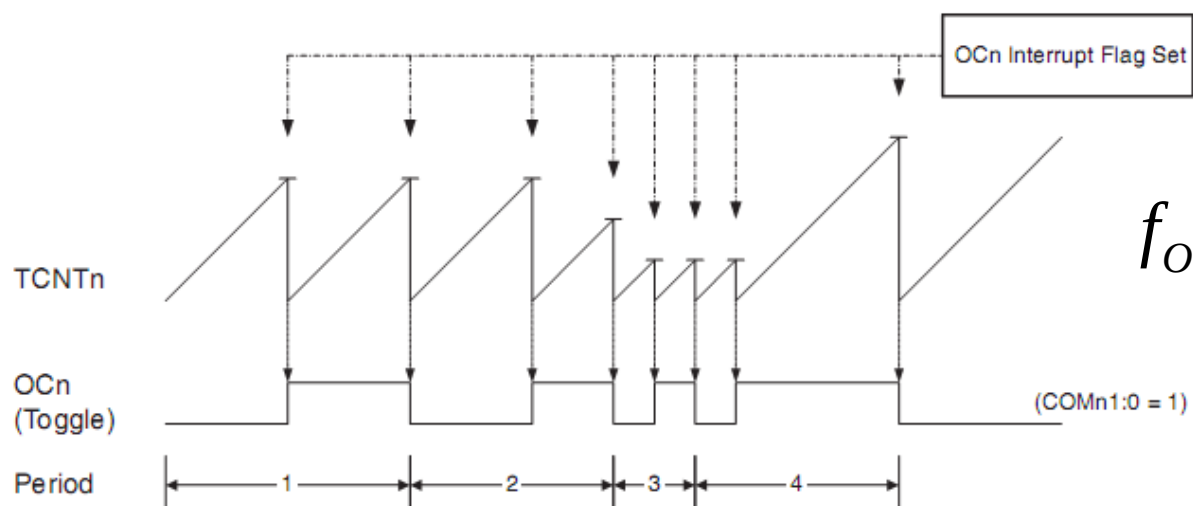


- **Normal**

- Numărare simplă: 0 → 255
- Când se produce saturarea → întrerupere de overflow, numărarea se reia de la 0

- **CTC – Clear Timer on Compare Match: 0 → OCR0**

- Când valoarea număratorului (TCNT0) ajunge la valoarea din OCR0, se produce resetarea
- Frecvența undelor generate se poate regla prin scrierea registrului OCR0
 - COM01:COM00 – setați pe 01 → basculare OC0 la egalitate



$$f_{OCn} = \frac{f_{clk_I/O}}{2N(1 + OCR_n)}$$

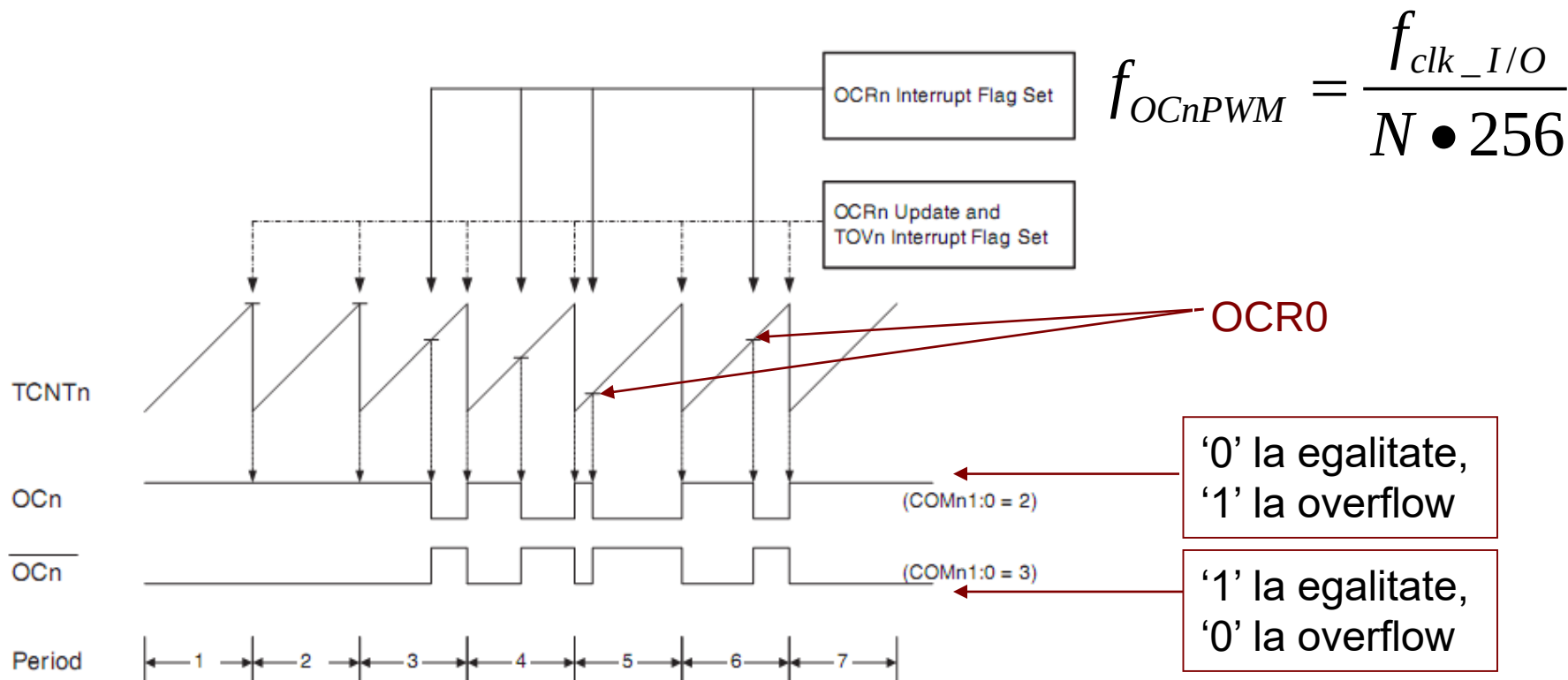


Tipuri de unde (moduri de functionare)



- **Fast Pulse Width Modulation (PWM)**

- Generare semnale PWM (modulare in latimea pulsului) pe OC0
- Factorul de umplere se regleaza prin scrierea registrului de comparare OCR0
- Frecventa este fixa, data de bitii de clock select
- Factorul de umplere = $OCR0 / 255$

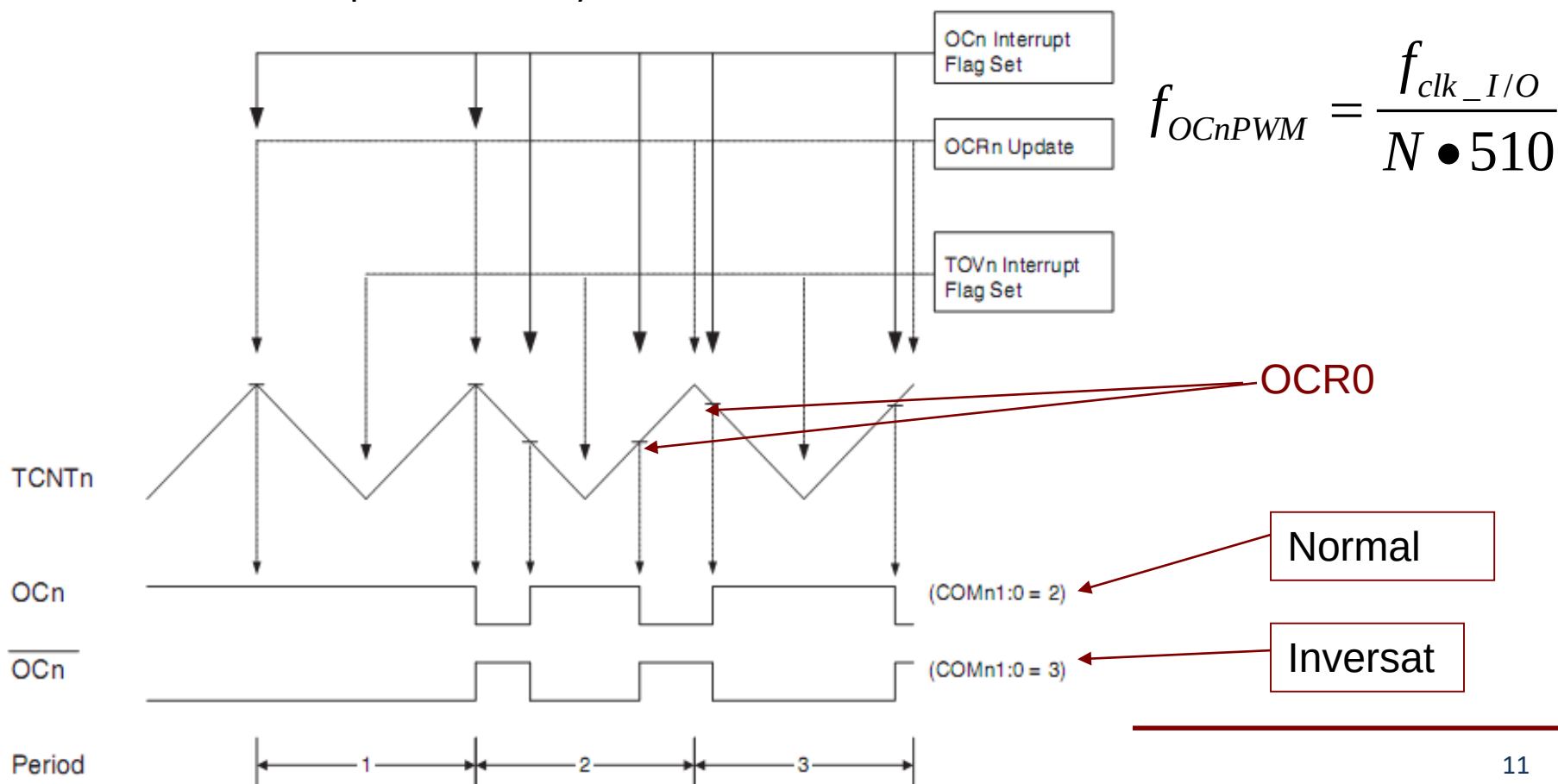




Tipuri de unde (moduri de functionare)



- **Phase Correct PWM mode** – Generare semnale PWM cu corecția fazei
 - Pulsul este simetric față de mijlocul perioadei
 - Factorul de umplere se reglează prin scrierea registrului de comparare OCR0
 - Numărare crescătoare / descrescătoare, ieșirea se modifică la egalități succesive
 - Factorul de umplere = $\text{OCR0} / 255$

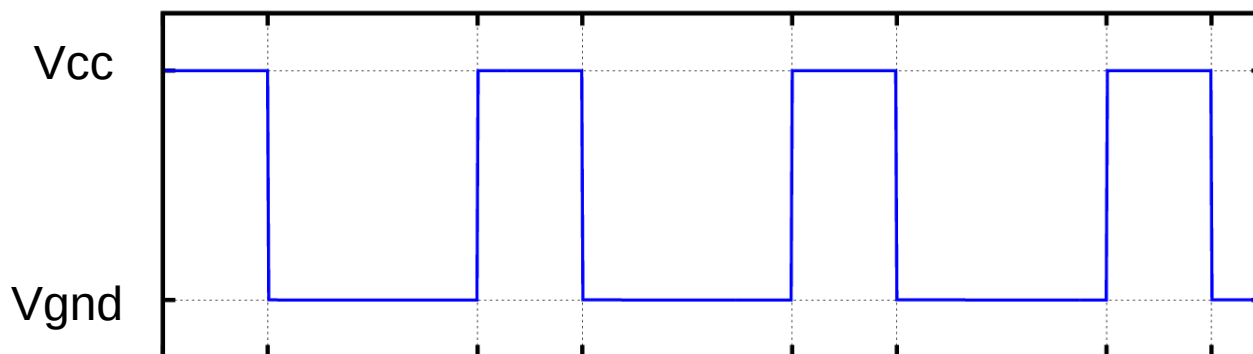




Pulse Width Modulation (PWM)



- Unele sisteme necesită control prin variația tensiunii de intrare
- Exemplu: turația unui motor, intensitatea unui LED
- Sistemele digitale pot produce la ieșire doar valori de 0 (GND) și 1 (Vcc)
- Factorul de umplere D (duty cycle)



$$D = \frac{T_{on}}{T_{on} + T_{off}}$$

- Tensiunea medie – poate fi oricât între cele doua extreme, depinde de D

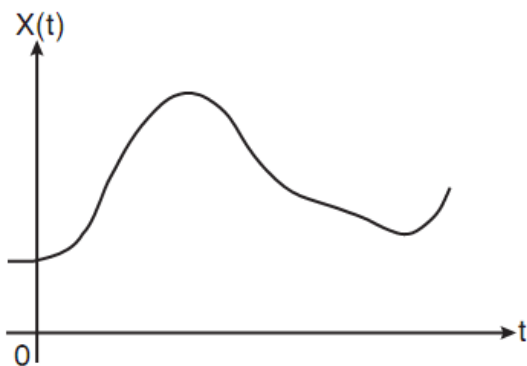
$$V_{MED} = DV_{CC} + (1-D)V_{GND}$$



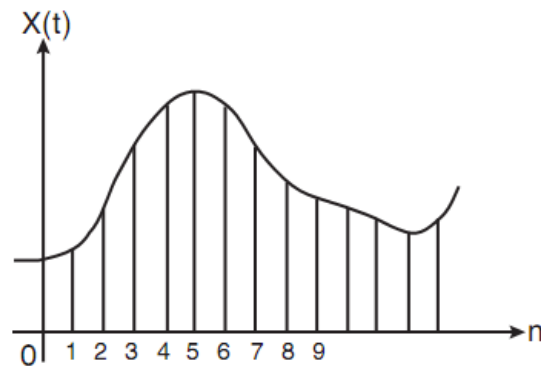
Pulse Width Modulation (PWM)



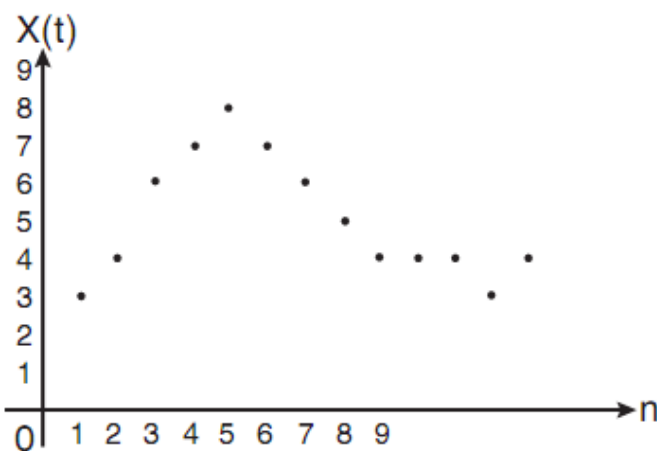
- Semnale analogice de joasă frecvență (ex. Sunet) pot fi codificate PWM
- Pași: eșantionare, digitizare, calculare D pe baza valorii digitale



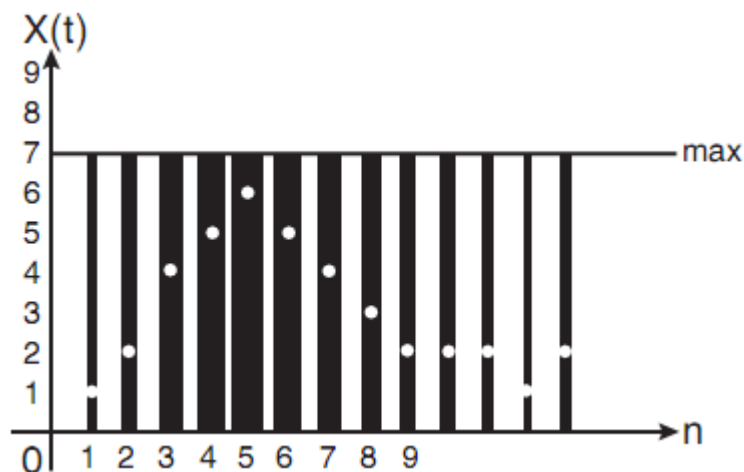
Semnal analogic



Eșantionare



Digitizare a eșantioanelor



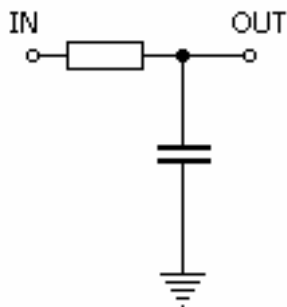
Calculare a factorului de umplere



Pulse Width Modulation (PWM)



- Utilizarea semnalului PWM
 - Se poate utiliza ca atare, dacă aplicația permite: variația luminozității unui bec, turația unui motor, etc – inerția dispozitivelor produce efectul de mediere a tensiunii
 - Se poate filtra printr-un filtru trece-jos, pentru a reconstrui semnalul analogic
 - Se reglează frecvența limită superioară (*cutoff frequency*) pentru domeniul de frecvență al semnalului analogic, care este mai mic decât frecvența semnalului purtător
 - Filtru trece jos simplu – filtrul RC



$$f_{cutoff} = \frac{1}{2\pi RC}$$



Întreruperi generate de temporizatoare



- Evenimentele care generează întreruperi:
 - Saturare (*overflow*)
 - Egalitate la comparare (*compare match*)
 - Eveniment extern (captură) – disponibil doar la temporizatoarele de 16 biți

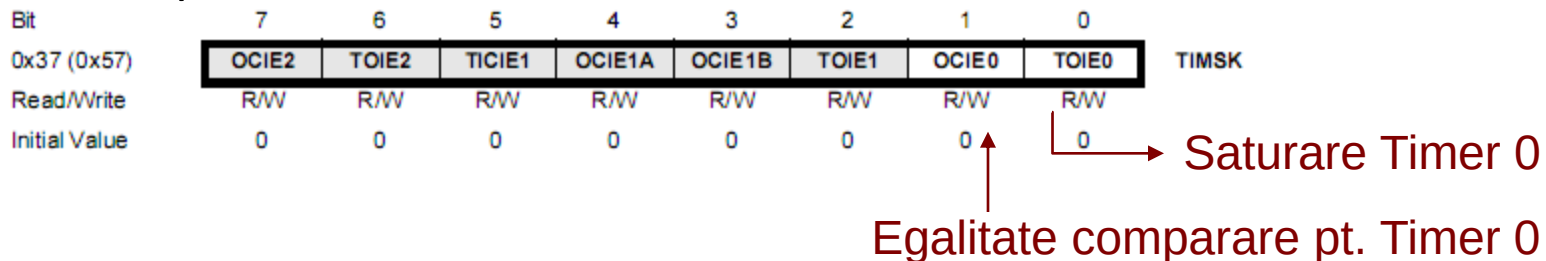
Adresa		Descriere
0x0012	TIMER2 COMP	Timer/Counter2 Compare Match
0x0014	TIMER2 OVF	Timer/Counter2 Overflow
0x0016	TIMER1 CAPT	Timer/Counter1 Capture Event
0x0018	TIMER1 COMPA	Timer/Counter1 Compare Match A
0x001A	TIMER1 COMPB	Timer/Counter1 Compare Match B
0x001C	TIMER1 OVF	Timer/Counter1 Overflow
0x001E	TIMER0 COMP	Timer/Counter0 Compare Match
0x0020	TIMER0 OVF	Timer/Counter0 Overflow
0x0030 ⁽³⁾	TIMER1 COMPC	Timer/Counter1 Compare Match C
0x0032 ⁽³⁾	TIMER3 CAPT	Timer/Counter3 Capture Event
0x0034 ⁽³⁾	TIMER3 COMPA	Timer/Counter3 Compare Match A
0x0036 ⁽³⁾	TIMER3 COMPB	Timer/Counter3 Compare Match B
0x0038 ⁽³⁾	TIMER3 COMPC	Timer/Counter3 Compare Match C
0x003A ⁽³⁾	TIMER3 OVF	Timer/Counter3 Overflow



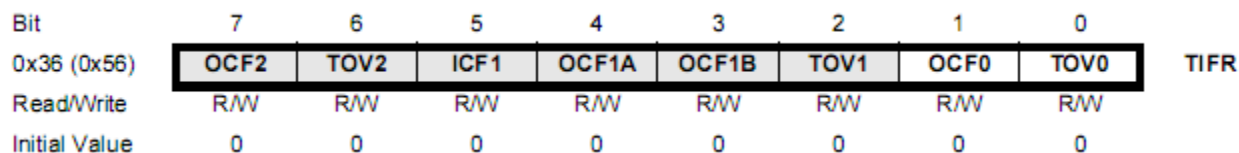
Întreruperi generate de temporizatoare



- Activare/ dezactivare întreruperi – registrul TIMSK (accesibil cu instrucțiuni I/O)



- Accesare stare întreruperi – registrul TIFR



- Utilizare posibilă a întreruperilor
 - Asigurarea unui interval regulat între diferite acțiuni programate – exemplu, comutarea între celulele unui SSD
 - Generare forme de undă software
 - Modificare parametri pentru generarea undelor hardware



Exemplu 1



- Generare semnal de o anumită frecvență specificată
 - Problema: să se genereze un semnal cu frecvența de 50 Hz
 - Modul de lucru ales: CTC (Clear on Compare Match) – permite reglarea perioadei prin setarea registrului de comparare
 - Calculul frecvenței:

$$f_{OCn} = \frac{f_{clk_I/O}}{2N(1 + OCR_n)}$$

- $f_{OCn} = 50$
- $N = 1024 \rightarrow$ divizarea maximă de frecvență permisă din prescaler
- $f_{clk_I/O} = 16$ MHz, frecvența plăcii
- $OCR0 = 16000000 / (2 * 1024 * 50) - 1 = 155$



Exemplu 1



- Generare semnal de frecvență specificată – cod sursa

```
.include "m64def.inc"
.org 0x0000
    jmp reset
```

```
reset:
    ldi r16, 0b00111111
    out TCCR0, r16; configurare Timer0

    ldi r16, 155 ; OCR calculat
    out OCR0, r16

    ldi r16, 0xff
    out DDRB, r16 ; activeaza iesirea OC0 (PB4)

loop:
    rjmp loop
```

Mode	WGM00:01	Mode
0	00	Normal
1	10	PWM, Phase Correct
2	01	CTC
3	11	Fast PWM

Normal, CTC

COM0[1:0]	Description
00	Normal, OC0 disconnected
01	Toggle OC0 on compare match
10	Clear OC0 on compare match
11	Set OC0 on compare match

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/counter stopped)
0	0	1	clk _{TOS} / (No prescaling)
0	1	0	clk _{TOS} /8 (From prescaler)
0	1	1	clk _{TOS} /32 (From prescaler)
1	0	0	clk _{TOS} /64 (From prescaler)
1	0	1	clk _{TOS} /128 (From prescaler)
1	1	0	clk _{TOS} /256 (From prescaler)
1	1	1	clk _{TOS} /1024 (From prescaler)



Exemplu 2



- Utilizarea întreruperii la egalitatea comparației
 - Problema: să se genereze un semnal cu perioada de 1 secundă – imposibil prin reglarea directă a timerelor, frecvența e prea joasă
 - Vom utiliza configurația stabilită în exemplul 1
 - Frecvența rezultată – 50 Hz, cu modul Toggle on CTC, implica două egalități în 1/50 secunde
 - Vom folosi întreruperea declansată la egalitate dintre numărator și OCR
 - Vom bascula (toggle) un semnal la fiecare 50 de astfel de evenimente
 - Acest semnal are perioada de 1 secundă



Exemplu 2



```
.include "m64def.inc"  
.org 0x0000  
    rjmp reset
```

```
.org 0x001E                                ; adresa vectorului pentru intreruperea Timer0 Comp  
    rjmp timercpm
```

reset:

```
    ldi r16, low(RAMEND) ; intreruperile necesita stiva  
    out SPL, r16  
    ldi r16, high(RAMEND)  
    out SPH, r16
```

```
    ldi r16, 0b00011111 ; configuratia anterioara  
    out TCCR0, r16  
    ldi r16, 155  
    out OCR0, r16
```

```
    ldi r16, 0xff        ; folosim LED-urile ca iesire  
    out DDRE, r16
```



Exemplu 2



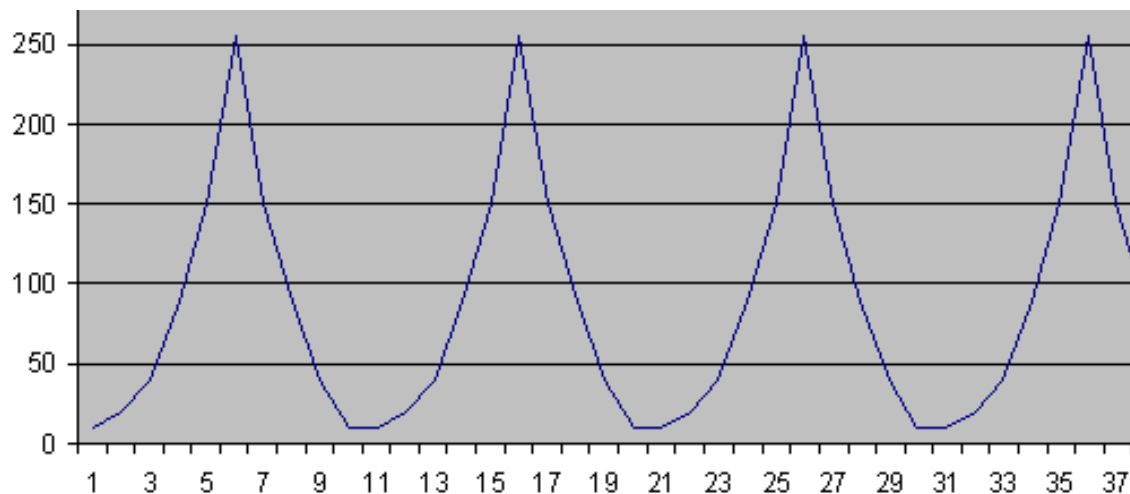
ldi r16, 0b00000010 ; activare punctuala intrerupere Timer0 Comp
out TIMSK, r16

ldi r18, 0 ; numaratorul de evenimente
ldi r19, 0 ; registru folosit pentru bascularea semnalului
sei ; activate globala intreruperi

loop:
 rjmp loop

timercpm: ; rutina de tratare a intreruperii
 inc r18 ; incrementare contor evenimente
 cpi r18, 50
 brne exit ; nu a ajuns la 50, iesire
 com r19 ; basculare r19
 out PORTE, r19 ; afisare
 ldi r18, 0 ; reinitializare contor
exit:
 reti

- Folosirea PWM
 - Problema: să se genereze o funcție analogică oarecare
 - Funcția va fi definită prin nivele discrete, stocate într-un LUT
 - Poate fi rezultat al unui proces de conversie Analog/Digital
 - Utilizăm modul de lucru PWM phase correct
 - OCR0 va defini lățimea pulsului
 - Vom modifica OCR0 la finalul fiecărei perioade de numărare
 - Valorile unei perioade: 10, 20, 40, 90, 150, 255, 150, 90, 40, 20, 10





Exemplu 3



```
.include "m64def.inc"
.org 0x0000
    rjmp reset
.org 0x0020 ; adresa vector Timer0Ovf
    rjmp timerovf

reset:
    ldi r16, low(RAMEND)
    out SPL, r16
    ldi r16, high(RAMEND)
    out SPH, r16

    ldi r16, 0b01100001 ; configurare Timer0
    out TCCR0, r16
    ldi r16, 0xff
    out DDRB, r16 ; activare iesire OC0

    ldi r16, 0b00000001 ; activare intrerupere
    out TIMSK, r16
```

Mode	WGM00:01	Mode
0	00	Normal
1	10	PWM, Phase Correct
2	01	CTC
3	11	Fast PWM

PWM, Phase Correct

COM0[1:0]	Description
00	Normal, OC0 disconnected
01	Reserved
10	Clear OC0 on compare match when up-counting. Set OC0 on compare match when down counting
11	Set OC0 on compare match when up-counting. Clear OC0 on compare match when down counting.

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/counter stopped)
0	0	1	$\text{clk}_{T0S}/(\text{No prescaling})$
0	1	0	$\text{clk}_{T0S}/8$ (From prescaler)
0	1	1	$\text{clk}_{T0S}/32$ (From prescaler)
1	0	0	$\text{clk}_{T0S}/64$ (From prescaler)
1	0	1	$\text{clk}_{T0S}/128$ (From prescaler)
1	1	0	$\text{clk}_{T0S}/256$ (From prescaler)
1	1	1	$\text{clk}_{T0S}/1024$ (From prescaler)



Exemplu 3



ldi r18, 0 ; iteratorul tabelii de valori
sei ; activare globala intreruperi

loop: **rjmp** loop ; programul principal se blocheaza aici

timerovf: ; Timer0Ovf ISR – se apeleaza la sfarsitul unei perioade

ldi r17, 0 ; calcul adresei in LUT

ldi zh, high(2*translut)

ldi zl, low(2*translut)

add zl, r18

adc zh, r17

lpm r17, Z

out OCR0, r17 ; valoarea din LUT se introduce in OCR

inc r18

cpi r18, 10 ; adresa maxima din LUT

brne exit

ldi r18, 0

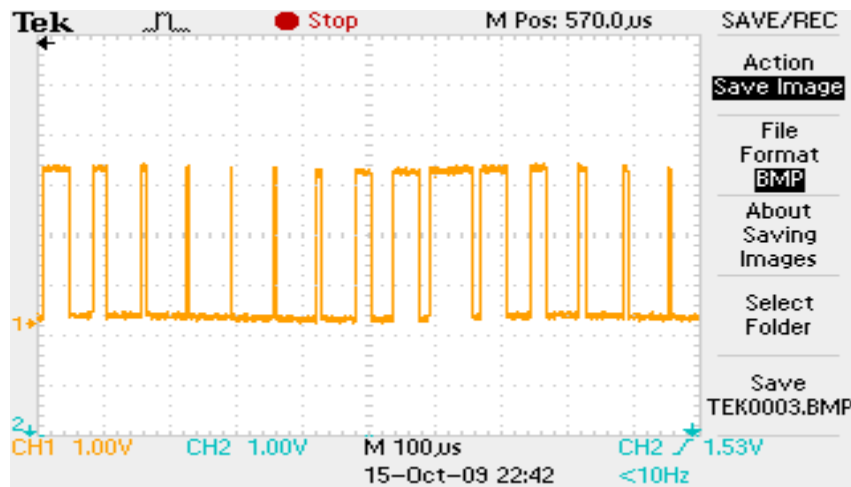
exit:

reti

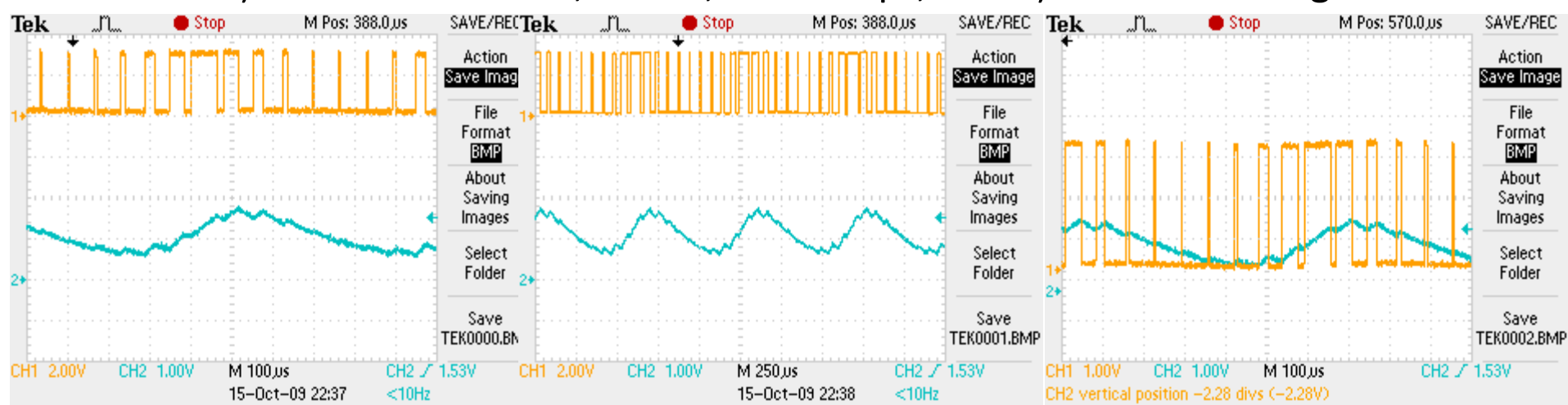
translut: ; tabela de valori, aici se defineste functia

.db 10, 20, 40, 90, 150, 255, 150, 90, 40, 20, 10

- Folosirea PWM – rezultate



- Prin atașarea unui filtru RC, $R = 1K$, $C = 0.22 \mu F$, se obține unda analogică





Exemplu 4



- Citirea stării temporizatorului
 - Problema: măsurarea perioadei dintre două evenimente externe
 - Configurația aleasă – Normal, cu frecvență minimă de numărare
 - Evenimentul extern provoacă o întrerupere externă, pe tranziția 0-1

```
.include "m64def.inc"
```

```
.org 0x0000
```

```
    rjmp reset
```

```
.org 0x0002
```

```
    rjmp int0ISR
```

; adresa vector Intrerupere externa 0

```
reset:
```

```
    ldi r16, low(RAMEND)
```

```
    out SPL, r16
```

```
    ldi r16, high(RAMEND)
```

```
    out SPH, r16
```

```
    ldi r16, 0b0000111    ; configurare Timer0
```

```
    out TCCR0, r16
```

```
    ldi r16, 0xff
```

```
    out DDRE, r16    ; activare iesire port E - leds
```



Exemplu 4



ldi r16, 0b00000011 ; configurare mod de tratare INT0 – front crescator

sts EICRA, r16

ldi r16, 0b00000001 ; activare intrerupere INT0

out EIMSK, r16

ldi r17, 0 ; ultima valoare numarator

sei

loop: **rjmp** loop

int0ISR:

in r16, **TCNT0** ; citirea starii registrului numarator

mov r18, r16

sub r16, r17 ; diferenta fata de valoarea precedenta

mov r17, r18 ; noua stare devine vechea stare

out PORTE, r16 ; afisam diferenta

reti



1. **Atmel ATmega640/V-1280/V-1281/V-2560/V-2561/V datasheet**
2. **Atmel Atmega64 datasheet**



Proiectarea cu Micro-Procesoare

Lector: Mihai Negru

An 3 – Calculatoare și Tehnologia Informației
Seria B

Curs 5: Temporizatoare la sistemele Arduino

<http://users.utcluj.ro/~negrum/>



- **Functii pentru intarziere**

- **delay**(unsigned long ms) – întârziere pentru un număr specificat de milisecunde
- **delayMicroseconds**(unsigned int us) – întârziere pentru un număr specificat de μ secunde

- **Functii pentru citirea timpului**

- unsigned long **millis**() – returnează timpul, în milisecunde, de la pornirea rulării programului curent. Ajunge la saturație după aproximativ 50 de zile
- unsigned long **micros**() – returnează timpul, în microsecunde, de la pornirea rulării programului curent. Ajunge la saturație după aproximativ 70 minute. La plăcile cu oscilator de 16 MHz (de exemplu, Arduino Mega), această funcție are rezoluția de 4 μ s.



- **Exemplu: temporizare fără a folosi funcția delay()**

```
const int ledPin = 13;           // pin-ul unde avem atasat un LED

int ledState = LOW;             // starea led-ului, initial stins
long previousMillis = 0;        // variabila in care stocam timpul ultimei actualizari

long interval = 1000;           // intervalul de clipire, in ms

void setup() {
  pinMode(ledPin, OUTPUT);      // configurare pin ca iesire
}

void loop()
{
  unsigned long currentMillis = millis(); // preluarea timpului curent

  if(currentMillis - previousMillis >= interval) { // daca a trecut mai mult timp decat intervalul prestabilit
    previousMillis = currentMillis; // actualizam timpul

    if (ledState == LOW) // comutam starea led-ului
      ledState = HIGH;
    else
      ledState = LOW;

    digitalWrite(ledPin, ledState); // scriem starea la iesire
  }
}
```

Sursa: <http://arduino.cc/en/Tutorial/BlinkWithoutDelay>



- **Exemplu: temporizare fără a folosi funcția delay() – pentru multitasking !**
 - **Doua led-uri, fiecare comută la 1 secundă, cu întârziere de 0.5 sec între ele**

```
long sequenceDelay = 500;           // intervalul dintre cele doua actiuni
long flashDelay = 1000;

boolean LED13state = false;         // starea celor doua LED-uri, la pinul 13 si la pinul 12
boolean LED12state = false;         // initial ambele sunt stinse
long waitUntil13 = 0;               // primul LED se aprinde imediat
long waitUntil12 = sequenceDelay;   // al doilea dupa 0.5 sec

void setup() {
    pinMode(13, OUTPUT);             // configurare pini ca iesire
    pinMode(12, OUTPUT);
}

void loop() {
    digitalWrite(13, LED13state);    // la fiecare iteratie, se seteaza valoarea actualizata pe pini
    digitalWrite(12, LED12state);

    if (millis() >= waitUntil13) {   // se verifica timpul pentru primul LED
        LED13state = !(LED13state); // daca a trecut 1 secunda, se comuta starea
        waitUntil13 += flashDelay;   // noul timp tinta, peste 1 secunda
    }

    if (millis() >= waitUntil12) {   // se verifica timpul pentru al doilea LED
        LED12state = !(LED12state); // se comuta starea daca a trecut suficient timp
        waitUntil12 += flashDelay;   // noul timp tinta, peste 1 secunda
    }
}
```

Sursa: <http://www.baldengineer.com/blog/2011/01/06/millis-tutorial/>



- Folosirea bibliotecii Timer pentru sincronizare / temporizare
- <http://playground.arduino.cc/Code/Timer>
- Metode:
- **int every(long period, callback):** rulează funcția 'callback' la intervale de 'period' milisecunde. Returnează identificatorul evenimentului programat.
- **int every(long period, callback, int repeatCount):** rulează funcția 'callback' la intervale de 'period' milisecunde, de 'repeatCount' ori.
- **int after(long duration, callback):** rulează funcția 'callback' o singură dată, după un interval de timp de 'duration' milisecunde.
- **int oscillate(int pin, long period, int startingValue):** generare de semnal. Modifică starea pinului 'pin' după fiecare 'period' milisecunde. Starea inițială a pinului este cea specificată de 'startingValue', HIGH sau LOW.
- **int oscillate(int pin, long period, int startingValue, int repeatCount):** variază starea pinului 'pin', la intervale specificate de 'period', de 'repeatCount' ori.
- **int pulse(int pin, long period, int startingValue):** schimbă starea lui 'pin' o singură dată, după 'period' milisecunde. Valoarea inițială este cea specificată în 'startingValue'.
- **int stop(int id):** toate funcțiile de mai sus returnează un identificator al evenimentului programat. Această funcție poate fi apelată pentru a opri evenimentul. Doar 10 evenimente pot fi atașate temporizatorului.
- **int update():** această funcție trebuie apelată în bucla principală (loop) a programului, pentru a actualiza starea obiectului de tip Timer.



- **Exemplu: generarea unui puls de durată îndelungată, fără a bloca activitatea sistemului**

```
#include "Timer.h"

Timer t;                                // declararea obiectului de tip Timer
int pin = 13;

void setup()
{
  pinMode(pin, OUTPUT);
  t.pulse(pin, 10 * 60 * 1000, HIGH);    // puls de 10 minute, cu valoare initiala HIGH
}

void loop()
{
  t.update();                            // necesar pentru functionarea timer-ului
                                          // apelul e de ordinul microsecundelor

  // restul ciclului ramane disponibil pentru alte procesari
}
```

<http://www.doctormonk.com/2012/01/arduino-timer-library.html>



- **Exemplu: apelarea unei funcții la intervale regulate de timp, și generarea unui semnal oscilator**

```
#include "Timer.h"
```

```
Timer t;                                // declararea obiectului de tip Timer  
int pin = 13;                           // pin-ul pentru oscilatie
```

```
void setup()  
{  
  Serial.begin(9600);                   // initializarea comunicatiei seriale  
  pinMode(pin, OUTPUT);                 // configurare pin pentru iesire  
  t.oscillate(pin, 100, LOW);            // initializarea generarii semnalului oscilatoriu (100 ms)  
  t.every(1000, takeReading);           // la fiecare 1000 ms se apeleaza functia takeReading  
}
```

```
void loop()  
{  
  t.update();                           // apelul necesar pentru functionarea timerului  
}
```

```
void takeReading()                      // functia care se apeleaza la fiecare 1 secunda  
{  
  Serial.println(analogRead(0));        // se trimite prin serial starea tensiunii analogice de pe un pin  
}
```



- **Exemplu: stoparea unui proces inițiat**

```
#include "Timer.h"
```

```
Timer t;
```

```
int ledEvent; // identificator eveniment
```

```
void setup()
```

```
{
```

```
  Serial.begin(9600);
```

```
// initializare interfata seriala
```

```
  int tickEvent = t.every(2000, doSomething); // se apeleaza doSomething la fiecare 2 secunde
```

```
  Serial.print("2 second tick started id="); // scrierea prin interfata seriala
```

```
  Serial.println(tickEvent); // a identificatorului procesului initiat
```

```
  pinMode(13, OUTPUT);
```

```
  ledEvent = t.oscillate(13, 50, HIGH); // pornire eveniment oscilare led la 50 ms
```

```
  Serial.print("LED event started id="); // scriere prin interfata seriala
```

```
  Serial.println(ledEvent); // a identificatorului ledEvent
```

```
  int afterEvent = t.after(10000, doAfter); // programare executie doAfter, dupa 10 secunde
```

```
  Serial.print("After event started id="); // scriere prin interfata seriala
```

```
  Serial.println(afterEvent); // a identificatorului pentru aceasta programare
```

```
}
```

<http://www.doctormonk.com/2012/01/arduino-timer-library.html>



- **Exemplu: stoparea unui proces inițiat**

```
void loop()
{
  t.update();           // actualizare timer
}

void doSomething()      // functia care se apeleaza la 2 secunde
{
  Serial.print("2 second tick: millis()="); // transmite numarul de milisecunde curent
  Serial.println(millis());                // pe interfata seriala
}

void doAfter()          // functia care se apeleaza dupa 10 secunde
{                       // o singura data
  Serial.println("stop the led event");
  t.stop(ledEvent);     // se opreste oscilarea led-ului
  t.oscillate(13, 500, HIGH, 5); // si se porneste o alta oscilare, la 500 ms
}                       // doar de 5 ori
```

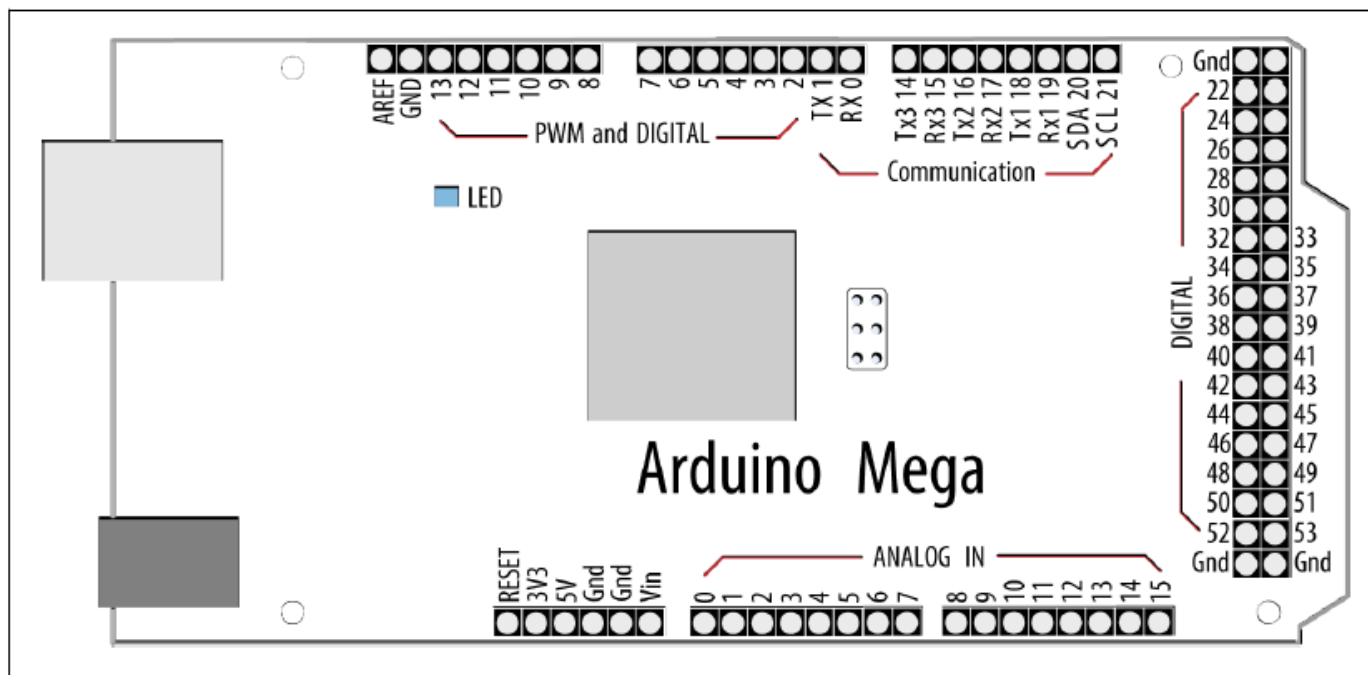
<http://www.doctormonk.com/2012/01/arduino-timer-library.html>



Generare de semnale PWM



- **Pulse Width Modulation (PWM) la Arduino:** o parte din pinii Arduino suportă funcția PWM, realizată intern prin intermediul temporizatoarelor
- La Arduino Mega, pinii 2-13 suporta funcția PWM
- Frecvența semnalului PWM este fixă, aproximativ 500 Hz

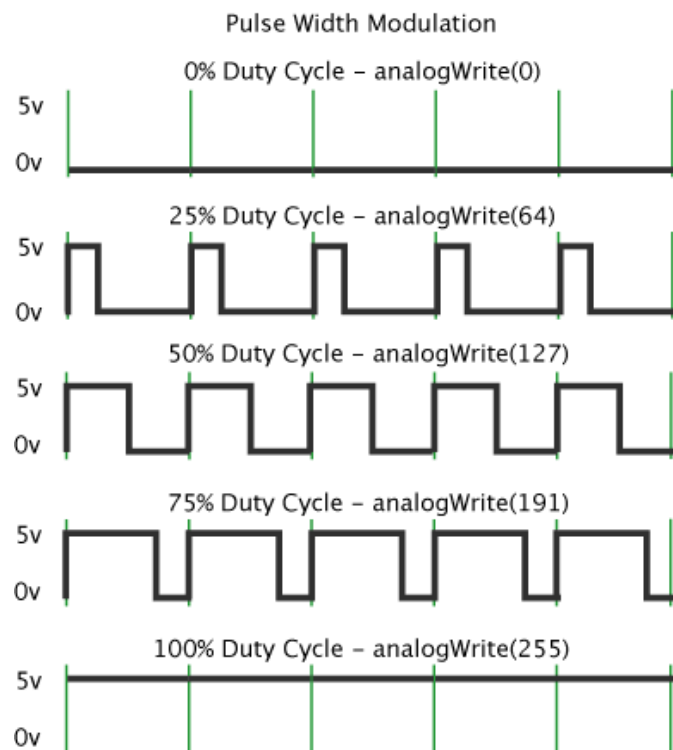




Generare de semnale PWM



- Funcția **analogWrite (pin, value)** determină generarea unui semnal PWM pe pinul 'pin', cu factorul de umplere specificat de 'value'.
- 'value' poate lua valori de la 0 la 255, corespunzătoare factorilor de umplere de la 0% la 100%
- Pinul pe care se dorește generarea semnalului PWM trebuie configurat ca ieșire.

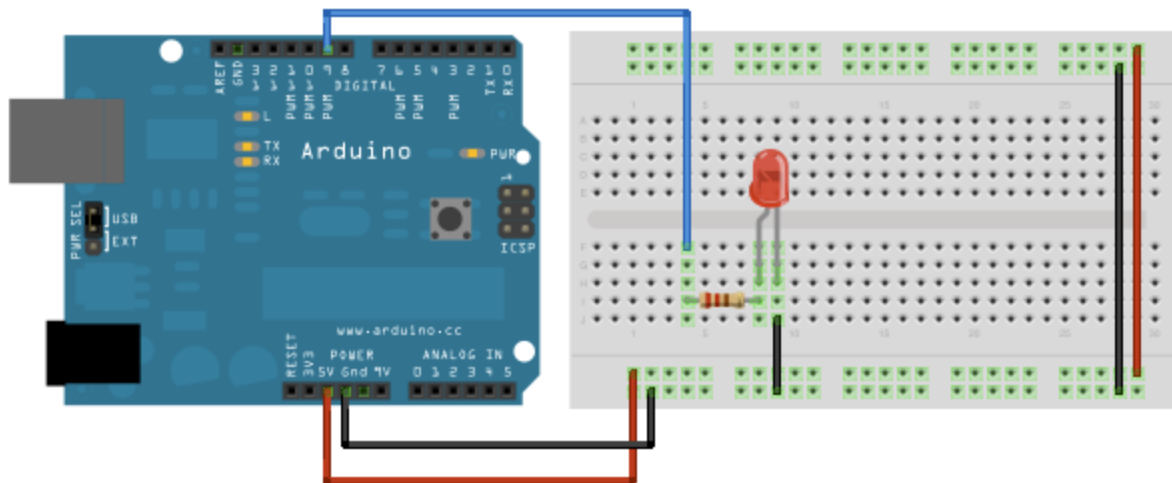
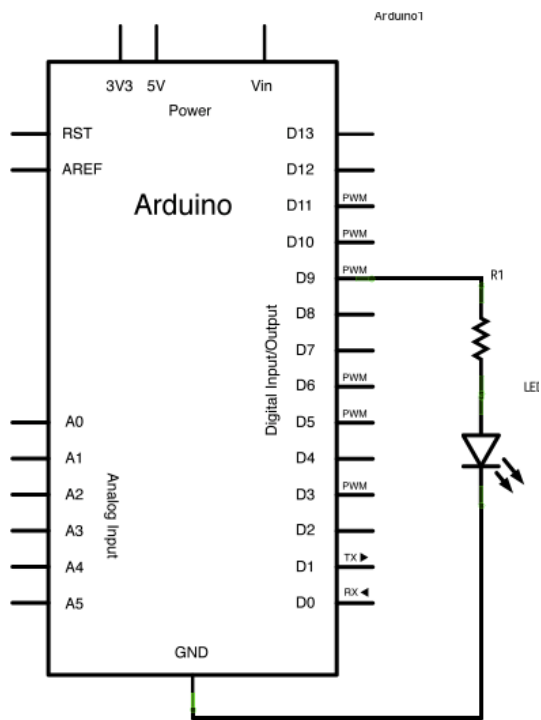




Generare de semnale PWM



- Exemplu: fade-in, fade-out cu un led extern



<http://arduino.cc/en/Tutorial/Fade>



- **Exemplu: fade-in, fade-out cu un led extern**

```
int led = 9;
int brightness = 0;           // starea curenta a led-ului, initial stins
int fadeAmount = 5;          // incrementul starii led-ului

void setup() {
  pinMode(led, OUTPUT);      // pin 9, iesire
}

void loop() {

  analogWrite(led, brightness); // configureaza factorul de umplere al PWM

  brightness = brightness + fadeAmount; // modifica starea curenta prin adaugarea incrementului

  if (brightness == 0 || brightness == 255) { // la capatul intervalului, schimba semnul incrementului
    fadeAmount = -fadeAmount ;
  }

  delay(30);                 // intarziere
}
```

<http://arduino.cc/en/Tutorial/Fade>



Generare de semnale PWM



- Funcția **tone ()** cauzează producerea de pulsuri cu factor de umplere 50%, și frecvență variabilă. Există două moduri de apelare:
- **tone(pin, frequency)** – produce un semnal de frecvență ‘frequency’ pe pinul ‘pin’, de durată nedefinită
- **tone(pin, frequency, duration)** – produce un semnal de frecvență ‘frequency’, pe pinul ‘pin’, cu durata de timp ‘duration’ în milisecunde.

- Funcția **noTone(pin)** oprește generarea semnalului pentru pinul ‘pin’.
- Doar un singur pin poate genera ton la un moment dat. Dacă se dorește generarea de ton pe alt pin, în timp ce un ton este activ, trebuie apelată funcția **noTone()** pentru oprirea tonului activ.
- La unele plăci Arduino, generarea tonului poate afecta generarea semnalelor PWM.



- **Exemplu: reproducerea unei melodii după note**

```
const int speakerPin = 9; // Difuzorul se conecteaza la pin-ul 9
char noteNames[] = {'C','D','E','F','G','a','b'}; // Numele notelor
unsigned int frequencies[] = {262,294,330,349,392,440,494}; // Frecventele asociate
const byte noteCount = sizeof(noteNames); // Numarul de note in tabela

// Partitura melodiei, spatiul reprezinta pauza
char score[] = "CCGGaaGFFEEDDC GGFFEEDGGFFEED CCGGaaGFFEEDDC ";
const byte scoreLen = sizeof(score); // Numarul de note in melodie

void setup()
{
}

void loop()
{
    for (int i = 0; i < scoreLen; i++) // Se parcurge partitura
    {
        int duration = 333; // Fiecare nota va fi cantata 0.33 secunde
        playNote(score[i], duration); // Apel functie cantare nota (slide urmator)
    }

    delay(4000); // Pauza inainte de repetarea melodiei
}
```



- **Exemplu: reproducerea unei melodii după note**

```
void playNote(char note, int duration)
{
    // Se parcurge lista de note
    for (int i = 0; i < noteCount; i++)
    {
        // Se cauta nota curenta in lista
        if (noteNames[i] == note) // Daca a fost gasita
            tone(speakerPin, frequencies[i], duration); // Se genereaza tonul corespunzator, cu frecventa notei
        // Daca nota nu a fost gasita, se face pauza oricum
        delay(duration);
    }
}
```



- Uneori este nevoie de mai multe opțiuni pentru configurarea temporizatoarelor.
- Două opțiuni: folosirea unor biblioteci dedicate, sau accesul direct la regiștrii de configurare ai temporizatoarelor de pe microcontrollerul AVR
- Biblioteca Timer1: <http://playground.arduino.cc/Code/Timer1>
 - Funcții pentru folosirea temporizatorului de 16 biți Timer 1.
 - La Arduino Mega, nu se pot folosi toți pinii de generare de semnal ai Timer 1, și se recomandă folosirea Timer 3
<http://playground.arduino.cc/uploads/Code/TimerThree.zip>, cu aceleași funcții.
- Cele mai importante metode ale clasei Timer:
- **initialize(period)** – inițializarea temporizatorului, cu perioada ‘period’, în microsecunde. Perioada este intervalul în care temporizatorul efectuează o număratoare completă.
- **setPeriod(period)** – modifică perioada unui temporizator deja inițializat.
- **pwm(pin, duty, period)** – generează un semnal PWM pe pinul ‘pin’, cu factorul de umplere ‘duty’ având valori de la 0 la 1023, și cu perioada specificată (opțional) de ‘period’, în microsecunde. Pentru ‘pin’ se acceptă doar valorile la care temporizatorul este conectat fizic (Timer 1 conectat la pinii 9 și 10, Timer 3 conectat la pinii 2, 3 și 5 la Arduino Mega).



- **attachInterrupt(function, period)** – atașează funcția ‘function’ pentru a fi apelată de fiecare dată când temporizatorul își termină un ciclu, sau la intervale specificate de parametrul opțional ‘period’.
- **detachInterrupt()** – dezactivează întreruperea și detașează funcția ISR specificată de attachInterrupt().
- **disablePwm(pin)** – dezactivează generarea semnalului PWM pe pin-ul specificat.
- **read()** – returnează intervalul trecut de la ultima saturare a număratorului temporizatorului, în microsecunde.

Relația dintre perioade și prescaler, pentru sisteme cu oscilator de 16MHz:

Prescaler	Timpul dintre două incrementari	Perioada maxima
1	0.0625 μ s	8.192 ms
8	0.5 μ s	65.536 ms
64	4 μ s	524.288 ms
256	16 μ s	2097.152 ms
1024	64 μ s	8388.608 ms



- **Exemplu de utilizare a bibliotecii Timer1**
- Activează generarea unui semnal PWM pe pinul 9, cu factor de umplere 50%, și activează o intrerupere care modifică starea pinului 10 la fiecare 0.5 secunde

```
#include "TimerOne.h"
```

```
void setup()
```

```
{  
  pinMode(10, OUTPUT);  
  Timer1.initialize(500000);  
  Timer1.pwm(9, 512);  
  Timer1.attachInterrupt(callback);  
}
```

```
// initializeaza timer 1, cu perioada de 0.5 secunde  
// activeaza pwm pe pinul 9, cu factor de umplere 50%  
// ataseaza functia callback() pentru tratarea intreruperii  
// declansata de saturarea temporizatorului
```

```
void callback()
```

```
{  
  digitalWrite(10, digitalRead(10) ^ 1);  
}
```

```
void loop()
```

```
{  
  // liber pentru alte procesari  
}
```




- Folosirea regiștrilor de configurare
- **Exemplu:** funcția setPeriod din biblioteca Timer1

```
#define RESOLUTION 65536 // Temporizatorul este pe 16 biti
```

```
void TimerOne::setPeriod(long microseconds)
```

```
{
    long cycles = (F_CPU / 2000000) * microseconds; // modul PWM phase correct, numara in sus si in jos pentru 1 perioada

    // verifica daca numarul de cicluri se potriveste in valoarea maxima a numaratorului
    if(cycles < RESOLUTION) clockSelectBits = _BV(CS10); // fara divizare
    else if((cycles >= 3) < RESOLUTION) clockSelectBits = _BV(CS11); // divizare cu 8
    else if((cycles >= 3) < RESOLUTION) clockSelectBits = _BV(CS11) | _BV(CS10); // divizare cu 64
    else if((cycles >= 2) < RESOLUTION) clockSelectBits = _BV(CS12); // divizare cu 256
    else if((cycles >= 2) < RESOLUTION) clockSelectBits = _BV(CS12) | _BV(CS10); // divizare cu 1024
    else cycles = RESOLUTION - 1, clockSelectBits = _BV(CS12) | _BV(CS10); // cerere imposibila, divizare maxima si setare
    // numarul de cicluri la maximul posibil

    oldSREG = SREG; // salvare registru stare
    cli(); // dezactivare intreruperi
    ICR1 = pwmPeriod = cycles; // configurare registru ICR1
    SREG = oldSREG; // refacere stare SREG (alterata de CLI)
    TCCR1B &= ~(_BV(CS10) | _BV(CS11) | _BV(CS12)); // stergere biti de clock select pentru Timer 1
    TCCR1B |= clockSelectBits; // configurare biti clock select pentru Timer 1, calculati mai sus
}
```



- Folosirea regiștrilor de configurare

Bit	7	6	5	4	3	2	1	0	
(0x81)	ICNC1	ICES1	–	WGM13	WGM12	CS12	CS11	CS10	TCCR1B
Read/write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W	
Initial value	0	0	0	0	0	0	0	0	

CS12	CS11	CS10	Description
0	0	0	No clock source (timer/counter stopped)
0	0	1	$\text{clk}_{\text{I/O}}/1$ (no prescaling)
0	1	0	$\text{clk}_{\text{I/O}}/8$ (from prescaler)
0	1	1	$\text{clk}_{\text{I/O}}/64$ (from prescaler)
1	0	0	$\text{clk}_{\text{I/O}}/256$ (from prescaler)
1	0	1	$\text{clk}_{\text{I/O}}/1024$ (from prescaler)
1	1	0	External clock source on T1 pin. Clock on falling edge.
1	1	1	External clock source on T1 pin. Clock on rising edge.

Registrul ICR1 este pe 16 biți, împărțit în ICR1H și ICR1L. Utilizat pentru specificarea valorii prag TOP până la care numărătorul se incrementează. După atingerea TOP, numărătorul se întoarce la 0, când se termină perioada.

Bit	7	6	5	4	3	2	1	0	
(0x87)	ICR1[15:8]								ICR1H
(0x86)	ICR1[7:0]								ICR1L
Read/write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial value	0	0	0	0	0	0	0	0	



Exerciții



- Presupunând că nu există funcțiile `delay()` și `millis()`, implementați-le folosind temporizatoare
- Având un afișor cu două cifre de 7 segmente, realizați afișarea unui număr dat folosind temporizatoare
- Realizați un sistem de control al iluminării. Fiecare oră din cele 24 ale unei zile este definită de utilizator (printr-un LUT) ca zi, seară sau noapte. În funcție de tipul orei, lumina va fi stinsă, aprinsă mediu, sau aprinsă maxim



1. Atmel ATmega640/V-1280/V-1281/V-2560/V-2561/V datasheet
2. Atmel Atmega64 datasheet
3. <http://arduino.cc/en/>
4. <http://www.baldengineer.com/blog/2011/01/06/millis-tutorial/>
5. <http://www.doctormonk.com/2012/01/arduino-timer-library.html>
6. <http://arduino.cc/en/Tutorial/Fade>



Proiectarea cu Micro-Procesoare

Lector: Mihai Negru

An 3 – Calculatoare și Tehnologia Informației
Seria B

Curs 6: Comunicare Serială

<http://users.utcluj.ro/~negrum/>



- **Universal Synchronous and Asynchronous serial Receiver and Transmitter (USART)**
 - Comunicare serială sincronă sau asincronă
 - Frecvența (baud rate) variabilă
 - Suportă pachete de date de 5-9 biți, cu sau fără paritate
 - Suportă întreruperi pentru controlul transmisiei
 - Detecția erorilor de transmisie
 - Comunicare între placa de dezvoltare și PC
- **Serial Peripheral Interface (SPI)**
 - Comunicare serială sincronă
 - Mod de funcționare full duplex
 - Configurare Master sau Slave
 - Frecvență de comunicare variabilă
 - Se poate folosi pentru conexiune între plăci de dezvoltare, sau între o placă și diferite module PMOD (ex. DAC extern, Serial Flash)



- **Two Wire Serial Interface (TWI)**
 - Protocol de comunicare complex, folosind doar două fire (clock și data)
 - Implementare Atmel a protocolului I2C (Inter Integrated Circuit)
 - Controllerul TWI integrat suportă moduri master și slave
 - Adresare pe 7 biți
 - Suport pentru arbitrare pentru mai multe dispozitive master
 - Adresa slave programabilă
- **PS2**
- **CAN**
- ...

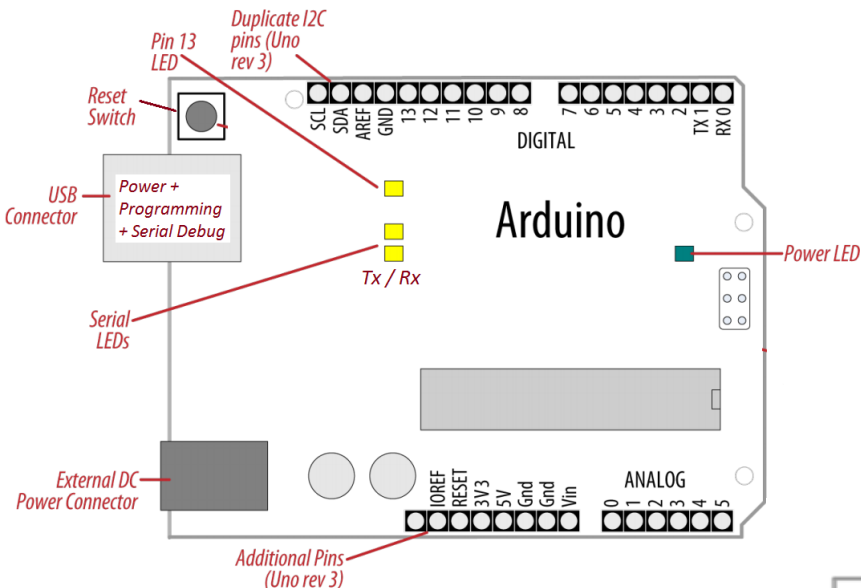


Interfețe seriale la Arduino



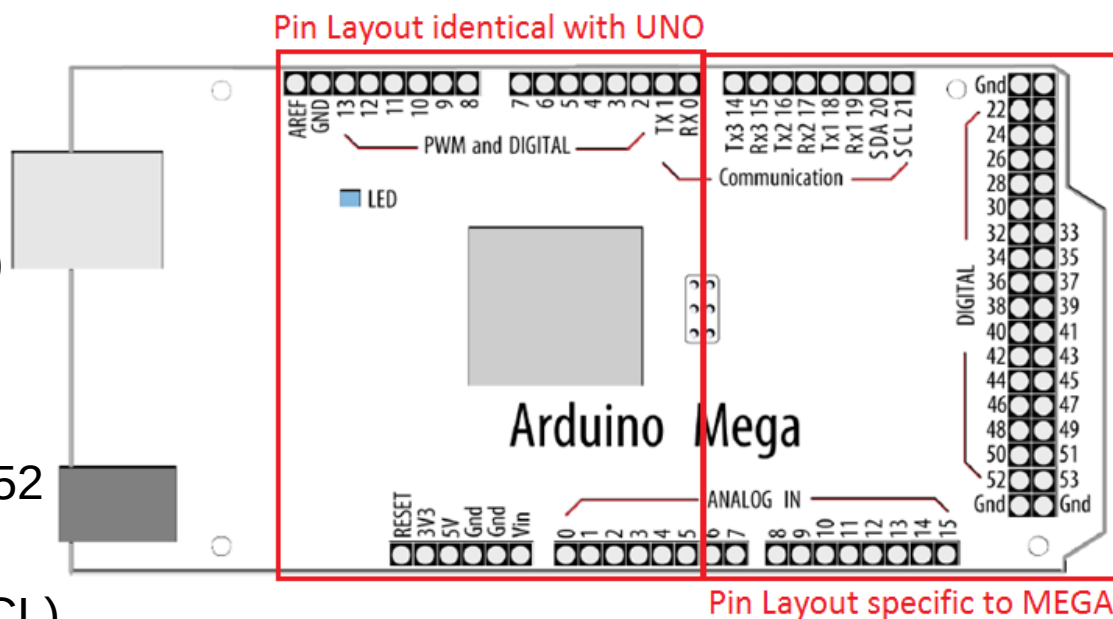
Arduino UNO (rev. 3)

- **UART:** pin 0 (RX) , pin 1 (TX) – **Utilizați pentru programare!**
- **SPI:** pin 10 (SS), pin 11 (MOSI), pin 12 (MISO), pin 13 (SCK).
- **TWI(I2C):** A4 sau pin SDA, A5 sau pin SCL



Arduino MEGA (rev. 3)

- **UART0:** pin 0 (RX), pin 1 (TX)
- **UART1:** pin 19 (RX), pin 18 (TX)
- **UART2:** pin 17 (RX) si 16 (TX)
- **UART3:** pin 15 (RX) si 14 (TX)
- **SPI:** pin 50 (MISO), 51 (MOSI), 52 (SCK), 53 (SS)
- **TWI(I2C):** pin 20 (SDA) si 21 (SCL)

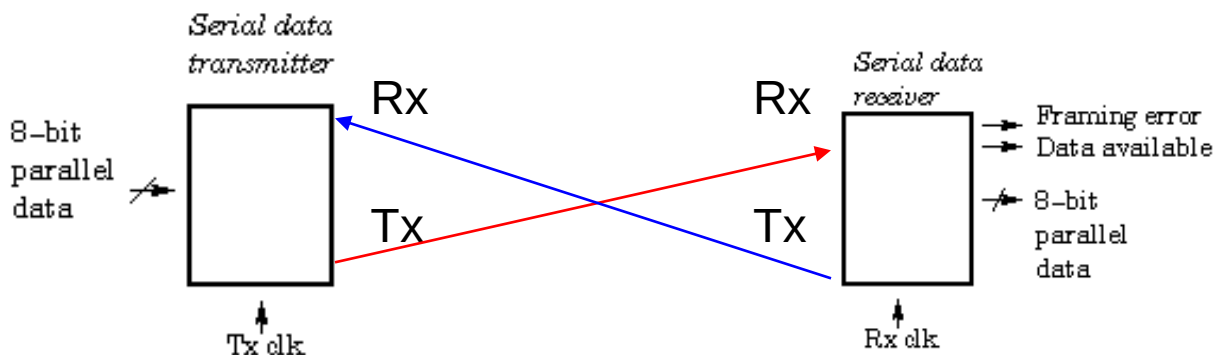




USART

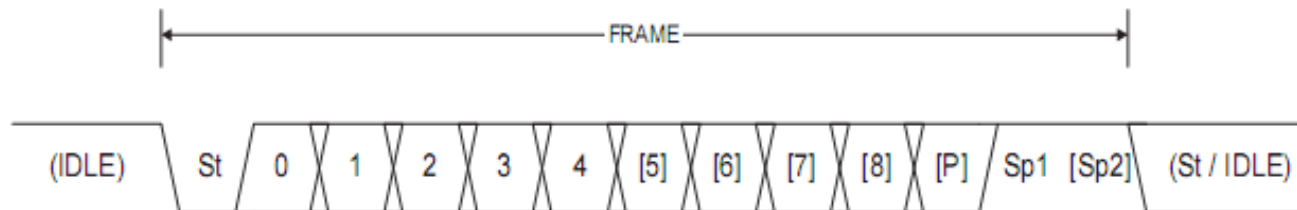


- **USART – UART cu posibilitate de sincronizare prin semnal de ceas**
- UART – Interfață pentru comunicare serială asincronă
 - **Asincron**: intervalul dintre pachetele de date poate fi nedefinit. Destinatarul transmisiei detectează când începe și când se termină un pachet
 - **Baud rate** (rata de transfer): intervalul de timp dintre biți este fix, și trebuie cunoscut de ambele părți
 - Transmisia și recepția se pot efectua simultan (full duplex).
 - O interfață UART are două semnale:
 - Rx – intrare, recepție
 - Tx – ieșire, transmisie
 - USART are un semnal în plus, xck (external clock) care poate fi intrare sau ieșire, și va sincroniza transmisia și recepția





- **Transmisia datelor** → pachet (frame)
 - **St**: 1 bit de start, cu valoare '0'
 - **D**: 5, 6, 7, 8, 9 biți de date
 - **P**: 1 bit de paritate. Paritatea poate fi:
 - Absentă: bitul P nu există
 - Pară (*Even*)
 - Impară (*Odd*)
 - **Sp**: 1, 1.5 sau 2 biți de stop, cu valoare '1'





- **Recepția datelor**

1. Se detectează o tranziție din '1' în '0' pe linia Rx (recepție)
 2. Se verifică mijlocul intervalului pentru bitul de start. Dacă este '0', se inițiază secvența de recepție, altfel tranziția se consideră zgomot.
 3. Se verifică mijlocul intervalului pentru biții următori (date, paritate, stop), și se reconstruiește pachetul de date
 4. Dacă în poziția unde trebuie să fie biții de stop se detectează valoarea zero, se generează eroare de împachetare (**framing error**)
 5. Dacă paritatea calculată la destinație nu corespunde cu bitul P, se generează eroare de paritate (**parity error**)
- Pentru robustețe, receptorul eșantionează semnalul de intrare la o frecvență de 8-16 ori mai mare decât *baud rate* – *oversampling*



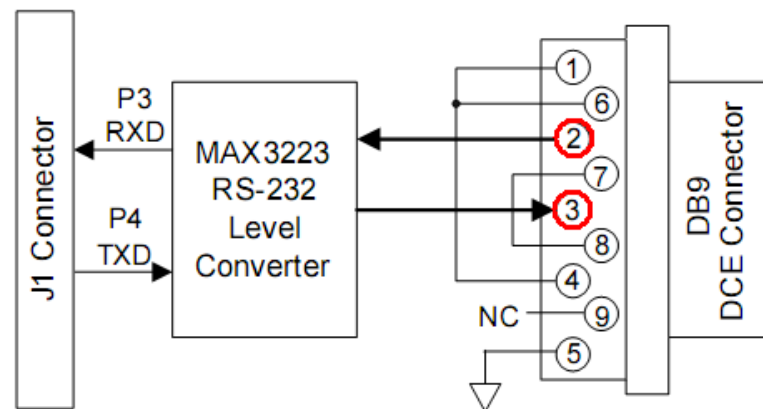
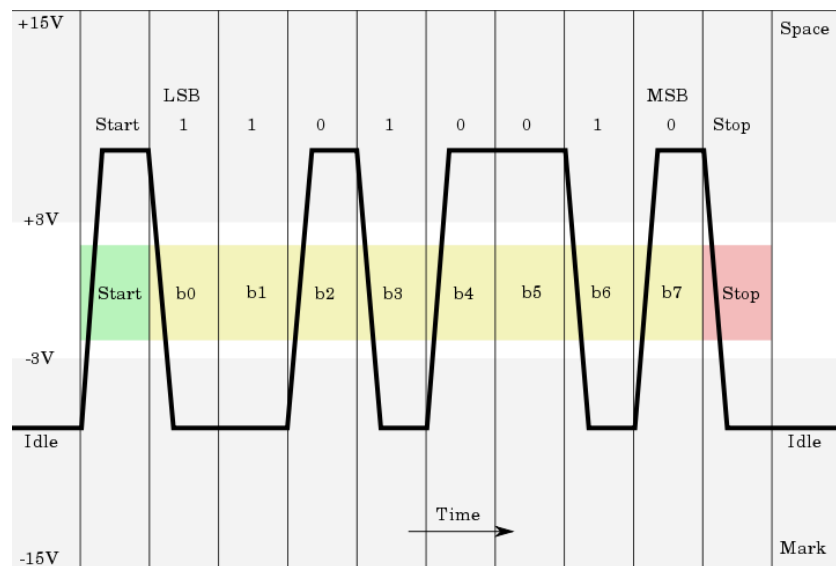
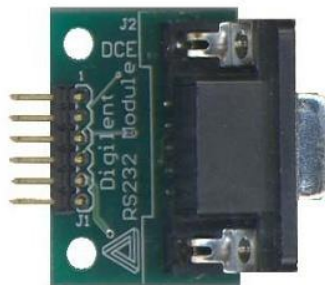
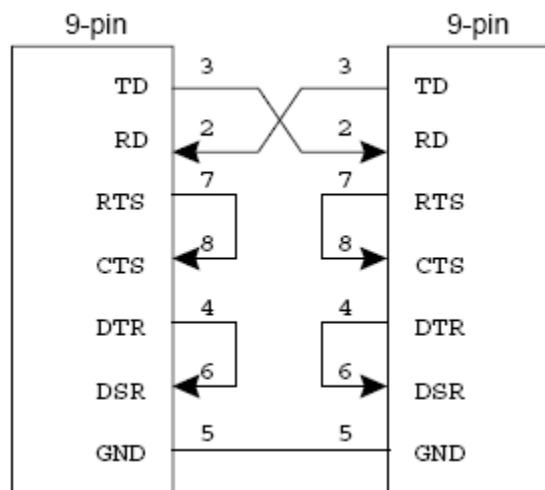
USART



- **UART și RS232**
- Adaptarea nivelelor de tensiune
 - RS232 logic '1' -5... -15 V
 - RS232 logic '0' +5...+15 V

Este nevoie de conversie de la nivelele Logice UART la RS232

Corespondenta pinilor



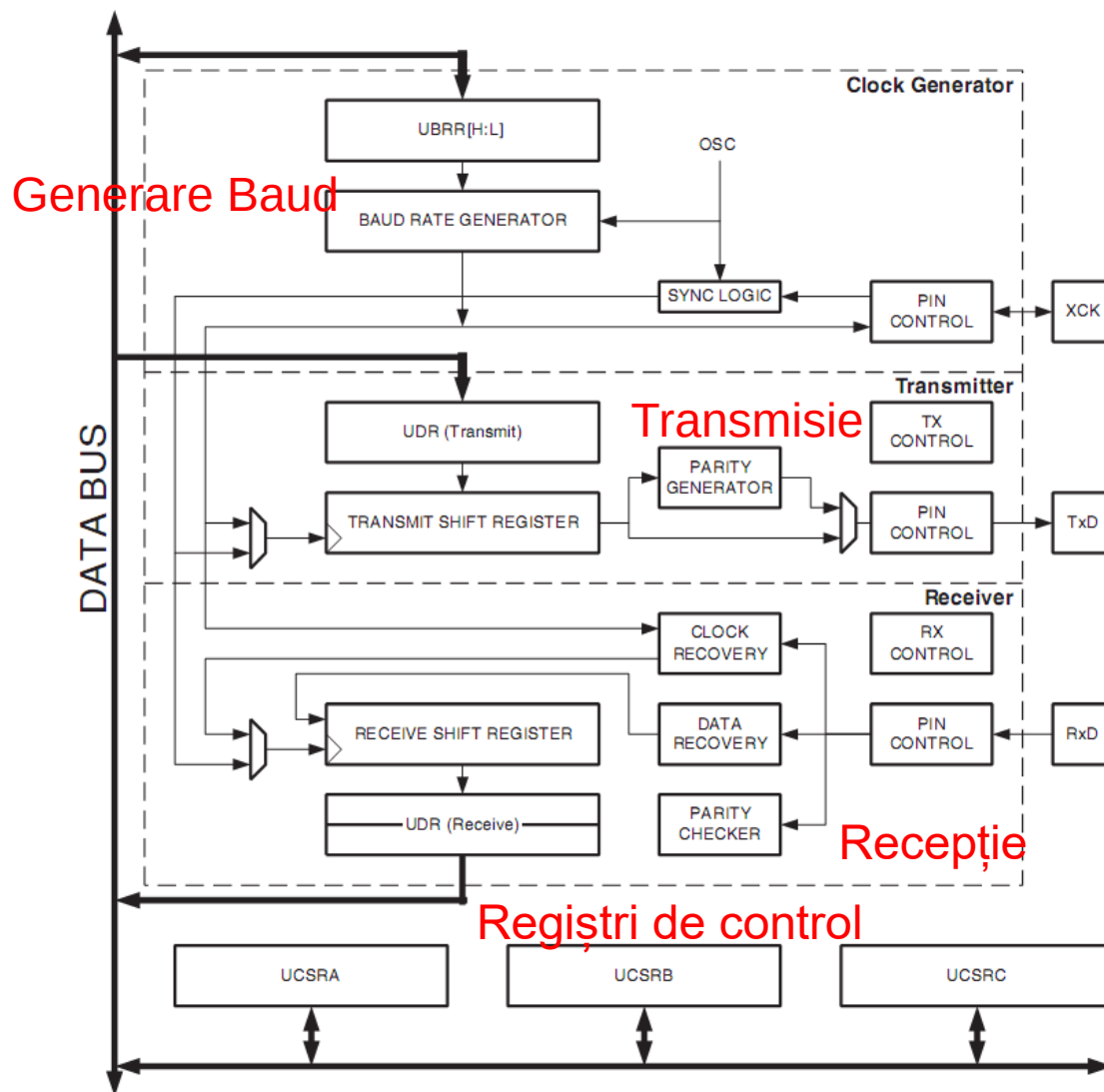


USART la AVR



- Două unități USART:
 - USART0 și USART1

- Arhitectura generală





USART la AVR – Configurare



- Registrul de control și stare **UCSRnA**

Bit	7	6	5	4	3	2	1	0	
	RXCn	TXCn	UDREN	FEn	DORn	UPEn	U2Xn	MPCMn	UCSRnA
Read/Write	R	R/W	R	R	R	R	R/W	R/W	
Initial Value	0	0	1	0	0	0	0	0	

- **RXCn** – Este ‘1’ cand recepția e completă. Poate genera întrerupere
- **TXCn** – Este ‘1’ cand transmisia e completă. Poate genera întrerupere
- **UDREN** – Data Register Empty, semnalează că registrul poate fi scris
- **FEn** – Semnalează eroare de impachetare (Frame Error)
- **DORn** – Data overrun – când se detectează un început de recepție înainte ca datele deja recepționate să fie citite din registrul de date
- **UPEn** – Eroare de paritate (Parity Error)
- **U2Xn** – Valoare ‘1’ → Dublare viteză de transmisie USART
- **MPCMn** – Activare mod de comunicare multiprocesor



USART la AVR – Configurare



- Registrul de control și stare **UCSRnB**

Bit	7	6	5	4	3	2	1	0	
	RXCIE_n	TXCIE_n	UDRIE_n	RXEN_n	TXEN_n	UCSZ_{n2}	RXB8_n	TXB8_n	UCSR_{nB}
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **RXCIE_n** – Dacă e setat '1', se generează întrerupere la terminarea receptiei
- **TXCIE_n** – Dacă e setat '1', se generează întrerupere la terminarea transmisiei
- **UDRIE_n** – Dacă e setat '1', se generează întrerupere când registrul de date e gol
- **RXEN** – activare receptie
- **TXEN** – activare transmisie
- **UCSZ_{n2}** – combinat cu UCSZ_{n1} si UCSZ_{n0} din **USCR_{nC}** stabilește mărimea pachetului
- **RXB8_n** – al 9-lea bit receptionat, cand pachetul are 9 biti
- **TXB8_n** – al 9-lea bit de transmis, cand pachetul are 9 biti



USART la AVR – Configurare



- Registrul de control și stare **UCSRnC**

Bit	7	6	5	4	3	2	1	0	
	–	UMSELn	UPMn1	UPMn0	USBSn	UCSZn1	UCSZn0	UCPOLn	UCSRnC
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	1	1	0	

- UMSELn** – Mod asincron ‘0’ sau sincron ‘1’
- UPMn1 si UPMn0 – Selectia modului de paritate
- USBSn** – configurare biti de stop: ‘0’ – 1 bit, ‘1’ – 2 biți
- UCSZn1:UCSZn0** – combinat cu UCSZn2 din UCSRnB → dimensiunea pachetului

UCSZn2	UCSZn1	UCSZn0	Character Size			
0	0	0	5-bit			
0	0	1	6-bit			
0	1	0	7-bit			
0	1	1	8-bit	UPMn1	UPMn0	Parity Mode
1	0	0	Reserved	0	0	Disabled
1	0	1	Reserved	0	1	Reserved
1	1	0	Reserved	1	0	Enabled, Even Parity
1	1	1	9-bit	1	1	Enabled, Odd Parity



USART la AVR – Configurare



- Regiștrii de control al frecvenței: **UBRRnH** si **UBRRnL**
 - Formează împreună valoarea **UBRRn**, de 12 biți

Bit	15	14	13	12	11	10	9	8	
	–	–	–	–	UBRRn[11:8]				UBRRnH
	UBRRn[7:0]								UBRRnL
	7	6	5	4	3	2	1	0	
Read/Write	R	R	R	R	R/W	R/W	R/W	R/W	
	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	
	0	0	0	0	0	0	0	0	

Operating Mode	Equation for Calculating Baud Rate ⁽¹⁾	Equation for Calculating UBRR Value
Asynchronous Normal mode (U2Xn = 0)	$BAUD = \frac{f_{osc}}{16(UBRR + 1n)}$	$UBRRn = \frac{f_{osc}}{16BAUD} - 1$
Asynchronous Double Speed mode (U2Xn = 1)	$BAUD = \frac{f_{osc}}{8(UBRRn + 1)}$	$UBRRn = \frac{f_{osc}}{8BAUD} - 1$

- Citire date recepționate / scriere date pentru transmis**
 - Ambele acțiuni se fac prin registrul **UDRn**



USART la AVR



- Pini comuni cu porturile I/O – Direcția este configurată automat prin activarea recepției și/sau a transmisiei!**

Port Pin	Alternate Function
PD7	T2 (Timer/Counter2 Clock Input)
PD6	T1 (Timer/Counter1 Clock Input)
PD5	XCK1 ⁽¹⁾ (USART1 External Clock Input/Output)
PD4	ICP1 (Timer/Counter1 Input Capture Pin)
PD3	INT3/TXD1 ⁽¹⁾ (External Interrupt3 Input or UART1 Transmit Pin)
PD2	INT2/RXD1 ⁽¹⁾ (External Interrupt2 Input or UART1 Receive Pin)
PD1	INT1/SDA ⁽¹⁾ (External Interrupt1 Input or I2C Data Input/Output)
PD0	INT0/SCL ⁽¹⁾ (External Interrupt0 Input or I2C Data Input/Output)

USART1, portul D

Port Pin	Alternate Function
PE7	INT7/ICP3 ⁽¹⁾ (External Interrupt 7 Input or Timer/Counter3 Input Capture Pin)
PE6	INT6/ T3 ⁽¹⁾ (External Interrupt 6 Input or Timer/Counter3 Clock Input)
PE5	INT5/OC3C ⁽¹⁾ (External Interrupt 5 Input or Output Compare and PWM Output C for Timer/Counter3)
PE4	INT4/OC3B ⁽¹⁾ (External Interrupt 4 Input or Output Compare and PWM Output B for Timer/Counter3)
PE3	AIN1/OC3A ⁽¹⁾ (Analog Comparator Negative Input or Output Compare and PWM Output A for Timer/Counter3)
PE2	AIN0/XCK0 ⁽¹⁾ (Analog Comparator Positive Input or USART0 external clock input/output)
PE1	PDO/TXD0 (Programming Data Output or UART0 Transmit Pin)
PE0	PDI/RXD0 (Programming Data Input or UART0 Receive Pin)

USART0, portul E



- **Exemplu:** comunicare între ATmega2560 si PC – ECHO simplu
- **Necesar:** cablu serial, modul **PMOD RS232**

1. Configurare

Baud: 9600

Marime pachet: 8 biti

Biți de stop: 2

Paritate: fără paritate

$$UBRRn = \frac{f_{osc}}{16BAUD} - 1$$

$$f_{osc} = 16 \text{ MHz} \rightarrow 16.000.000$$

$$UBRRn = 103$$

2. Așteptare recepție caracter

- Verificare **RXCn** din **UCSRnA**, așteptare până devine 1

3. Citire caracter recepționat, din **UDRn**

4. Scriere caracter de transmis, în **UDRn**

5. Așteaptă transmisie caracter

- Verificare **TXCn** din **UCSRnA**, așteptare până devine 1

6. Salt la 2



- **Exemplu:** comunicare între ATmega64 si PC – ECHO simplu

```
ldi r16, 0b0001000          ; activare Rx si Tx
sts UCSR1B, r16
ldi r16, 0b000110          ; dimensiune frame 8 biti, fara paritate, 2 biti de stop
sts UCSR1C, r16
ldi r16, 103                ; Baud rate calculat, incape in primii 8 biti
ldi r17, 0                  ; Bitii superiori la UBRR sunt zero
sts UBRR1H, r17
sts UBRR1L, r16

mainloop:
  recloop:
    lds r20, UCSR1A
    sbrc r20, 7              ; bitul 7 din UCSR1A – receptie completa
    rjmp recloop

    lds r16, UDR1            ; citire date receptionate
    sts UDR1, r16           ; scriere date spre transmisie

  txloop:
    lds r20, UCSR1A          ; asteptare terminare transmisie
    sbrc r20, 5
    rjmp txloop
  rjmp mainloop
```



- **Toate placile Arduino** au cel puțin un port serial (port UART sau USART), accesibil prin obiectul C++ **Serial**
- **Comunicare $\mu\text{C} \leftrightarrow \text{PC}$** prin conexiunea USB, folosind adaptorul USB-UART integrat pe placă – comunicare folosită și pentru programarea plăcii!!
- **Comunicarea între plăci** folosind pinii digitali 0 (RX) și 1 (TX) – **nerecomandată**. Pentru a putea folosi aceste tipuri de comunicație, nu folosiți pinii 0 și 1 pentru operații generale de I/O digital !!!
- **Placa Arduino MEGA** are trei porturi seriale suplimentare: **UART1** folosește pinii 19 (RX) și 18 (TX), **UART2** pinii 17 (RX) și 16 (TX), **UART3** pinii 15 (RX) și 14 (TX)
- Pentru a comunica cu un PC, este nevoie de un adaptor USB-UART extern, sau un adaptor UART-RS232, deoarece aceste interfețe nu folosesc adaptorul integrat pe placa
- Pentru a comunica cu un dispozitiv care folosește interfața serială **cu nivele logice TTL**, se conectează pinul TX al plăcii la pinul RX al dispozitivului, pinul RX al plăcii la pinul TX al dispozitivului, și pinii de masă (GND) împreună



- Biblioteca **Serial** integrată în mediul de dezvoltare Arduino (<http://arduino.cc/en/Reference/Serial>) – folosită pentru facilitarea comunicației prin interfețele seriale disponibile.
- **Metodele clasei Serial (selectie):**
 - **Serial.begin(speed)** – configurează viteza de transmisie (speed) și formatul implicit de date (8 data bits, no parity, one stop bit)
 - **Serial.begin(speed, config)** – configurează viteza (speed) + selectează un alt format al datelor (config): SERIAL_8N1 (implicit), SERIAL_7E2, SERIAL_5O1
 - **Serial.print(val)** – trimite valoarea val ca un șir de caractere citibil de către utilizator (ex: Serial.print(20) va trimite caracterele '2' și '0')
 - **Serial.print(val, format)** – format specifică baza de numerație folosită (BIN, OCT, DEC, HEX. Pentru numere în virgulă mobilă, format specifică numărul de zecimale folosit.
 - **Serial.println** – Trimite datele + \r\n (ASCII 13 + ASCII 10)
 - **Example:**

Serial.print(78) transmite "78"	Serial.print(78, BIN) transmite "1001110"
Serial.print(1.23456) transmite "1.23"	Serial.println(1.23456, 4) transmite "1.2346"
Serial.print("Hello") transmite "Hello"	



UART la Arduino



- byte IncomingByte = **Serial.read()** – citește un byte prin interfața serială
- int NoOfBytesSent = **Serial.write(data)** – scrie date în format binar prin interfața serială. Datele se pot scrie ca un byte (val) sau ca un șir de octeți specificat ca un string (str) sau ca un șir buf, de lungime specificată len (buf, len)
- **Serial.flush()** – așteaptă până când transmisia datelor pe interfața serială este completă.
- int NoOfBytes = **Serial.available()** – Returnează numărul de octeți disponibili pentru a fi citați prin interfața serială. Datele sunt deja primite și stocate într-o zonă de memorie buffer (capacitate maximă 64 octeți)
- **serialEvent()** – funcție definită de utilizator, care este apelată automat când există date disponibile în zona buffer. Folosiți **Serial.read()** în această funcție, pentru a citi aceste date.
- serialEvent1(), serialEvent2(), serialEvent3() – Pentru Arduino Mega, funcții care se apelează automat pentru celelalte interfețe seriale.



Exemplul 1:

```
void setup() {  
    Serial.begin(9600);           // deschide portul serial, configureaza viteza la 9600 bps  
    Serial.println("Hello");  
}  
void loop() {}
```

Exemplul 2 (doar pentru Arduino Mega):

// Arduino Mega foloseste patru porturi seriale (Serial, Serial1, Serial2, Serial3),
// se pot configura cu viteze diferite:

```
void setup(){  
    Serial.begin(9600);  
    Serial1.begin(38400);  
    Serial2.begin(19200);  
    Serial3.begin(4800);  
  
    Serial.println("Hello Computer");  
    Serial1.println("Hello Serial 1");  
    Serial2.println("Hello Serial 2");  
    Serial3.println("Hello Serial 3");  
}  
void loop() {}
```




Comunicație $\mu\text{C} \leftrightarrow \text{PC}$, Exemplul 3 – recepționează o cifră (caracter de la '0' la '9') și modifică starea unui LED proporțional cu cifra citită

```
const int ledPin = 13;           // pin LED
int blinkRate=0;                 // rata de modificare a starii
void setup()
{
  Serial.begin(9600); // initializare port serial
  pinMode(ledPin, OUTPUT); // configurare pin LED ca iesire
}
void loop() {
  if ( Serial.available())       // Verifica dacă avem date de citit
  {
    char ch = Serial.read(); // citește caracterul receptionat
    If( isDigit(ch) )         // Este cifră ?
    {
      blinkRate = (ch - '0'); // Se convertește în valoare numerică
      blinkRate = blinkRate * 100; // Rata = 100ms * cifra citită
    }
  }
  blink();
}
```



Exemplul 3 – cont.

```
// modifica stare LED pe baza ratei calculate
void blink()
{
  digitalWrite(ledPin,HIGH);
  delay(blinkRate); // intarzierea calculata
  digitalWrite(ledPin,LOW);
  delay(blinkRate);
}
```

- **Pentru a utiliza acest exemplu:**
- Folositi Serial Monitor, inclus in mediul Arduino, activându-l din meniul Tools sau apăsând <CTRL+SHIFT+M>)
- Selectați aceeași viteză cu cea pe care ați configurat-o apelând Serial.begin()
- Tastați cifre, și apăsați butonul “Send”.



Exemplul 4 – exemplul 3 modificat să folosească serialEvent()

```
void loop()
{
  blink();
}

void serialEvent() // functia se apeleaza automat, cand exista date de citit
{
  while(Serial.available()) // cat timp exista date disponibile
  {
    char ch = Serial.read(); // acestea se citesc
    Serial.write(ch); // echo – trimitem datele inapoi pentru verificare
    if( isDigit(ch) ) // e cifra ?
    {
      blinkRate = (ch - '0'); // convertire la valoare numerica
      blinkRate = blinkRate * 100; // calcul timp de intarziere
    }
  }
}
```



- În afara interfețelor seriale hardware, Arduino permite comunicarea serială și pe alți pini digitali, realizând procesul de serializare / de-deserializare a datelor prin program
- Se folosește biblioteca `SoftwareSerial`, inclusă în pachetul software Arduino (necesită includerea header-ului `softwareserial.h`)
- Pinul de receptie (**RX**) **trebuie conectat la un pin digital care suporta întreruperi externe** declanșate prin schimbarea stării acestuia:
 - La Arduino Mega, acești pini sunt: 10, 11, 12, 13, 14, 15, 50, 51, 52, 53, A8 (62), A9 (63), A10 (64), A11 (65), A12 (66), A13 (67), A14 (68), A15 (69)
- Se pot crea mai multe obiecte C++ de tip `SoftwareSerial`, dar numai unul poate fi activ la un moment dat
- Crearea unui obiect – trebuie specificați pinii de recepție și transmisie:
`SoftwareSerial mySerial = SoftwareSerial(rxPin, txPin)`
- Sunt implementate funcțiile **`begin`**, **`read`**, **`write`**, **`print`**, **`println`**, care se folosesc în același mod ca în cazul interfețelor seriale hardware



Software UART la Arduino



- **Exemplu:** comunicarea folosind două interfețe seriale, una hardware, conectată la PC, și una software, conectată la un alt dispozitiv compatibil UART
 - Arduino joaca rolul de releu de comunicație: ce primește pe o interfată, transmite pe cealaltă.

```
#include <SoftwareSerial.h>
```

```
SoftwareSerial mySerial(10, 11); // Interfata software foloseste pin 10 pt RX, pin 11 pt TX
```

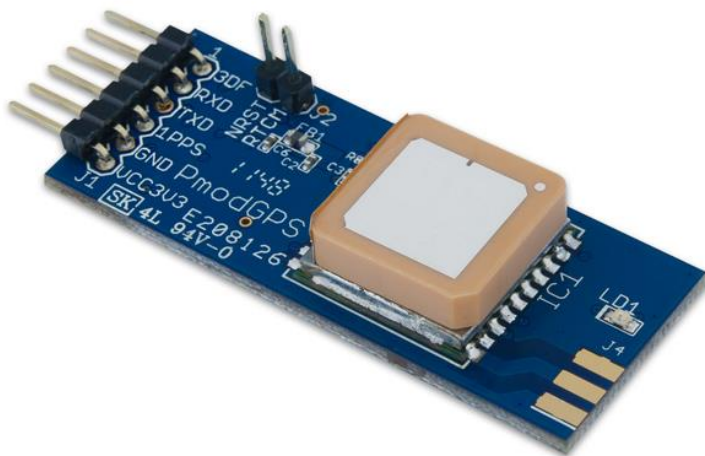
```
void setup()
```

```
{  
  Serial.begin(9600);           // Configurare interfata hardware  
  mySerial.begin(9600);         // Configurare interfata software  
}
```

```
void loop()
```

```
{  
  if (mySerial.available())  
    Serial.write(mySerial.read()); // citire de pe interfata software, transmisie prin cea hardware  
  if (Serial.available())  
    mySerial.write(Serial.read()); // citire de pe interfata hardware, transmisie prin cea software  
}
```

- **Exemplu:** comunicarea folosind două interfețe seriale, una hardware, conectată la PC, și una software, conectată la un alt dispozitiv compatibil UART
- Posibilă utilizare a programului anterior: vizualizarea datelor transmise de către un dispozitiv compatibil UART pe terminalul Serial Monitor
- Exemplu: receptor GPS Digilent PMod GPS:



```
$GPGGA,064951.000,2307.1256,N,12016.4438,E,1,8,0.95,39.9,M,17.8,M,,*65
$GPGSA,A,3,29,21,26,15,18,09,06,10,,,,,2.32,0.95,2.11*00
$GPGSV,3,1,09,29,36,029,42,21,46,314,43,26,44,020,43,15,21,321,39*7D
$GPRMC,064951.000,A,2307.1256,N,12016.4438,E,0.03,165.48,260406,3.05,W,A*55
$GPVTG,165.48,T,,M,0.03,N,0.06,K,A*37
```



Codurile ASCII



Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

Source: www.LookupTables.com



Codurile ASCII Extinse



128	Ç	144	É	161	í	177	␣	193	⌞	209	⌞	225	β	241	±
129	û	145	æ	162	ó	178	␣	194	⌞	210	⌞	226	Γ	242	≥
130	é	146	Æ	163	ù	179		195	⌞	211	⌞	227	π	243	≤
131	â	147	ô	164	ñ	180	⌞	196	—	212	⌞	228	Σ	244	∫
132	ä	148	ö	165	Ñ	181	⌞	197	+	213	⌞	229	σ	245	∫
133	à	149	ò	166	²	182	⌞	198	⌞	214	⌞	230	μ	246	÷
134	å	150	û	167	°	183	⌞	199	⌞	215	⌞	231	τ	247	≈
135	ç	151	ù	168	¿	184	⌞	200	⌞	216	⌞	232	Φ	248	°
136	ê	152	—	169	—	185	⌞	201	⌞	217	⌞	233	Θ	249	·
137	ë	153	Ö	170	¬	186	⌞	202	⌞	218	⌞	234	Ω	250	·
138	è	154	Ü	171	½	187	⌞	203	⌞	219	■	235	δ	251	√
139	ï	156	£	172	¼	188	⌞	204	⌞	220	■	236	∞	252	—
140	î	157	¥	173	¡	189	⌞	205	=	221	■	237	φ	253	²
141	ì	158	—	174	«	190	⌞	206	⌞	222	■	238	ε	254	■
142	Ä	159	ƒ	175	»	191	⌞	207	⌞	223	■	239	∩	255	
143	Å	160	á	176	␣	192	⌞	208	⌞	224	α	240	≡		

Source: www.LookupTables.com



1. **Atmel ATmega640/V-1280/V-1281/V-2560/V-2561/V datasheet**
2. **Atmel Atmega64 datasheet**
3. **Atmel Atmega2560 datasheet**



Proiectarea cu Micro-Procesoare

Lector: Mihai Negru

An 3 – Calculatoare și Tehnologia Informației
Seria B

Curs 7: Comunicare Serială 2

<http://users.utcluj.ro/~negrum/>

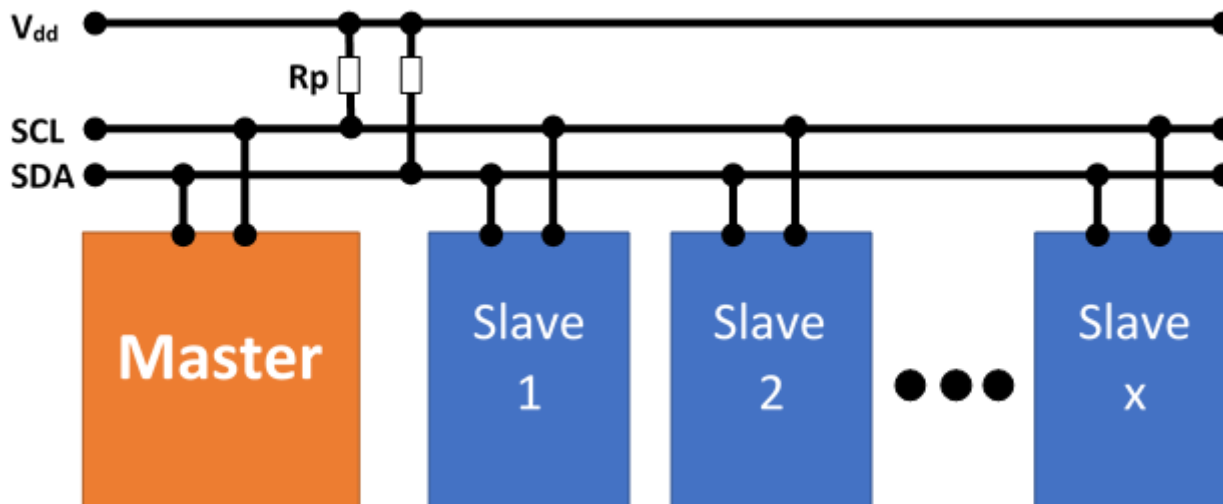


I²C (Inter-Integrated Circuit)



- **Structura**

- *SDA*, *SCL* – date și clock, bidirecționale, conectate “open collector” sau “open drain”
- Rezistente “pull up” țin linia la V_{cc}
- O linie poate fi trasă la ‘0’ de orice dispozitiv – ea va deveni ‘0’ dacă cel puțin un dispozitiv scrie ‘0’, altfel e ‘1’
- Fiecare dispozitiv are o adresă de 7 biți
- Exista 16 adrese rezervate → 112 adrese disponibile → max 112 dispozitive





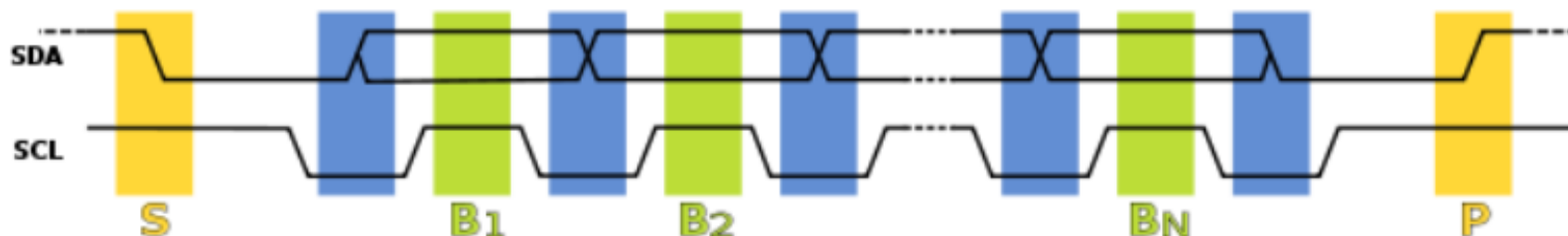
- **Tipuri de dispozitive (noduri)**
 - *Master* – generează semnalul de ceas, și adresele
 - *Slave* – primește semnal de ceas, și adresă
 - Rolul de master și slave se poate schimba pentru același dispozitiv
- **Moduri de operare**
 - *master transmit* — nodul master trimite date la slave
 - *master receive* — nodul master recepționează date de la slave
 - *slave transmit* — nodul slave trimite date la master
 - *slave receive* — nodul slave primește date de la master



I²C (Inter-Integrated Circuit)



- **Diagrame de timp**



- **Moduri de operare**

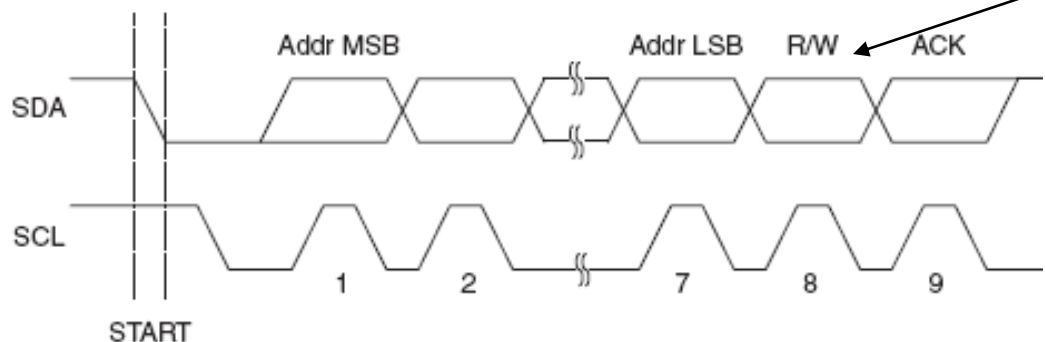
- *Start*: o tranziție a SDA din '1' în '0', cu SCL menținut pe '1'
- *Transfer biți*: valoarea bitului în SDA se schimbă când SCL e '0', și este menținută stabilă pentru preluare când SCL e '1'
- *Stop*: o tranziție a SDA din '0' în '1' când SCL e '1'



I²C (Inter-Integrated Circuit)

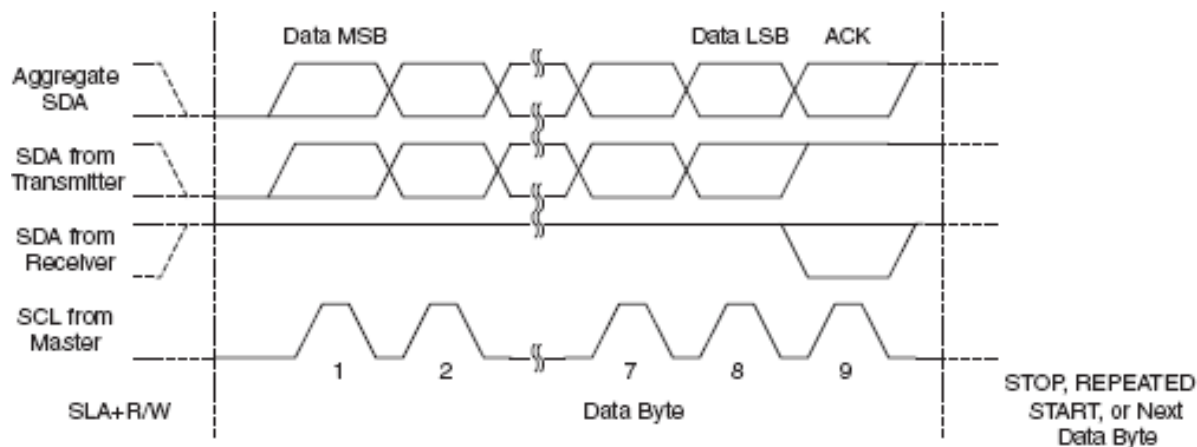


- **Formatul pachetelor – adresa**



Specifică dacă se dorește citire sau scriere

- **Formatul pachetelor – date**

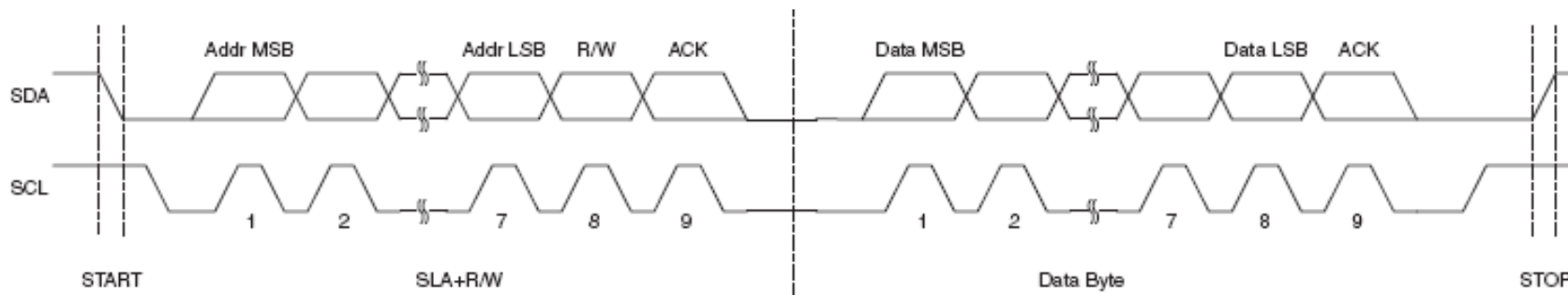




I²C (Inter-Integrated Circuit)



- **Transmisie tipică**



- **Arbitrare**

- Fiecare master monitorizează semnalele de START și STOP, și nu inițiază un mesaj cât timp alt master ține bus-ul ocupat
- Dacă doi master inițiază transmisie în același timp, se produce arbitrarea, pe baza liniei SDA
 - Fiecare master verifică linia SDA, și dacă nivelul acesteia nu este cel așteptat (cel scris), acel master pierde arbitrarea
 - Primul master care pune pe linie '1' când altul pune '0' pierde
 - Master-ul care pierde așteaptă un semnal de stop, apoi încearcă din nou



- **Reglarea vitezei de transmisie**

- “Clock stretching”
- Un dispozitiv slave poate ține linia de ceas (SCL) la ‘0’ mai mult timp, indicând necesitatea unui timp mai lung pentru procesarea datelor
- Dispozitivul master va încerca să pună linia SCL pe ‘1’, dar va eșua din cauza configurației ‘open collector’
- Dispozitivul master va verifica dacă linia SCL a fost pusă pe ‘1’ și va continua transmisia când acest lucru se întâmplă
- Nu toate dispozitivele suportă “clock stretching”
- Există limite pentru intervalul de stretching



- **Aplicații**

- Citirea datelor de configurare din SPD EEPROM-ul din SDRAM, DDR SDRAM, DDR2 SDRAM
- Interfațare senzori digitali
- Accesarea convertoarelor DAC și ADC.
- Schimbarea setărilor în monitoare video (Display Data Channel).
- Schimbarea volumului în boxe inteligente.
- Citirea stării senzorilor de diagnostic hardware, precum termostatul CPU și viteza ventilatorului.
- Posibilitatea ca un microcontroller să controleze multiple dispozitive atașate prin doar două fire.
- Posibilitatea ca dispozitive să fie atașate sau eliminate de pe bus în timpul funcționării



- Se utilizează biblioteca **Wire**, din pachetul software Arduino
- O placa Arduino poate fi I2C master sau I2C slave
- Metodele obiectului Wire:
 - **Wire.begin(address)** – activează interfața I2C. Parametrul address indică adresa de 7 biți cu care Arduino se atasează la bus-ul I2C ca slave. Dacă funcția este apelată fără parametri, dispozitivul este master.
 - **Wire.beginTransmission(address)** – începe procesul de transmisie dinspre master către un slave specificat de **address**. După apelul acestei funcții, datele pot fi scrise cu **write()**.
 - **Wire.endTransmission()** – apelat de master pentru a realiza efectiv transmisia începută cu **beginTransmission**, cu datele pregătite prin apelul **Wire.write()**.



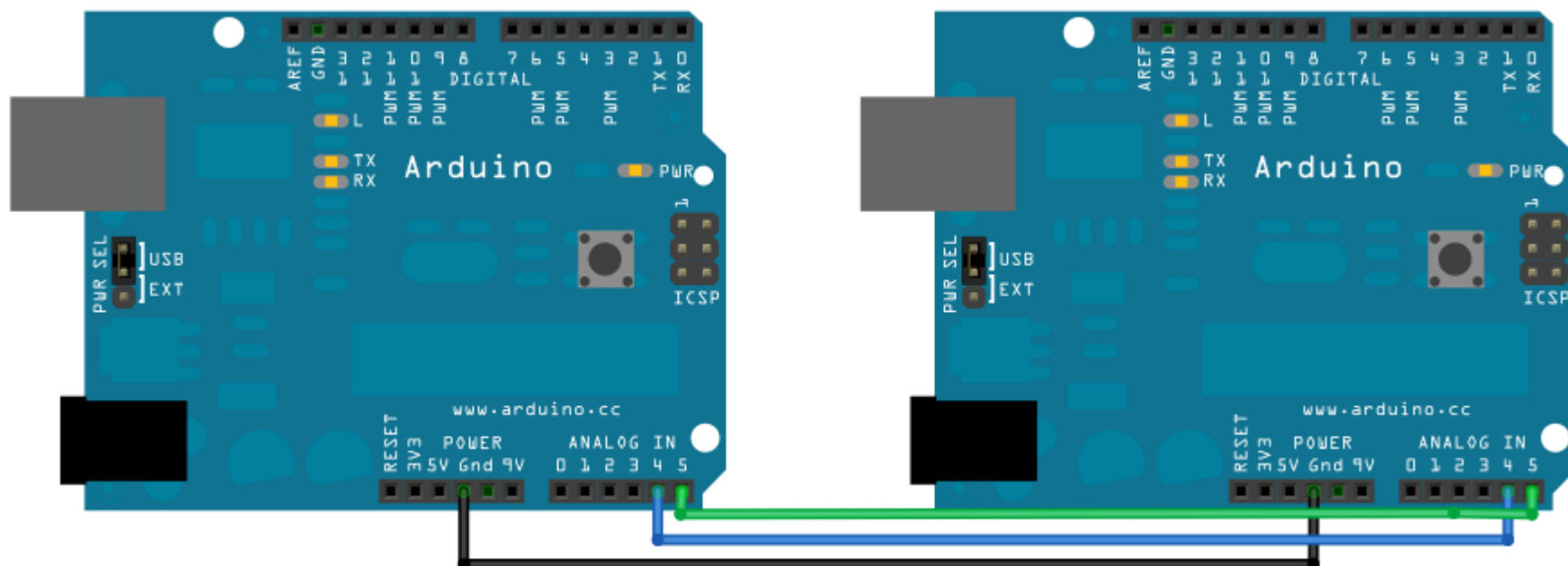
- **Wire.write(value)** – scrie un byte. Funcția poate fi apelată de slave, dacă a fost solicitat de către master, sau de master după ce a apelat beginTransmission. Alternative: Wire.write(string), sau Wire.write(data, length).
- **Wire.requestFrom(address, quantity)** – master cere o cantitate (**quantity**) de date de la un slave identificat prin **address**. Funcția poate fi apelată și ca **Wire.requestFrom(address, quantity, stop)**, stop fiind o valoare booleană specificând dacă masterul va elibera bus-ul, sau va menține conexiunea activă.
- **Wire.available()** – returnează numărul de bytes disponibili pentru a fi citați.
- **Wire.read()** – citește un byte, dacă available() > 0. Apelabilă și de master, și de slave.
- **Wire.onReceive(handler)** – configurează o **funcție handler**, la dispozitivul slave, care va fi apelată automat la primirea datelor de la master.
- **Wire.onRequest(handler)** – configurează o funcție handler, la dispozitivul slave, care va fi apelată automat atunci când masterul cere date.



Comunicare I2C la Arduino



- **Exemplu:** conectarea a două plăci Arduino prin I2C, una având rol de master, care va transmite datele, și una de slave, care le va recepționa.
- Sursa: <http://arduino.cc/en/Tutorial/MasterWriter>





- **Exemplu:** conectarea a două plăci Arduino prin I2C, una având rol de master, care va transmite datele, și una de slave, care le va recepționa.

Cod master:

```
#include <Wire.h>
```

```
void setup() {
```

```
    Wire.begin(); // activeaza interfata I2C ca master
```

```
}
```

```
byte x = 0; // valoarea de transmis, se va incrementa
```

```
void loop() {
```

```
    Wire.beginTransmission(4); // incepe un proces de transmisie, catre adresa slave 4
```

```
    Wire.write("x is ");    // scriere string
```

```
    Wire.write(x);          // scriere un byte
```

```
    Wire.endTransmission(); // finalizeaza tranzactia de scriere
```

```
    x++; // incrementare valoare
```

```
    delay(500);
```

```
}
```



- **Exemplu:** conectarea a două plăci Arduino prin I2C, una având rol de master, care va transmite datele, și una de slave, care le va recepționa.

Cod slave:

```
#include <Wire.h>

void setup() {
    Wire.begin(4);                // activeaza interfata i2c ca slave, cu adresa 4
    Wire.onReceive(receiveEvent); // inregistreaza functia receiveEvent pentru a fi apelata la venirea datelor
    Serial.begin(9600);           // activeaza interfata seriala, pentru a afisa pe PC datele primite
}

void loop() { // functia loop nu face nimic
    delay(100);
}

void receiveEvent(int howMany) // functie apelata cand slave primeste date
{
    while(1 < Wire.available()) // parcurge cantitatea de date, lasand ultimul octet separat
    {
        char c = Wire.read(); // citeste caracter cu caracter
        Serial.print(c);      // trimite caracterul la PC
    }
    int x = Wire.read(); // ultimul caracter este tratat ca o valoare numerica
    Serial.println(x);   // si va fi tiparit ca atare
}
```

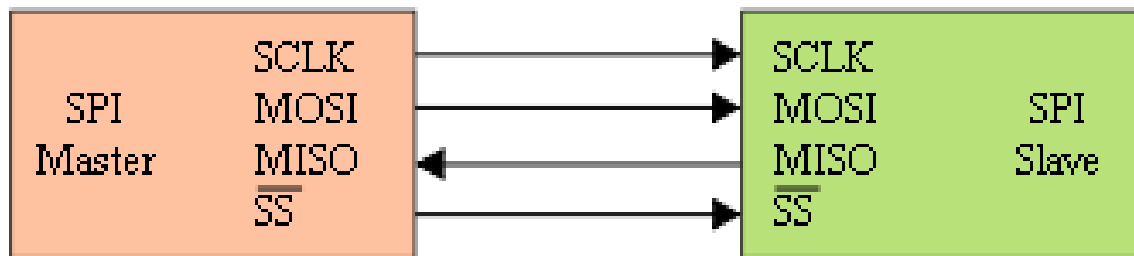


Serial Peripheral Interface (SPI)



- **Semnale**

- SCLK – Serial clock, generat de Master
- MOSI – Master Output Slave Input, date transmise de Master
- MISO – Master Input Slave Output, date receptionate de Master
- SS – Slave Select, activarea dispozitivului Slave de catre Master, activ pe zero



- **Funcționare**

- Master-ul inițiază comunicația prin activarea SS ($SS \rightarrow 0$)
- Master generează semnalul de ceas SCLK
- La fiecare perioadă de ceas un bit se transmite de la master la slave, și un bit de la slave la master – sincron
- După fiecare pachet de date (8, 16 biți,...) SS este dezactivat ($SS \rightarrow 1$), pentru sincronizarea transmisiei

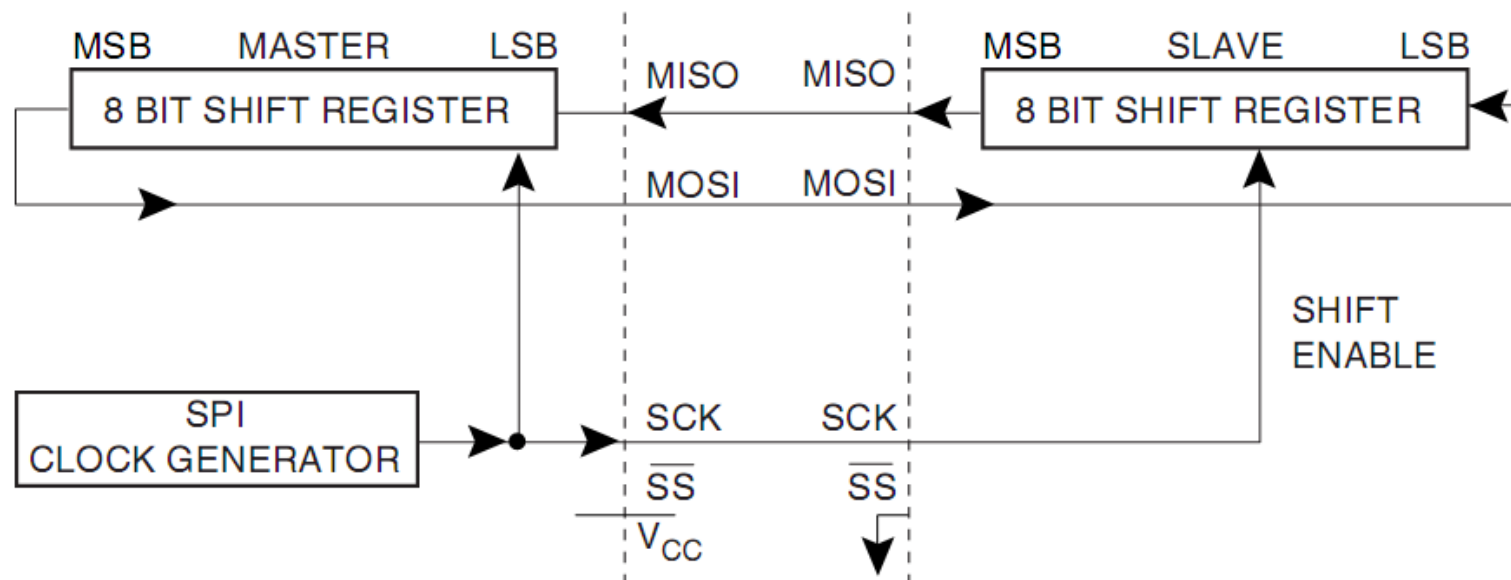


Serial Peripheral Interface (SPI)



- **Principiul de functionare**

- Ambii parteneri au câte un registru de deplasare intern, ieșirile și intrările fiind conectate prin MISO/MOSI
- Ambii registri au același ceas, SCLK
- Cei doi regiștri formează împreună un registru de rotație
- După un număr de perioade de ceas egal cu dimensiunea unui registru, Master și Slave fac schimb de date



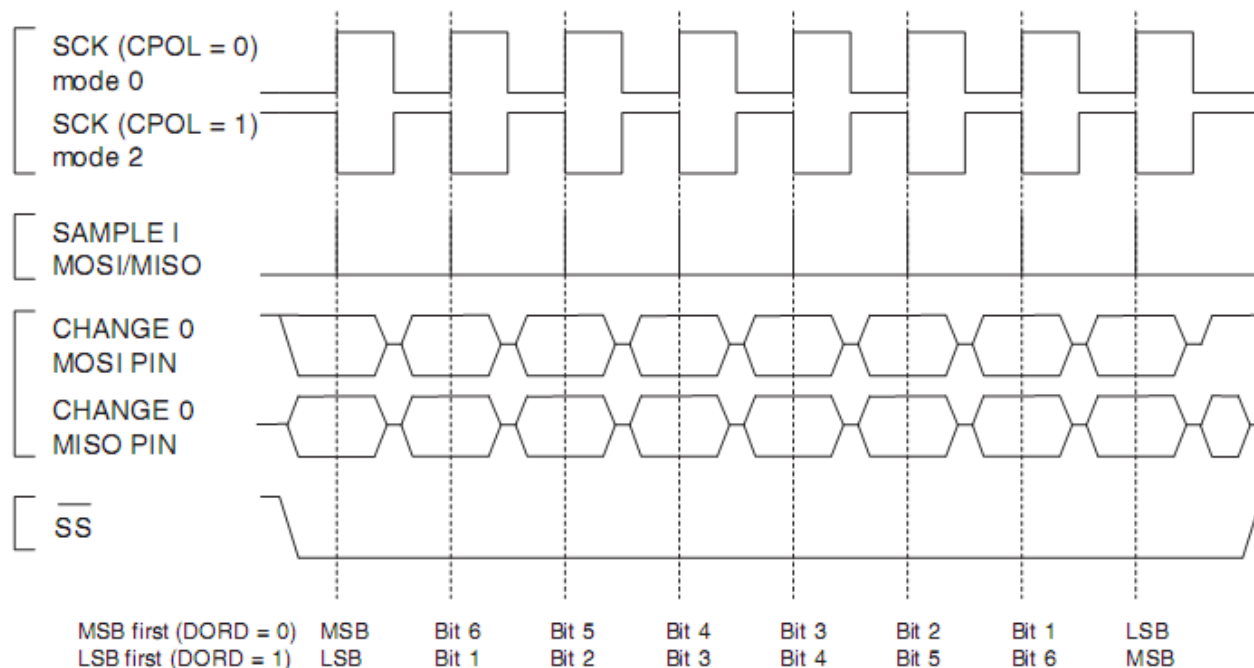


Serial Peripheral Interface (SPI)



- **Sincronizarea datelor cu semnalul de ceas**

- Deplasarea (shiftare) datelor și preluarea lor se fac pe fronturi opuse
- CPOL – clock polarity: primul front e crescător sau descrescător
- CPHA – clock phase
- Pentru CPHA = 0
 - Pe primul front se face preluarea datelor
 - Pe al doilea front se face stabilizarea (deplasarea)

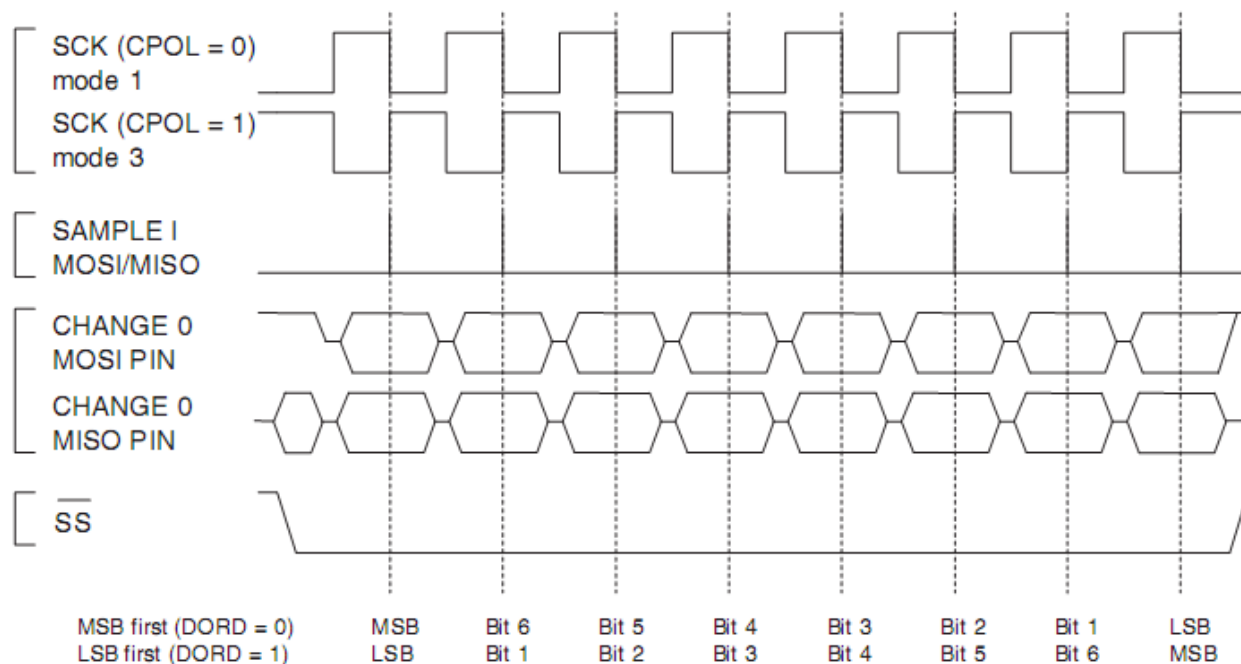




Serial Peripheral Interface (SPI)



- **Sincronizarea datelor cu semnalul de ceas**
 - Deplasarea (shiftare) datelor și preluarea lor se fac pe fronturi opuse
 - CPOL – clock polarity: primul front e crescător sau descrescător
 - CPHA – clock phase
 - Pentru CPHA = 1
 - Pe primul front se face deplasarea
 - Pe al doilea front se face preluarea datelor



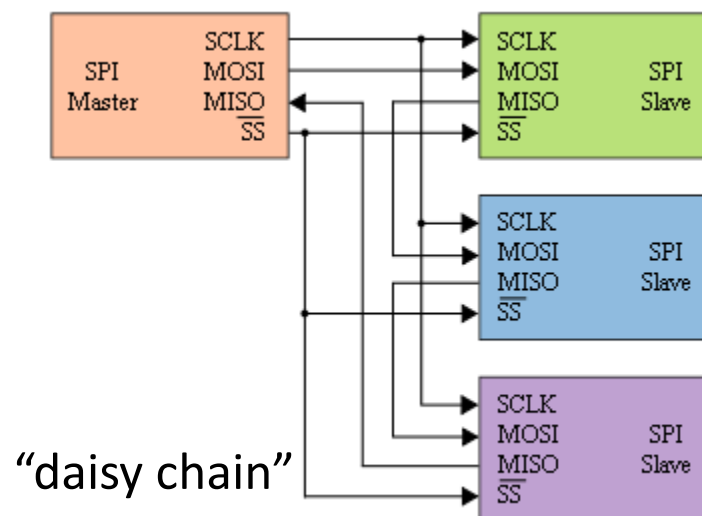
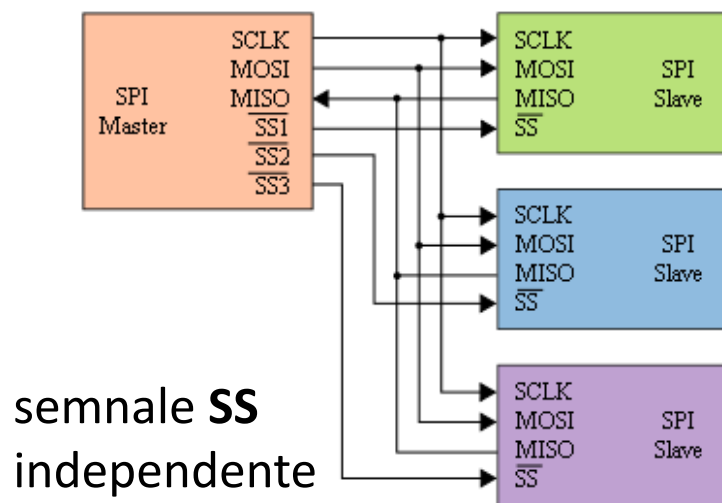


Serial Peripheral Interface (SPI)



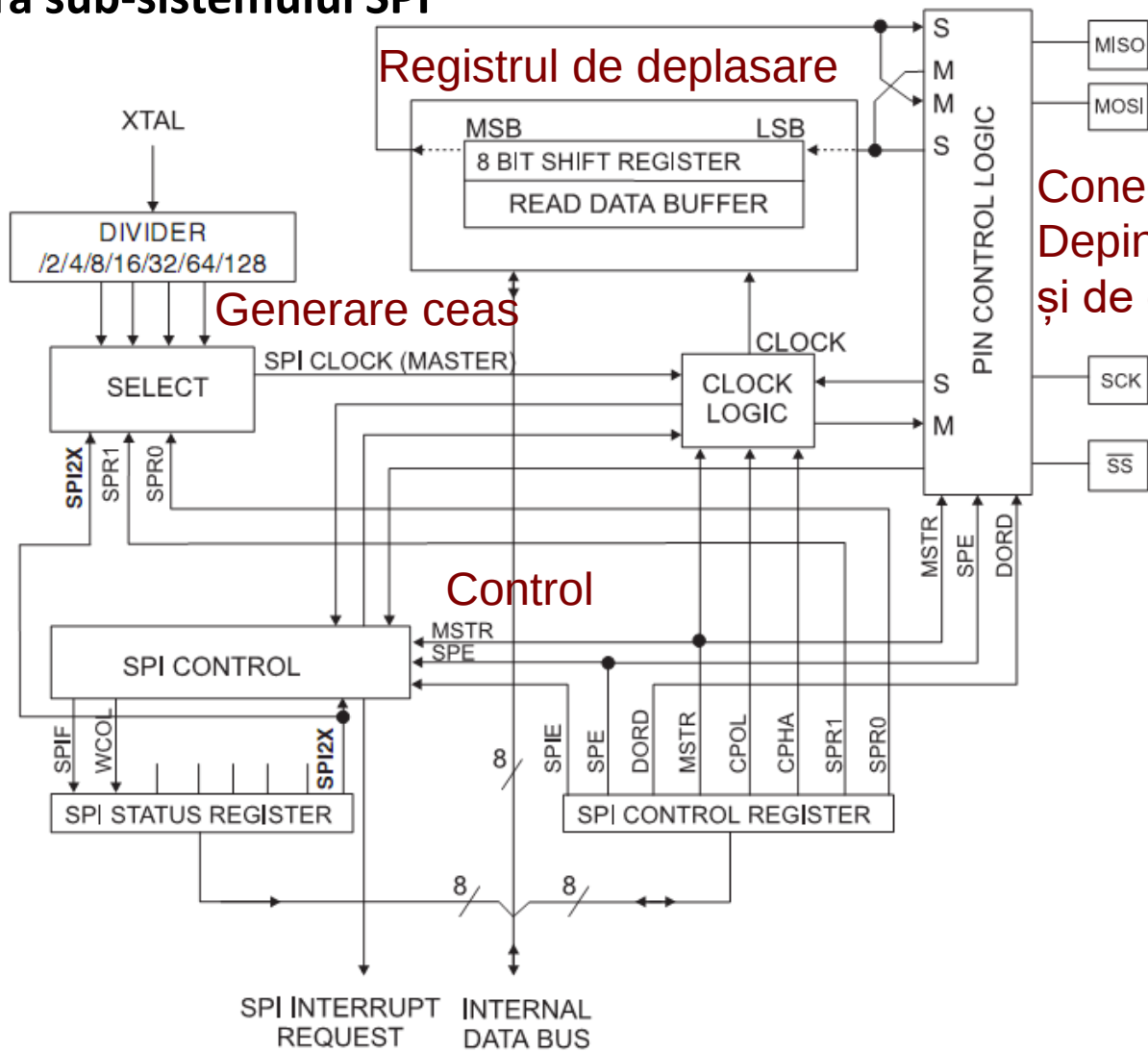
• Utilizarea semnalului SS

- Pentru un dispozitiv “slave” **SS** este semnal de intrare
 - $SS = 0 \rightarrow$ activarea dispozitivului slave. O tranziție din 0 în 1 $\rightarrow$ marchează sfârșitul unui pachet (resetarea ciclului de transfer)
 - $SS = 1 \rightarrow$ dispozitiv slave inactiv
- Pentru un dispozitiv “master” **SS** poate fi:
 - ieșire – prin el se activează dispozitivul “slave” pentru comunicare
 - intrare – dacă se permit mai multe dispozitive master, o valoare ‘0’ la intrarea SS trece dispozitivul curent în modul “Slave”
- Configurații cu mai multe dispozitive





- **Arhitectura sub-sistemului SPI**



Conectare pini
Depinde de mod (M/S)
și de ordinea datelor



Serial Peripheral Interface (SPI) la AVR



- Configurare SPI

Bit	7	6	5	4	3	2	1	0	
0x0D (0x2D)	SPIE	SPE	DORD	MSTR	CPOL	CPHA	SPR1	SPR0	SPCR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- Registrul **SPCR**:

- SPIE – SPI Interrupt Enable, generare întrerupere la terminarea transmisiei
- SPE – SPI Enable. Trebuie setat pe 1 pentru orice operație cu SPI
- DORD – Data Order. 1=LSB first, 0 = MSB first
- MSTR – 1: Master, 0: Slave
- CPOL, CPHA – selectează polaritatea și faza semnalului SCLK

	Leading Edge	Trailing Edge
CPOL = 0, CPHA = 0	Sample (Rising)	Setup (Falling)
CPOL = 0, CPHA = 1	Setup (Rising)	Sample (Falling)
CPOL = 1, CPHA = 0	Sample (Falling)	Setup (Rising)
CPOL = 1, CPHA = 1	Setup (Falling)	Sample (Rising)

- SPR1, SPR0 – reglează viteza SPI împreună cu SPI2X din registrul SPSR



Serial Peripheral Interface (SPI) la AVR



- Configurare SPI

Bit	7	6	5	4	3	2	1	0	
0x0E (0x2E)	SPIF	WCOL	–	–	–	–	–	SPI2X	SPSR
Read/Write	R	R	R	R	R	R	R	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- Registrul **SPSR**:

- SPI2X – Reglare frecvență ceas, împreună cu SPR1 și SPR0 din SPCR
- WCOL – Write collision: setat dacă scriem în **SPDR** înainte ca SPI să transfere datele
- SPIF – SPI Interrupt flag: setat când se termină transmisia. Dacă SPIE este setat, se generează cerere de întrerupere

SPI2X	SPR1	SPR0	SCK Frequency
0	0	0	$f_{osc}/4$
0	0	1	$f_{osc}/16$
0	1	0	$f_{osc}/64$
0	1	1	$f_{osc}/128$
1	0	0	$f_{osc}/2$
1	0	1	$f_{osc}/8$
1	1	0	$f_{osc}/32$
1	1	1	$f_{osc}/64$



Serial Peripheral Interface (SPI) la AVR



- **Utilizare SPI (Master)**

- Configurare direcție pini I/O: Pinii SPI sunt comuni cu pinii portului B

Port Pin	Alternate Functions
PB7	OC2/OC1C ⁽¹⁾ (Output Compare and PWM Output for Timer/Counter2 or Output Compare and PWM Output C for Timer/Counter1)
PB6	OC1B (Output Compare and PWM Output B for Timer/Counter1)
PB5	OC1A (Output Compare and PWM Output A for Timer/Counter1)
PB4	OC0 (Output Compare and PWM Output for Timer/Counter0)
PB3	MISO (SPI Bus Master Input/Slave Output)
PB2	MOSI (SPI Bus Master Output/Slave Input)
PB1	SCK (SPI Bus Serial Clock)
PB0	$\overline{SS}$ (SPI Slave Select input)

- Configurare: scriere SPCR și SPSR cu valorile corespunzătoare pentru modul de lucru
- Activare SS (PB(0) $\leftarrow$ '0', explicit!)
- Scriere date în SPDR – declanșează transmisia
- Așteptare până SPIF din SPSR este setat – transmisie completă
- Citire date din SPDR – datele trimise de 'slave'
- Dezactivare SS (PB(0) $\leftarrow$ '1')



Serial Peripheral Interface (SPI) la AVR



- **Utilizare SPI (Master) – Cod sursă**

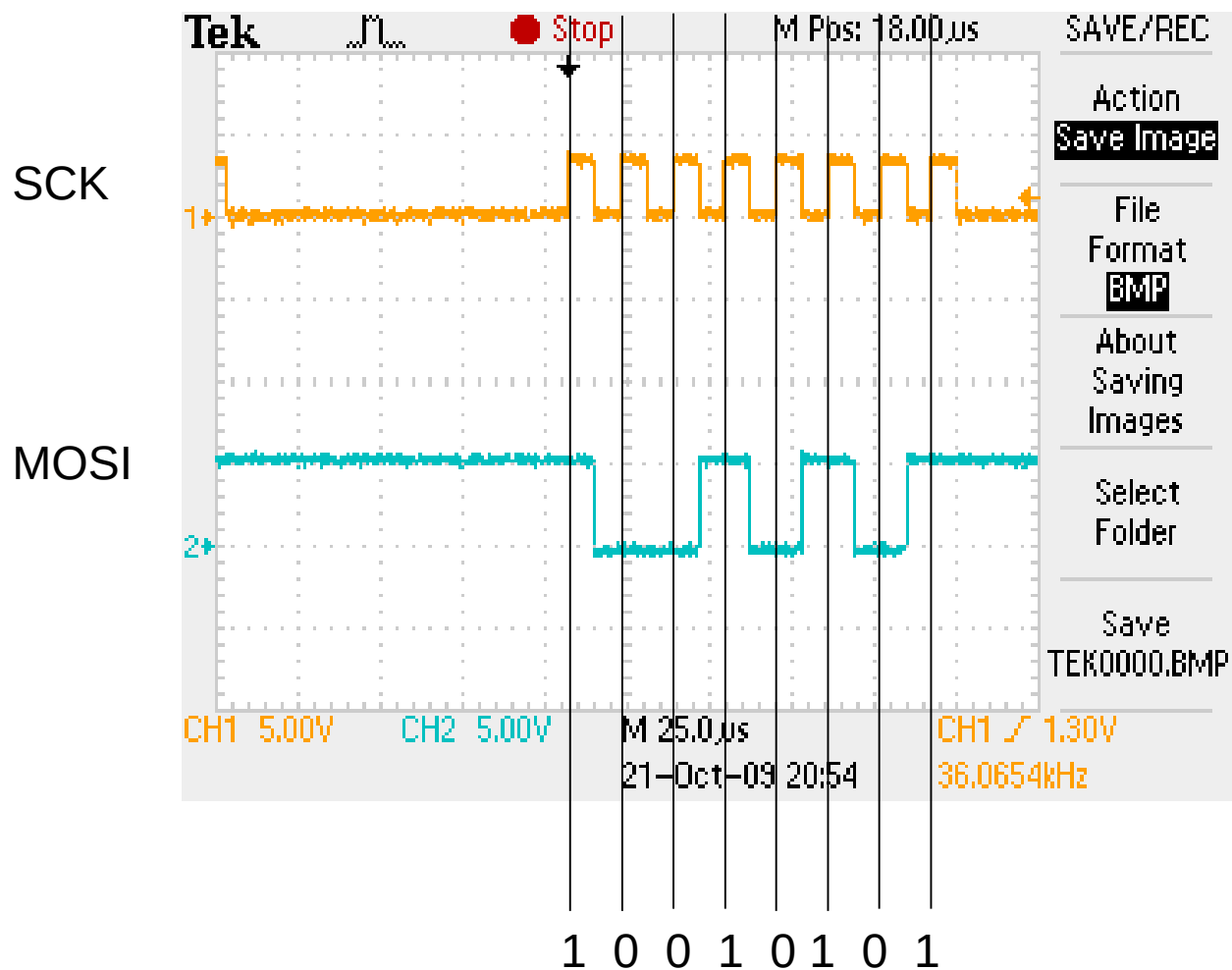
```
.org 0x0000
    jmp reset
reset:
    ldi r16,0b00000111           ; MISO intrare, MOSI, SCK si SS iesire
    out DDRB,r16
    ldi r16, 0b00000001         ; Initial, SS<--1, SPI Slave inactiv
    out PORTB, r16
    cbi SPSR, 0                 ; pune bitul zero din SPSR pe zero - pentru frecventa
    ldi r16,0b01010011        ; Intreruperi dezactivate, SPI Enabled, MSB first, Master,
                                ; CPOL=0, CPHA = 0, Frecventa cea mai lenta
    out SPCR,r16
loop:
    cbi PORTB, 0                 ; SS ← 0
    ldi r16, 0b10010101       ; datele de transmis
    out SPDR, r16
wait:
    sbis SPSR, 7                 ; bitul 7 din SPSR - transmisie completa
    rjmp wait
    in r16,SPDR
    sbi PORTB, 0                 ; SS ← 1
    ldi r18, 0
wait2:
                                ; pauza intre transmisii
    dec r18
    brne wait2
    rjmp loop
```




Serial Peripheral Interface (SPI) la AVR



- Utilizare SPI (Master) – Rezultat

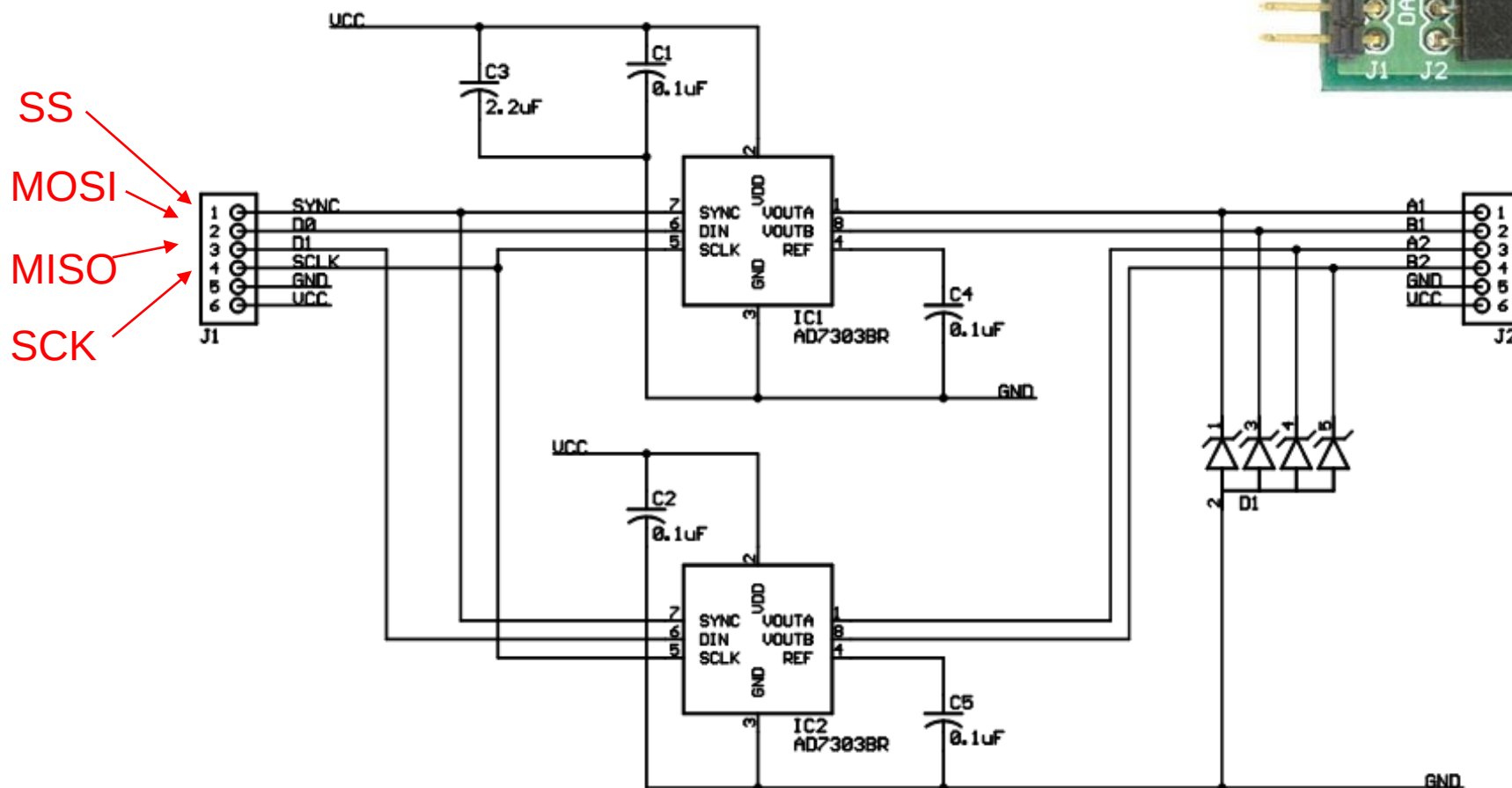
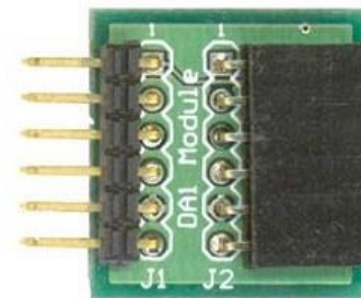




Serial Peripheral Interface (SPI) la AVR



- Conectare module prin SPI
- Digilent **PMOD DA1** – Digital to Analog Converter

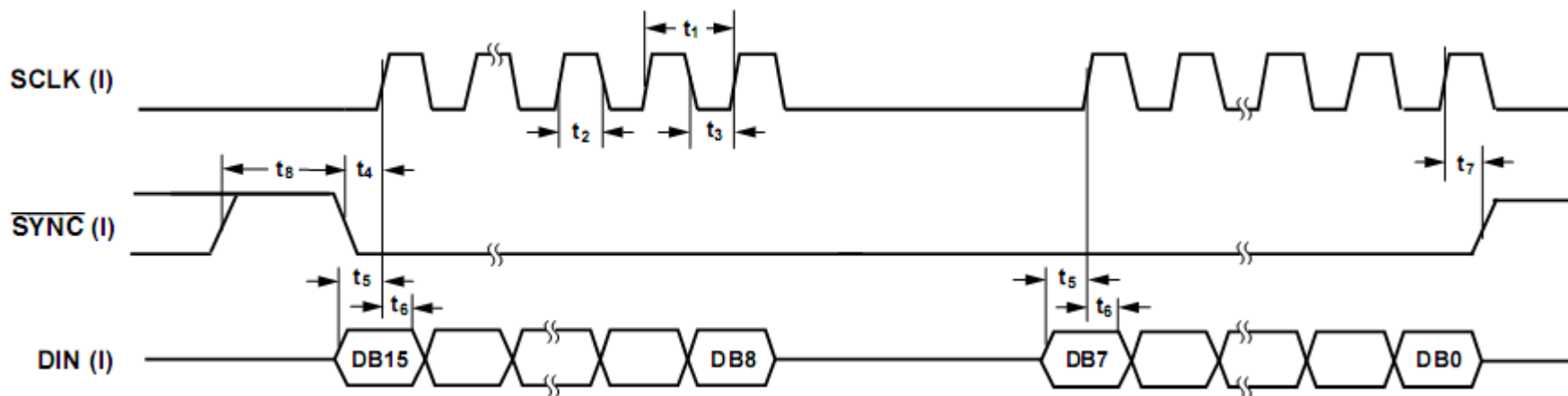
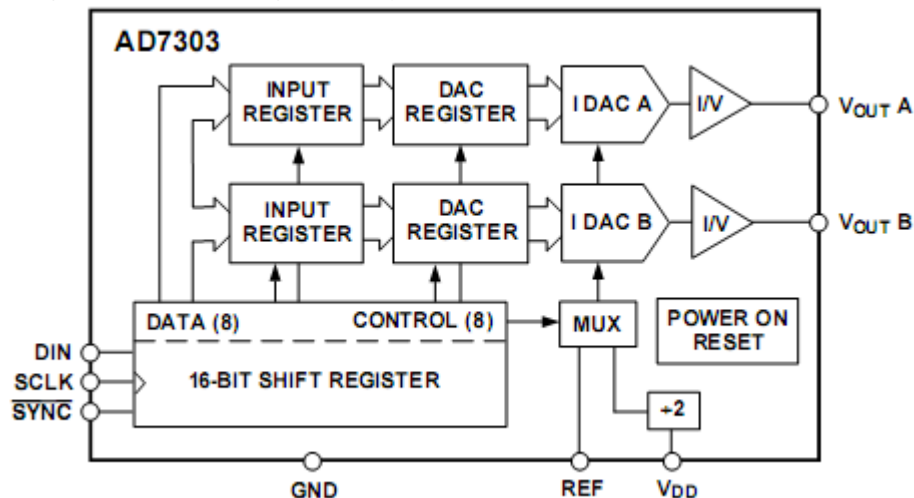




Serial Peripheral Interface (SPI) la AVR



- Conectare module prin SPI – PMOD DA1
- Transmisie 16 biți (2x8 biți) – primii 8 date, următorii control

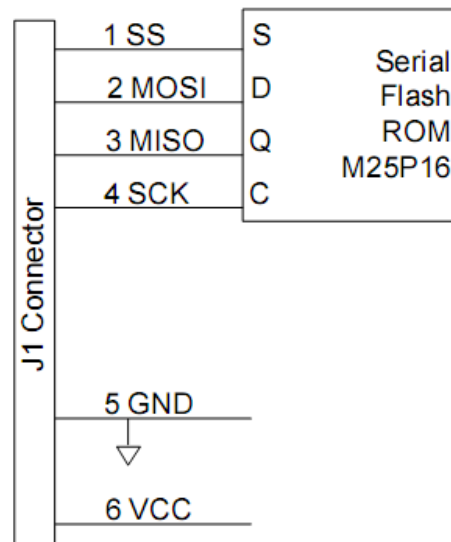




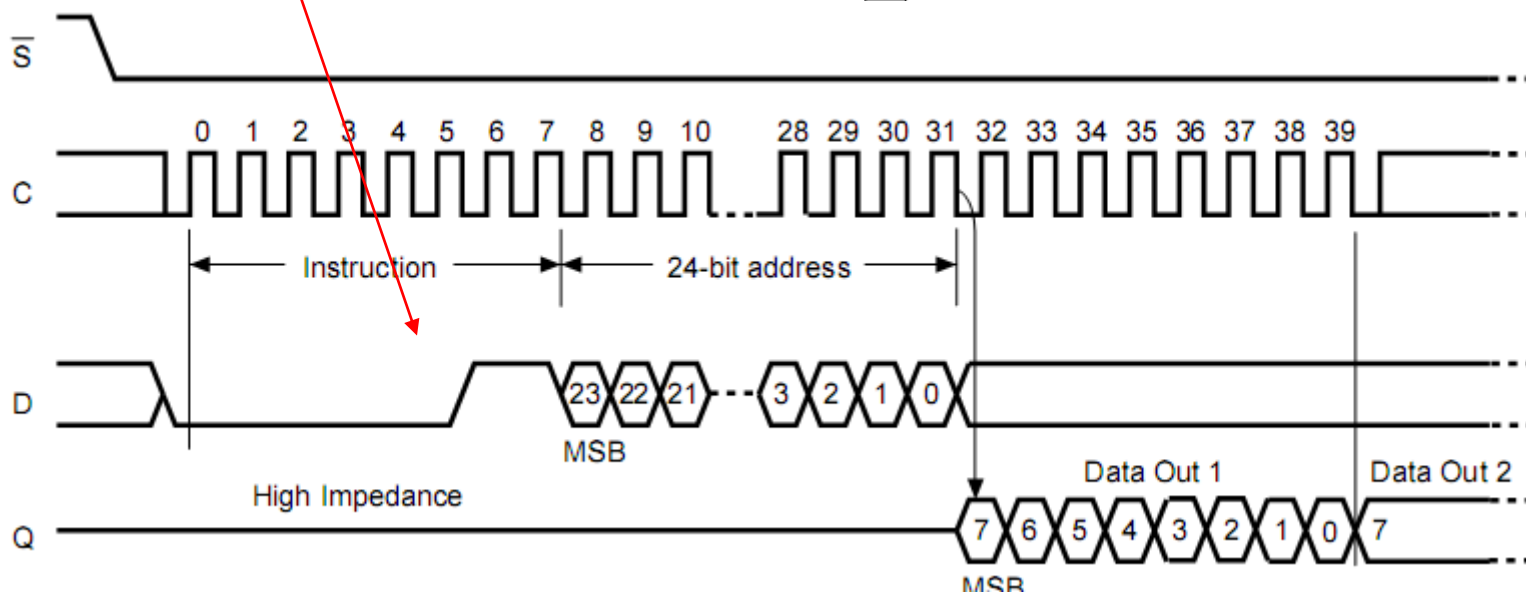
Serial Peripheral Interface (SPI) la AVR



- Conectare module prin SPI
- Digilent PMOD SF – Serial Flash



00000011 = 'READ'





- Se folosește biblioteca SPI (<http://arduino.cc/en/Reference/SPI>) [3]
- Funcții oferite:
 - SPI.setBitOrder(order) – ordinea biților: LSBFIRST sau MSBFIRST
 - SPI.setDataMode(mode) – faza și polaritatea transmisiei: SPI_MODE0, SPI_MODE1, SPI_MODE2 sau SPI_MODE3 (combinațiile CPHA și CPOL)
 - SPI.setClockDivider() – divizorul de frecvență: SPI_CLOCK_DIV(2 .. 128)
 - SPI.begin() – initializare interfață SPI, configurând pinii SCK, MOSI, și SS ca ieșire, punând SCK și MOSI pe LOW, și SS pe HIGH.
 - SPI.end() – dezactivează SPI, dar lasă pinii în modul în care au fost setați la inițializare
 - ReturnByte SPI.transfer(val) – transfera un byte (val) pe magistrala SPI, receptionând în același timp octetul ReturnByte.
- Biblioteca SPI suportă doar modul master
- Orice pin poate fi folosit ca Slave Select (SS).



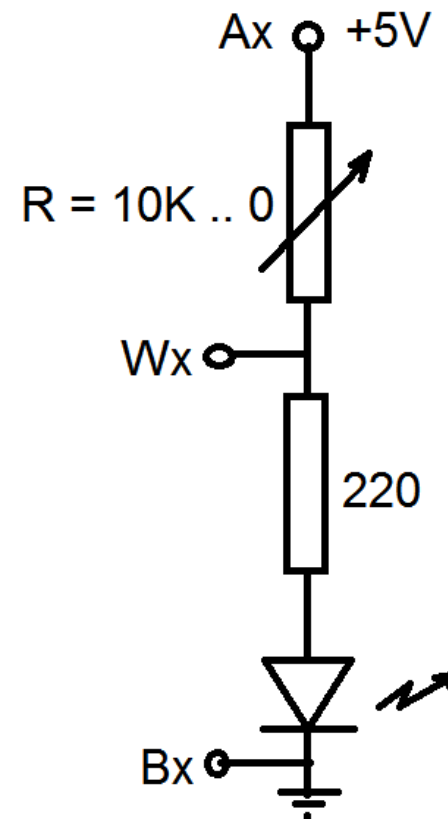
- **Exemplu (SPI)** – Controlul unui potențiometru digital folosind SPI [4]

<http://www.youtube.com/watch?v=1nO2SSExEnQ>

AD5206 datasheet: <http://datasheet.octopart.com/AD5206BRU10-Analog-Devices-datasheet-8405.pdf>

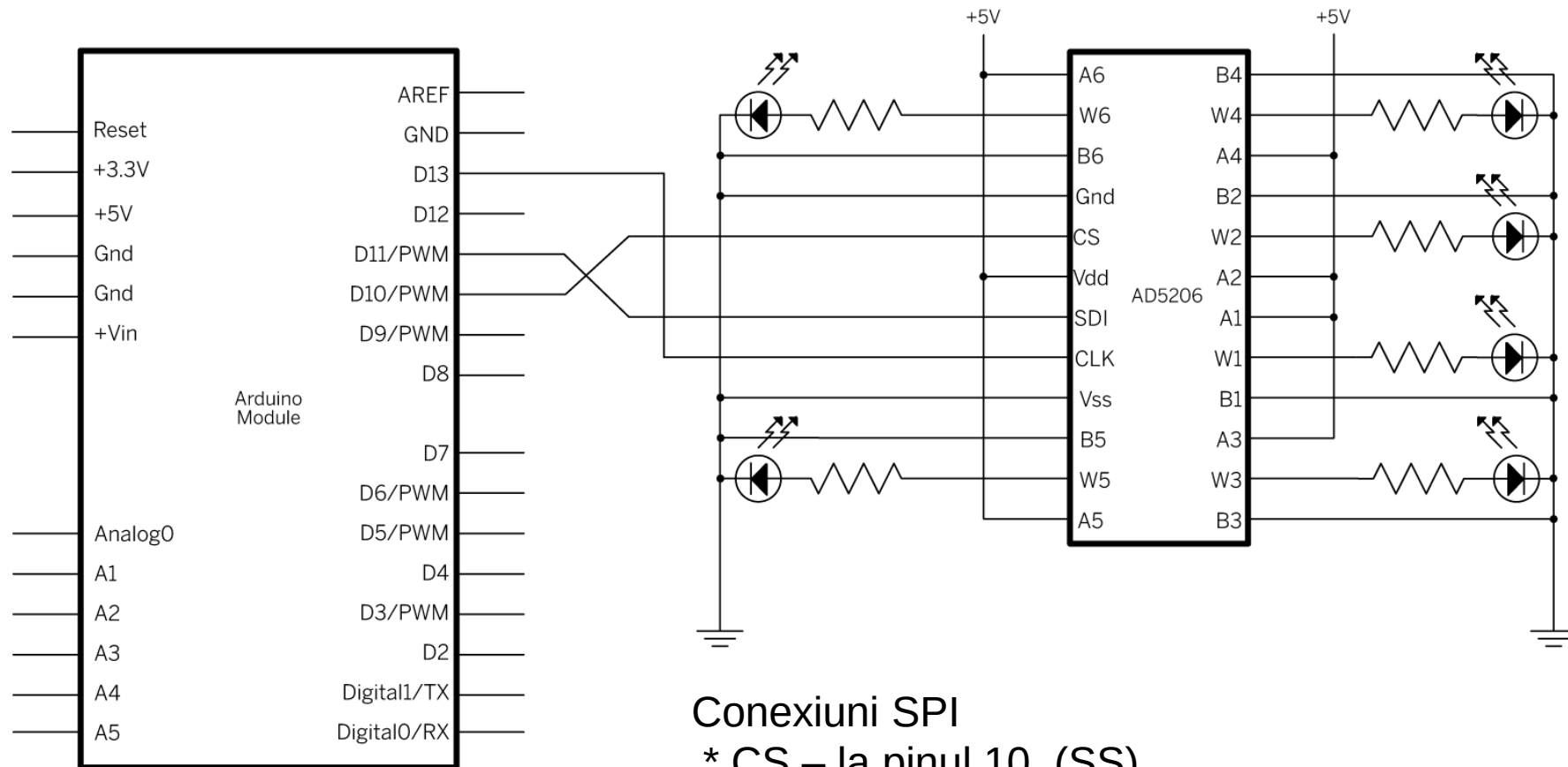
AD5206 este un potențiometru digital cu 6 canale (echivalent cu șase potențiometre individuale).

- 3 pini pe chip pentru fiecare element: Ax, Bx și Wx (Wiper – cursorul).
- pin A = HIGH, pin B = LOW și pinul W = tensiune variabilă. R are rezistența maximă de 10 Kohm, împărțită în 255 pași.
- Pentru controlul rezistenței, se trimite pe SPI doi octeți: primul pentru selecția canalului (0 – 5) și al doilea cu valoarea rezistenței pentru fiecare canal (0 – 255) .





- **Exemplu (SPI)** – Controlul unui potențiometrul digital folosind SPI [4]



Conexiuni SPI

- * CS – la pinul 10 (SS)
- * SDI – la pinul 11 (MOSI)
- * CLK – la pinul 13 (SCK)



- **Exemplu (SPI)** – Controlul unui potențiometru digital folosind SPI [4]

```
#include <SPI.h>
const int slaveSelectPin = 10; // pin 10 ca SS

void setup() {
    // SS trebuie configurat ca iesire
    pinMode (slaveSelectPin, OUTPUT);
    SPI.begin(); // activare SPI
}

void loop() {
    // se iau cele 6 canale la rand
    for (int channel = 0; channel < 6; channel++) {
        // se schimba rezistenta fiecarui canal de la min la max
        for (int level = 0; level < 255; level++) {
            digitalPotWrite(channel, level);
            delay(10);
        }
        delay(100); // asteptam 100 ms cu rezistenta maxima
        // se schimba rezistenta de la max la min
        for (int level = 0; level < 255; level++) {
            digitalPotWrite(channel, 255 - level);
            delay(10);
        }
    }
}
```

```
// functia care foloseste SPI pentru actualizarea
// rezistentelor
void digitalPotWrite(int address, int value) {
    // activeaza SS prin scriere LOW
    digitalWrite(slaveSelectPin, LOW);
    // se trimite pe rand canalul si valoarea:
    SPI.transfer(address);
    SPI.transfer(value);
    // inactiveaza SS prin scriere HIGH
    digitalWrite(slaveSelectPin, HIGH);
}
```




Referințe



1. Atmel ATmega640/V-1280/V-1281/V-2560/V-2561/V datasheet
2. Atmel Atmega64 datasheet
3. Arduino SPI reference guide: <http://arduino.cc/en/Reference/SPI>
4. Arduino SPI Tutorials: <http://arduino.cc/en/Tutorial/SPIDigitalPot>
5. Arduino I2C Library: <http://arduino.cc/en/reference/wire>
6. Arduino I2C tutorial: <http://arduino.cc/en/Tutorial/MasterWriter>



Proiectarea cu Micro-Procesoare

Lector: Mihai Negru

An 3 – Calculatoare și Tehnologia Informației

Seria B

Curs 8: Controlul Motoarelor. Senzori percepție

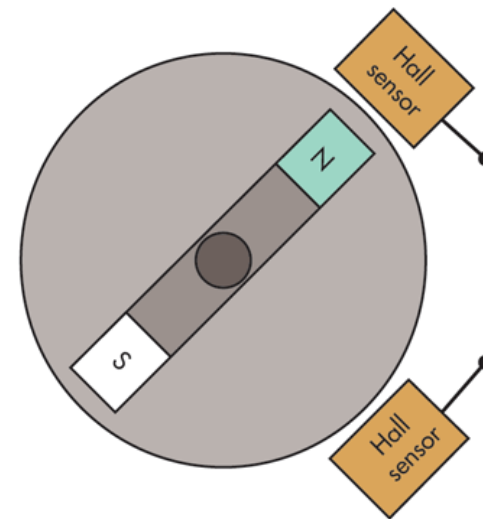
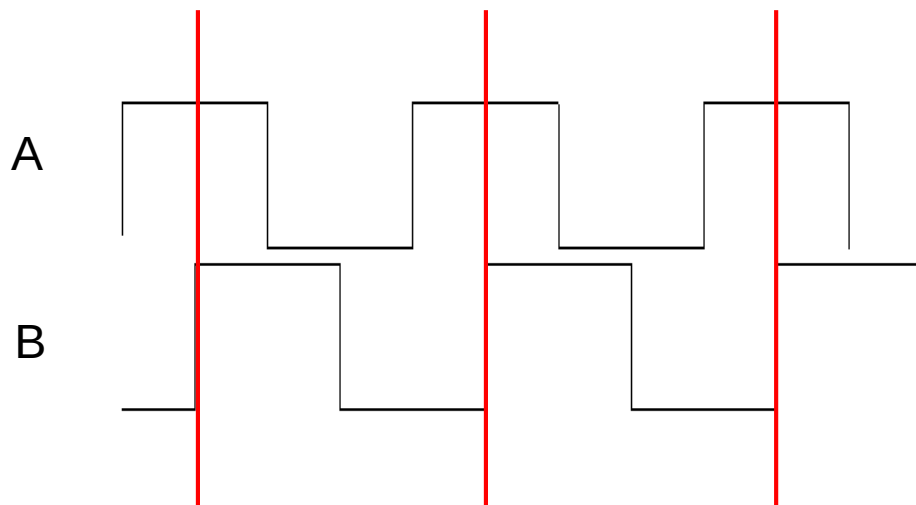
<http://users.utcluj.ro/~negrum/>



- **Motor/cutie de viteze Digilent MT-Motor**
 - Motor clasic DC, viteza e dată de tensiune, direcția de polaritate
 - Rotație continuă, cât timp motorul este sub tensiune
 - Cutie de viteze (angrenaj de roți dințate) cu raport 1:19, 1:53, 1:48, etc.
 - Majorare a forței (cuplu) în dauna vitezei de rotație



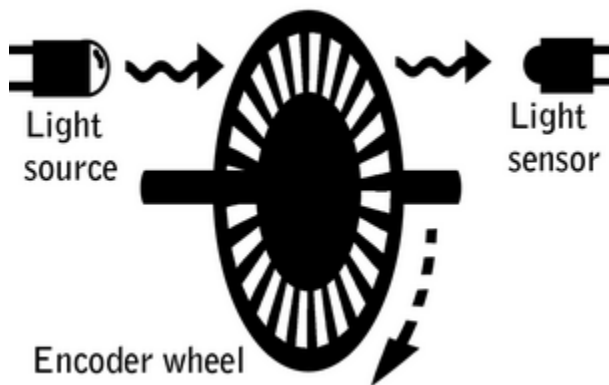
- **Măsurarea turației motorului**
 - Senzor Hall (magnetic) în cuadratură



- Orientare: se monitorizează fronturile crescătoare sau descrescătoare ale unui semnal
- Starea celuilalt semnal în momentul tranziției dă orientarea



- **Măsurarea turației motorului**
 - Roată cu perforații + senzor de lumină IR

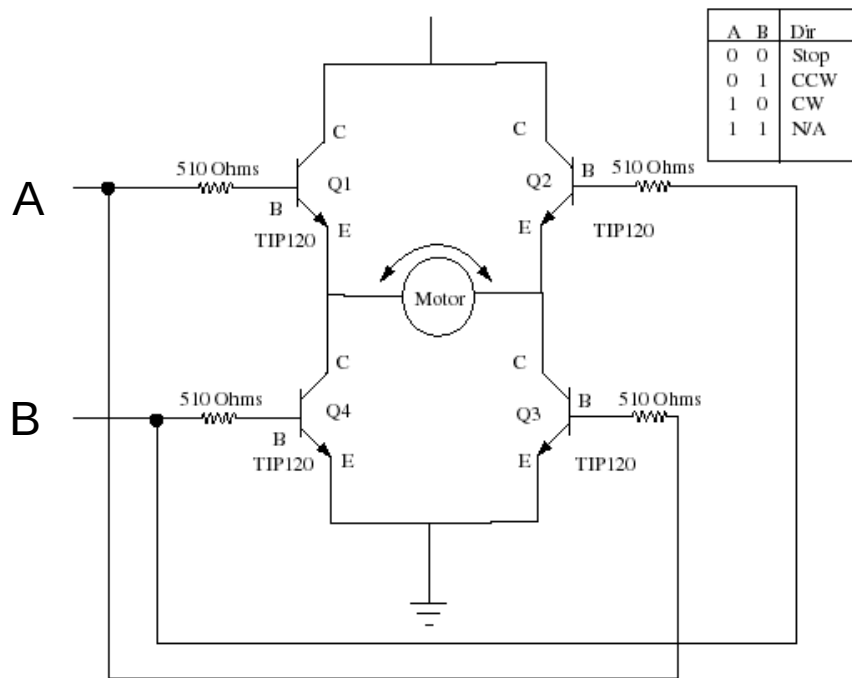
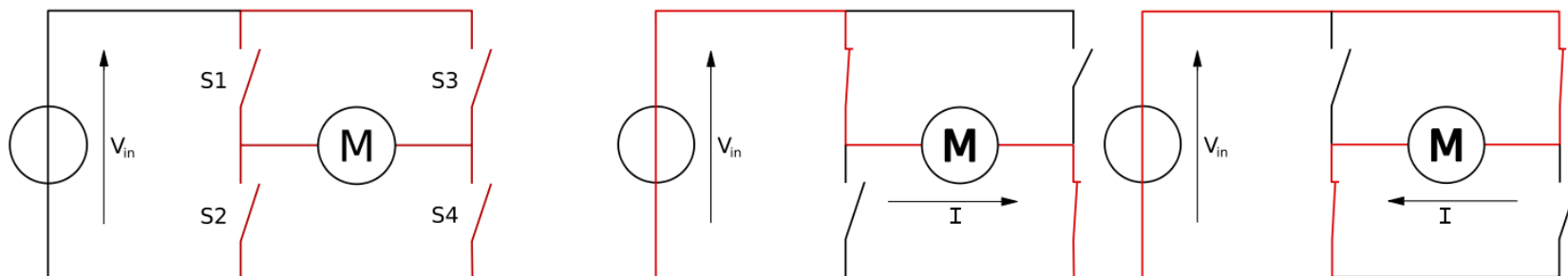


- Trecerea sau blocarea razei IR produce un tren de pulsuri pentru măsurarea turației.
- Cum putem măsura și orientarea ?



- **Puntea H**

- Controlul pornirii-opririi și al direcției unui motor





Controlul motoarelor de curent continuu (DC)



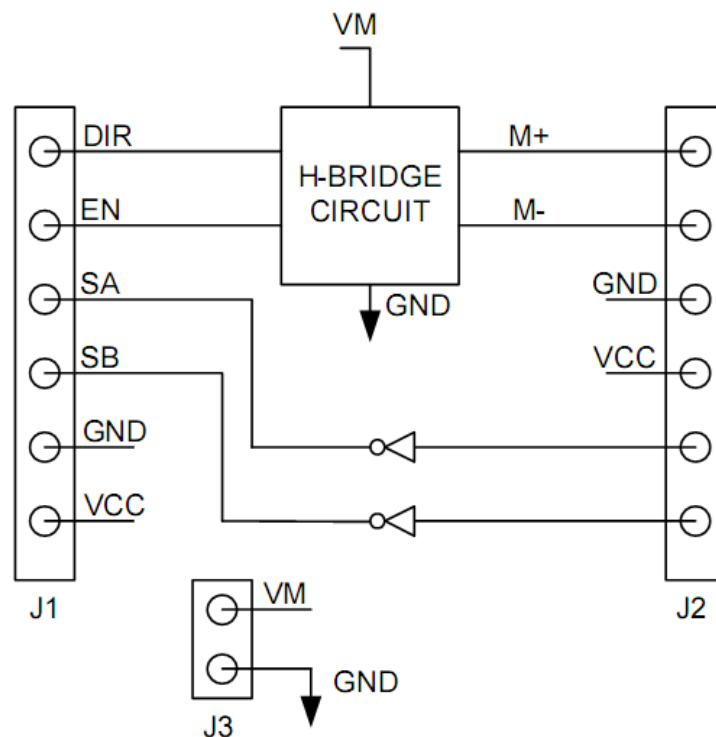
- **Puntea H**

- Digilent PMOD HB5
- DIR – control directie
- EN – dacă e '1', motorul funcționează – se poate atașa PWM pentru viteza variabilă
- **A = EN and DIR, B = EN and (not DIR) – Previne scurtcircuitul.**
- SA, SB – semnale de la motor, pentru a monitoriza starea acestuia

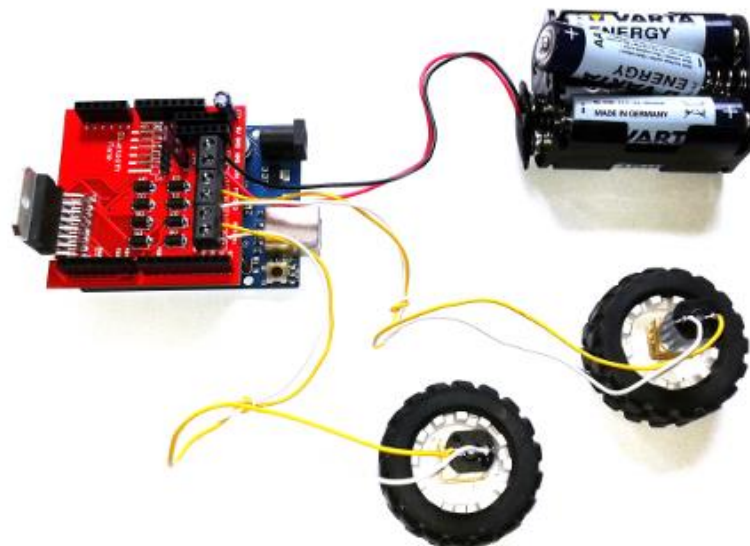
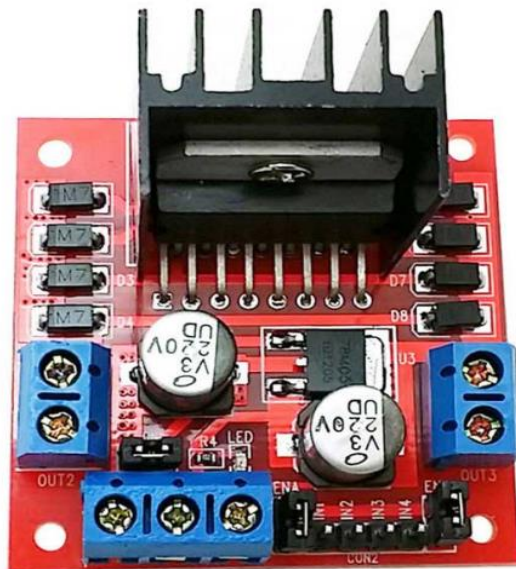


- | | |
|--------------------------|----------------|
| ☐ | Direction |
| ☐ | Enable |
| ☐ | Sensor A |
| ☐ | Sensor B |
| ☐ | GND |
| ☐ | Vcc (3.3 - 5v) |

HB5 6-Pin Header, J1



- **Driver-ul de motoare L298 (Shield) pentru Arduino**
 - <http://www.robofun.ro/shields/shield-motoare-l298-v2>
 - Driver-ul se conectează la platforma Arduino folosind 4 pini digitali (3, 5, 6 și 9) conectați la pinii In1, In2, In3 și In4.
 - Tensiune de alimentare motoare: 5... 35 V
 - Tensiune circuit logic: 5 V (poate genera această tensiune pentru alimentare Arduino)
 - Poate controla motoare care necesită cel mult 2 Amperi (2000 mA).
 - 2 Motoare DC, sau un motor pas cu pas (Stepper)





- Exemplu: Rotație a două motoare, în ambele sensuri**

```
int MOTOR2_PIN1 = 3; // fiecare motor are 2 pini. Diferenta de polaritate dintre ei
int MOTOR2_PIN2 = 5; // cauzeaza motorul sa se deplaseze, intr-un sens sau in celalalt
int MOTOR1_PIN1 = 6;
int MOTOR1_PIN2 = 9;

// functie control, viteza pentru M1 si pentru M2
void go(int speedLeft, int speedRight) {

    if (speedLeft > 0) { // viteza pozitiva, pe pin 1
        analogWrite(MOTOR1_PIN1, speedLeft);
        analogWrite(MOTOR1_PIN2, 0);
    }
    else
    {
        analogWrite(MOTOR1_PIN1, 0);
        analogWrite(MOTOR1_PIN2, -speedLeft); // viteza negativa,
                                                // val absoluta pe pin2
    }

    if (speedRight > 0) {
        analogWrite(MOTOR2_PIN1, speedRight);
        analogWrite(MOTOR2_PIN2, 0);
    }
    else
    {
        analogWrite(MOTOR2_PIN1, 0);
        analogWrite(MOTOR2_PIN2, -speedRight);
    }
}

void setup() {
    // pinii motor, configurati ca iesire
    pinMode(MOTOR1_PIN1, OUTPUT);
    pinMode(MOTOR1_PIN2, OUTPUT);
    pinMode(MOTOR2_PIN1, OUTPUT);
    pinMode(MOTOR2_PIN2, OUTPUT);
}

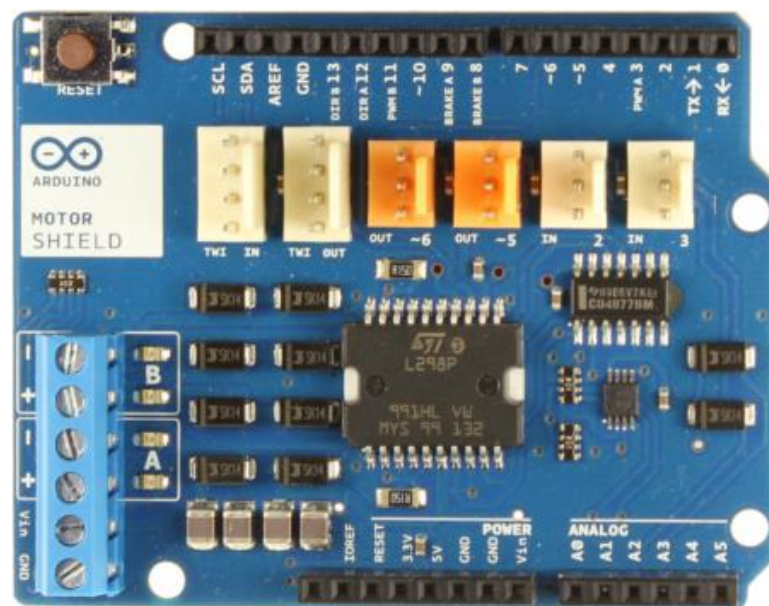
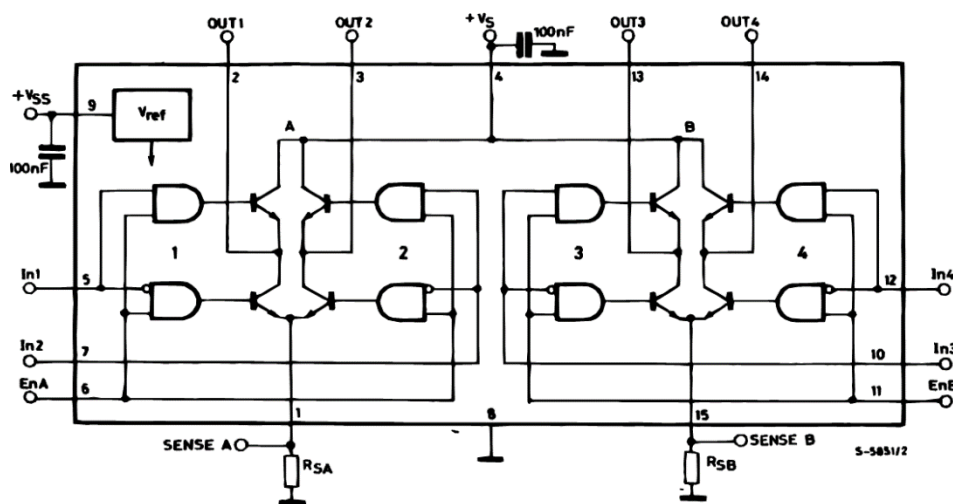
void loop() {
    // 2 motoare, 2 directii, 4 comutatii de cate 1 secunda
    go(255,-255);
    delay(1000);
    go(-255,-255);
    delay(1000);
    go(-255,255);
    delay(1000);
    go(255,255);
    delay(1000);
}
```



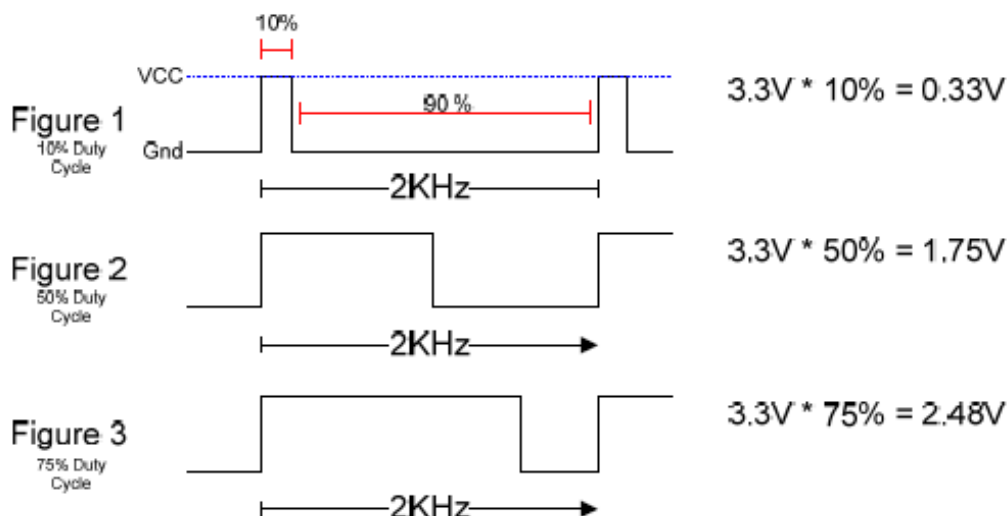
Arduino Motor Shield



- <https://www.arduino.cc/en/Main/ArduinoMotorShieldR3>
- Bazat pe driver-ul L298 $\Rightarrow$ pentru sarcini inductive: relee, bobine, motoare DC, motoare pas cu pas (max. 2A / canal) [6]
- Poate comanda 2 motoare DC (control viteză și direcție pentru fiecare, independent), sau 1 motor pas cu pas
- Funcții: mers continuu / stop / frână; măsură curent absorbit.



- **Controlul vitezei folosind PWM**
- Într-un circuit analogic, viteza motorului este controlată de nivelul tensiunii. Într-un circuit digital, avem doar două soluții:
 - Folosirea unui circuit de rezistență variabilă pentru a controla tensiunea aplicată motorului (soluție complicată, care irosește energie sub formă de căldură)
 - Aplicarea intermitentă a tensiunii sub formă PWM.



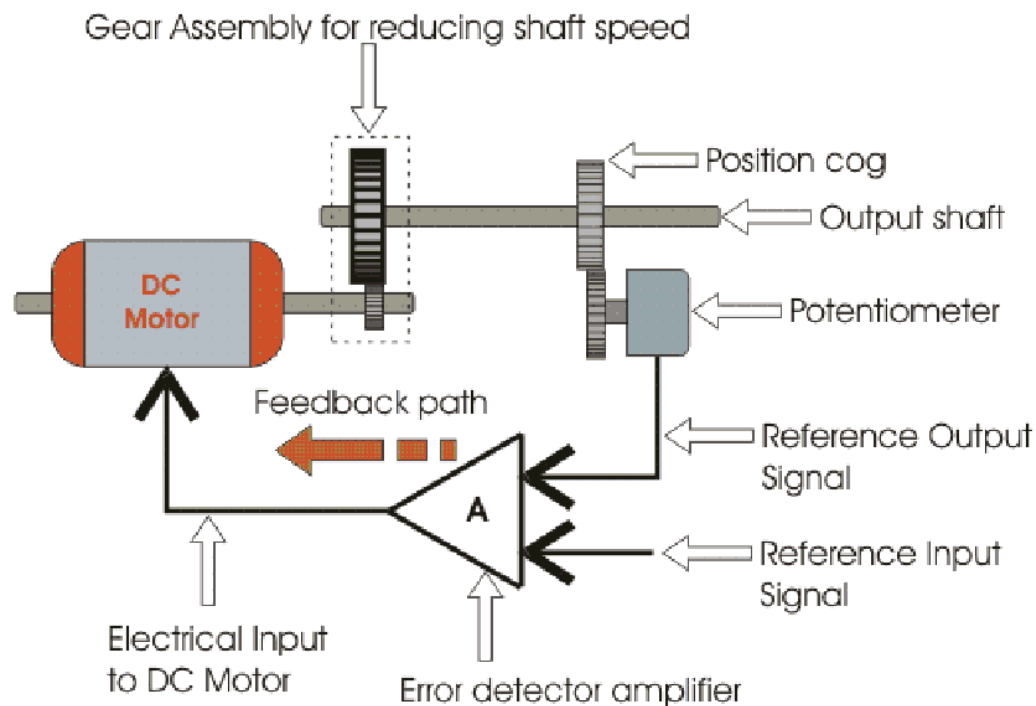
- Când tensiunea este aplicată, motorul este acționat de forța electromagnetică.
- Când tensiunea e oprită, inerția cauzează motorul să continue rotație pentru scurt timp.
- Dacă frecvența pulsurilor este suficient de mare, acest proces de pornire+mers din inerție permite motorului o rotație uniformă, controlabilă prin logica digitală.



- **Efectele folosirii PWM**
- PWM are două efecte importante asupra motoarelor de curent continuu:
 - Rezistența inerțială la pornire este învinsă mai ușor, deoarece pulsurile scurte de tensiune maximă au un cuplu de forță mai mare decât tensiunea echivalentă intermediară.
 - Se generează o cantitate mai mare de căldură în interiorul motorului.
- Dacă un motor controlat PWM este folosit pentru o perioadă mai mare de timp, sunt necesare sisteme de disipare a căldurii, pentru a evita distrugerea motorului. Din acest motiv PWM este recomandat în sisteme de cuplu mare și utilizare intermitentă, precum acționarea suprafețelor de control la avioane, sau acționarea brațelor robotice.
- Circuitele PWM pot crea interferență radio. Aceasta poate fi minimizată prin scurtarea căilor dintre motor și controllerul PWM (folosirea unor cabluri scurte).
- Zgomotul electronic creat prin acționarea intermitentă a motorului poate să interfereze cu celelalte componente din circuit, și de aceea este recomandat să fie filtrat. Plasarea de condensatoare ceramice la terminalele motorului, și între terminalele motorului și stator poate fi o soluție pentru a filtra aceste interferențe.

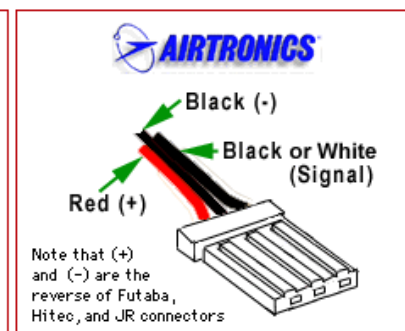
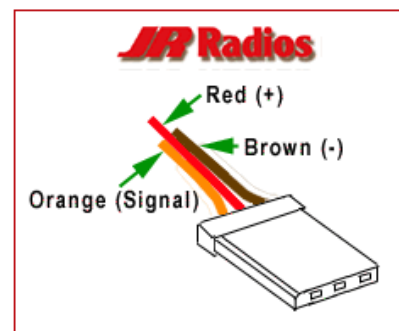
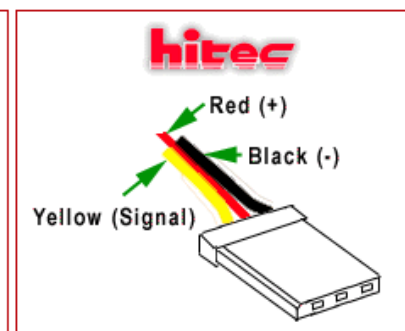
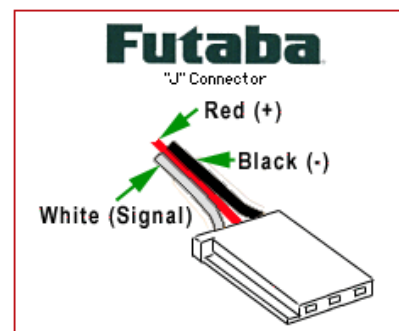
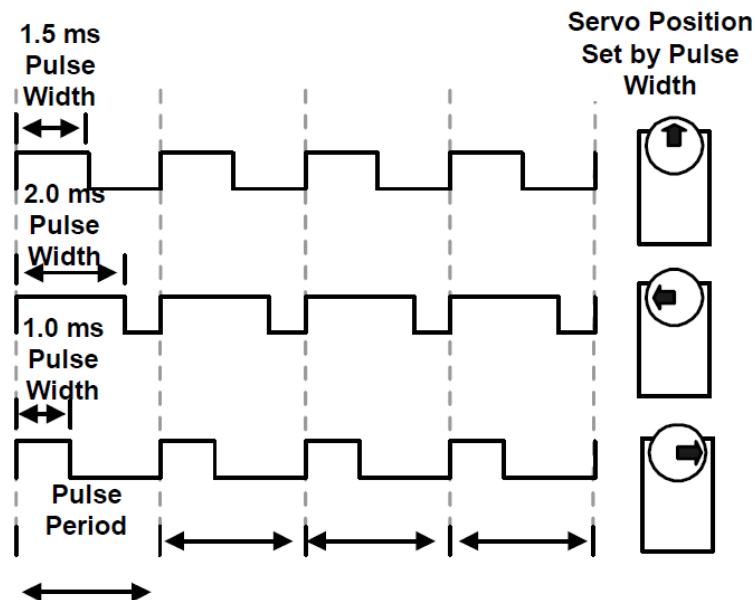
- **Motorul servo**

- Folosește un mecanism de feedback (reacție negativă) pentru a menține o poziție dată printr-un semnal de control (analog sau digital)
- Conține un motor DC, un angrenaj de roți dințate și un circuit de control



- **Motorul servo (ex: GWS Servo Kit)**

- Lăţimea pulsului controlează amplitudinea rotaţiei
- 1.5 ms – poziţia neutră
- 1 ms – poziţie maxim stânga (dreapta)
- 2 ms – poziţie maxim dreapta (stânga)
- Codificare PWM, frecvenţa purtătoare între 30 Hz şi 60 Hz





- Biblioteca **Servo**:
 - Poate controla până la **12 motoare pe majoritatea plăcilor Arduino**
 - **48 motoare** pe Arduino Mega.
- Folosirea bibliotecii va dezactiva analogWrite() (PWM) pe pinii 9 și 10, indiferent dacă există sau nu motor servo conectat la acești pini (**exceptând placa Arduino Mega**).
- La Arduino Mega, se pot utiliza până la 12 motoare servo fără a afecta funcționarea PWM; folosirea mai multor motoare va dezactiva PWM pe pinii 11 și 12.
- **Conectarea Servo la Arduino (3 fire):** Vcc, Gnd, semnal.
 - **Vcc** → la pinul de 5V al placii.
 - Gnd (negru sau **maro**) → la GND de pe Arduino.
 - Pinul de semnal (**galben**, **portocaliu** sau **alb**) conectat la un pin digital.
- **Notă:** motoarele necesită putere considerabilă! Pentru a acționa mai mult de 2 motoare servo, folosiți o sursă de alimentare externă, nu de la +5V de pe Arduino.



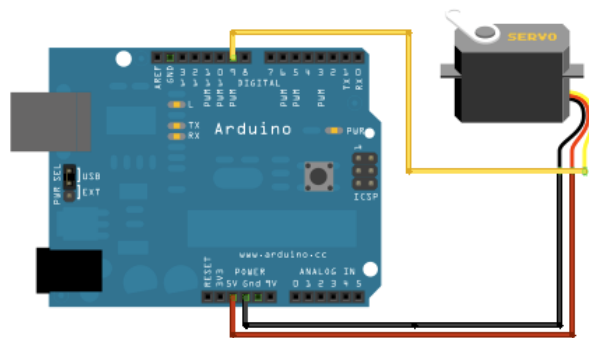
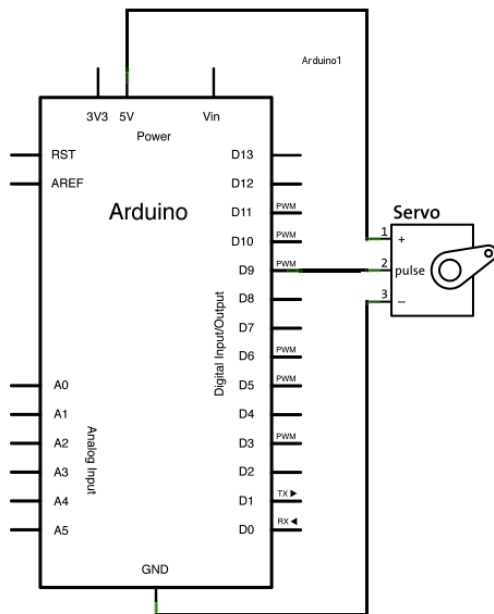
- **Metode ale bibliotecii Servo:**
- **`servo.attach(pin)` / `servo.attach(pin, min, max)`** – atașează obiectul Servo la pin
 - `servo`: un obiect instanță a clasei Servo
 - `pin`: numărul pinului digital unde va fi atașat semnalul pentru motorul Servo
 - `min` (opțional): lățimea pulsului, în microsecunde, corespunzătoare unghiului minim (0 grade) al motorului servo (implicit 544)
 - `max` (optional): lățimea pulsului, în microsecunde, corespunzătoare unghiului maxim (180 grade) al motorului servo (implicit 2400)
- **`servo.detach()`** – detașează obiectul de tip Servo de la pin.
- **`boolean val servo.attached()`** – verifică dacă obiectul de tip Servo este atașat unui pin. Returnează adevărat sau fals.
- **`servo.write (angle)`** – scrie o valoare (0 ... 180) către servo, controlând mișcarea:
 - *Servo standard* ⇒ setează unghiul axului [grade] cauzând motorul să se orienteze în direcția specificată.
 - *Servo cu rotație continuă* ⇒ configurează viteza de rotație (0: viteza maximă într-o direcție; 180: viteza maxima in directia opusa; ≈ 90 : oprit)
- **`int val servo.read()`** – citește unghiul curent al servo, configurat la ultimul apel al `write()`.



Controlul Motoarelor Servo – Arduino



- Exemplu:** Orientează axul unui servo parcurgând înainte și înapoi 180 grade (<http://arduino.cc/en/Tutorial/Sweep>)



```
#include <Servo.h>
```

```
Servo myservo;           // obiect pentru controlul servo  
int pos = 0;             // variabila ce tine pozitia curenta a axului
```

```
void setup() {  
  myservo.attach(9);      // ataseaza obiectul servo la pinul 9  
}
```

```
void loop() {  
  for(pos = 0; pos < 180; pos += 1) // de la 0 la 180 grade  
  {  
    myservo.write(pos); // configureaza pozitia dorita  
    delay(15);           // asteapta 15 ms pentru ca motorul sa se  
                          // pozitioneze  
  }  
  for(pos = 180; pos >= 1; pos -= 1) // baleiere inapoi  
  {  
    myservo.write(pos);  
    delay(15);  
  }  
}
```



Controlul Motoarelor Servo – Arduino



- **Exemplu:** Controlul poziției unui servo motor cu Arduino și un potentiometru (<http://arduino.cc/en/Tutorial/Knob>)

```
#include <Servo.h>
```

```
Servo myservo; // obiect pentru controlul motorului servo
```

```
int potpin = 0; // pin analogic pentru citirea potentiometrului
```

```
int val; // variabila in care se va citi starea pinului analogic
```

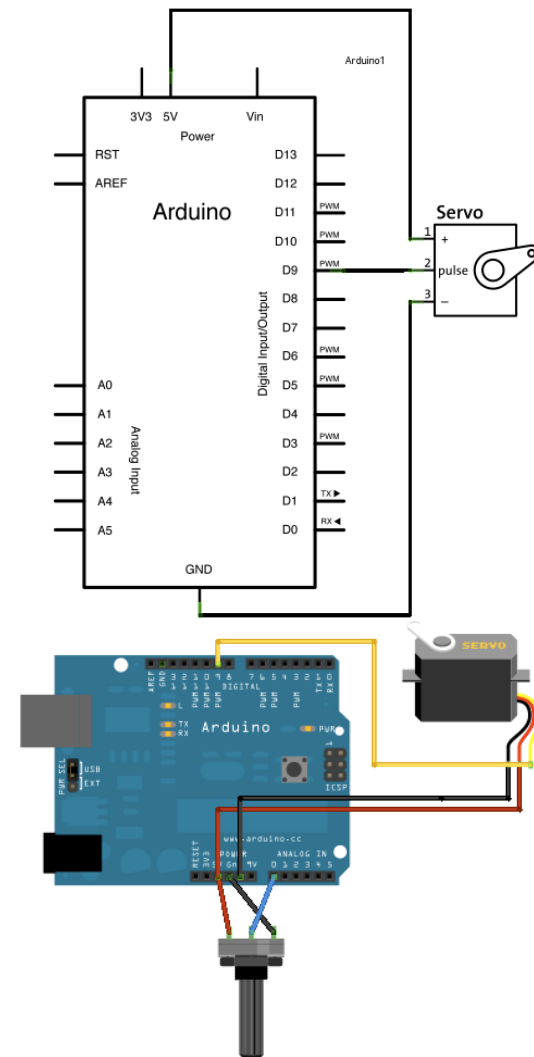
```
int angle; // unghiul servo
```

```
void setup()
```

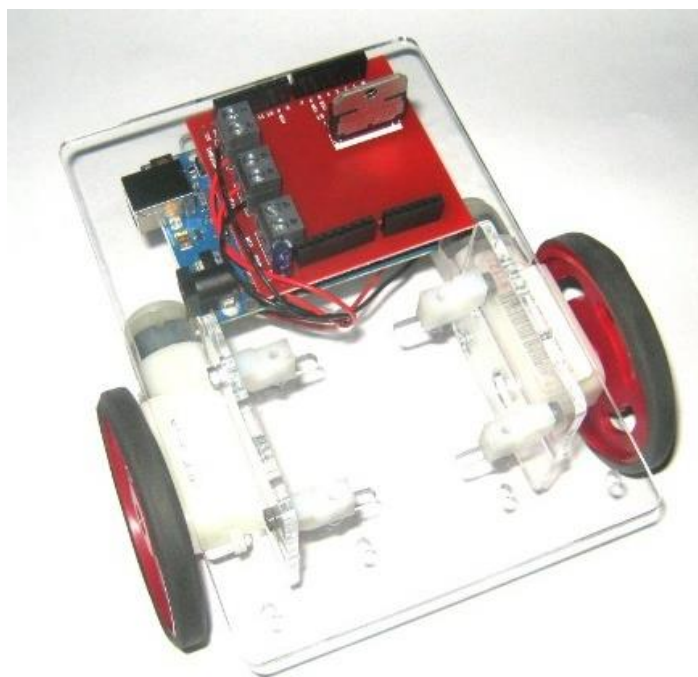
```
{  
  myservo.attach(9); // ataseaza obiectul servo la pinul 9  
}
```

```
void loop()
```

```
{  
  val = analogRead(potpin); // citeste stare potentiometru  
  angle = map(val, 0, 1023, 0, 179); // scalarea valorii citite (0...1023)  
                                     // in domeniul 0... 179  
  myservo.write(angle); // scrie noua pozitie pentru servo  
  delay(15); // asteapta pozitionarea motorului  
}
```

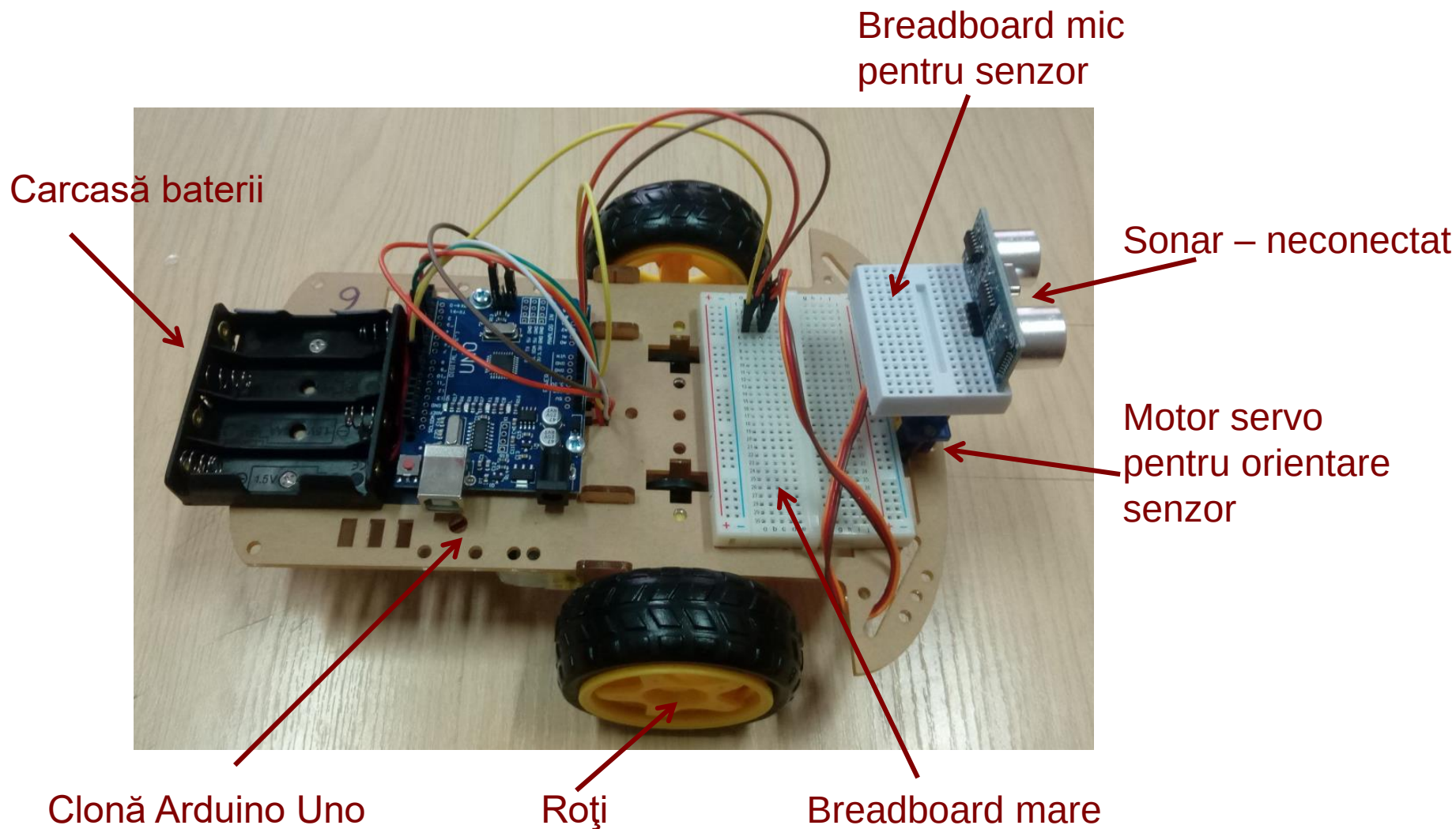


- <http://www.robofun.ro/kit-roboti/magician-robot-arduino-driver>
- <http://www.robofun.ro/kit-roboti/kit-robot-senile-arduino-driver-sharp>





Robotul Experimental



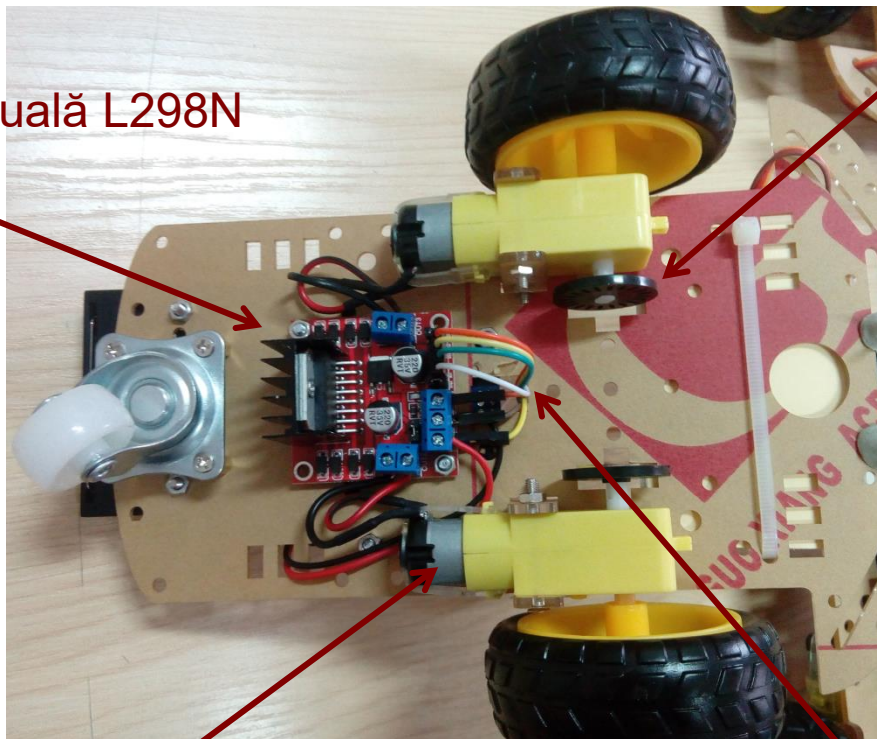


Robotul Experimental



Punte H duală L298N

Roată perforată pentru turație
Fără senzor IR !



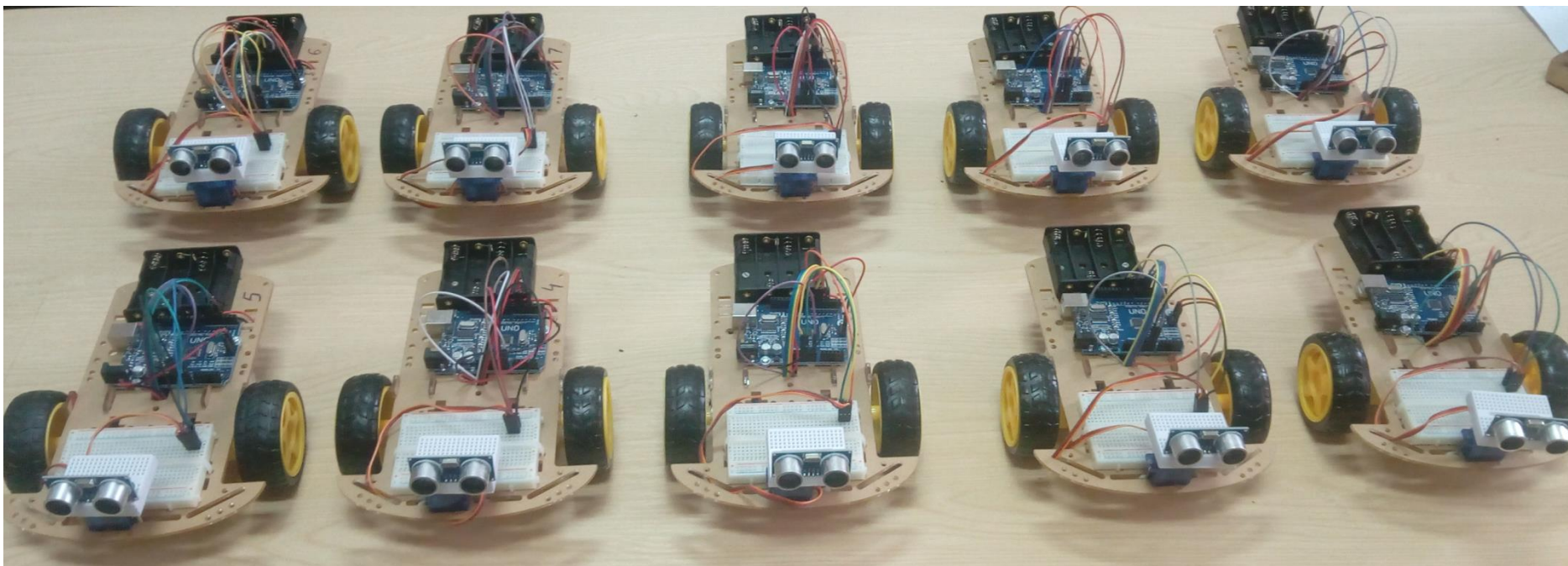
Motor DC

Roți

Fire control (In1, ... In4)



Robotul Experimental

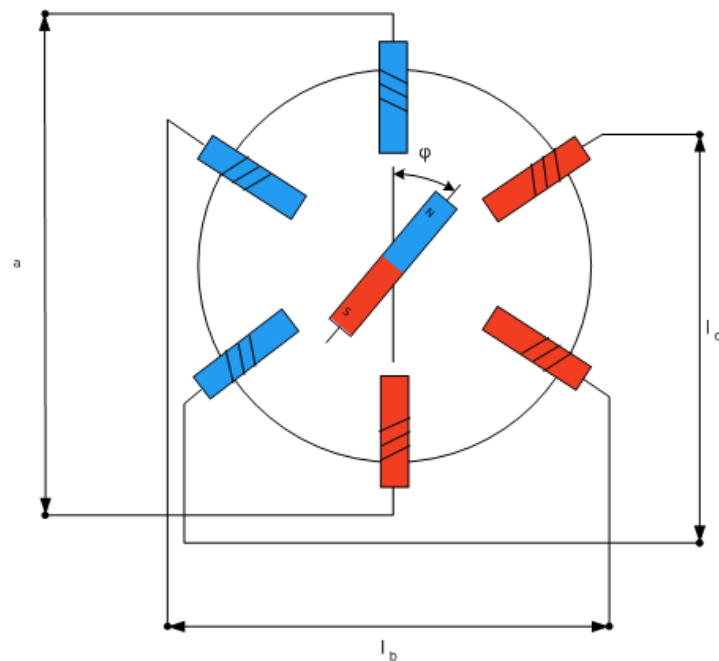
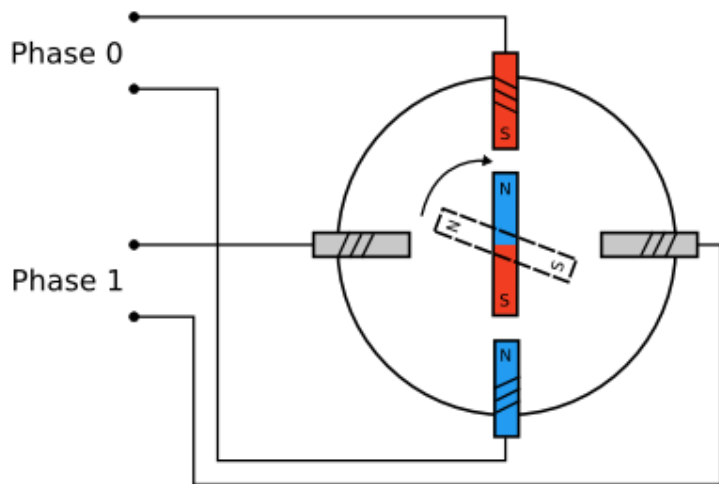




Motor Pas cu Pas



- Rotația se face pas cu pas, prin activarea selectivă a bobinelor de pe stator
- Curenții din bobine se schimbă prin control electronic, spre deosebire de motoarele clasice, la care schimbarea se face prin control mecanic, cu perii

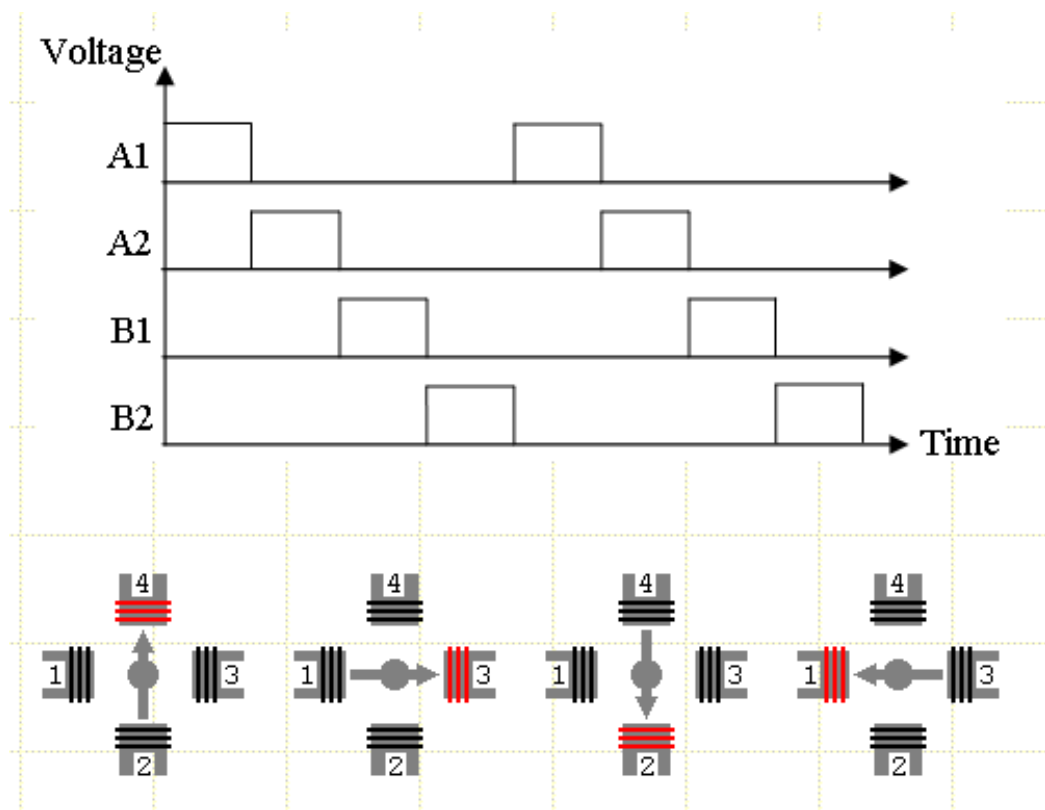




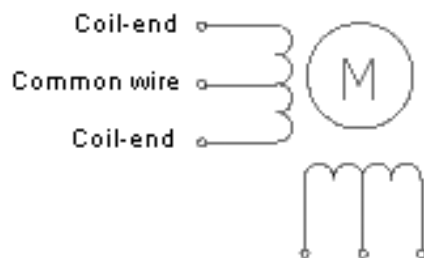
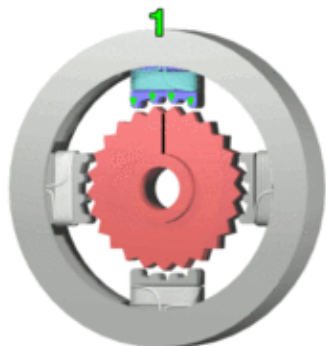
Motor Pas cu Pas



- Un controller de motor pas cu pas trebuie să genereze secvența corectă pentru activarea bobinelor
- Se pot efectua rotații complete, sau părți de rotație, în funcție de numărul de pulsuri – control precis al cantității de rotație!



• Motor pas cu pas unipolar



• Comandă tip undă, sau Pas întreg cu o singură fază

- Cuplu motor mic, se folosește rar. 25 dinți / 4 pași pentru a roti o poziție a unui dinte $\Rightarrow 25 \cdot 4 = 100$ pași pentru o rotație completă $\Rightarrow$ fiecare pas va avea $360/100 = 3.6^\circ$

• Comandă cu pas întreg cu două faze

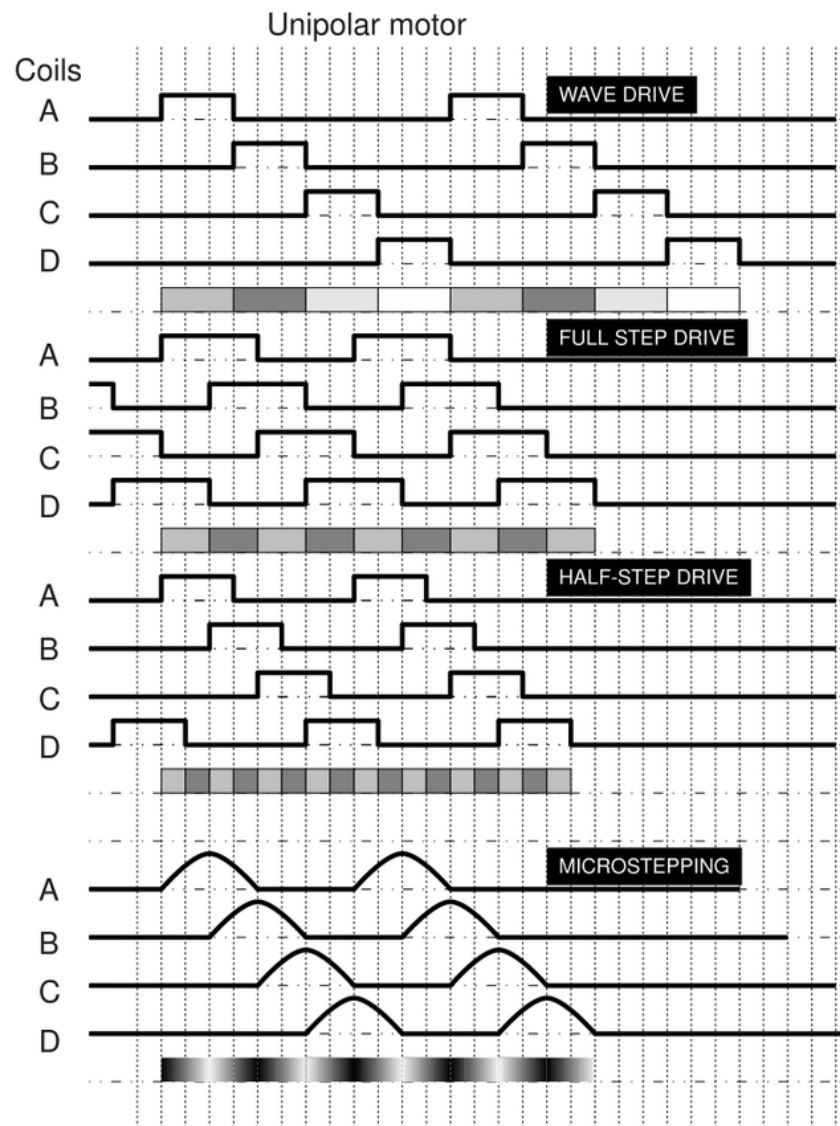
- Cuplu motor maxim, comanda cea mai folosită

• Comandă cu jumătate de pas

- Cuplu mai mic (70%) / rezoluție x2 (8 pași pentru a deplasa un dinte $\Rightarrow 25 \cdot 8 = 200$ pași pentru rotație întreagă $\Rightarrow$ un pas = 1.8°)

• Micro-pas

- Operare mai fină



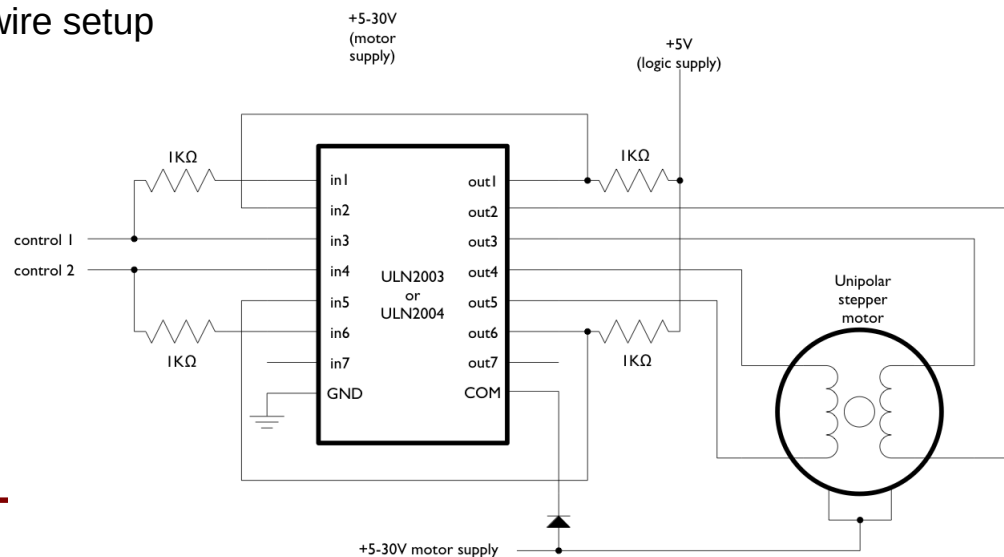


Motor Pas cu Pas – Arduino



- **Biblioteca Arduino Stepper** (<http://arduino.cc/en/reference/stepper>)
 - Permite controlul motoarelor pas cu pas unipolare sau bipolare. Pentru a putea folosi această bibliotecă, e nevoie de un motor pas cu pas și de o interfață hardware pentru acesta.
 - Pentru a crea un obiect de clasa Stepper, se apelează constructorul:
 - **Stepper(steps, pin1, pin2)** - ex: Stepper myStepper = Stepper(100, 5, 6);
 - **Stepper(steps, pin1, pin2, pin3, pin4)**
 - int steps: numărul de pași per rotație completă (ex: $360 / 3.6 = 100$ pași)
 - int pin1, pin2: doi pini atașați interfeței hardware (montaj cu 2 pini)
 - int pin3, pin4: *opțional* pini atașați interfeței hardware, pentru montaj cu 4 pini

2 pin/wire setup



Secvență de control (2 pini):

Pas	pin 1	pin 2
1	low	high
2	high	high
3	high	low
4	low	low



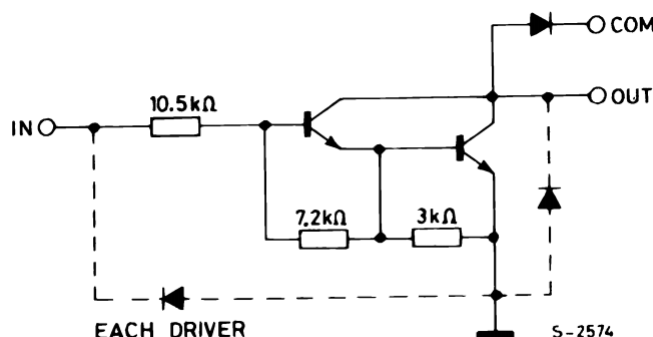
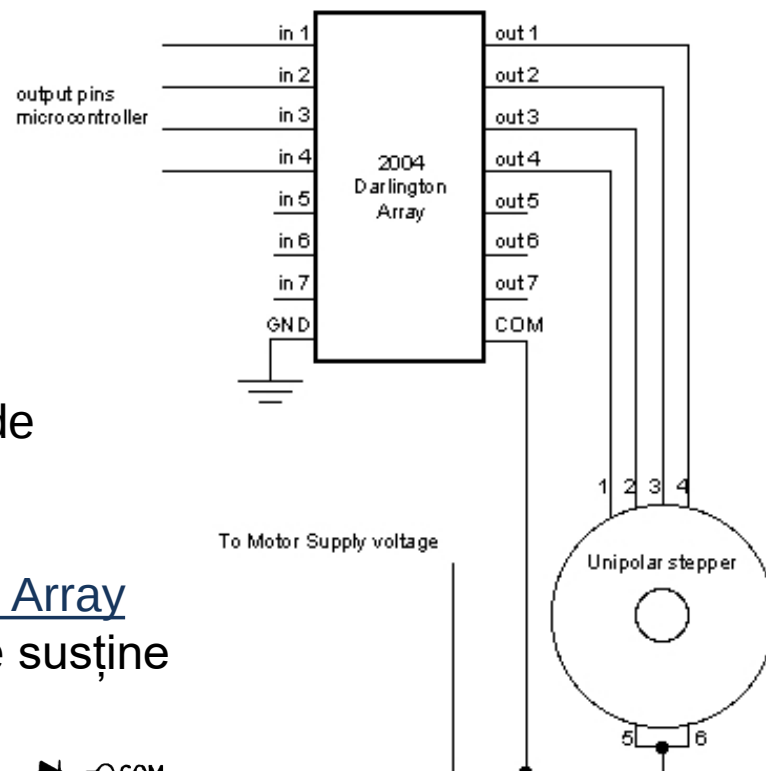
- **Secvență de control (4 pini):**

Pas	pin 1	pin 2	pin 3	pin 4
1	High	low	high	low
2	low	high	high	low
3	low	high	low	high
4	high	low	low	high

Dacă se folosește biblioteca Stepper, semnalele de control sunt generate de către bibliotecă!

Exemplu de interfata hardware: U2004 Darlington Array

- Tensiune și amperaj mare. Fiecare canal poate susține 500 mA, cu vârfuri acceptate de 600 mA.





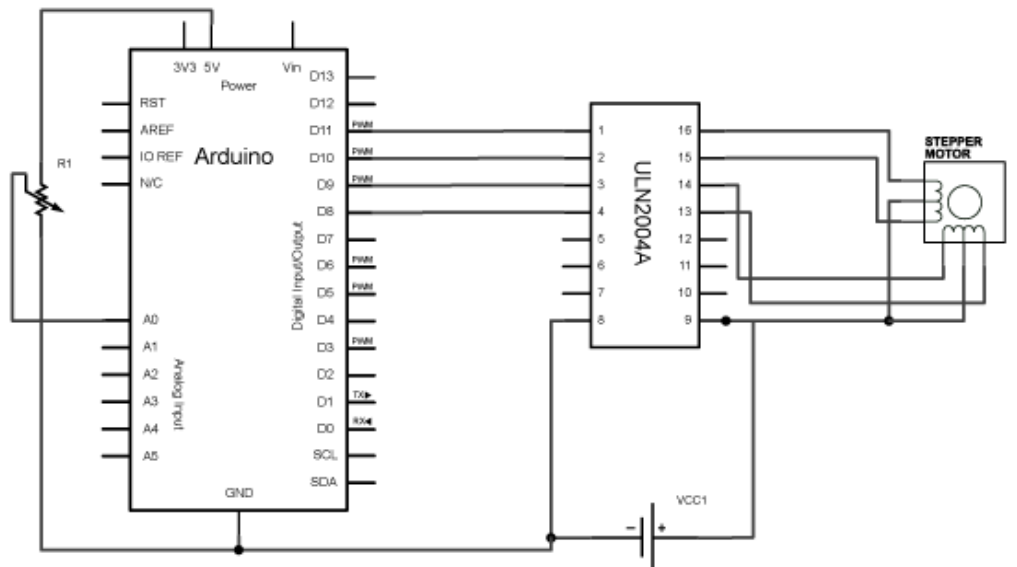
- **Funcții ale bibliotecii Stepper** (<http://arduino.cc/en/reference/stepper>)
- **setSpeed(long rpms)** – configurează viteza de rotație a motorului, în rotații pe minut (RPMs). Această funcție nu pornește motorul, ci doar configurează viteza cu care se va roti când se apelează funcția step().
- **step(int steps)** – Rotește motorul un număr specificat de pași, cu viteza configurată.
 - int steps: numărul de pași pe care motorul îi va executa – pozitiv (+) rotație într-o direcție, negativ (-) în direcția opusă
 - **Funcția este blocantă:** va aștepta până când motorul va termina rotația, pentru a ieși. (Ex: la viteza = 1 RPM, apelată cu parametrul steps = 100 pentru un motor cu rotație completă în 100 de pași, funcția va bloca programul timp de 1 minut).
 - Pentru un control mai bun, apăsați doar un număr mic de pași odată cu o viteză mare.



Motor Pas cu Pas – Arduino



- Exemplu: Motor pas cu pas controlat cu potențiomtru Knob (<http://arduino.cc/en/Tutorial/MotorKnob>)



```
#include <Stepper.h>
#define STEPS 100
```

```
Stepper stepper(STEPS, 8, 9, 10, 11);
```

```
// citirea anterioara de la potentiometru
int previous = 0;
```

```
void setup()
{
  // viteza motor, 30 RPM
  stepper.setSpeed(30);
}
```

```
void loop()
{
  // citire stare potentiometru
  int val = analogRead(0);
  // deplasare motor cu diferenta dintre citiri
  stepper.step(val - previous);
  // valoarea curenta devine valoare anterioara
  previous = val;
}
```

- Motorul pas cu pas se poate controla și prin puntea H duală
- Sursa: <https://coeleveld.com/arduino-stepper-l298n/>

```
#include <Stepper.h>
```

```
const int stepsPerRevolution = 200; // modificati pentru
// specificatiile motorului propriu
```

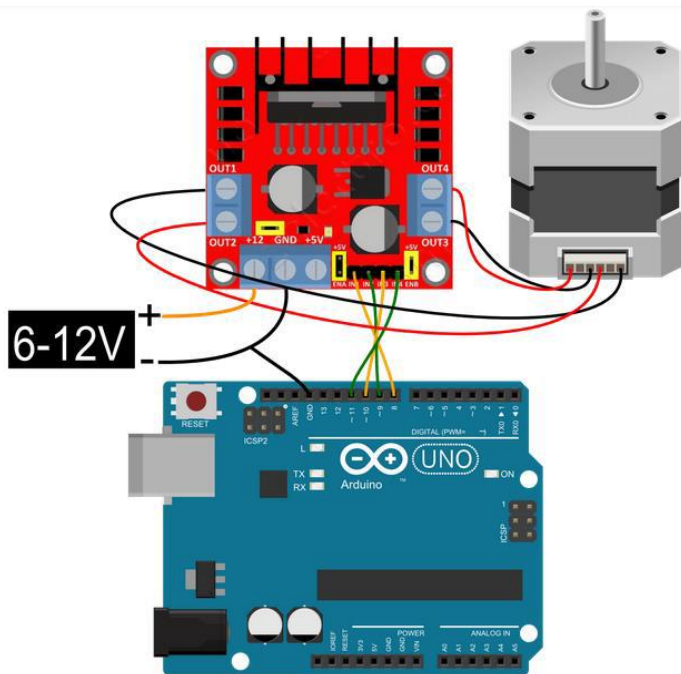
```
// se initializeaza biblioteca stepper pe pinii 8 ...11:
```

```
Stepper myStepper(stepsPerRevolution, 8, 9, 10, 11);
```

```
void setup() {
    // regleaza viteza de rotatie la 60 rpm:
    myStepper.setSpeed(60);
    // initializeaza interfata serial
    Serial.begin(9600);
```

```
void loop() {
    // o rotatie completa in directia orara
    Serial.println("clockwise");
    myStepper.step(stepsPerRevolution);
    delay(500);

    // o rotatie completa in directia antiorara
    Serial.println("counterclockwise");
    myStepper.step(-stepsPerRevolution);
    delay(500);
}
```





1. Atmel ATmega640/V-1280/V-1281/V-2560/V-2561/V data-sheet
2. Atmel Atmega64 datasheet
3. <http://arduino.cc/en/>
4. <http://www.robofun.ro/shields/shield-motoare-l298-v2>
5. <https://www.arduino.cc/en/Main/ArduinoMotorShieldR3>
6. http://www.st.com/web/en/catalog/sense_power/FM142/CL851/SC1790/SS1555/PF63147
7. <http://arduino.cc/en/Tutorial/Sweep>
8. <http://arduino.cc/en/Tutorial/Knob>
9. <http://arduino.cc/en/Tutorial/MotorKnob>



Proiectarea cu Micro-Procesoare

Lector: Mihai Negru

An 3 – Calculatoare și Tehnologia Informației
Seria B

Curs 9: Procesarea Semnalelor Analogice

<http://users.utcluj.ro/~negrum/>



- **Comparator analogic:** compară un semnal de intrare cu alt semnal de intrare, sau cu tensiuni de referință, indicând printr-un bit relația dintre aceste tensiuni (care este mai mare), sau producând cereri de întrerupere
- **Convertor analog/digital pe 10 biți,** cu 16 intrări analogice multiplexate: convertește semnalul analogic într-o valoare numerică între 0 și 1023.



- **Conversia semnalului analogic în semnal digital**

- Transformarea unui semnal analogic (tensiune) de intrare într-o valoare digitală pe **10 biți**
- Intrarea poate fi “single end” – tensiunea de intrare se calculează între pinul de intrare și GND, sau diferențială – diferența de tensiune între doi pini de intrare
- Pentru intrarea diferențială, se poate utiliza amplificare (Gain)

$$ADC = \frac{V_{IN} * 1024}{V_{REF}}$$

ADC: 0...1023

$$ADC = \frac{(V_{POS} - V_{NEG}) \cdot Gain \cdot 512}{V_{REF}}$$

ADC: -512...511, complement față de 2

- V_{REF} poate fi:
 - AVCC – conectat în placă la VCC
 - A_{REF} – tensiune externă de referință
 - Tensiune de referință internă **2.56 V**



Convertor Analog/Digital



ATmega 328P

Rezoluție – 10 biți

Single ended – 8 canale multiplexate

Intrare senzor temperatura

Optional – ajustare stanga / dreapta
pentru citirea rezultatului ADC

Interval tensiune intrare: 0 ... VCC

Tensiune de referinta: 1.1 V interna

Moduri de functionare:

- Free running sau Single Conversion
- Interrupt on ADC Conv. Complete
- Sleep mode noise canceller

ATmega 2560

Rezoluție – 10 biți

Single ended – 16 canale multiplexate

Diferențială – 14 canale

Diferențială – 4 canale cu amplificare
(optionala 10x sau 200x)

Optional – ajustare stanga / dreapta
pentru citirea rezultatului ADC

Interval tensiune intrare: 0 ... VCC

Interval tensiune intrare: 2.7 V – VCC

Tensiune de referinta: Selectabil 2.56 V
sau 1.1 V (interne)

Moduri de functionare:

- Free running sau Single Conversion
- Interrupt on ADC Conv. Complete
- Sleep mode noise canceller

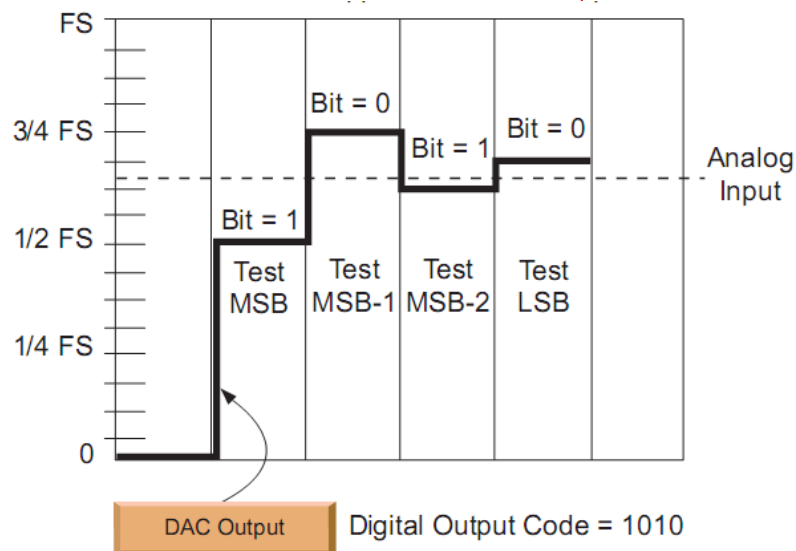
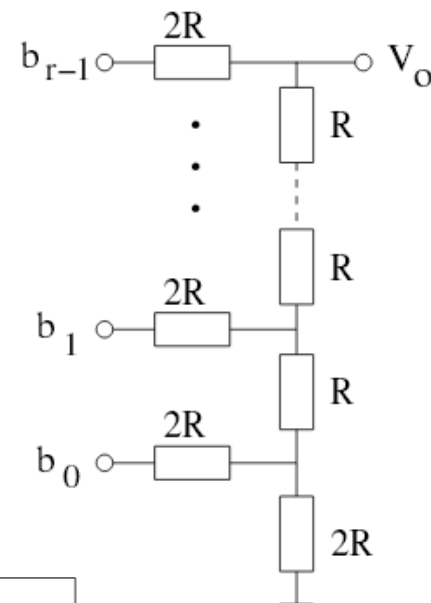
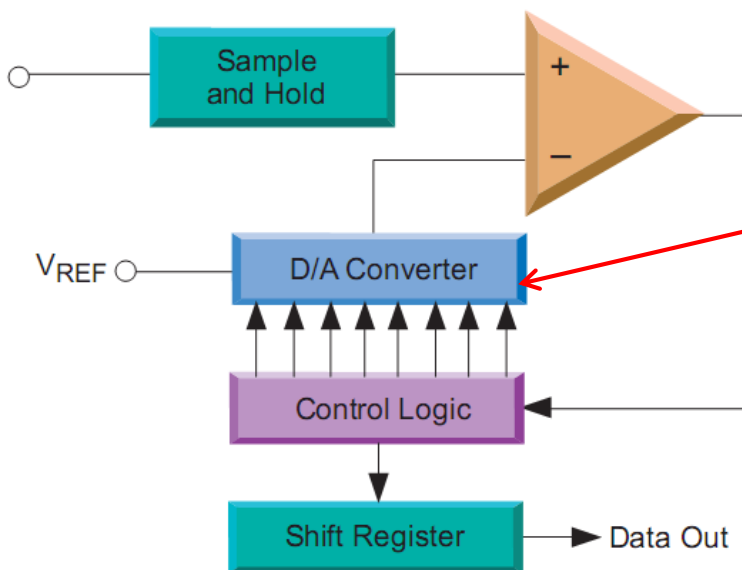


Convertor Analog/Digital



- **Principiul de funcționare**

- Comparații succesive cu o tensiune de referință
- Principiul este similar cu căutarea binară





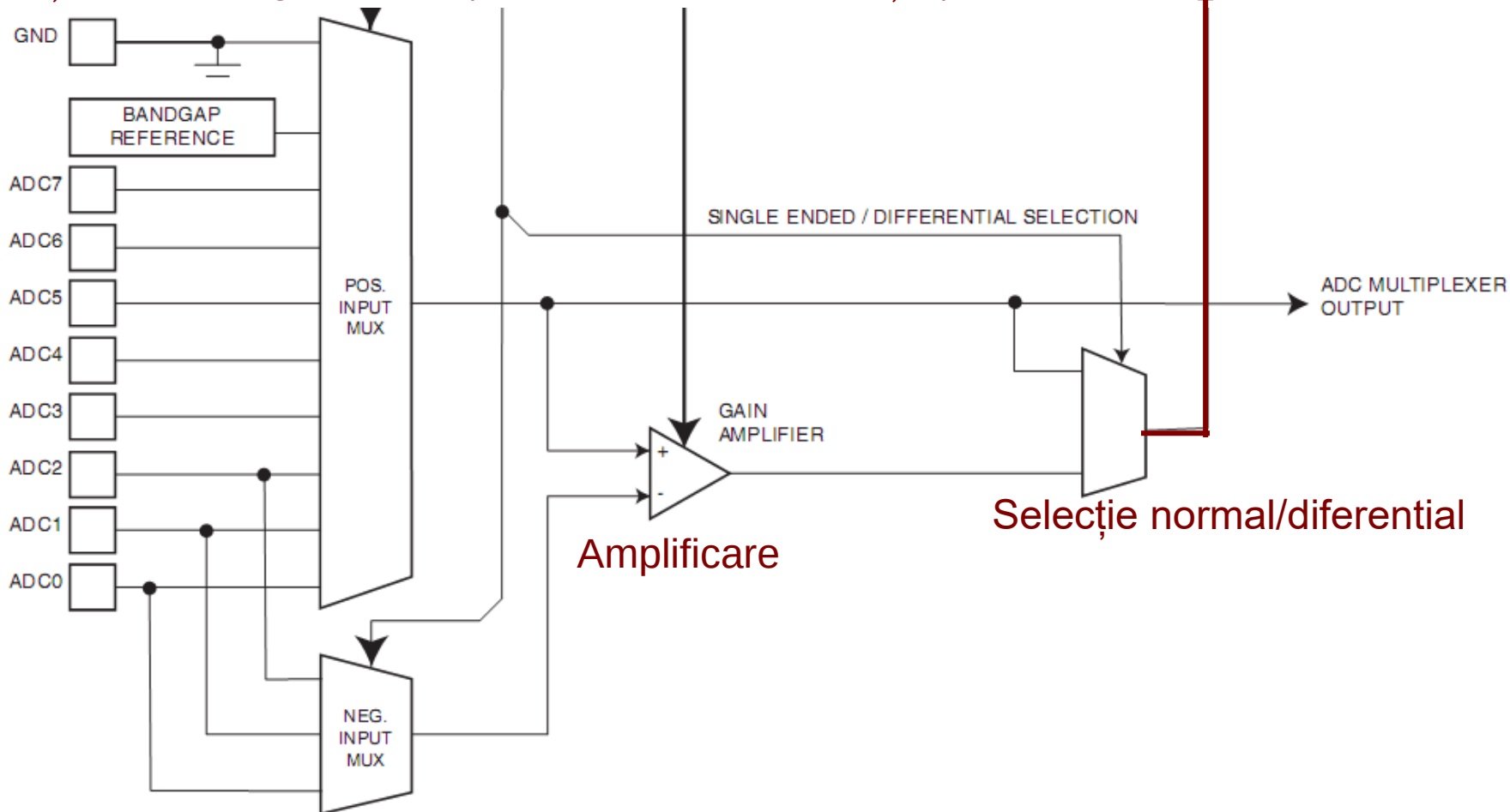


Convertor Analog/Digital



- **Schema bloc – secțiunea de selecție a intrării**

Selecție intrări single ended (sau pozitive pt. diferențial)



Către modul de conversie

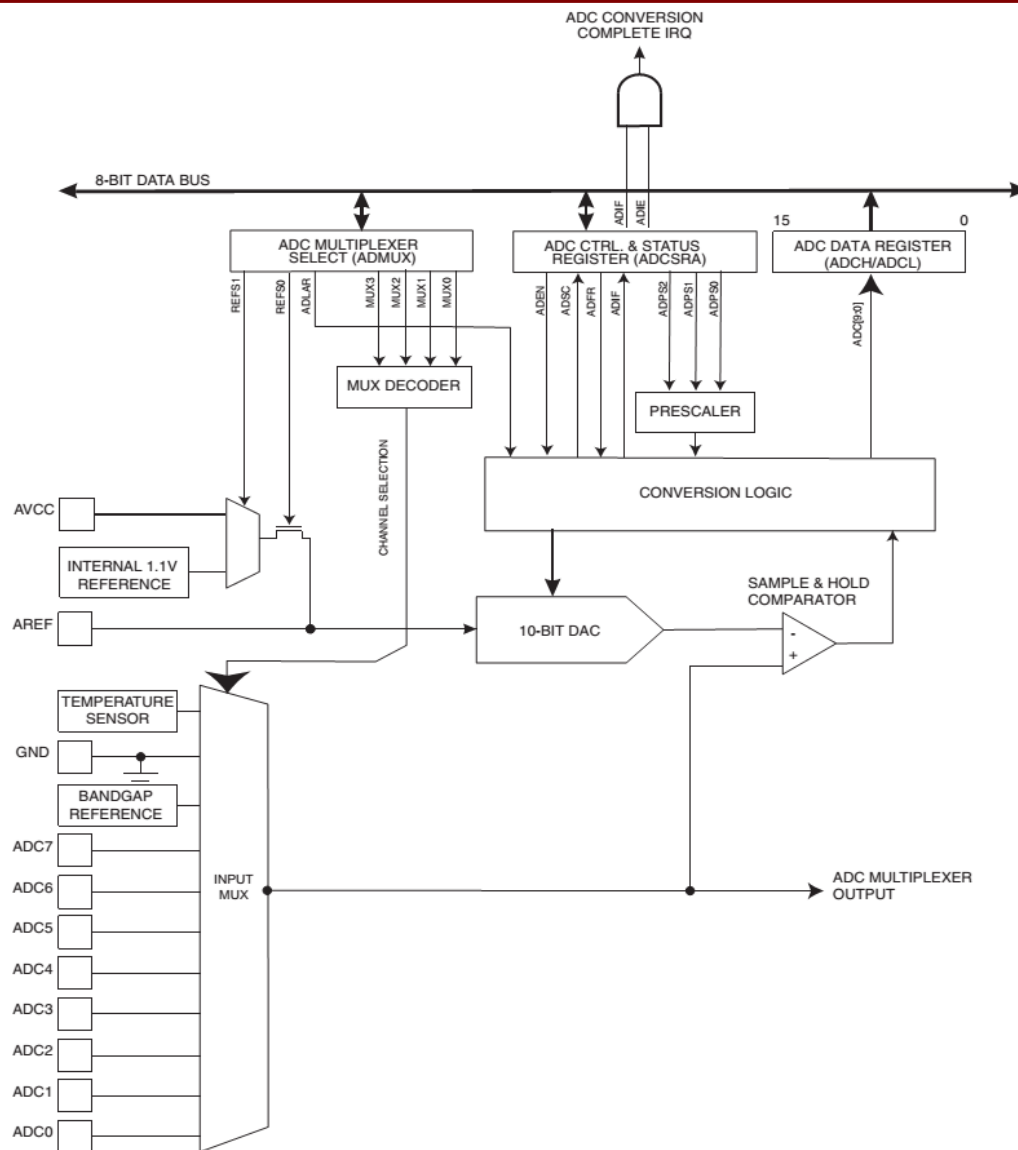
Amplificare

Selecție normal/diferential

Selecție intrări negative (pt. Diferențial)

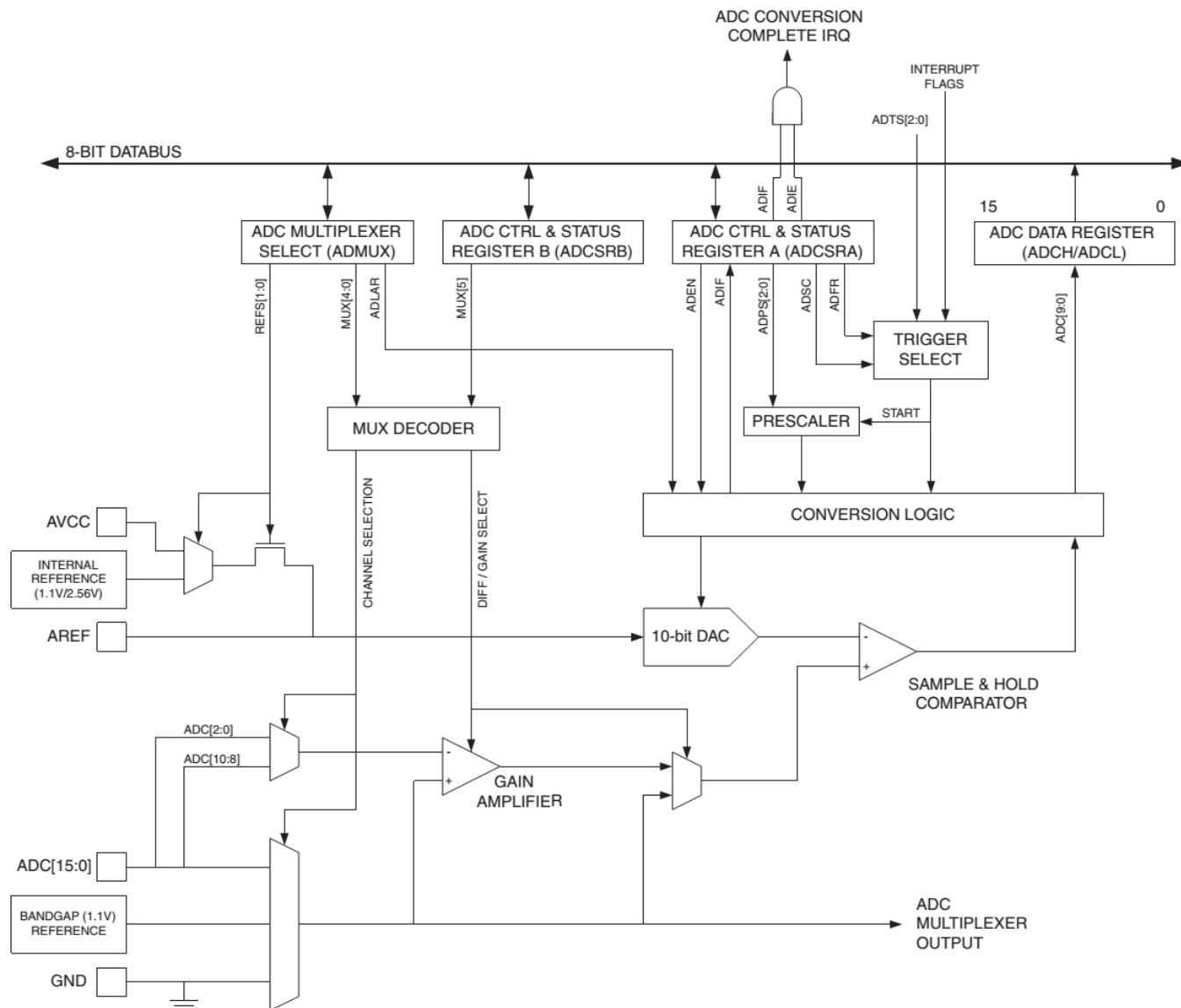


Convertor Analog/Digital ATmega 328P





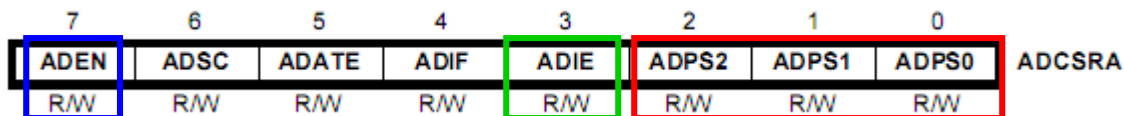
Convertor Analog/Digital ATmega 2560





- **Configurare convertor**

- Activare – setarea bitului **ADEN** din registrul **ADCSRA**
- Folosire întreruperi la terminarea conversiei – setarea bitului **ADIE** din **ADCSRA**
- Selecție frecvență pentru ceasul convertorului: bitii **ADPS2:0**



ADPS2	ADPS1	ADPS0	Division Factor
0	0	0	2
0	0	1	2
0	1	0	4
0	1	1	8
1	0	0	16
1	0	1	32
1	1	0	64
1	1	1	128

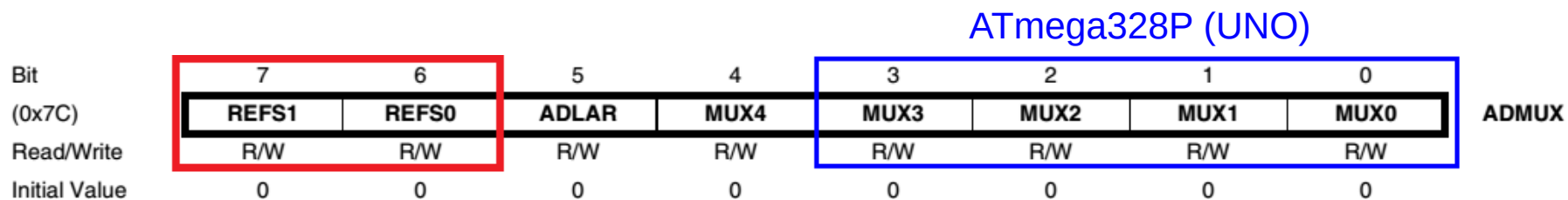


Convertor Analog/Digital



Selecția tensiunilor de referință:

- biții **REFS1:0** din **ADMUX** (**ADC Multiplexer Selection Register**)



ATmega2560 (MEGA)

REFS1	REFS0	Voltage Reference Selection ⁽¹⁾
0	0	AREF, Internal V_{REF} turned off
0	1	AVCC with external capacitor at AREF pin
1	0	Internal 1.1V Voltage Reference with external capacitor at AREF pin
1	1	Internal 2.56V Voltage Reference with external capacitor at AREF pin

ATmega328P (UNO)

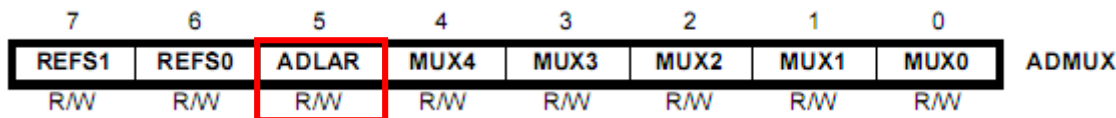
REFS1	REFS0	Voltage Reference Selection
0	0	AREF, Internal V_{ref} turned off
0	1	AV _{CC} with external capacitor at AREF pin
1	0	Reserved
1	1	Internal 1.1V Voltage Reference with external capacitor at AREF pin



Convertor Analog/Digital



- Configurare date de ieșire și accesare rezultate
- Dispunerea celor 10 biți de date este controlată de bitul **ADLAR** (ADC left adjust) din **ADMUX**



ADLAR = 0 – ajustare la dreapta

15	14	13	12	11	10	9	8
-	-	-	-	-	-	ADC9	ADC8
ADC7	ADC6	ADC5	ADC4	ADC3	ADC2	ADC1	ADC0
7	6	5	4	3	2	1	0

ADLAR = 1 – ajustare la stânga

15	14	13	12	11	10	9	8
ADC9	ADC8	ADC7	ADC6	ADC5	ADC4	ADC3	ADC2
ADC1	ADC0	-	-	-	-	-	-
7	6	5	4	3	2	1	0

- Citirea datelor: prima data se citește **ADCL**, apoi **ADCH**. La citirea ADCL, registrul ADCH ramane cu aceeași valoare până este citit
- Dacă ADLAR = 1, se poate citi doar **ADCH**, ca rezultat pe 8 biți (rezoluție mai scăzută)
$$ADCH = V_{in} * 256 / V_{ref}$$



Convertor Analog/Digital



- **Selecția intrărilor** – Configurarea biților **MUX** din **ADMUX**

MUX3..0	Single Ended Input
0000	ADC0
0001	ADC1
0010	ADC2
0011	ADC3
0100	ADC4
0101	ADC5
0110	ADC6
0111	ADC7
1000	ADC8 ⁽¹⁾
1001	(reserved)
1010	(reserved)
1011	(reserved)
1100	(reserved)
1101	(reserved)
1110	1.1V (V_{BG})
1111	0V (GND)

ATmega328P

Aceste tabele nu conțin toate combinațiile!
→ datasheet-ul corespunzător chipului

MUX5:0	Single Ended Input	Positive Differential Input	Negative Differential Input	Gain
000000	ADC0	N/A	N/A	N/A
000001	ADC1			
000010	ADC2			
000011	ADC3			
000100	ADC4			
000101	ADC5			
000110	ADC6			
000111	ADC7			
001000 ⁽¹⁾	N/A	ADC0	ADC0	10×
001001 ⁽¹⁾		ADC1	ADC0	10×
001010 ⁽¹⁾		ADC0	ADC0	200×
001011 ⁽¹⁾		ADC1	ADC0	200×
001100 ⁽¹⁾		ADC2	ADC2	10×
001101 ⁽¹⁾		ADC3	ADC2	10×
001110 ⁽¹⁾		ADC2	ADC2	200×
001111 ⁽¹⁾		ADC3	ADC2	200×
010000			ADC0	ADC1

ATmega2560

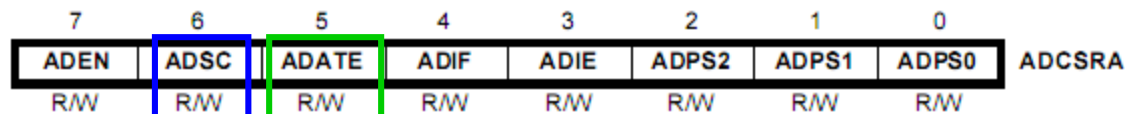
N/A



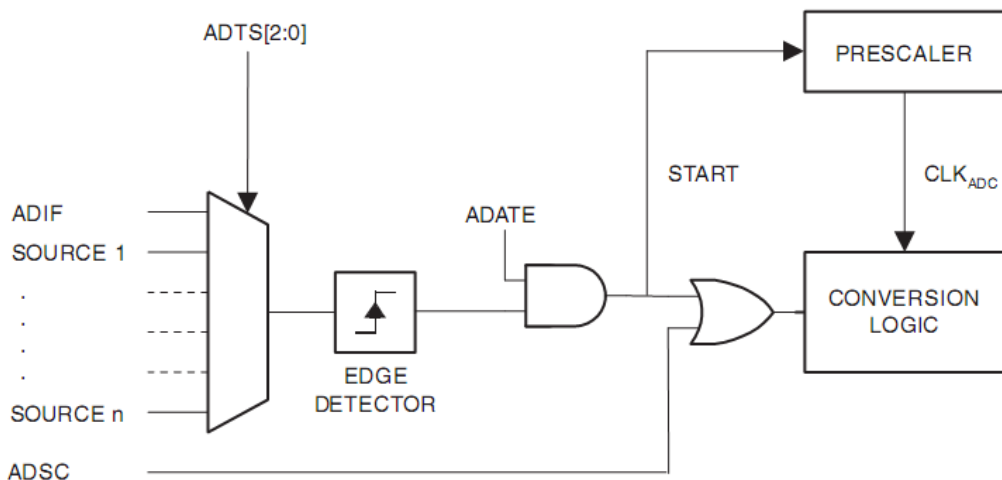
Convertor Analog/Digital



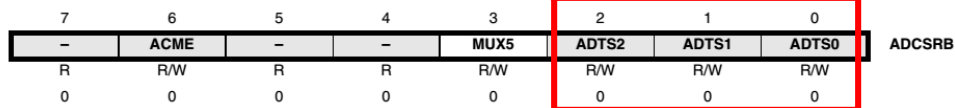
- Declanșare conversie



- La cerere – scrierea bitului **ADSC** din **ADCSCRA**. Acest bit rămâne '1' în timpul conversiei, și se șterge la terminare
- Automat, declanșat de variații ale semnalelor de intrare – dacă bitul **ADATE** din **ADCSCRA** e setat. Una din surse este **ADIF** – flag-ul care semnalează terminarea unei conversii, și eventual cererea unei întreruperi. În acest caz, o nouă conversie începe când se termină cea anterioară.
- Sursele pentru declanșare automată → biții **ADTS** din **ADCSCRB**



ADTS2	ADTS1	ADTS0	Trigger Source
0	0	0	Free Running mode
0	0	1	Analog Comparator
0	1	0	External Interrupt Request 0
0	1	1	Timer/Counter0 Compare Match
1	0	0	Timer/Counter0 Overflow
1	0	1	Timer/Counter1 Compare Match B
1	1	0	Timer/Counter1 Overflow
1	1	1	Timer/Counter1 Capture Event

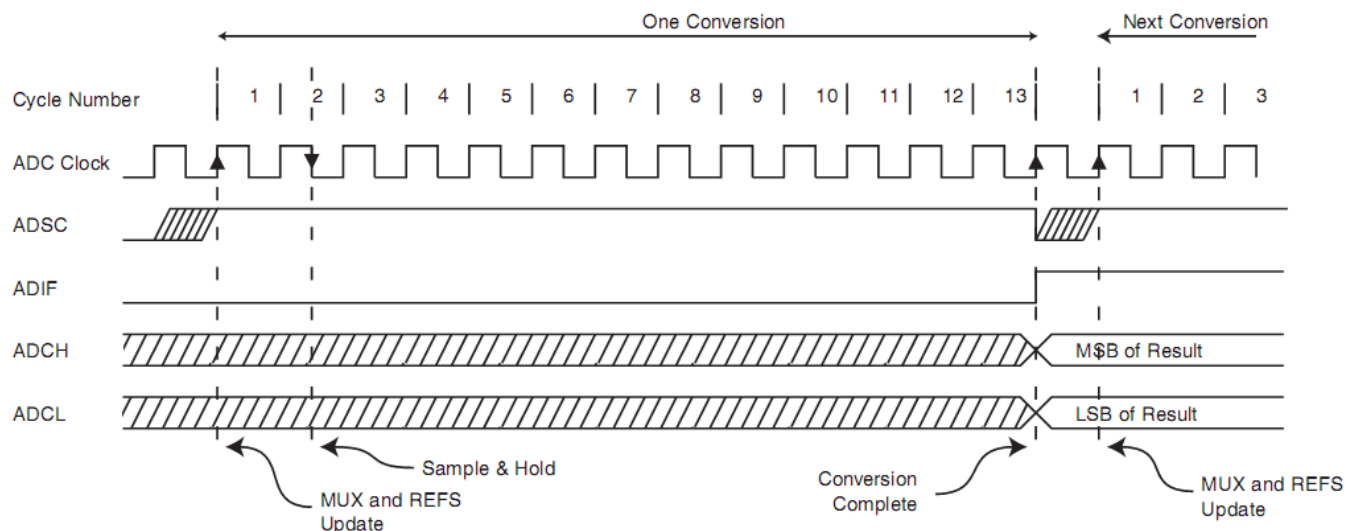




Convertor Analog/Digital



- Timpi de conversie, diagrame de timp



Condition	Sample & Hold (Cycles from Start of Conversion)	Conversion Time (Cycles)
First conversion	13.5	25
Normal conversions, single ended	1.5	13
Auto Triggered conversions	2	13.5
Normal conversions, differential	1.5/2.5	13/14



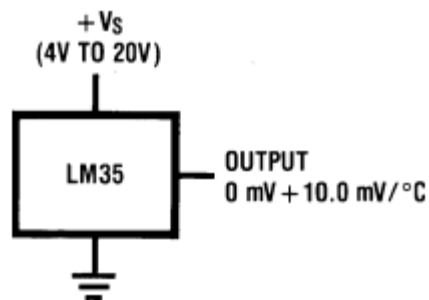
Convertor Analog/Digital



- **Măsurarea temperaturii**
- Senzor: National Semiconductor LM 35 (<http://www.ti.com/lit/ds/symlink/lm35.pdf>)
- $V_{\text{output}} = T * 0.01 \text{ [V]} / [^{\circ}\text{C}]$
- Intrare single ended:

$$ADC = \frac{V_{IN} * 1024}{V_{REF}}$$

- Dacă $ADLAR = 1$, putem scrie: $ADCH = V_{in} * 256 / V_{ref}$
- $ADCH = V_{\text{output}} * 256 / V_{ref}$
- $ADCH = T * 2,56 / V_{ref}$
- Dacă V_{ref} este **tensiunea internă de referință de 2,56 V**, atunci **$ADCH = T [^{\circ}\text{C}]$**



DS005516-3





- Exemplu – masurarea temperaturii ATmega 2560

rcall ADC_init

loop:

rcall start_ADC_conversion ; Starts a conversion

rcall wait_ADC_complete ; Wait to complete the current conversion

rcall ADC_read ; Read the result in r16

rjmp loop

ADC_Init:

ldi r16, 0b11100011 ; Vref=2,56 V internal, ADLAR=1 (Data Shift left) ADC3 single ended

out ADMUX, r16

ldi r16, 0b10000000 ; Activate ADC, max. speed (clock div. ratio = 2)

out ADCSRA, r16

ret

start_ADC_conversion: sbi ADCSRA, ADSC ; ADC start, set ADSC bit in ADCSRA

ret

wait_ADC_complete: sbic ADCSRA, ADSC ; When ADSC=0, conversion is finished

rjmp wait_ADC_complete

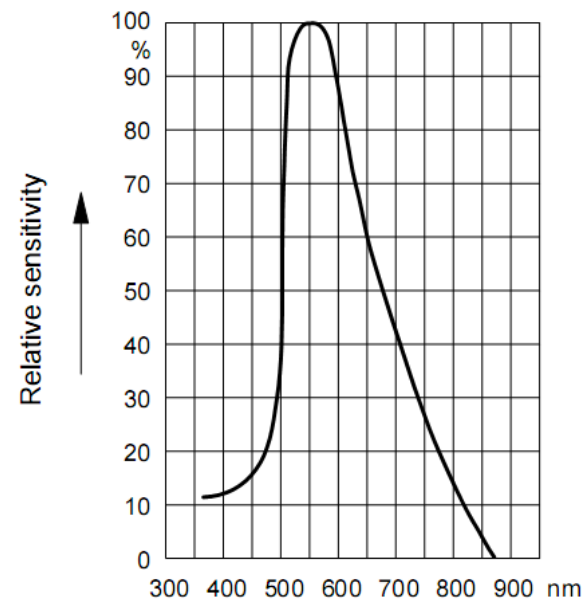
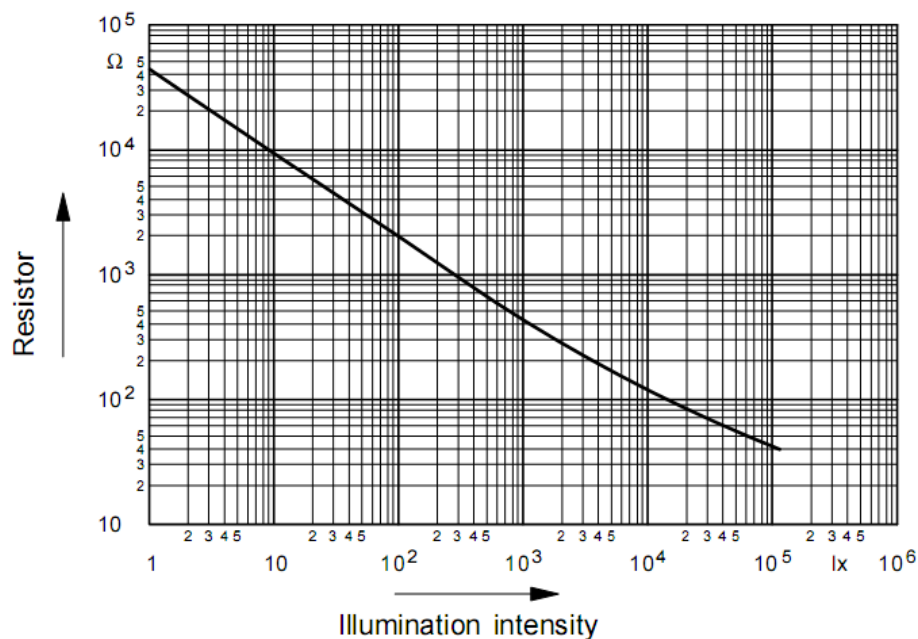
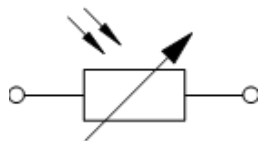
ret

ADC_read: in r16, ADCH ; ADCH – temperature on 8 bits

ret

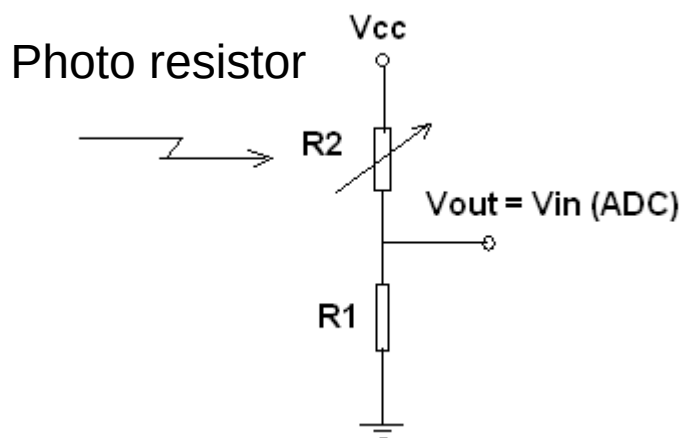
- **Măsurarea intensității luminoase**

- Soluția cea mai simplă: folosirea unei fotorezistențe (sulfură de cadmiu)
- Rezistența scade pe măsură ce intensitatea luminoasă crește



• Măsurarea intensității luminoase (continuare)

- Fotorezistența împreună cu o rezistență fixă formează un divizor de tensiune
- Relația dintre R_F și intensitatea luminoasă trebuie calibrată
- V_{OUT} se introduce la o ADC intrare single ended, GND comun
- ADLAR = 1 (low resolution): $ADCH = V_{in} * 256 / V_{ref}$



$R2 = 200 \, \Omega$ (bright light) $1.4 \, M\Omega$ (dark)

$R1 = \text{constant}$ (ex: 20 K)

$$V_{OUT} = V_{CC} \frac{R_1}{R_1 + R_2}$$

Calibrare sensor:

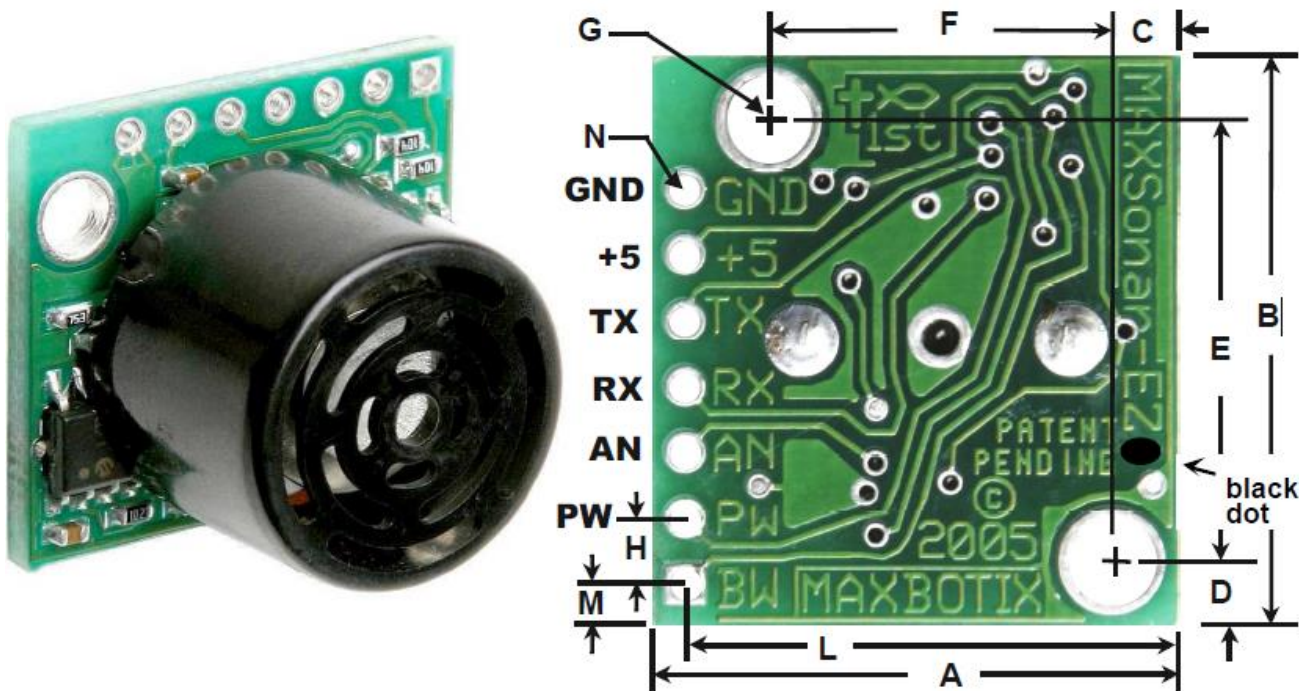
- Se măsoară ADCH pentru cea mai mică luminozitate (dark): $ADCH_{MIN}$
- Se măsoară ADCH pentru cea mai mare luminozitate: $ADCH_{MAX}$

Measurement:

- Compute the light brightness B [%]:

$$B[\%] = \frac{ADC - ADC_{MIN}}{ADC_{MAX} - ADC_{MIN}} * 100$$

- **LV-MaxSonar-EZ0 Sonar pentru detecția obstacolelor**
 - Comunicare serială (UART), Baud 9600, 8 biți de date, 1 bit stop, fără paritate
 - ieșire analogică, **Vcc/512** Volți per inch (1 inch = 2.54 cm)
 - ieșire PWM, **0.147 ms / inch**
 - Frecvența ultrasunetelor: 42 KHz
 - Distanța: 0-6.45 m, depinde foarte mult de dimensiunea obstacolului
 - Utilizarea cea mai simplă: folosind **convertorul ADC**

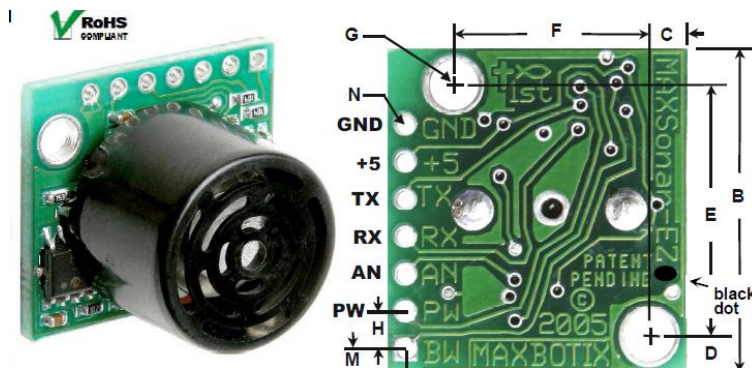


- **Exemplu: masurarea distantei cu senzor Sonar**
 - LV-MaxSonar®-EZ0™ High Performance Sonar Range Finder [7]

AN – ieșire analogică, cu factor de scalare ($V_{cc}/512$) per inch. O alimentare de 3.3V produce $\sim 6.4\text{mV/in} \approx 2.56\text{ mV/cm}$

Domeniu de distanta: 6-in (15 cm) .. 254 in (645 cm)
rezolutie 1-inch

$$ADC = \frac{V_{IN} \times 1024}{V_{REF}} = \frac{2.56\text{mV} \times d[\text{cm}] \times 1024}{2.56[\text{V}]} \approx d[\text{cm}]$$



ADC_Init:

```
ldi r16, 0b11000011
out ADMUX, r16
ldi r16, 0b10000000
out ADCSRA, r16
```

ret

ADC_read:

```
in r20, ADCL
in r21, ADCH
```

ret

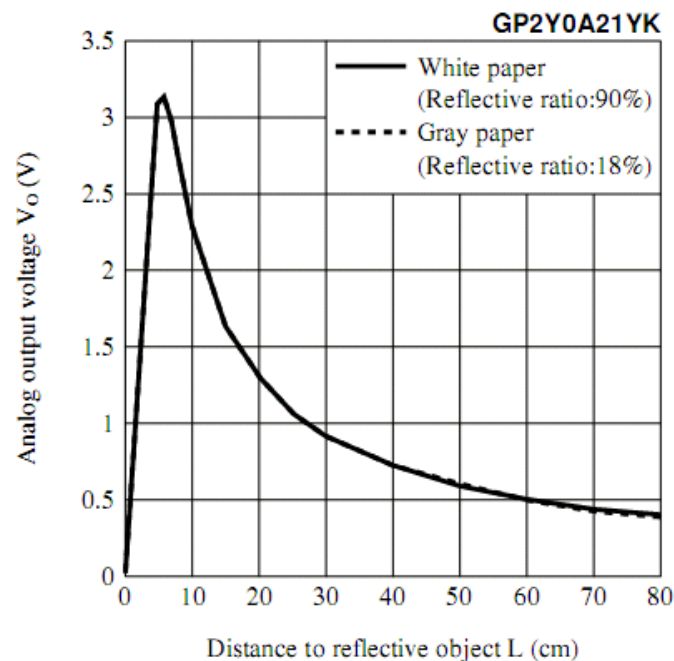
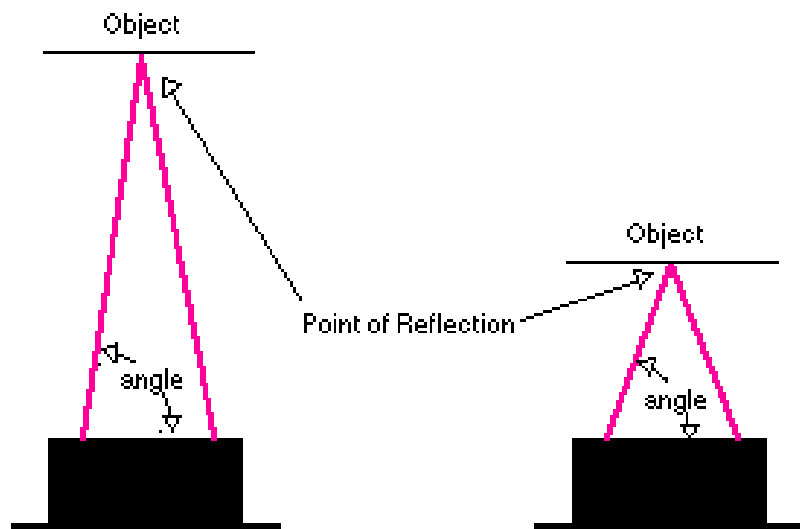
```
; Vref=2,56 V internal, ADLAR=0 (Data Shift right – full 1024 bit resolution),
; ADC3 single ended
; Activare ADC, viteza maxima
```

```
//ADC citire registru inferior
```

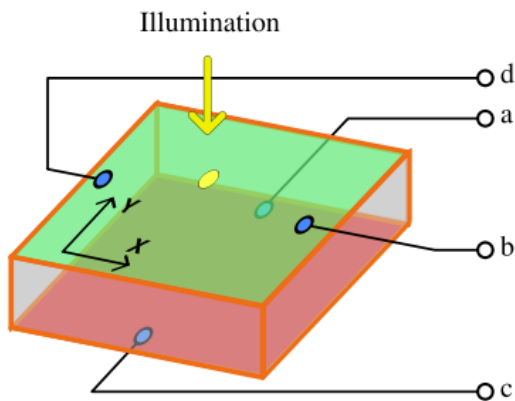
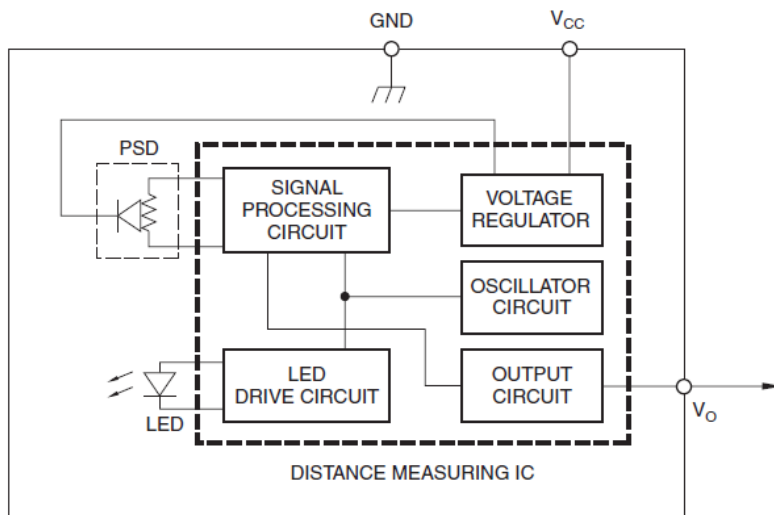
```
//ADC citire registru superior
```

```
// r21:r20 = d[cm]
```

- **SHARP GP2XX, familie de senzori pentru distanță bazați pe reflexia IR**
 - Folosește triangulația pentru calculul distanței
 - Se măsoară unghiul sub care se întoarce raza emisă
 - Leșire analogică, **neliniară**
 - Cost redus (approx. 10 usd)
 - Ușor de montat, robust



- **SHARP GP2XX, familie de senzori pentru distanță bazați pe reflexia IR**
 - Folosește o fotodiodă sensibilă la poziție
 - Pe baza răspunsului ei, se determină unghiul de reflexie



$$x = k_x \cdot \frac{I_b - I_d}{I_b + I_d}$$

$$y = k_y \cdot \frac{I_a - I_c}{I_a + I_c}$$

• Accelerometru ADXL335

- Măsoară accelerația pe 3 axe, de la -3 g ... 3 g
- Alimentare între 1.8 V ... 3.6 V
- Ieșire pentru 0 G: $V_{cc} / 2$
- Sensibilitate tipică pentru $V_{cc} = 3.3V$: 300 mV/G

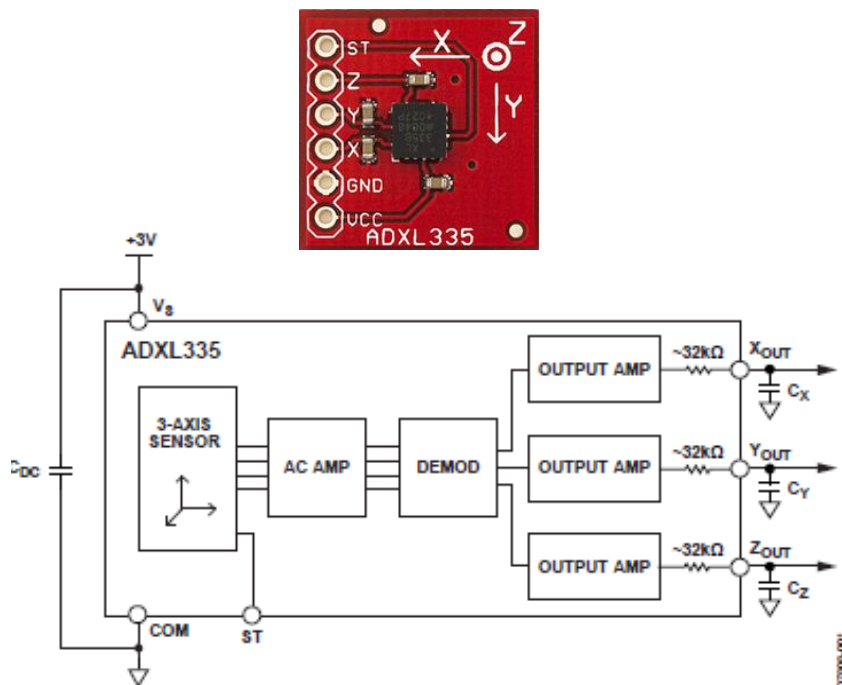
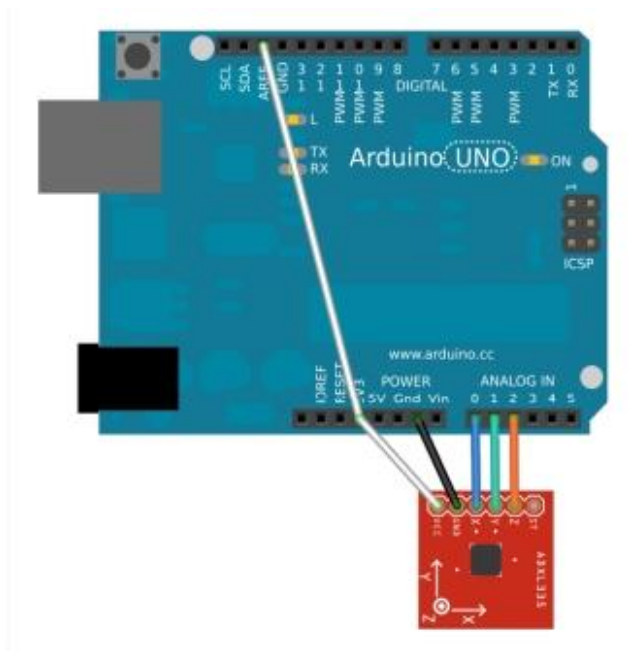


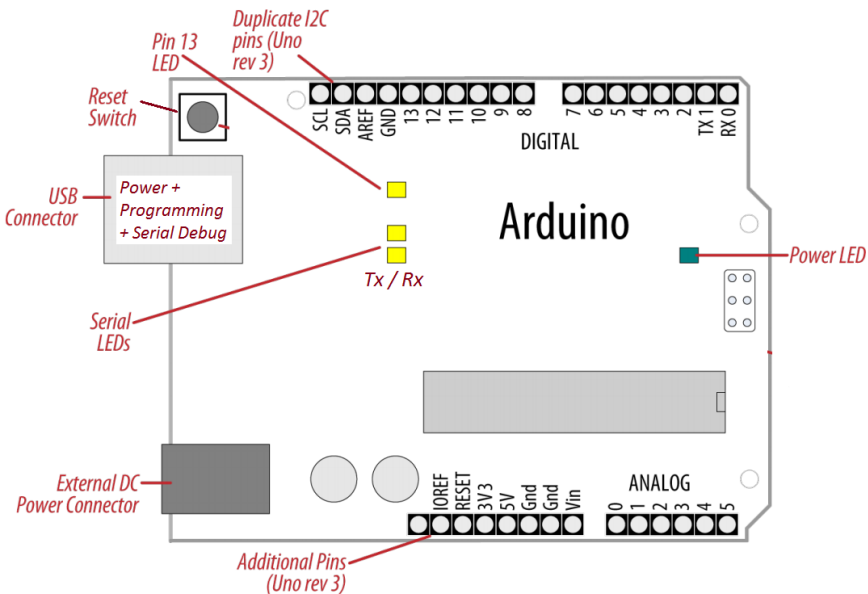
Figure 1.



Arduino 3.3 V	ADXL335 VCC
Arduino GND	ADXL335 GND
Arduino Analog0	ADXL335 X
Arduino Analog1	ADXL335 Y
Arduino Analog2	ADXL335 Z
Arduino 3.3	Arduino AREF



Procesare semnal analogic cu Arduino



Arduino UNO: A0 .. A5

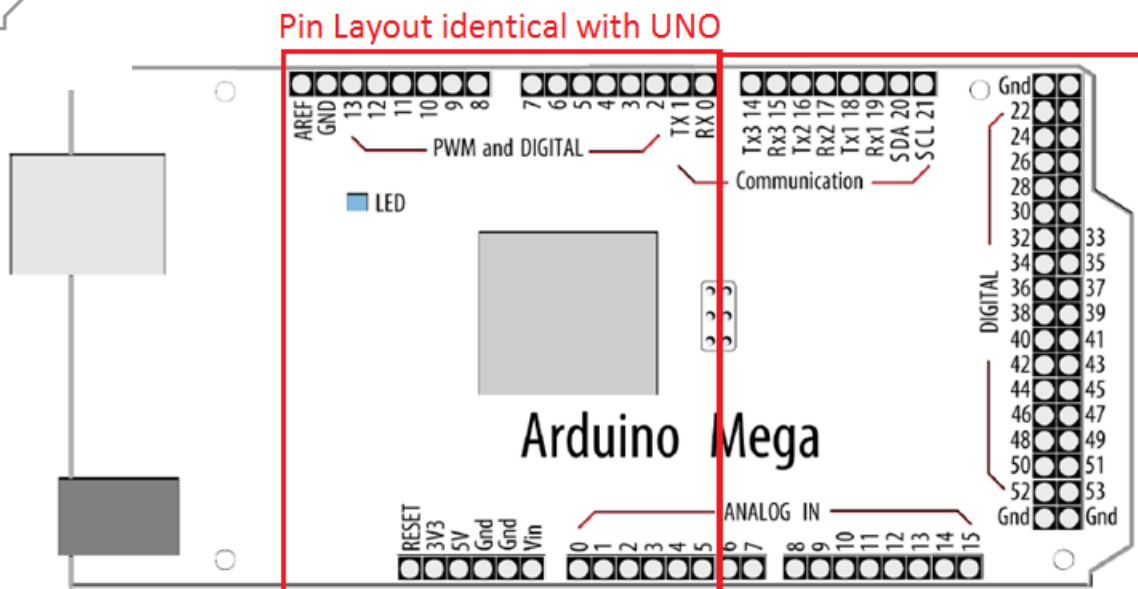
Arduino MEGA: A0 .. A15

- Pinii analogici sunt intrări pentru convertorul ADC al microcontrollerului.
- ADC are rezoluția de 10 biți, returnând valori între 0 și 1023

Alți pini:

A_{REF} (intrare) – tensiune de referință externă pentru ADC

IOREF (ieșire) – tensiune de referință pentru shield-uri



Pin Layout specific to MEGA



- **Funcția principală a pinilor analogici:** citirea semnalelor analogice
- Pinii analogici au și funcția de pin digital de uz general, ca și pinii digitali. Exemplu:
 - `pinMode(A0, OUTPUT);`
 - `digitalWrite(A0, HIGH);`
- Pinii analogici au de asemenea rezistențe **pull up resistors**, care funcționează în același fel ca rezistențele pinilor digitali. Ele sunt activate scriind HIGH pe pinul configurat ca intrare.
 - `digitalWrite(A0, HIGH);` // activare pullup la A0 configurat ca *input*.
- **Activarea unei rezistențe pull up va influența valorile citite cu `analogRead()` !!!**
- **Funcții:**
 - **`analogRead(pin)`** – citește o valoare de pe un pin analogic
 - **`analogReference(type)`** – configurează tensiunea de referință care va fi folosită pentru intrarea analogică (i.e. valoarea maximă a tensiunii de intrare măsurabilă pe pin-ul analogic)



- **analogReference(type)** – configurează tensiunea de referință care va fi folosită pentru intrarea analogică.
- **type** – stabilește referința folosită:
 - DEFAULT: tensiunea referință implicită, de 5 V (pentru UNO & MEGA)
 - INTERNAL: tensiune internă de referință, 1.1 V la UNO (*nu există la Arduino Mega*)
 - INTERNAL1V1: tensiune internă de referință 1.1V (*doar Arduino Mega*)
 - INTERNAL2V56: tensiune internă 2.56V (*doar Arduino Mega*)
 - EXTERNAL: tensiune de referință externă, aplicată la pinul AREF (**0 ... 5V**).
- După schimbarea tensiunii de referință, prima citire cu analogRead() poate fi eronată !!!
- **Nu folosiți o tensiune de referință externă negativă (<0V) sau mai mare de 5V pe pinul AREF! Dacă folosiți o tensiune externă de referință, configurați referința ca externă apelând analogReference() înainte de a apela funcția analogRead().** În caz contrar, veți pune în contact tensiunea de referință internă, generată în mod activ, cu tensiunea externă, putând cauza scurtcircuit și distrugerea microcontrollerului. !!!



Procesare semnal analogic cu Arduino



- `int digital_value analogRead(pin)` – citește o valoare de pe pinul analogic specificat
- O valoare analogica între 0 .. RANGE va produce un numar *digital_value* între 0 și 1023.
- Rezoluția de măsurare este deci: $\text{RANGE volți} / 1024 \text{ unități}$.
- Pentru referința DEFAULT (5V) rezoluția devine:

$$\text{resolutionADC} = .0049 \text{ volti (4.9 mV) / unitate.}$$

- Pentru a converti valoarea citită *digital_value* la tensiunea analogică:

$$\text{Voltage} = \text{resolutionADC} * \text{digital_value}$$

- Pentru a converti voltajul la o valoare fizica masurata in [X] folositi:

$$\text{Measurement [X]} = \text{Voltage [V]} / \text{Sensor_resolution [V]} / [\text{X}]$$

- Durează aproximativ 100 microsecunde (0.0001 s) pentru a citi o intrare analogică, astfel încât rata maximă de citire este 10,000 valori pe secundă.
- Dacă pinul analogic nu este conectat la nimic, valoarea returnată de `analogRead()` va fluctua în functie de mai multi factori (e.g. ce tensiuni sunt pe ceilalți pini analogici, apropierea mâinii de placă...) !!!



- **Exemplu** – Citirea tensiunii unui potențiometru conectat la intrarea analogică (<http://arduino.cc/en/Reference/AnalogRead>)

```
int analogPin = 3;           // aici se va conecta cursorul potentiometrului
                             // celelalte terminale ale potentiometrului se conecteaza la +5V si GND
int val = 0;                 // variabila in care se va citi valoarea analogica
float voltage;               // tensiunea calculata, in [mV]
float resolutionADC = 4.9;   // rezolutia in mV pentru referinta implicita de 5 V

void setup()
{
  Serial.begin(9600);
}

void loop()
{
  val = analogRead(analogPin); // citire intrare analogica, pentru referinta 5 V
  voltage = val * resolutionADC; // conversie in mV
  Serial.print("Digital value = ");
  Serial.println(val);         // transmitere la PC
  Serial.print("Voltage [mV] = ");
  Serial.println(voltage);
}
```



Procesare semnal analogic cu Arduino

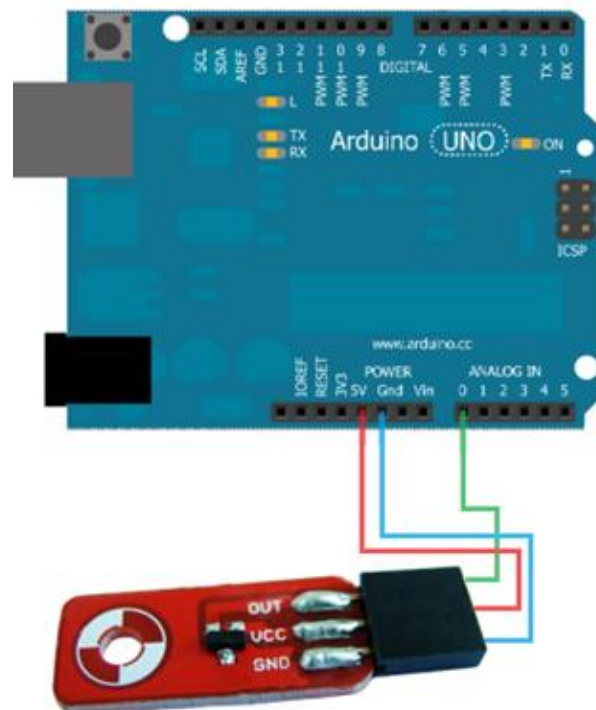
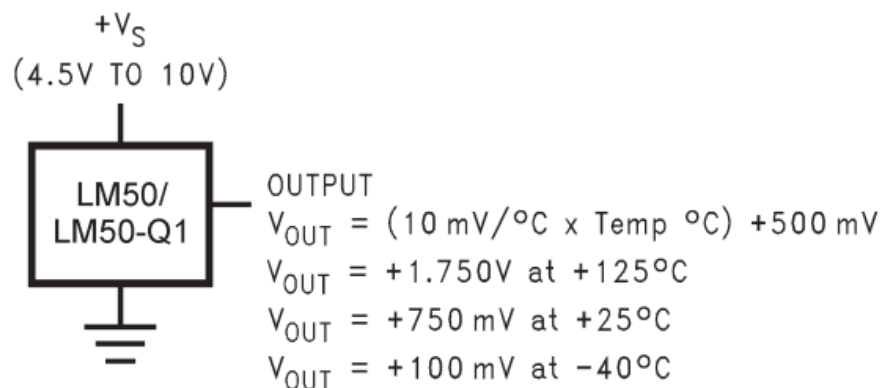


- **Senzor de temperatură folosind LM50 [5]**

<http://www.ti.com/lit/ds/symlink/lm50.pdf>

- Caracteristici:

- Iesire liniara $+10.0 \text{ mV}/^{\circ}\text{C} = 0.01\text{V}/^{\circ}\text{C}$
- Domeniu de temperaturi $-40^{\circ}\text{C} \dots +125^{\circ}\text{C}$
- Deplasament constant $+500 \text{ mV}$ pentru citirea temperaturilor negative
- Circuitul LM50 este inclus in senzorul de temperatura Brick [6]





- **Exemplu** – Citire temperatură de la senzor, face media a 10 citiri consecutive, și trimite către PC

```
float resolutionADC = .0049 ;      // rezolutia implicita (pentru referinta 5V) = 0.049 [V] / unitate
float resolutionSensor = .01 ;     // rezolutie senzor = 0.01V/°C

void setup() {
    Serial.begin(9600);
}

void loop(){
    Serial.print("Temp [C]: ");
    float temp = readTempInCelsius(10, 0); // citeste temperatura de 10 ori, face media
    Serial.println(temp);                 // afisare
    delay(200);
}

float readTempInCelsius(int count, int pin) { // citeste temperatura de count ori de pe pinul analogic pin
    float sumTemp = 0;
    for (int i = 0; i < count; i++) {
        int reading = analogRead(pin);
        float voltage = reading * resolutionADC;
        float tempCelsius = (voltage - 0.5) / resolutionSensor ; // scade deplasament, converteste in grade C
        sumTemp = sumTemp + tempCelsius;                          // suma temperaturilor
    }
    return sumTemp / (float)count;                                // media returnata
}
```



- **Exemplu** – Măsurare distanțe cu sonarul LV EZ0 (rezoluție 10mV / inch $\cong$ 0.01V)

```
const int sensorPin = 1;           // lesire sonar conectata la A1
float resolutionADC = .0049;       // rezolutia implicita (pentru referinta 5V) = 0.049 [V] / unitate
float resolutionSensor = .01;      // rezolutie senzor = 0.01V/ inch

void setup() {
    Serial.begin(9600);
}

void loop() {
    float distance = readDistance(10, sensorPin);           // distanta in inch, media a 10 citiri
    Serial.print("Distance [inch]: "); Serial.println(distance); // afisare distanta in inch
    Serial.print("Distance [cm]: "); Serial.println(distance*2.54); // afisare distanta in cm, 1 inch=2.54 cm
    delay(200);
}

float readDistance(int count, int pin) {
    // citeste de 10 ori distanta, si face media
    float sumDist = 0;
    for (int i = 0; i < count; i++) {
        int reading = analogRead(pin);
        float voltage = reading * resolutionADC;
        float distance = voltage / resolutionSensor; // conversie tensiune in distanta
        sumDist = sumDist + distance;
    }
    return sumDist / (float)count;
}
```



Referințe



1. Atmel ATmega640/V-1280/V-1281/V-2560/V-2561/V datasheet
2. Atmel Atmega64 datasheet
3. <http://arduino.cc/en/>
4. <http://arduino.cc/en/Reference/AnalogRead>
5. <http://www.ti.com/lit/ds/symlink/lm50.pdf>
6. <http://www.robofun.ro/senzori/vreme/senzor-temperatura-brick>
7. http://maxbotix.com/documents/LV-MaxSonar-EZ_Datasheet.pdf



Proiectarea cu Micro-Procesoare

Lector: Mihai Negru

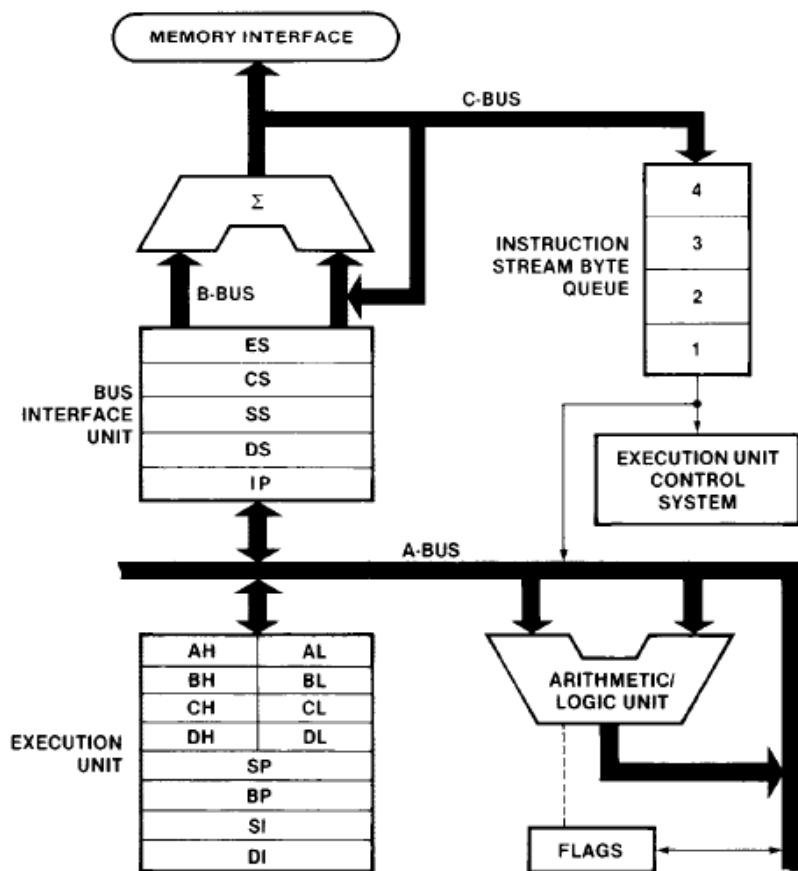
An 3 – Calculatoare și Tehnologia Informației
Seria B

Curs 10: Intel 8086 – I/O și întreruperi

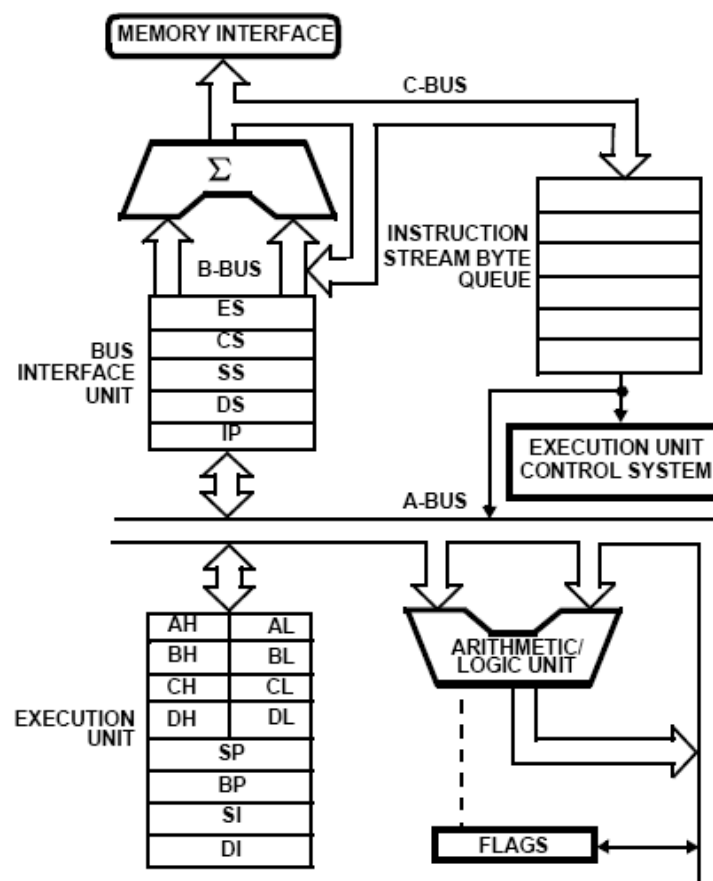
<http://users.utcluj.ro/~negrum/>



Diagrame bloc 8088 si 8086



8088



8086

Pipeline cu două etaje, EU (execution unit) și BIU (Bus Interface Unit) – decuplate printr-un FIFO (4/6 bytes)

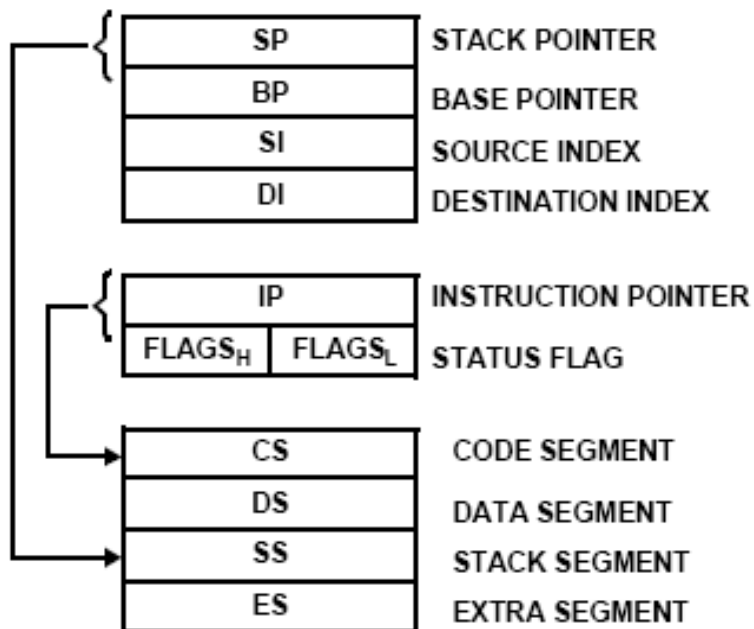


Regiștri 8086



AX	AH	AL	ACCUMULATOR
BX	BH	BL	BASE
CX	CH	CL	COUNT
DX	DH	DL	DATA

Regiștri de uz general (GPR)



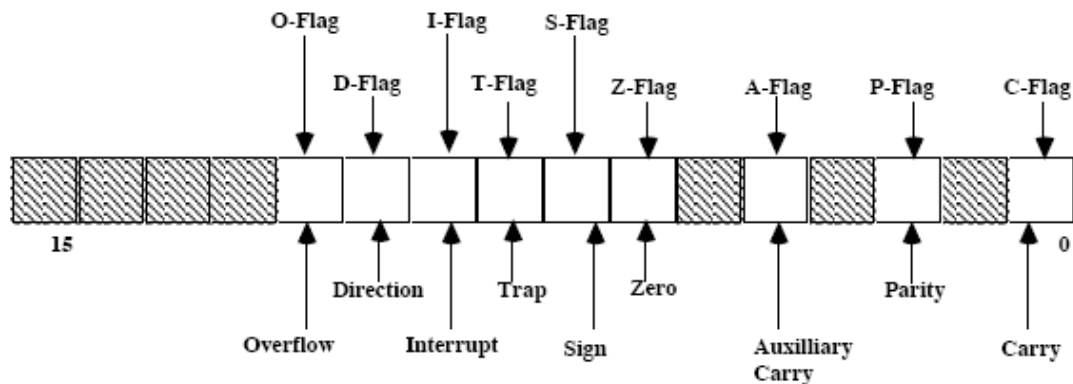
Regiștri pointer

Regiștri segment



Registrul de stare (Flags)

- Overflow Flag (OF) – dacă rezultatul unei operații este prea mare (ca pozitiv) sau prea mic (ca negativ) pentru a putea fi reprezentat în locația destinație
- Direction Flag (DF) – dacă este '1', operațiile cu șiruri vor decrementa automat regiștrii index. Dacă e '0', indecșii se vor autoincrementa.
- Interrupt-enable Flag (IF) – setarea acestui bit activează întreruperile mascabile.
- Single-step Flag (TF) – Generează o întrerupere după fiecare instrucțiune dacă e setat. Folosit cu programele de tip debugger (Trap Flag).
- Sign Flag (SF) – Este setat dacă rezultatul unei operații e negativ.
- Zero Flag (ZF) – Setat dacă rezultatul unei operații este zero.
- Auxiliary carry Flag (AF) – Setat dacă se produce carry între biții 0-3 și biții 4-7 din AL
- Parity Flag (PF) – dacă numărul biților cu valoare '1' din rezultatul unei operații este par, are valoarea '1'.
- Carry Flag (CF) – dacă se produce carry în urma unei operații.

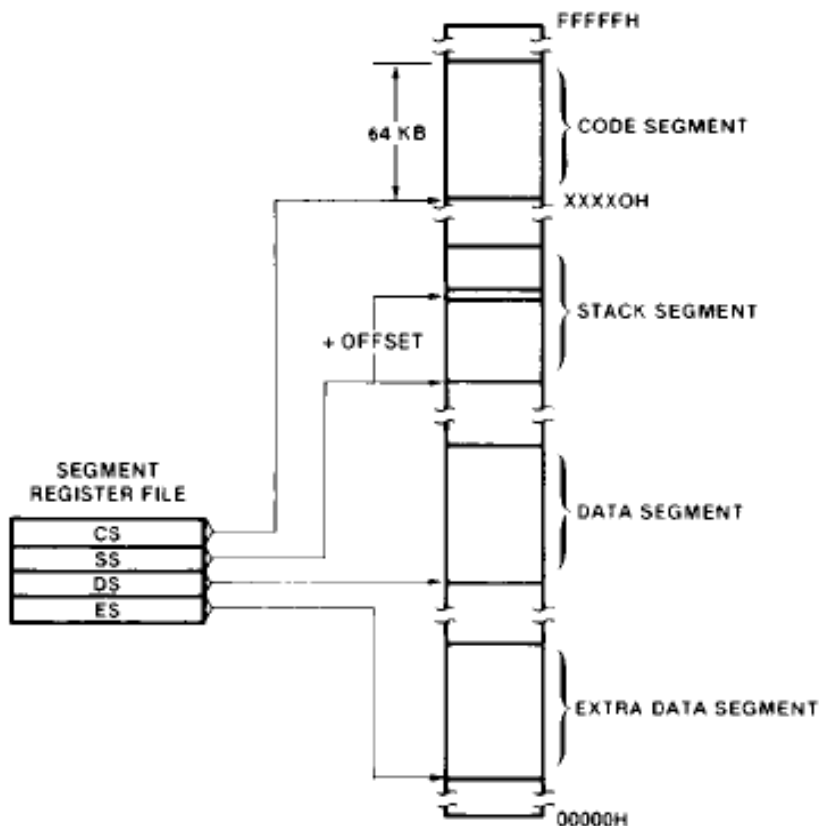




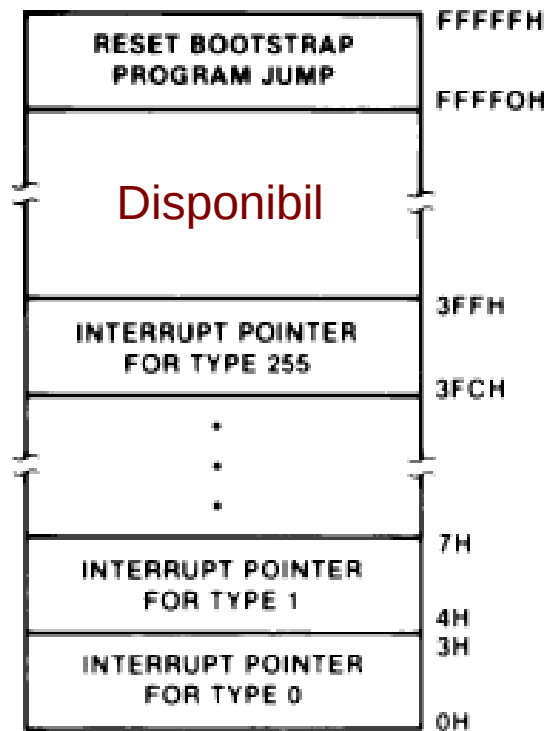
Spații de adresă – Memorie



- Memoria totala adresabilă: 1 MB
 - 20 linii de adresă
- Adresare la nivel de byte



Adresa cod RESET
CS: FFFFh, IP: 0000h



Adrese ISR, de la adresa 00000h
4 bytes / tip întrerupere,
256 tipuri întrerupere posibile



Moduri de adresare – Memorie

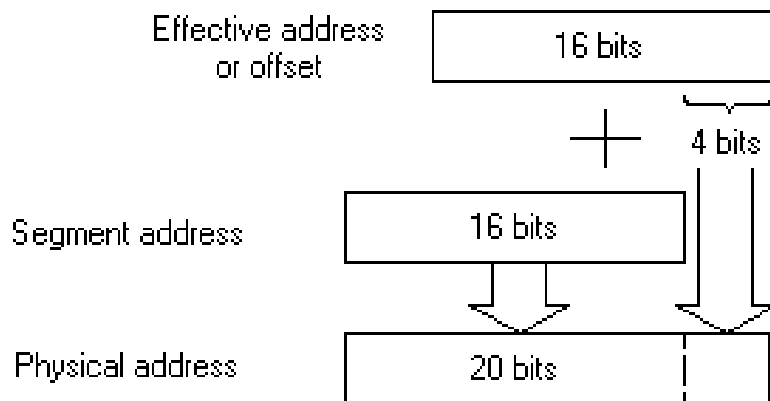


- Adresă efectivă = Bază + Index + Deplasament constantă
- Există multiple combinații, fiecare din cei trei termeni este opțional
- Adresa fizică: Segment x 16 + Adresa efectivă
- Adresa efectivă: 16 biți
- Adresa fizică: 20 biți

$$PA = \left\{ \begin{matrix} CS \\ SS \\ DS \\ ES \end{matrix} \right\} : \left\{ \begin{matrix} BX \\ BP \end{matrix} \right\} + \left\{ \begin{matrix} SI \\ DI \end{matrix} \right\} + \left\{ \begin{matrix} \text{8-bit displacement} \\ \text{16-bit displacement} \end{matrix} \right\}$$

Exemple:

```
mov    al, [bx]
mov    al, [si]
mov    al, [bp][di]
mov    al, [bx][di]200
mov    al, ss: [di]470
mov    al, [bp+si+100]
```

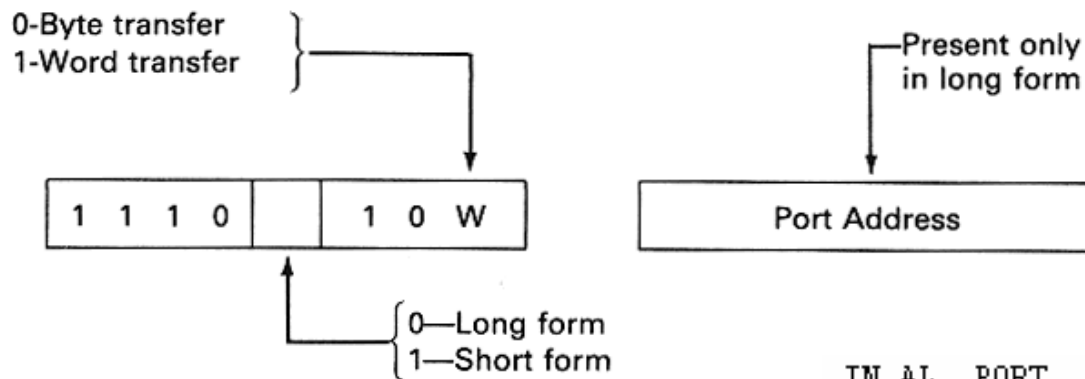




Adresare I/O



- I/O (spațiu separat), domeniu adresabil 0000 – FFFF (64 K-octeți)
- Două tipuri de instrucțiuni – scurtă și lungă. Instrucțiunea lungă permite specificarea portului ca o constantă între 0 și 255
- Registrul sursă sau destinație pentru operația I/O este întotdeauna AX (AL)
- Portul este specificat de o constantă, sau adresa lui este in DX



Exemple

```
IN AL, PORT
IN AX, PORT
IN AL, DX
IN AX, DX
```

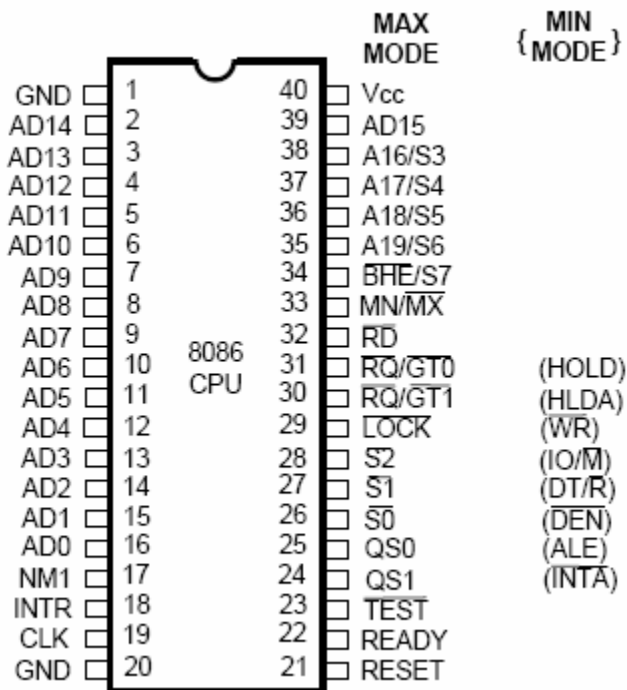
```
OUT PORT, AL
OUT PORT, AX
OUT DX, AL
OUT DX, AX
```

```
(AL) <- (PORT)
(AX) <- (PORT+1:PORT)
(AL) <- ((DX))
(AX) <- ((DX)+1:(DX))

(PORT) <- (AL)
(PORT+1:PORT) <- (AX)
((DX)) <- (AL)
((DX)+1:(DX)) <- (AX)
```



Diagrama pinilor



AD15:0 – Adrese și date, multiplexate pe aceiași pini
A19:A16 – Liniile de adresă superioară

BHE – Byte high enable

INTR – Interrupt request

MN/MX – Selecție între modul minim și maxim

WR, RD – write, read

DT/R – data transmit/receive

DEN – Data enable

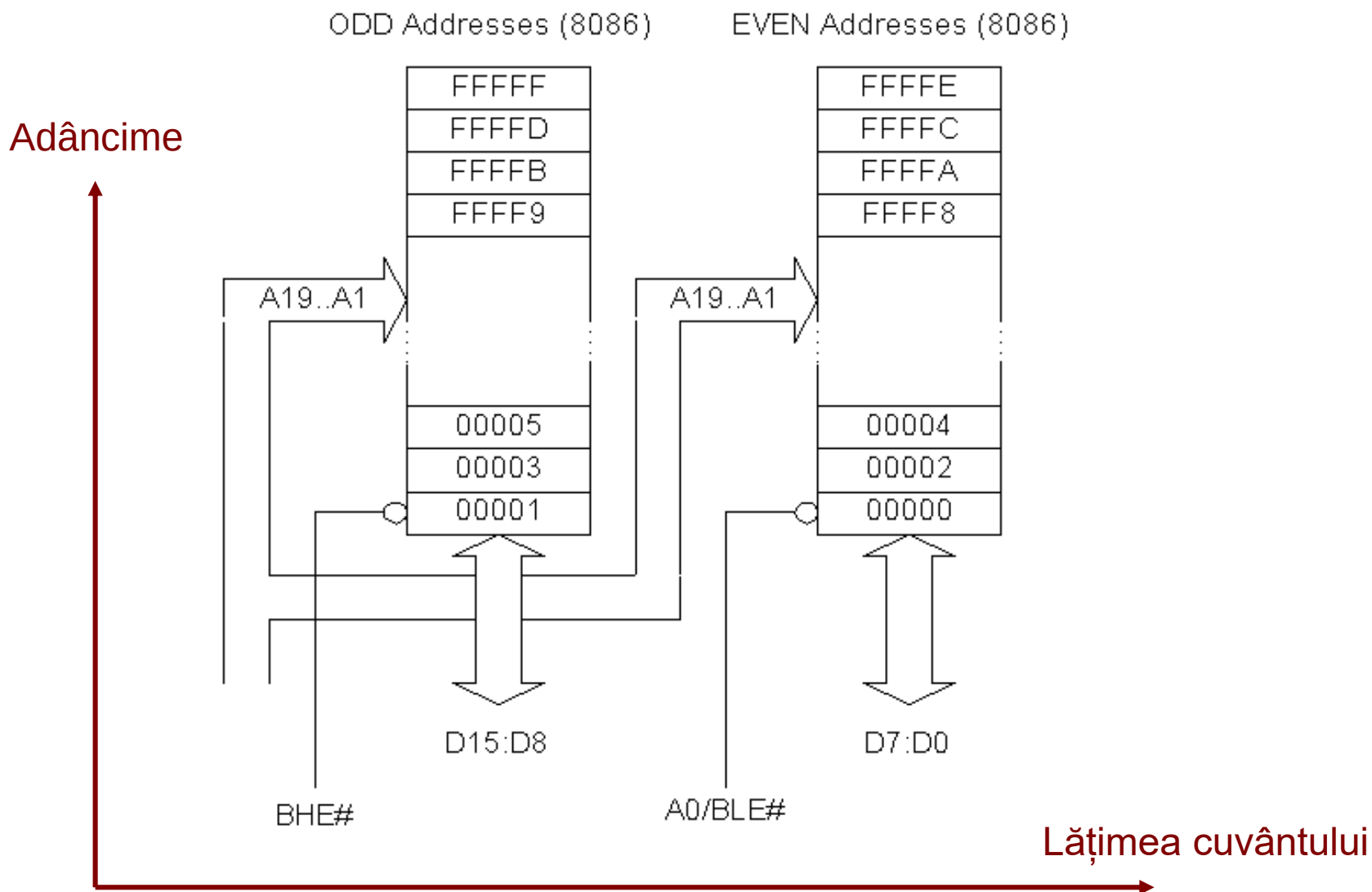
ALE – Address latch enable

IO/M – semnal care specifică dacă liniile de adresă indică o locație de memorie sau de I/O

#BHE	A0	Explicatie
0	0	Acces pe 16 biți (aliniat)
0	1	Byte superior, de la adresă impară
1	0	Byte inferior, de la adresă pară
1	1	Combinație nepermisă

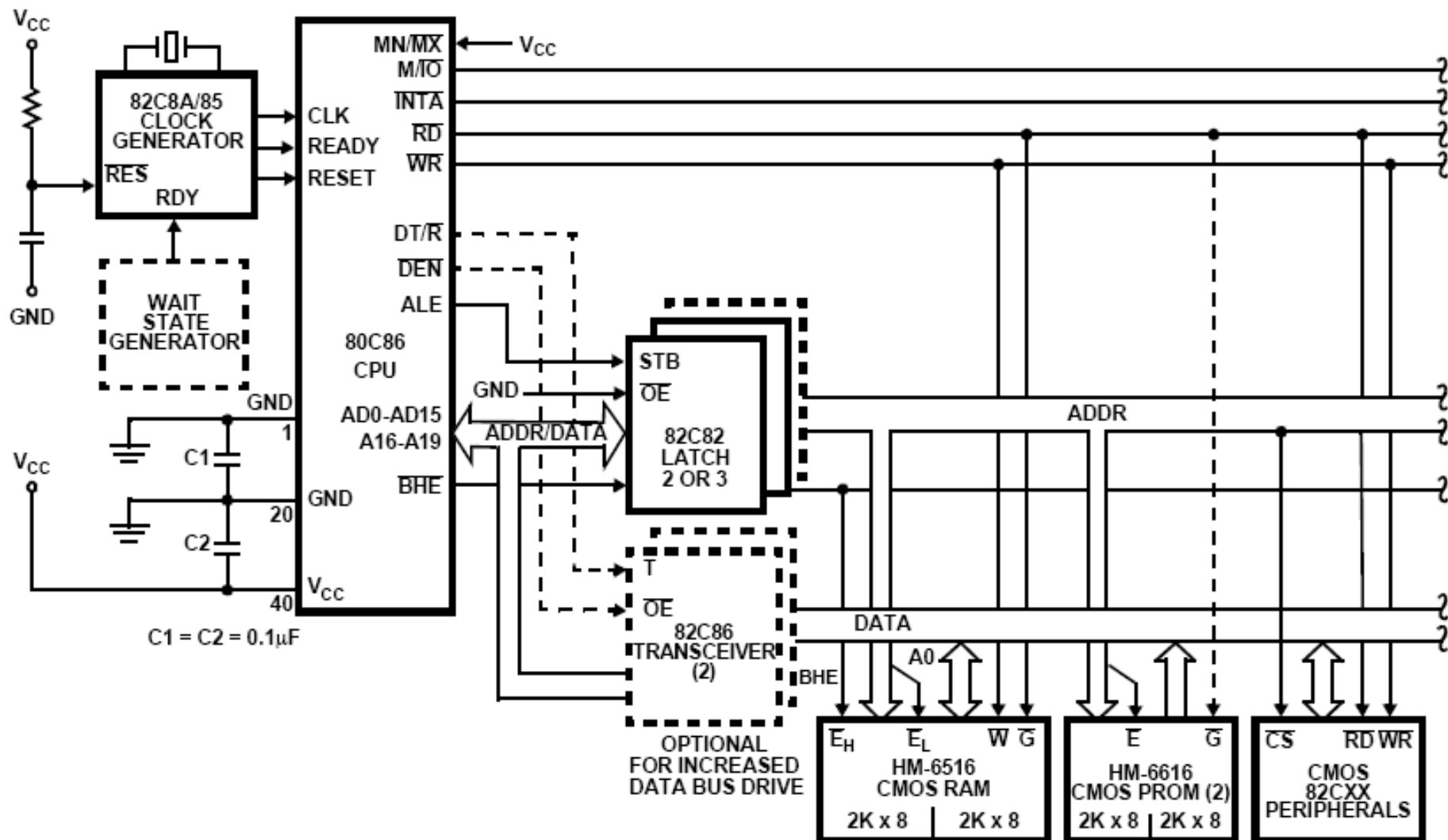


Organizarea memoriei



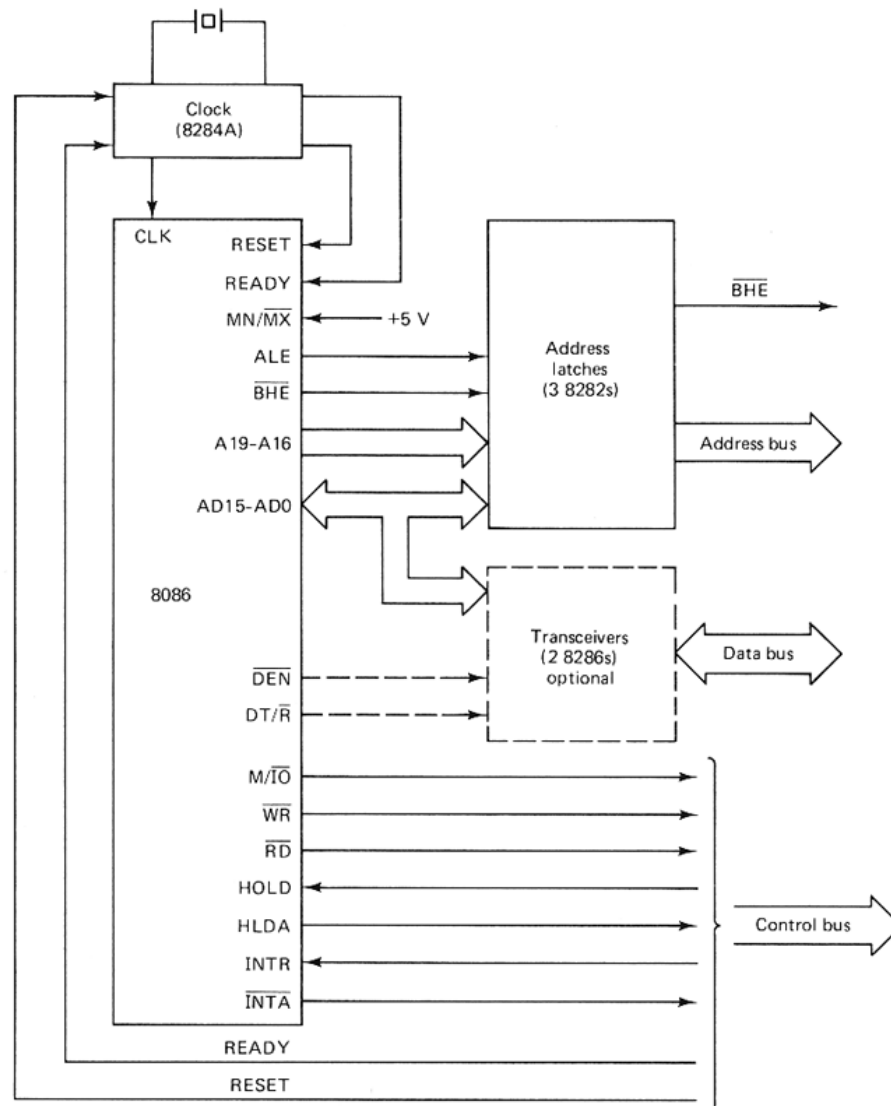


Sistem 8086 în mod minim



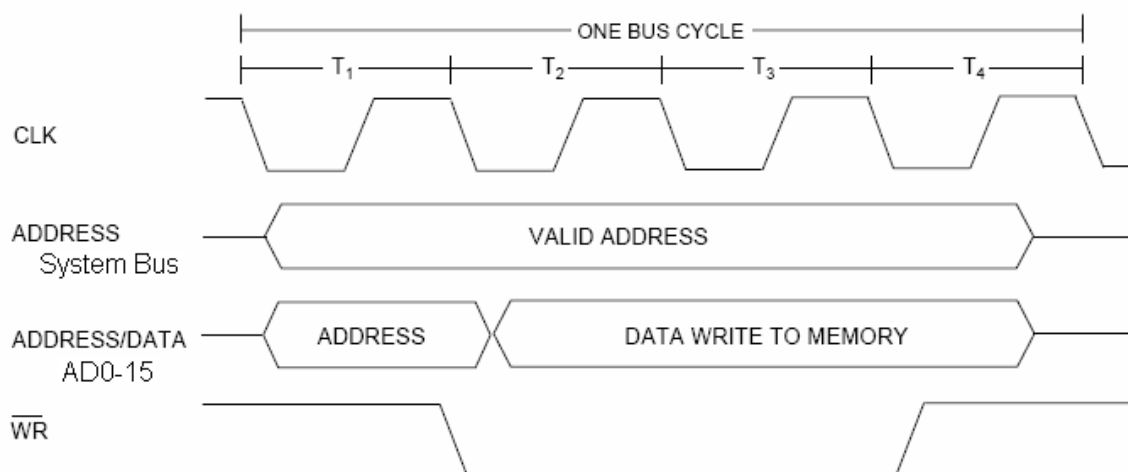
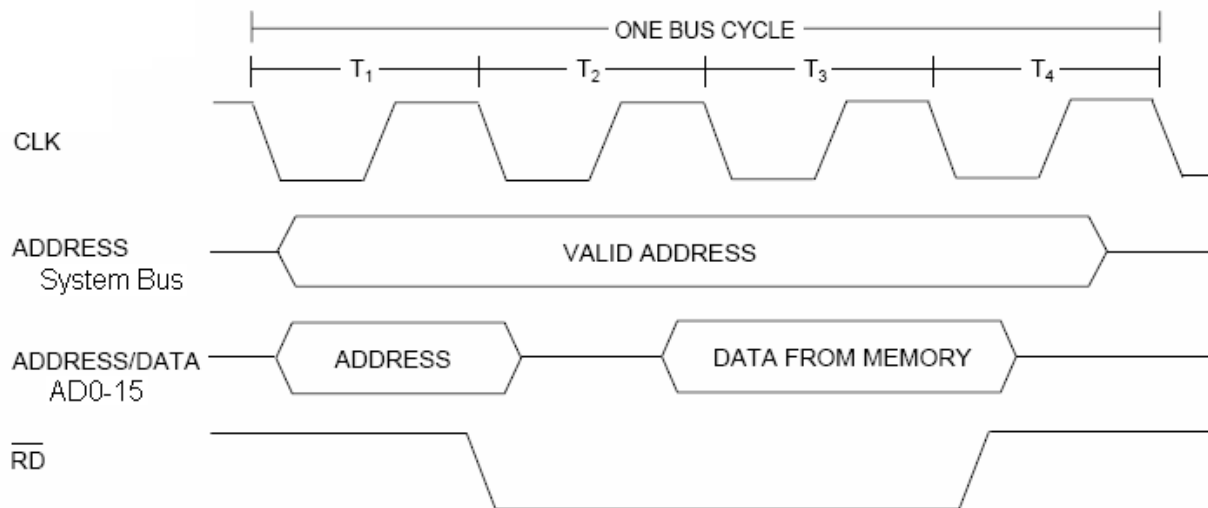


Formarea magistralelor (Bus) la 8086, mod minim



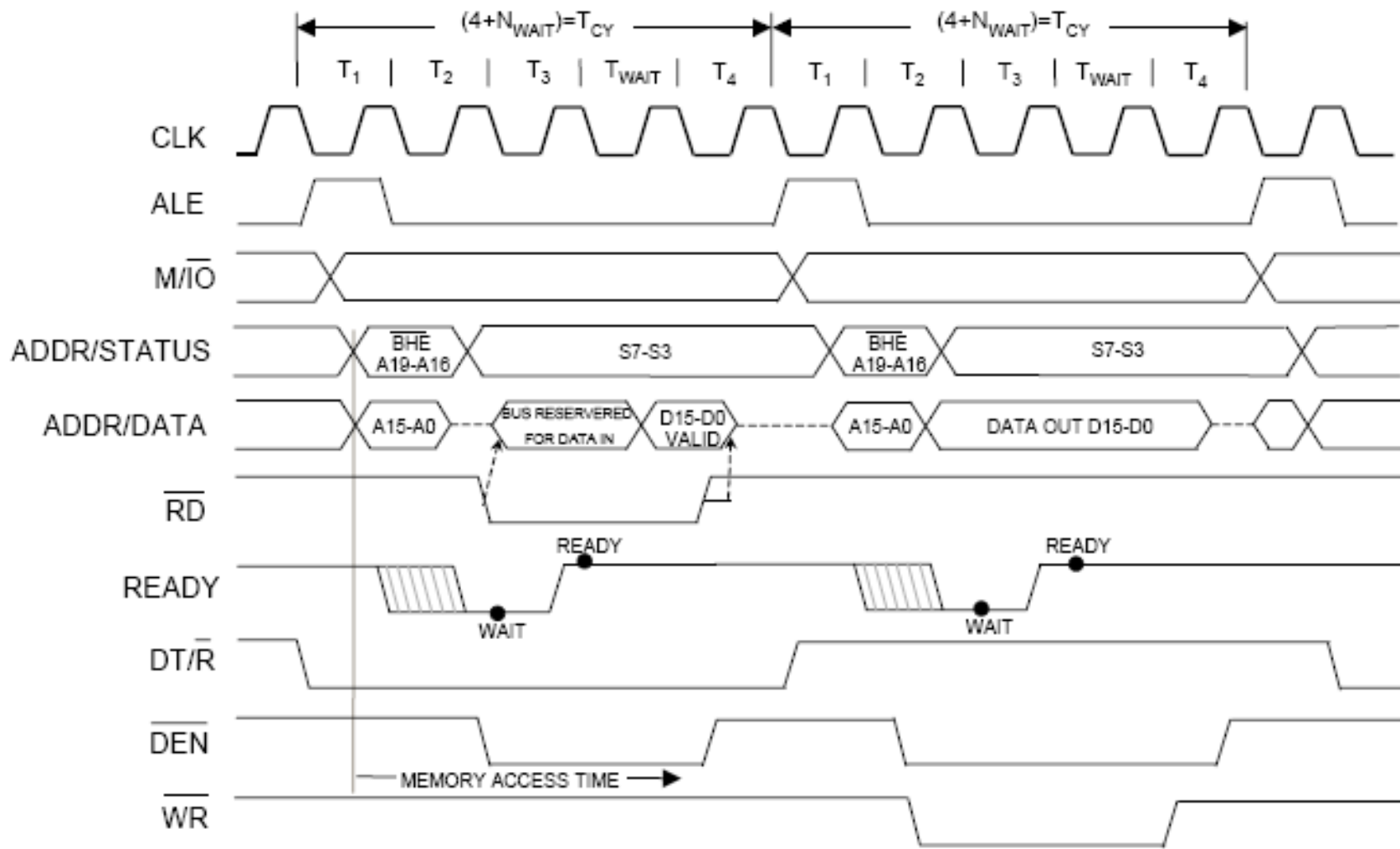


Diagrame de timp simplificate



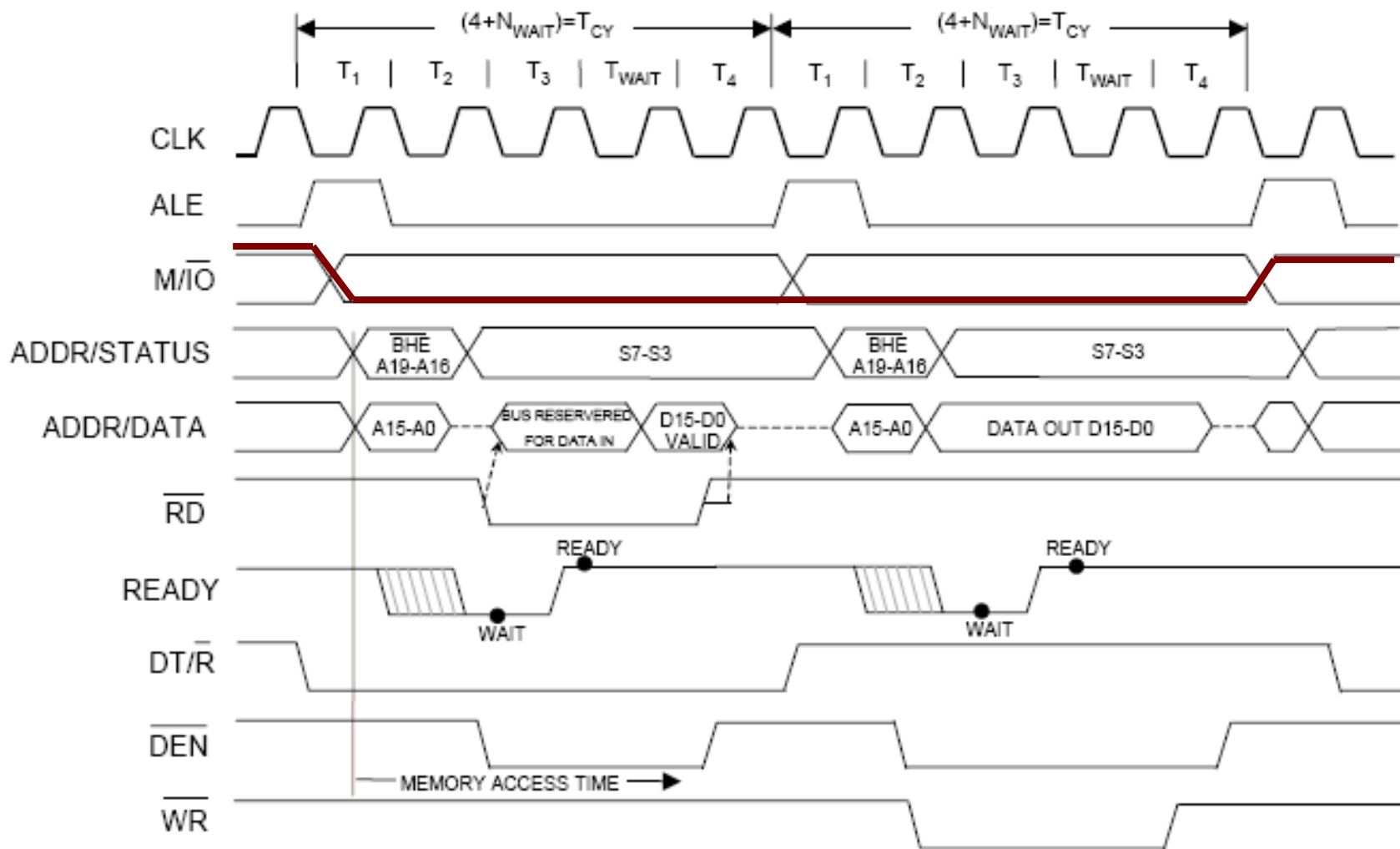


Diagrame de timp – detaliu





I/O Citire și Scriere – Diagrame de Timp



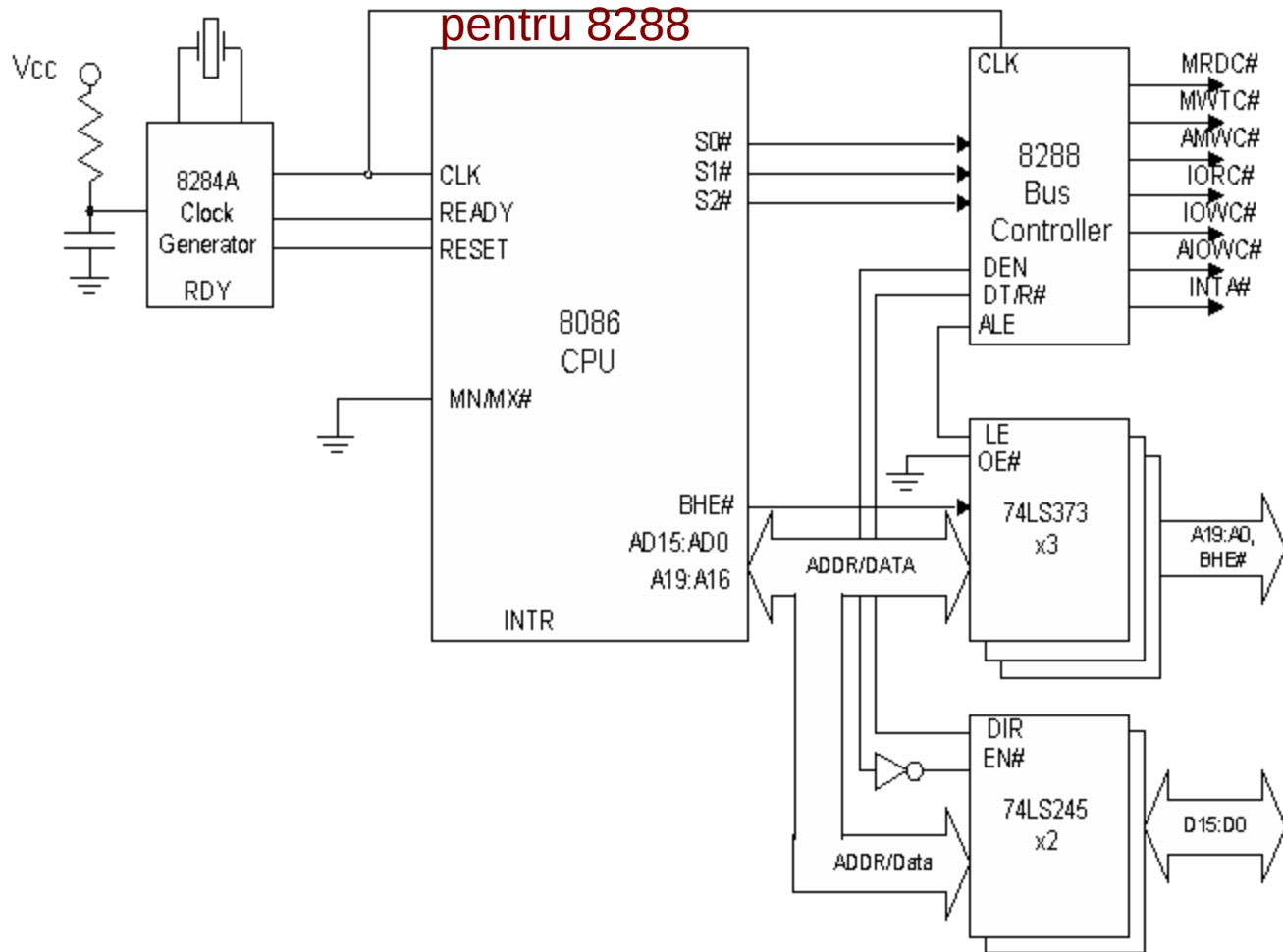
În modul minim, linia M/I0 face diferența dintre accesul la memorie sau I/O, restul semnalelor fiind identice



Sistem 8086 în mod maxim



Procesorul indică
starea, prin S2:S0,
care este comandă
pentru 8288



Comenzile pe bus
sunt generate de
controllerul 8288

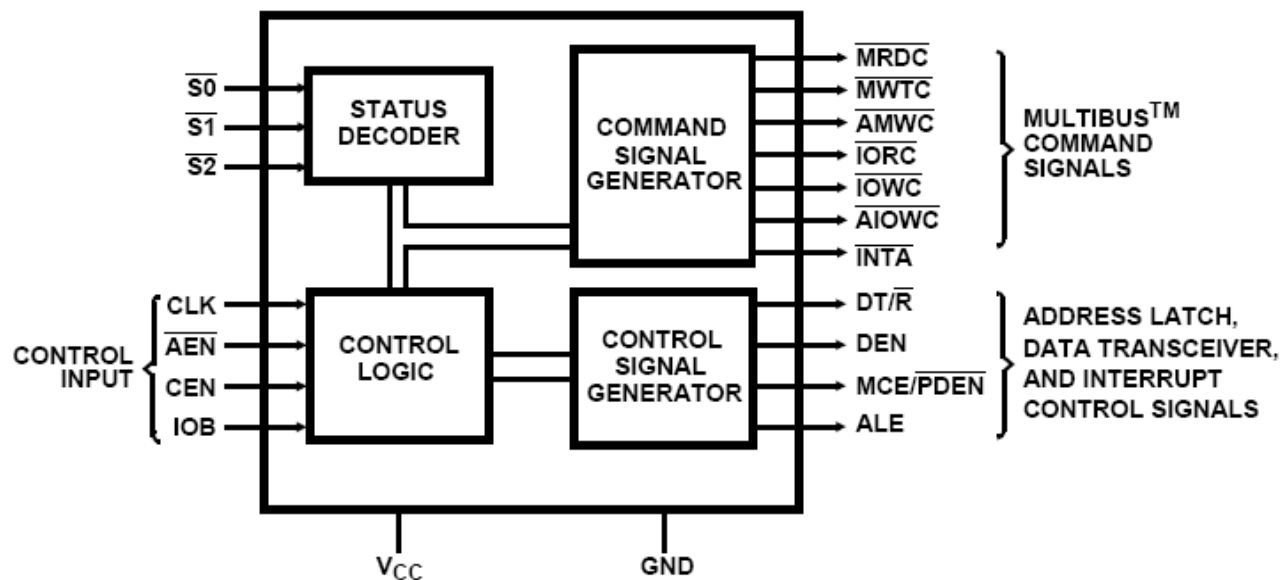
Bus de date și
adrese, similar cu
modul minim



Semnalele modului maxim

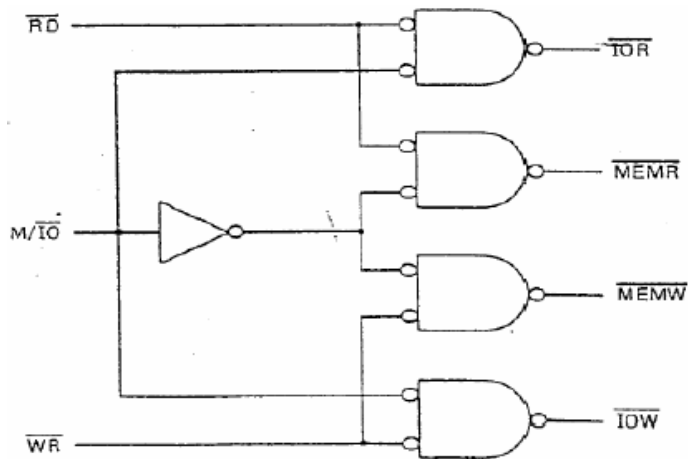


#S2	#S1	#S0	Explicație
0	0	0	Interrupt Acknowledge
0	0	1	Citire port I/O
0	1	0	Scriere port I/O
0	1	1	Halt
1	0	0	Instruction Fetch
1	0	1	Citire din memorie
1	1	0	Scriere în memorie
1	1	1	Pasiv (Nu se accesează bus-ul)

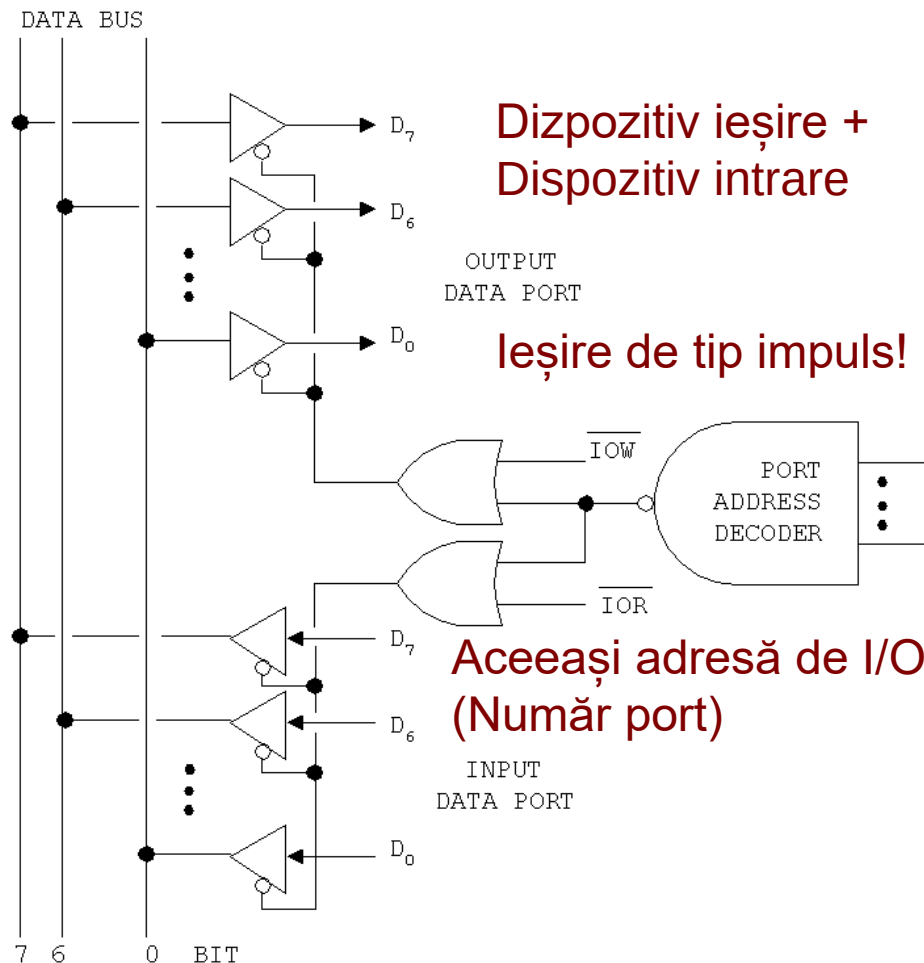


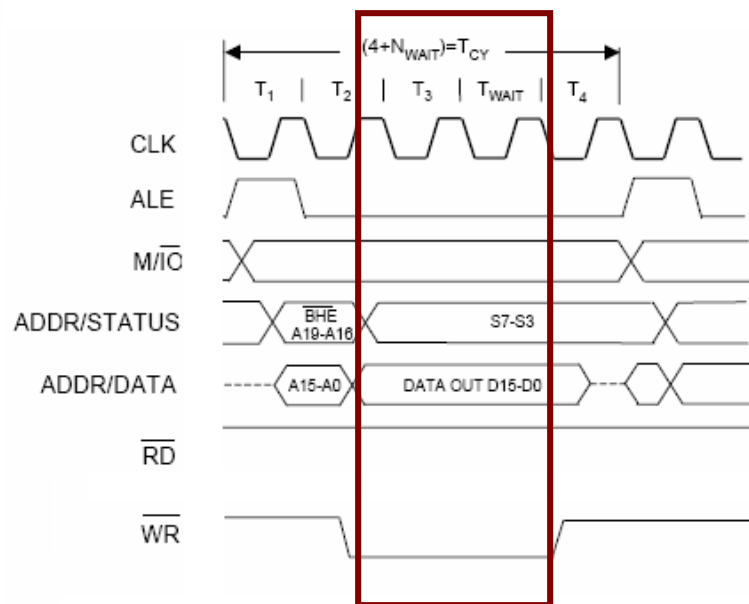
Dispozitivul este activat cand:

- Adresa transmisă de 8086 corespunde cu adresa proprie – Decodificare!
- Comanda transmisă de 8086 corespunde cu natura dispozitivului – I/O
- Două dispozitive pot avea aceeași adresă dacă sunt de tipuri diferite



Din combinația semnalelor RD, WR, și M/IO se pot genera semnale explicite pentru IOR, IOW, MEMR, MEMW, similare cu cele din modul maximal → se poate face o prezentare unitară a celor două moduri



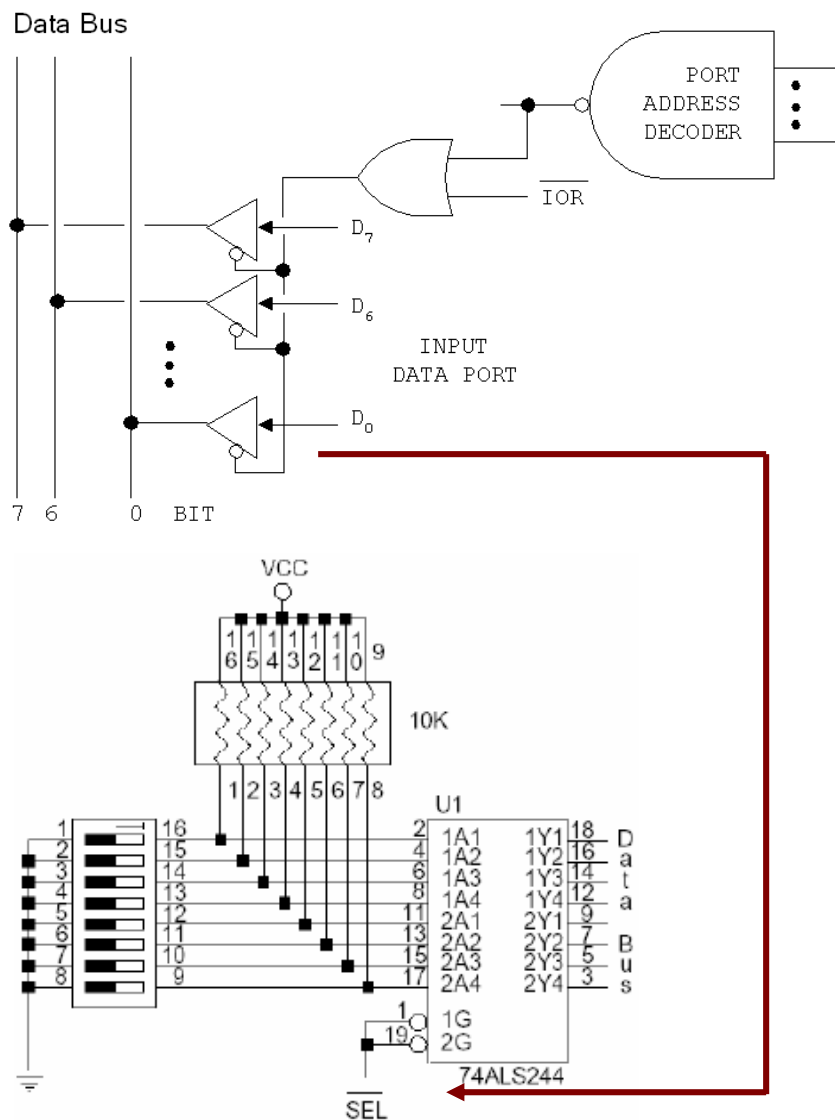


Soluția – utilizarea unui registru pentru mentinerea datelor după terminarea ciclului de bus

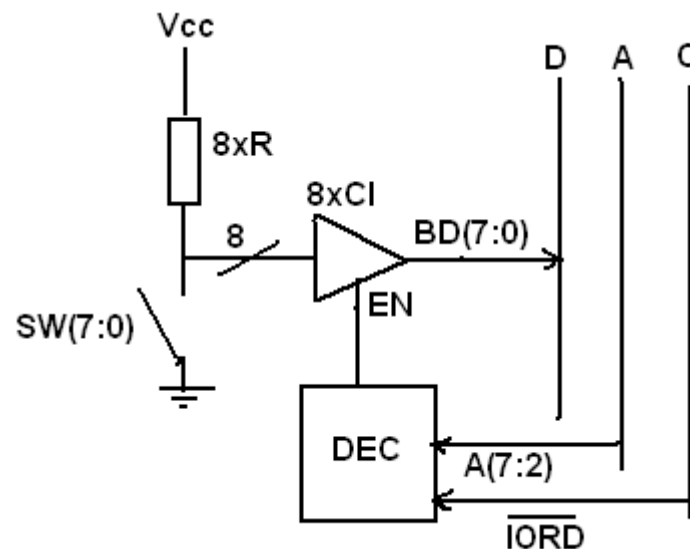
Latch sau Flip-Flop?



Problema intrării – decuplarea

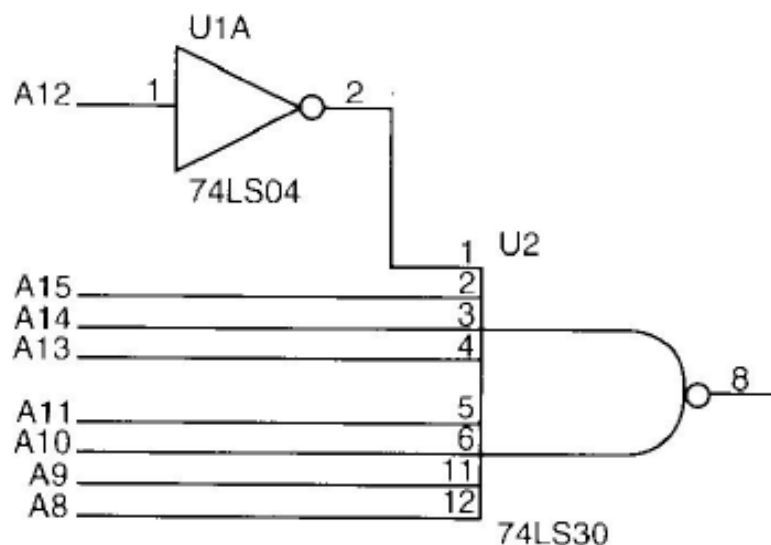


- Bus-ul de date este partajat (mai multe dispozitive impart aceleași linii de intrare)
- Un dispozitiv scrie date pe bus doar atunci cand este solicitat prin instructiuni I/O Read (in) !
- Folosire buffere 3-state
- Datele trebuie să fie stabile în momentul citirii.



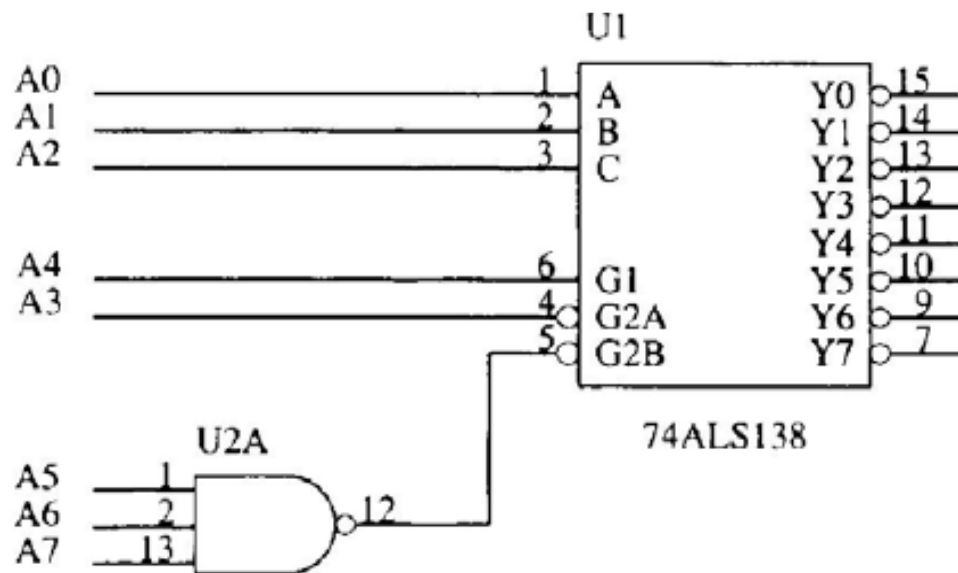


Decodificarea adreselor I/O



Decodificarea cu circuite logice discrete:
AND, OR, NAND, NOR, NOT

Adresa EFXh = 11101111XXXXXXXX



Decodificarea cu un decodicator 74LS138

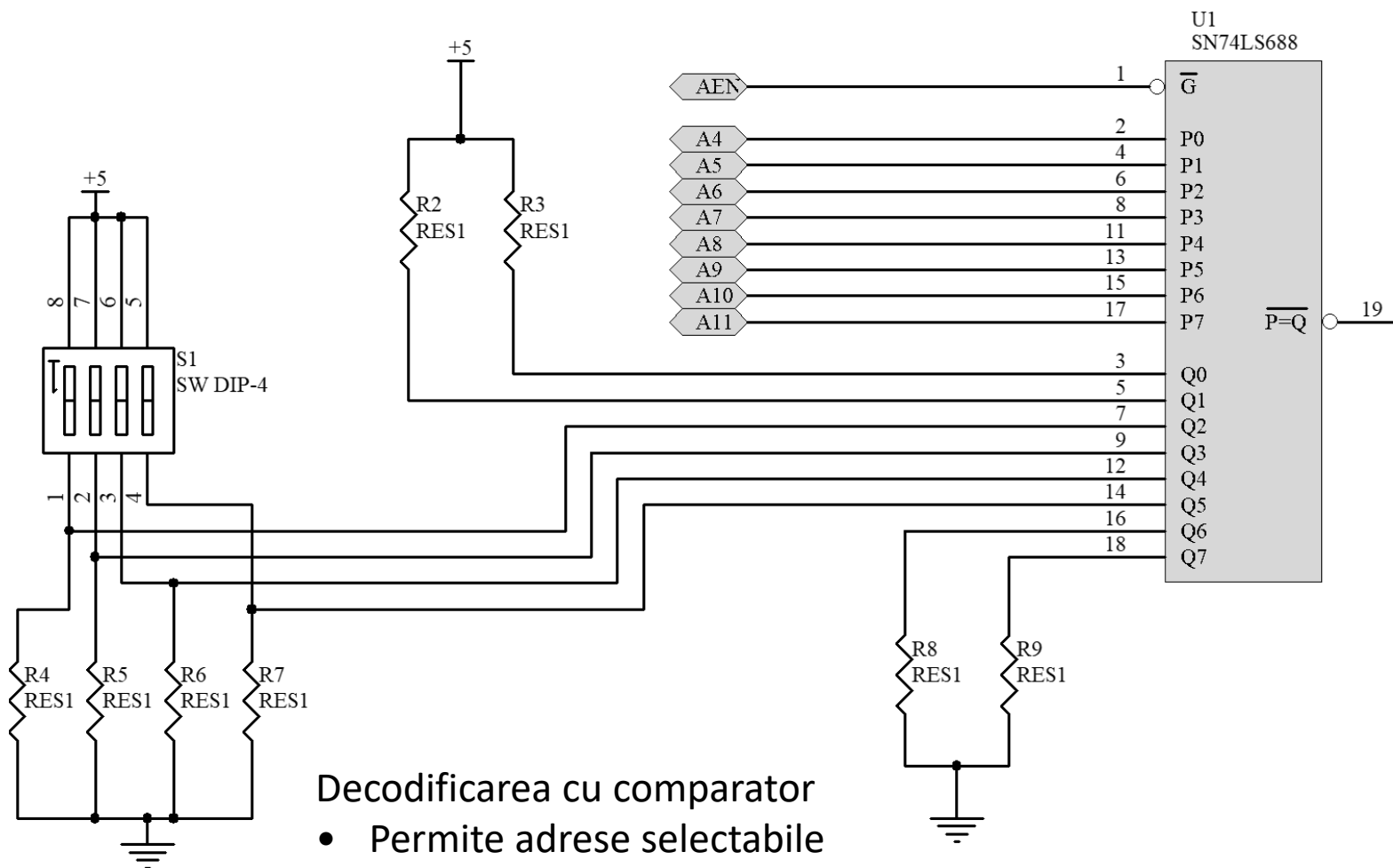
- G1, G2A, G2B – validari
- A, B, C – selecția unei linii
- Permite selectia a 8 dispozitive I/O
- A7:A5 trebuie sa fie '1'
- A3 trebuie sa fie '0'
- A4 trebuie sa fie '1'



Decodificarea adreselor I/O



A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0
0	0	Sw	sw	sw	Sw	1	1	X	X	X	X



Decodificarea cu comparator

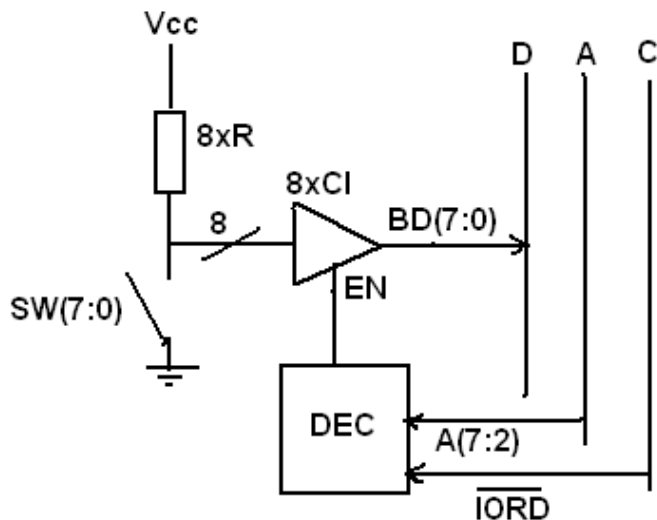
- Permite adrese selectabile
- Se poate selecta o adresă de bază între 030H – 3F0H



Acces pe Byte, acces pe Word

- Combinăția BHE/A0 se aplică oricarui transfer pe bus – memorie sau I/O

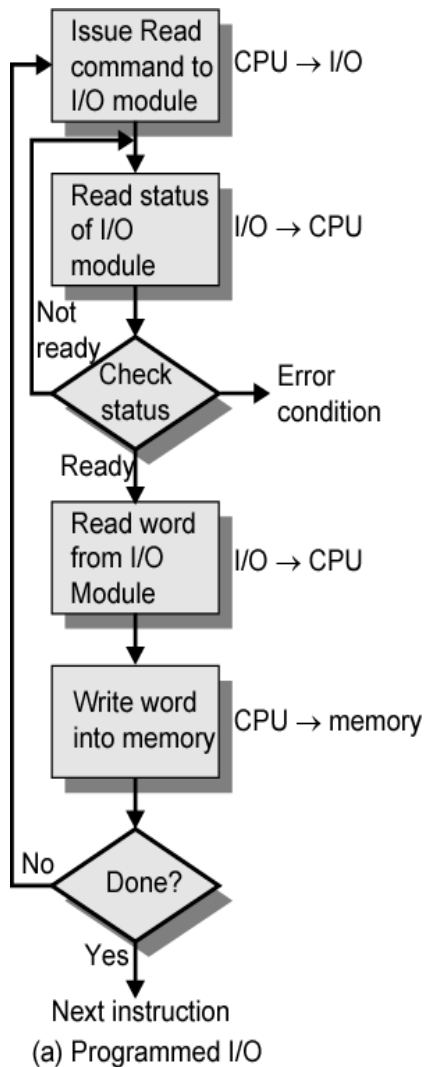
#BHE	A0	Explicatie
0	0	Acces pe 16 biți (aliniat)
0	1	Byte superior, de la adresă impară
1	0	Byte inferior, de la adresă pară
1	1	Combinăție nepermisă



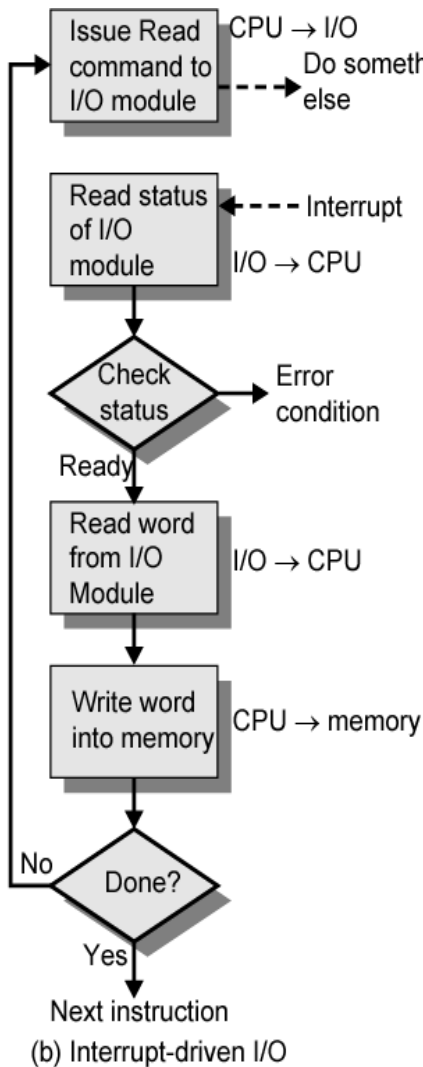
Conectarea la BD7:0 obligă A0 să fie '0'



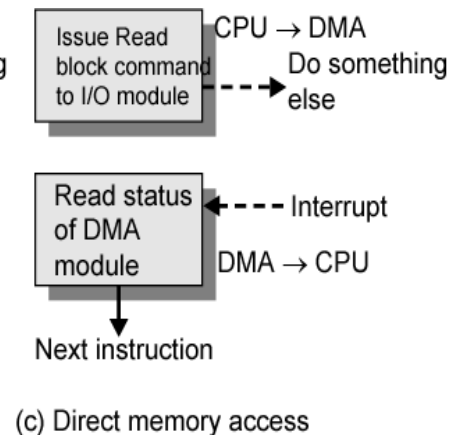
Tipuri de transfer



Programat



Întreruperi



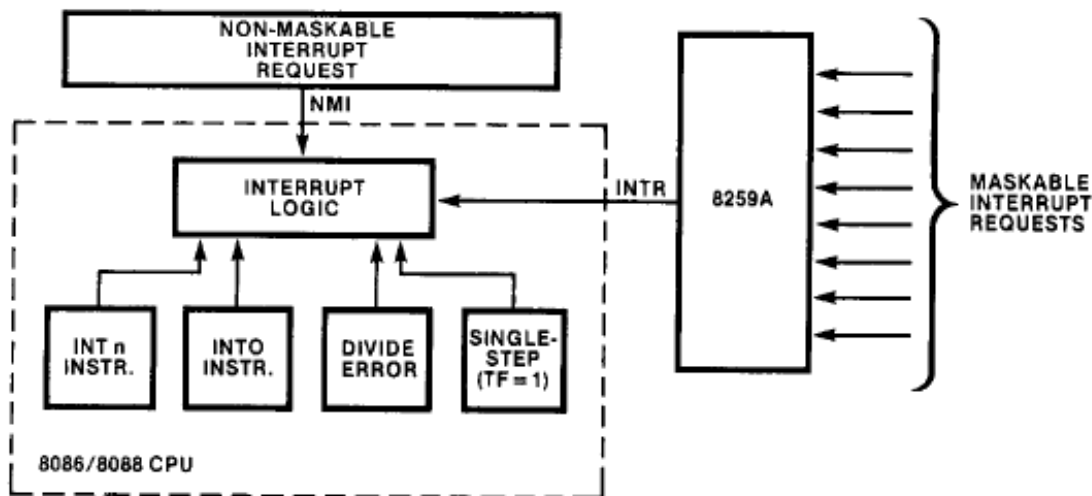
DMA



Întreruperi – Surse de întrerupere



- Inițiate prin Software – instrucțiuni INT n
- Inițiate prin Hardware
 - Excepții – semnalizare erori, (impartire cu 0,..)
 - Externe – dispozitive periferice, memorii
- Întreruperile pot fi:
 - Mascabile (Afectate de flag-ul IF)
 - Nemascabile



Prioritatea întreruperilor

INTERRUPT	PRIORITY
Divide error, INT n, INTO NMI INTR Single-step	highest lowest

Timpi

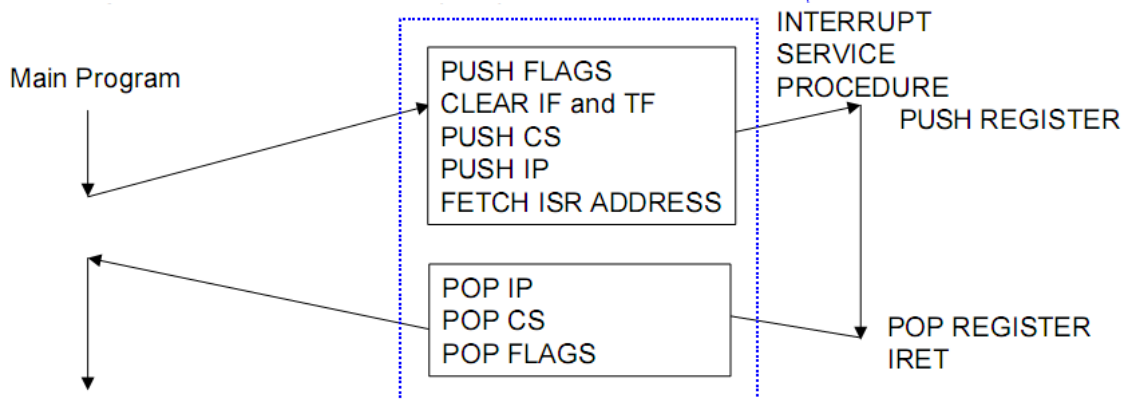
Interrupt Class	Processing Time
External Maskable Interrupt (INTR)	61 clocks
Non-Maskable Interrupt (NMI)	50 clocks
INT (with vector)	51 clocks
INT Type 3	52 clocks
INTO	53 clocks
Single Step	50 clocks



Înteruperi – Instrucțiuni ASM



Mnemonic	Meaning	Format	Operation	Flags Affected
CLI	Clear interrupt flag	CLI	$0 \rightarrow (IF)$	IF
STI	Set interrupt flag	STI	$1 \rightarrow (IF)$	IF
INT n	Type n software interrupt	INT n	$(Flags) \rightarrow ((SP - 2))$ $0 \rightarrow TF, IF$ $(CS) \rightarrow ((SP) - 4)$ $(2 + 4 \cdot n) \rightarrow (CS)$ $(IP) \rightarrow ((SP) - 6)$ $(4 \cdot n) \rightarrow (IP)$	TF, IF
IRET	Interrupt return	IRET	$((SP)) \rightarrow (IP)$ $((SP) + 2) \rightarrow (CS)$ $((SP) + 4) \rightarrow (Flags)$ $(SP) + 6 \rightarrow (SP)$	All
INTO	Interrupt on overflow	INTO	INT 4 steps	TF, IF
HLT	Halt	HLT	Wait for an external Interrupt or reset to occur	None
WAIT	Wait	WAIT	Wait for $\overline{TEST}$ input to go active	None





Vectori de Întrerupere

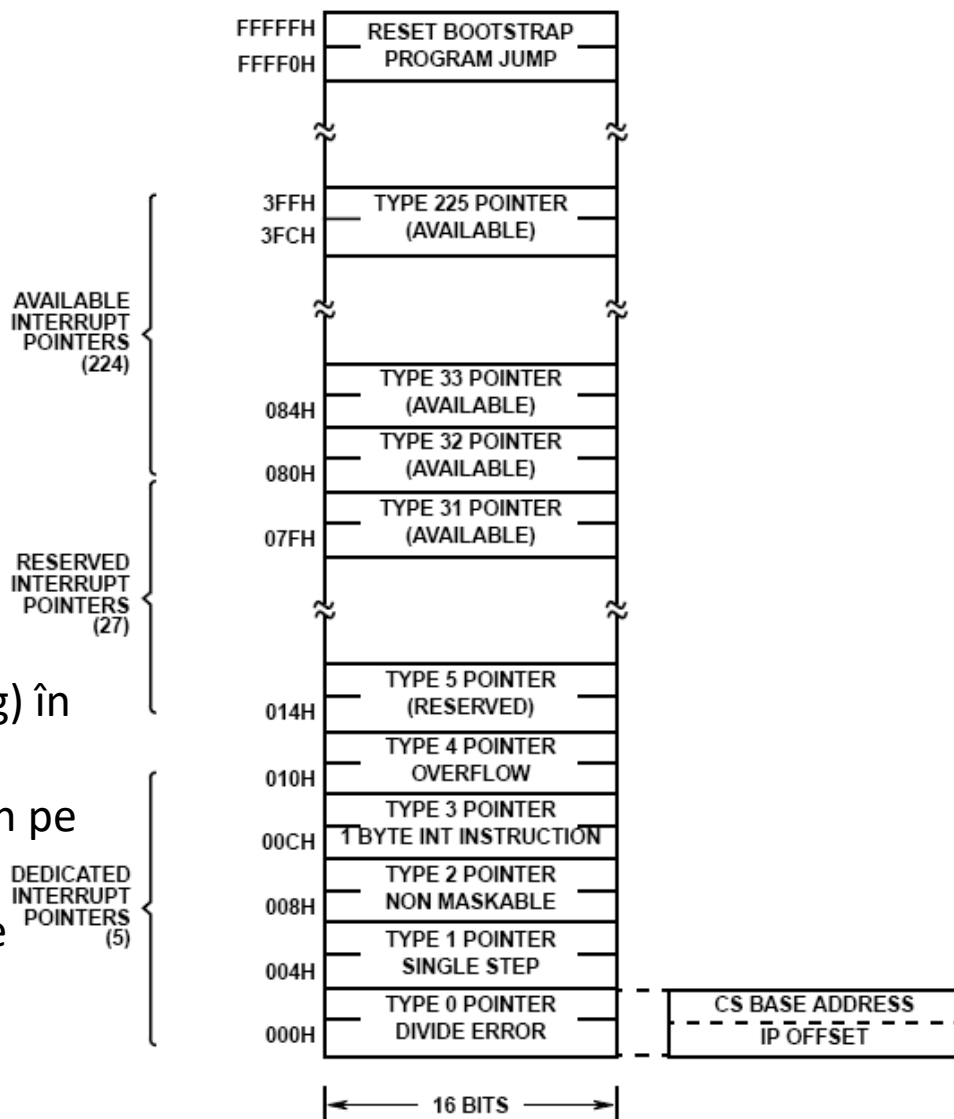


Tabela de vectori se află în RAM

- adresele de salt pentru fiecare tip de întrerupere se pot configura de către programator

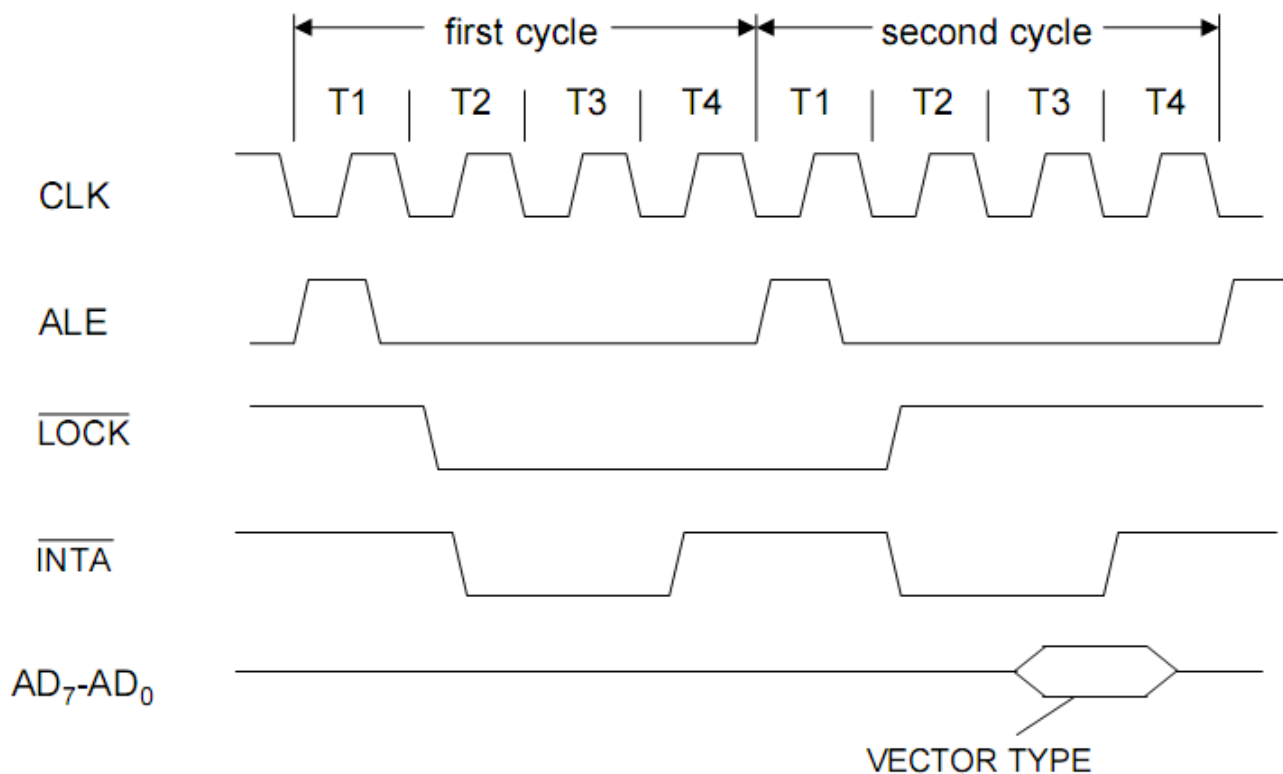
Tratarea întreruperilor

- Mascare – bitul IF (interrupt enable flag) în registrul de stare
- INTR – level triggered, sincronizat intern pe CLK ↑
- Durata INTR High – inclusiv perioada de ceas înainte de terminarea instrucțiunii curente





Ciclul INTA pentru întreruperi externe mascabile

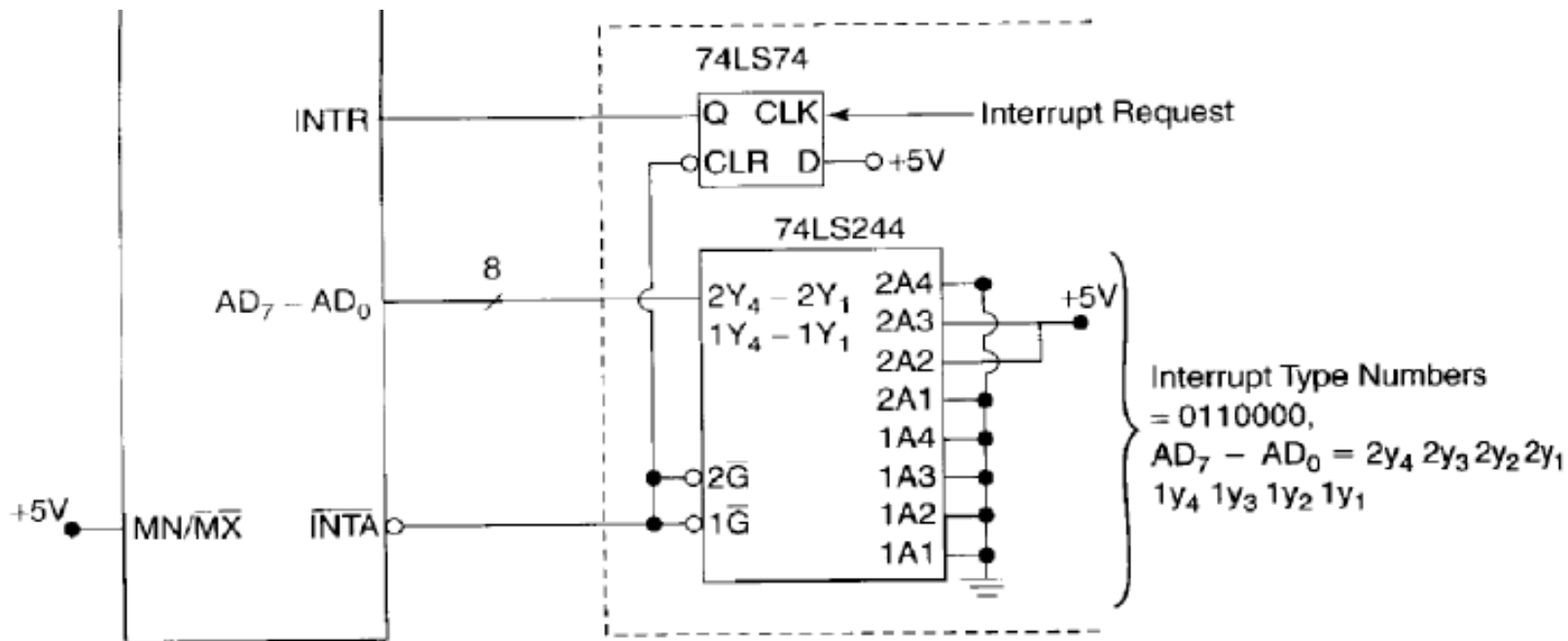


Un dispozitiv care generează întreruperi trebuie să fie capabil să:

- Scribe pe BUS-ul de date tipul intreruperii (indexul in tabela de vectori)
- Să dezactiveze INTR cand primește confirmarea INTA



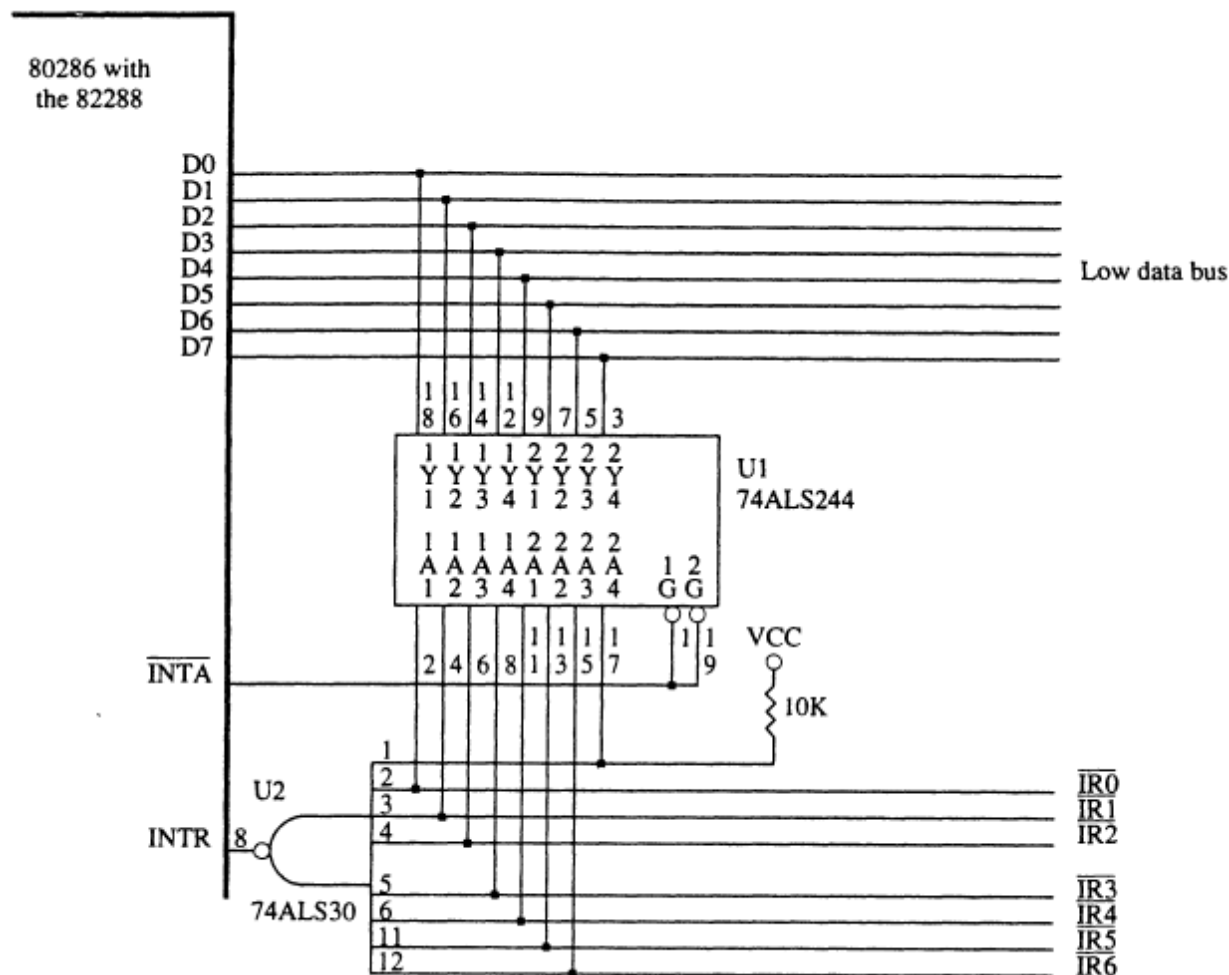
Întreruperi – Conectare dispozitive simple



- Generarea tipului de **întrerupere 60h**, când linia "Interrupt Request" este activă
- INTA produce scrierea tipului de întrerupere pe BUS, și ștergerea bistabilului care produce INTR
- 74LS244 – 3-STATE Buffer/Line Driver/Line Receiver

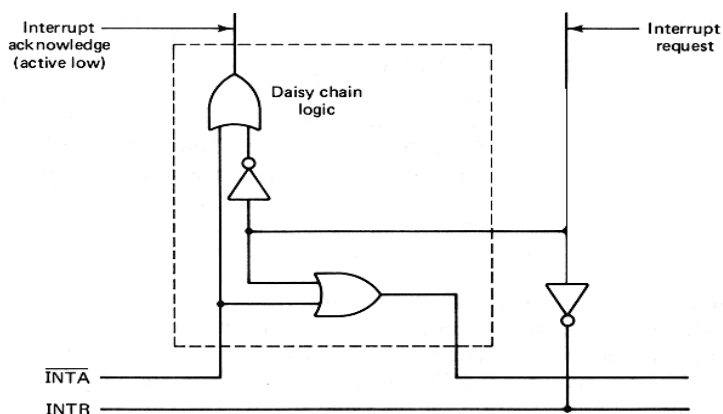
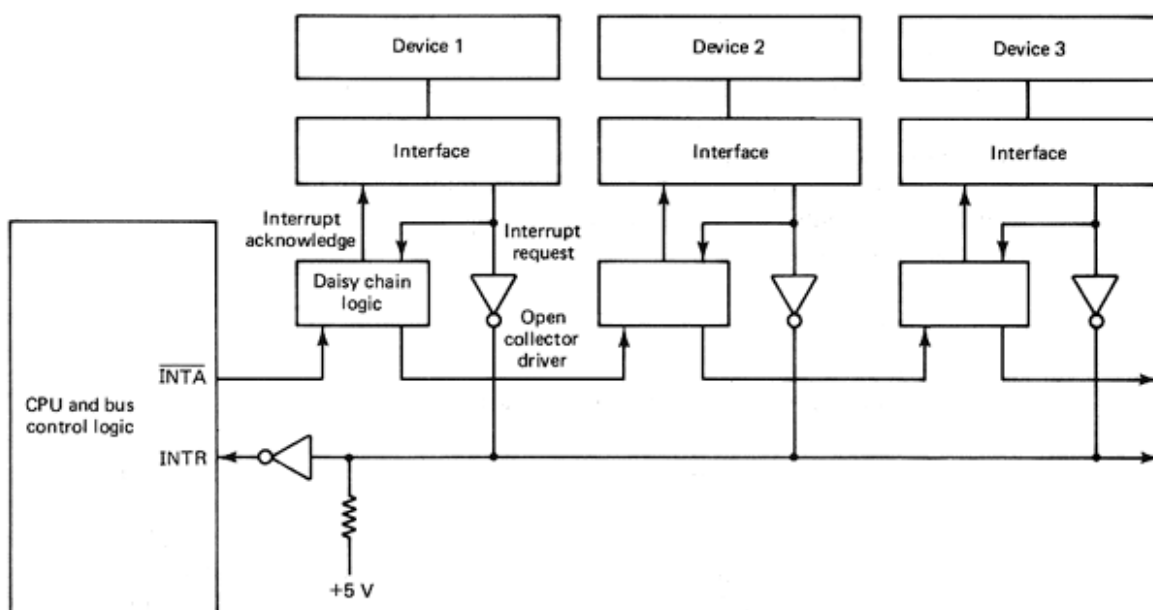


Întreruperi – Conectare dispozitive simple



Generare de multiple tipuri de întrerupere, în funcție de dispozitiv

- **Sondare (Polling)** – la apariția unei cereri de întrerupere, procesorul interoghează fiecare sursă potențială, iar aceasta raspunde. Ordinea de interogare a dispozitivelor este ordinea priorității întreruperilor.
- **Daisy Chain** – implementarea metodei polling in hardware. Limitare la lungimea lanțului din cauza întârzierilor de propagare

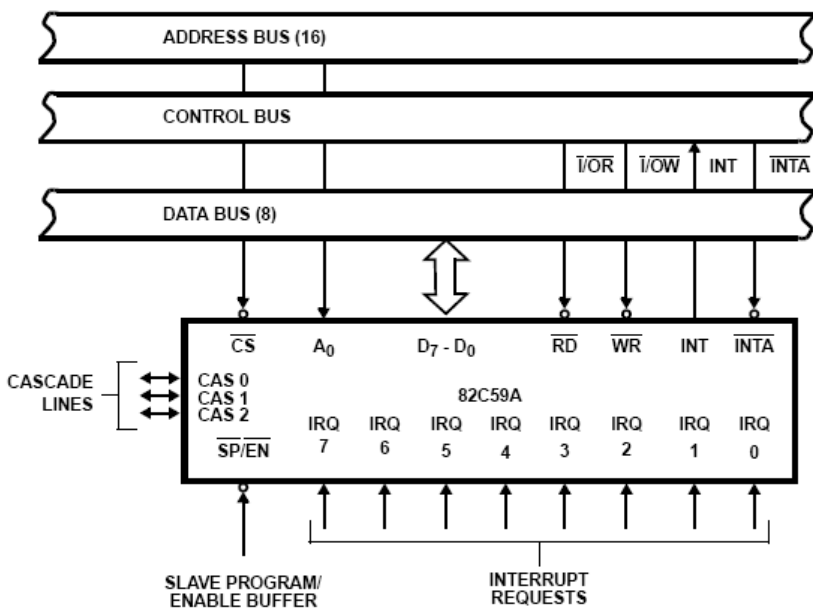
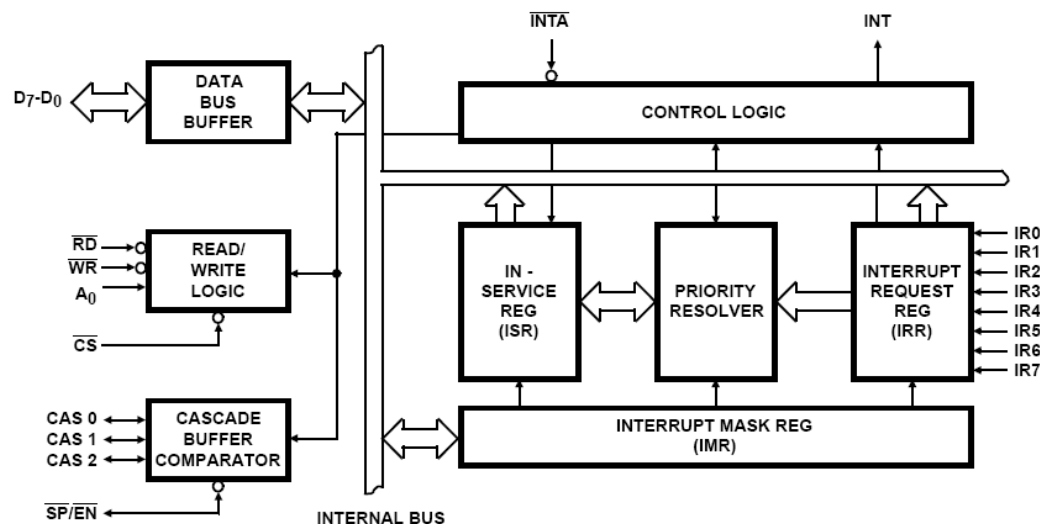




Controller de întreruperi 8259A



- 8 linii de cerere de întrerupere
- Prioritățile sunt programabile
- Se pot cascada – 8 slaves, 64 cereri de intrerupere in total





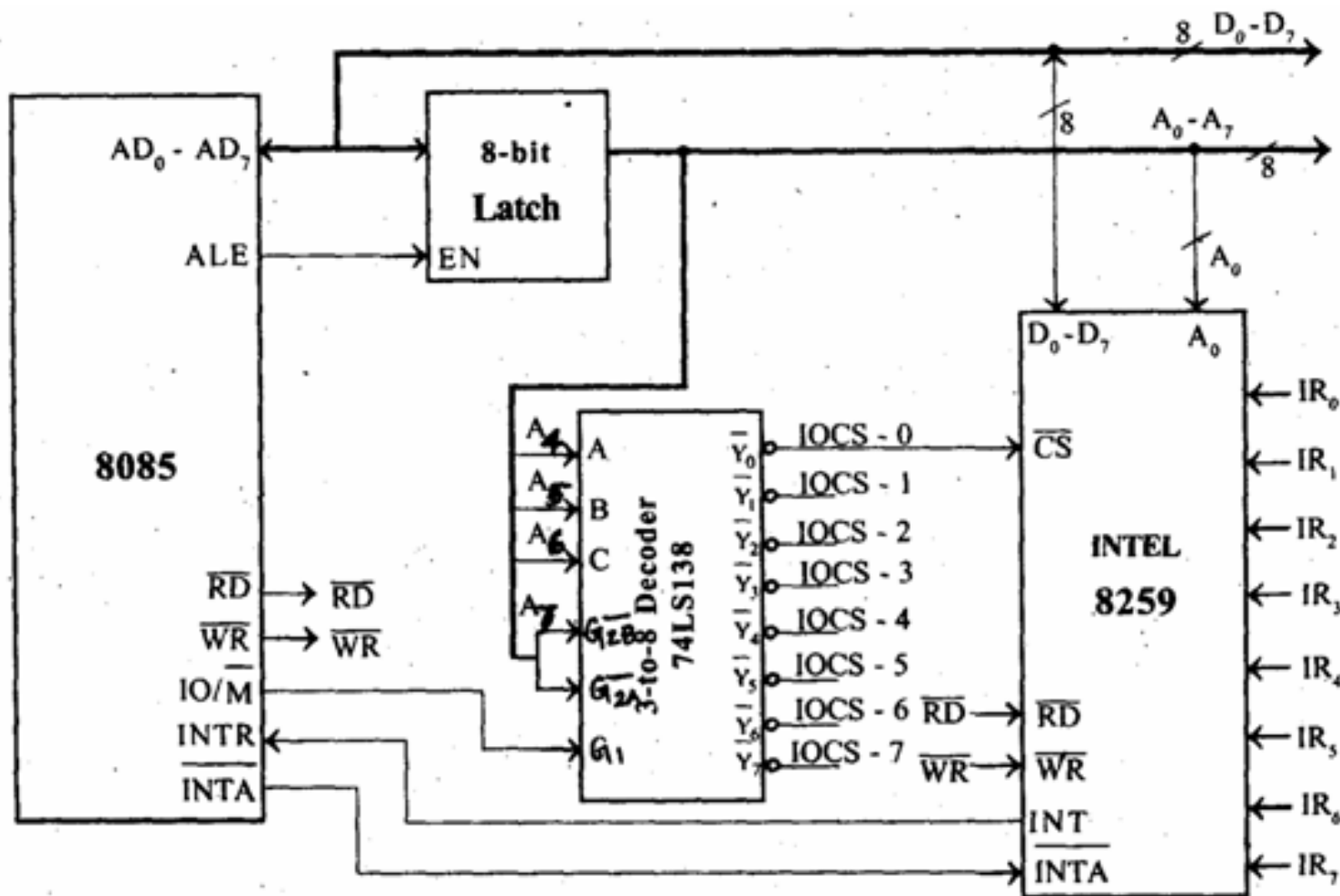
Controller de întreruperi 8259A



- PIC 8259 informează procesorul despre cererea de întrerupere, activând pinul INTR al procesorului
- Procesorul termină execuția instrucțiunii curente
- Procesorul trimite semnalul de Acknowledgment (INTA) către PIC 8259
- PIC 8259 transmite procesorului numărul vectorului pentru întreruperea cerută
- Procesorul folosește acest vector pentru a determina adresa unde este stocată rutina de tratare a întreruperii (ISR)
- Procesorul salvează pe stivă flagurile, CS și IP (în aceasta ordine).
- Procesorul pune IF pe zero.
- Procesorul setează CS:IP la adresa ISR și începe execuția acestei subrutine



Controller de întreruperi 8259A





Referințe



1. 8086/8088 datasheet
2. 8259 datasheet



Proiectarea cu Micro-Procesoare

Lector: Mihai Negru

An 3 – Calculatoare și Tehnologia Informației

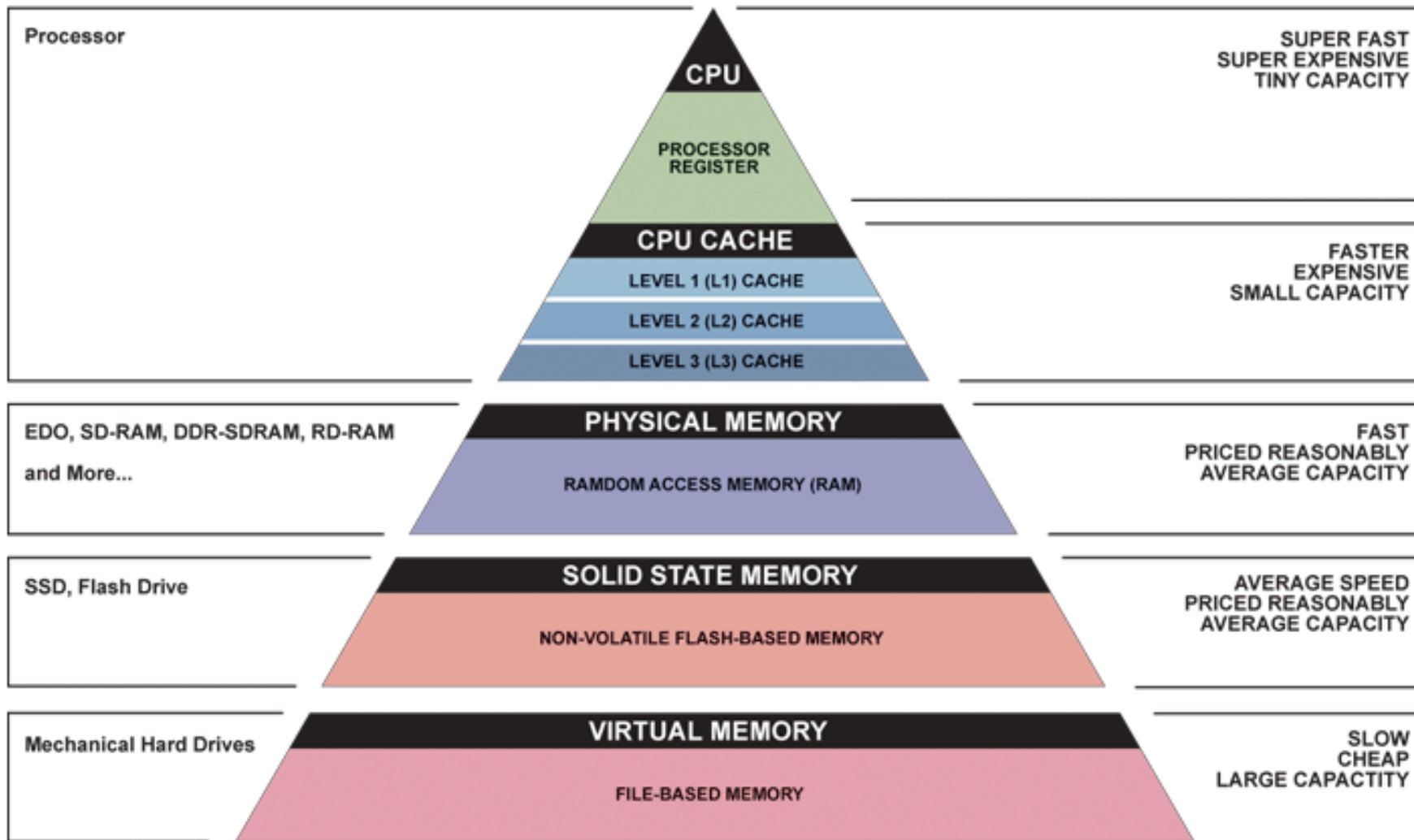
Seria B

Curs 11: Interfațarea Memoriei

<http://users.utcluj.ro/~negrum/>



Ierarhia Tipică a Memoriei



▲ Simplified Computer Memory Hierarchy
Illustration: Ryan J. Leng



Clasificarea Memoriilor



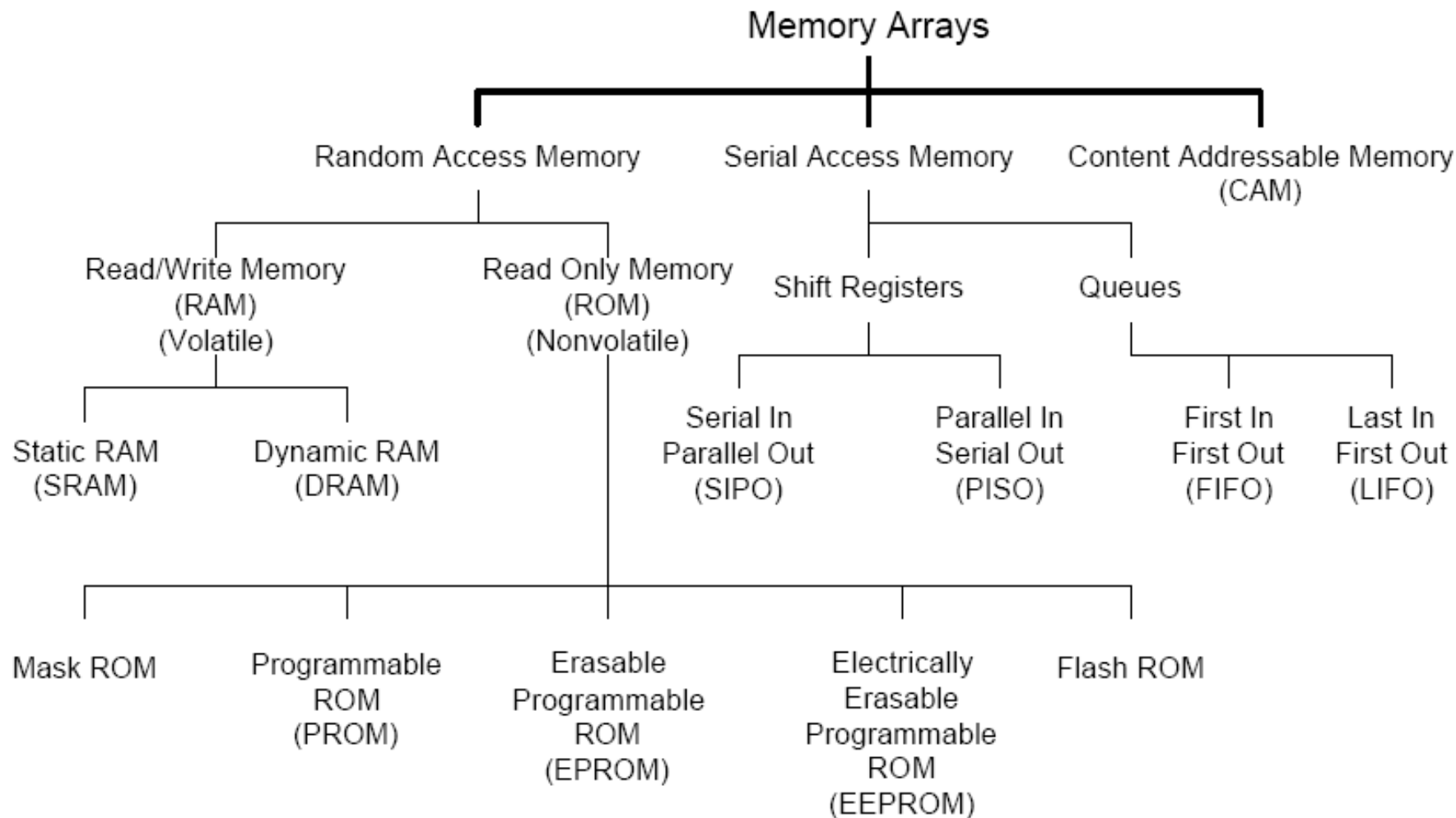
Read Write (RWM)		NVRWM	ROM
Random Access	Non-Random Access	EPROM	Mask-prog. ROM
SRAM (cache, regiștri) DRAM (memoria principală)	FIFO, LIFO Registru de deplasare	EEPROM FLASH	Electrically- prog. PROM

- Memorie statică / Memorie dinamică
- Memorie volatilă / Memorie nonvolatilă (NV)
- Read only (ROM) / Random Access (RAM)

Modul de acces – aleator, serial, adresabil prin continut

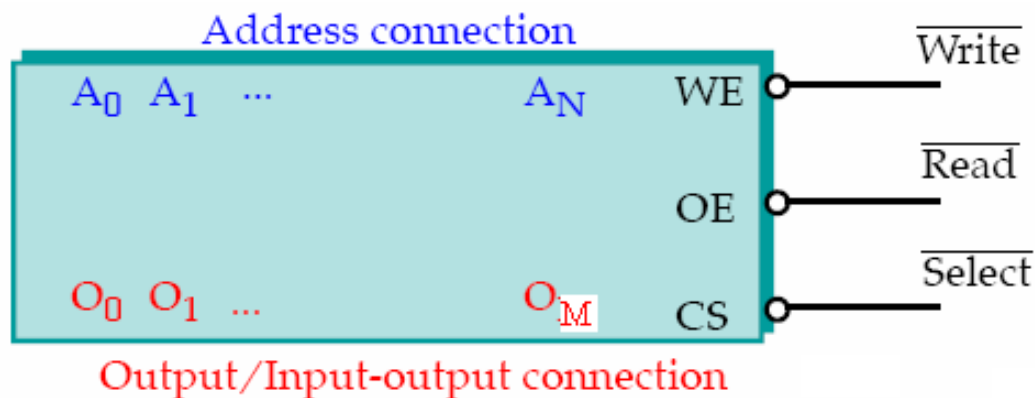


Clasificarea Memoriilor





Configurația generală a pinilor



- **Pini de adresă**

- $A_0 \dots A_N \Rightarrow$ numărul locațiilor de memorie = 2^{N+1}

- **Pini de date**

- $M \Rightarrow$ dimensiunea locațiilor de memorie
- Bidirecționali, ieșire 3-state (#OE)

- **Pini de control**

- Chip select/enable $\Rightarrow$ activează dispozitivul
- Read (#OE) / Write (#WE) $\Rightarrow$ selecția operației (citire / scriere)

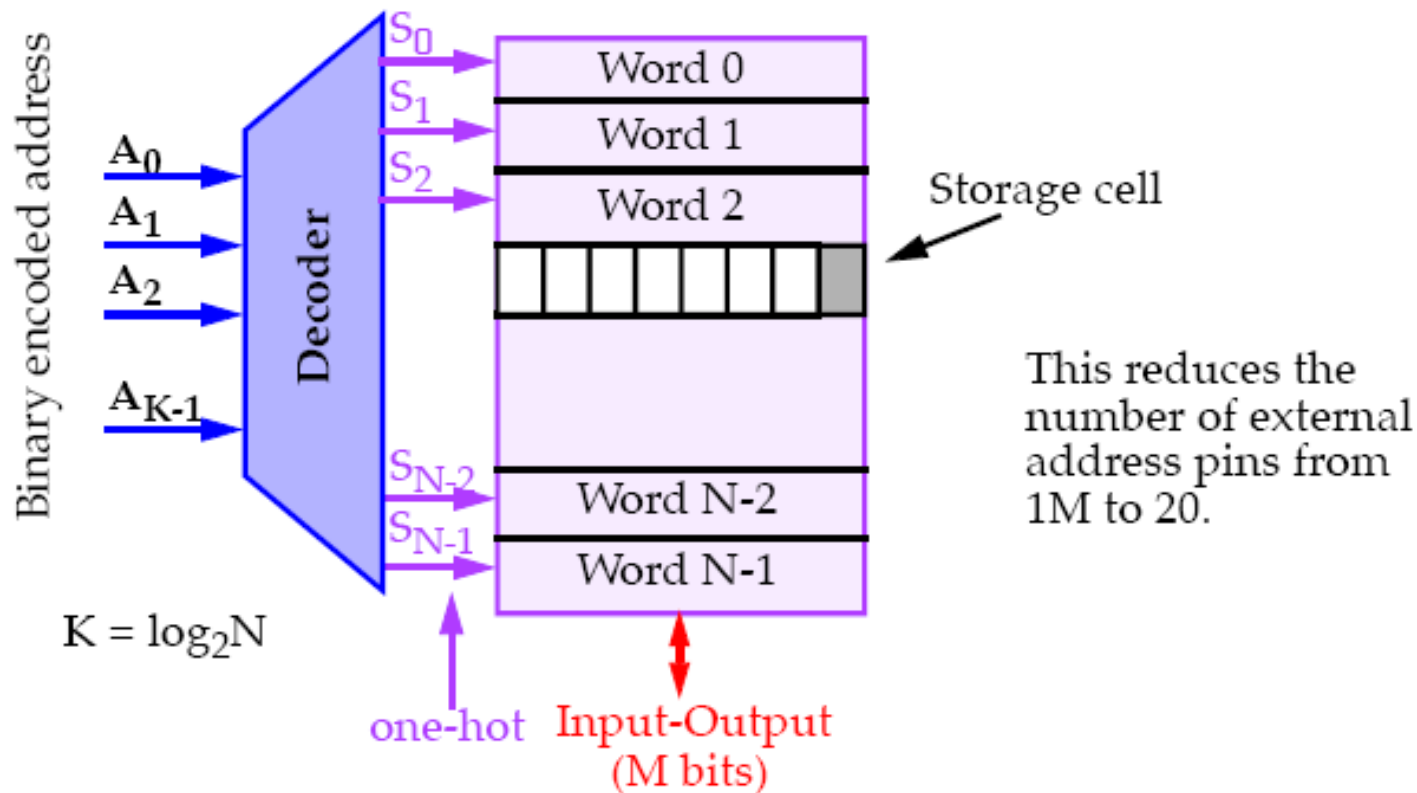


Arhitectura memoriei



Memory Architecture

Add a decoder to solve the package problem:

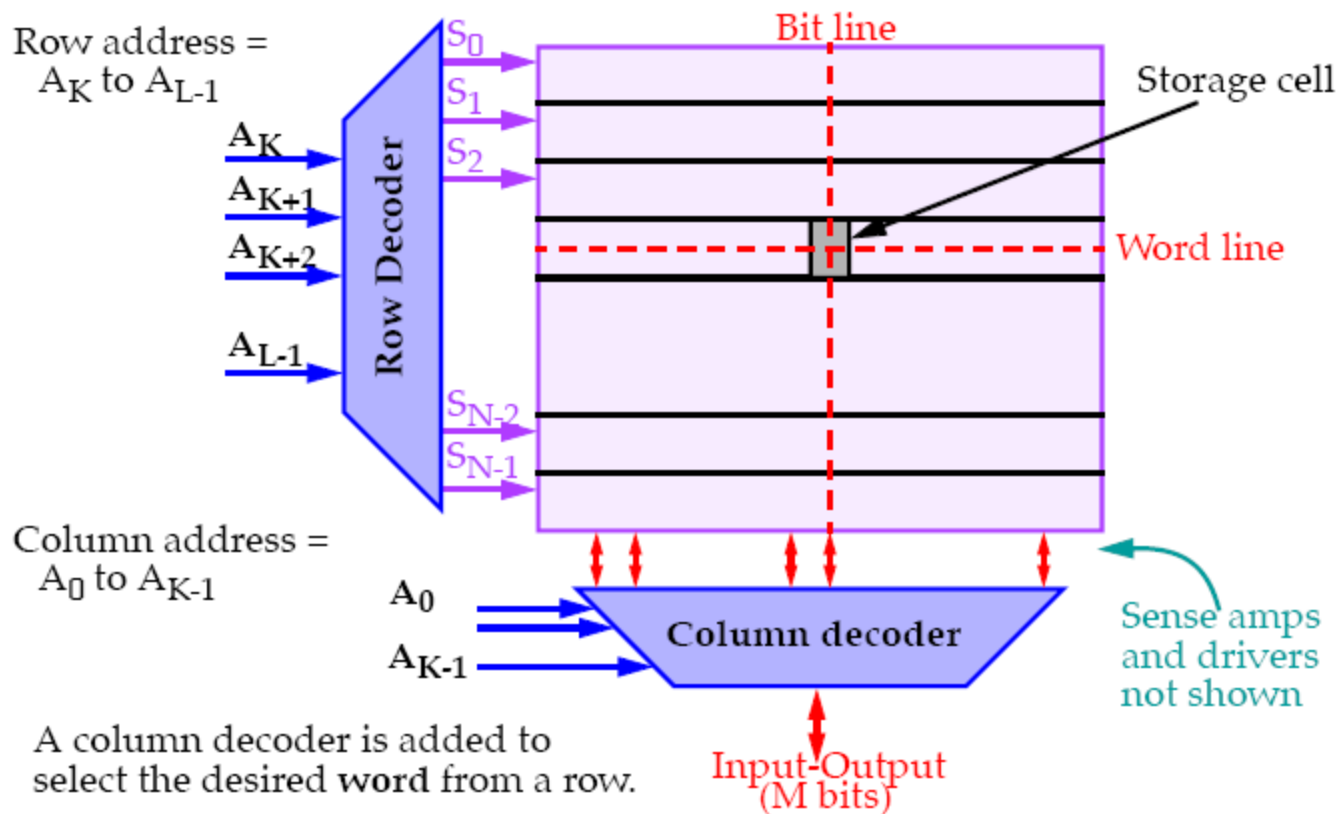


Nu se rezolvă problema raportului dintre lăţimea şi lungimea matricii

Această organizare este foarte lentă, deoarece firele verticale sunt foarte lungi



Arhitectura memoriei



Dimensiunile verticală și orizontală sunt de obicei similare

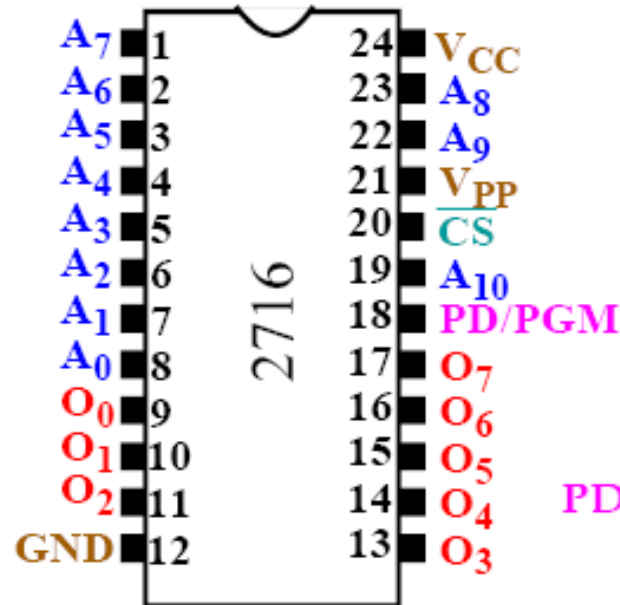
Mai multe cuvinte sunt stocate pe aceeași linie



ROM / EPROM



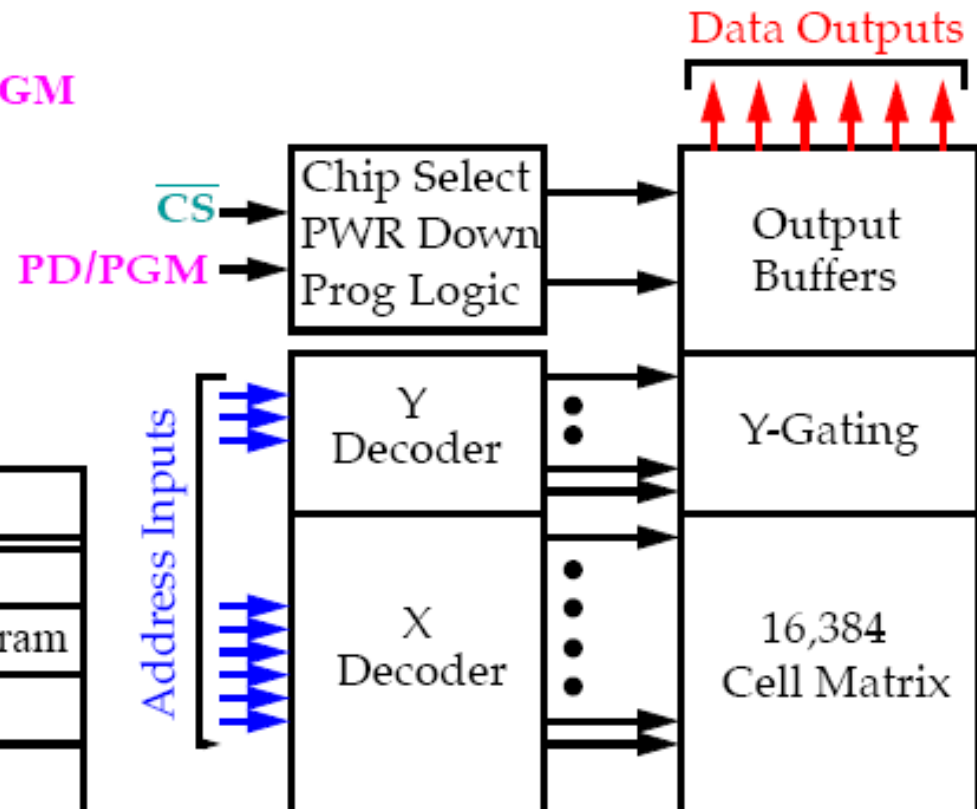
Intel 2716 EPROM (2K X 8):



2K x 8 EPROM

Pin(s)	Function
A_0-A_{10}	Address
PD/PGM	Power down/Program
$\overline{CS}$	Chip Select
O_0-O_7	Outputs

V_{PP} is used to program the device by applying 25V and pulsing PGM while holding $\overline{CS}$ high.

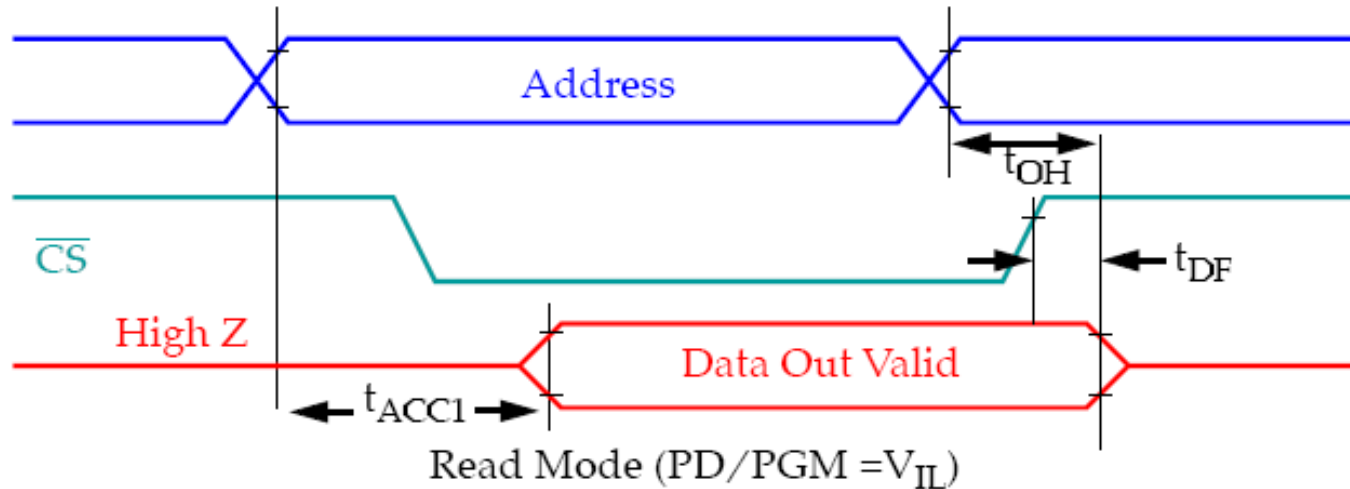




ROM / EPROM



2716 Timing diagram:



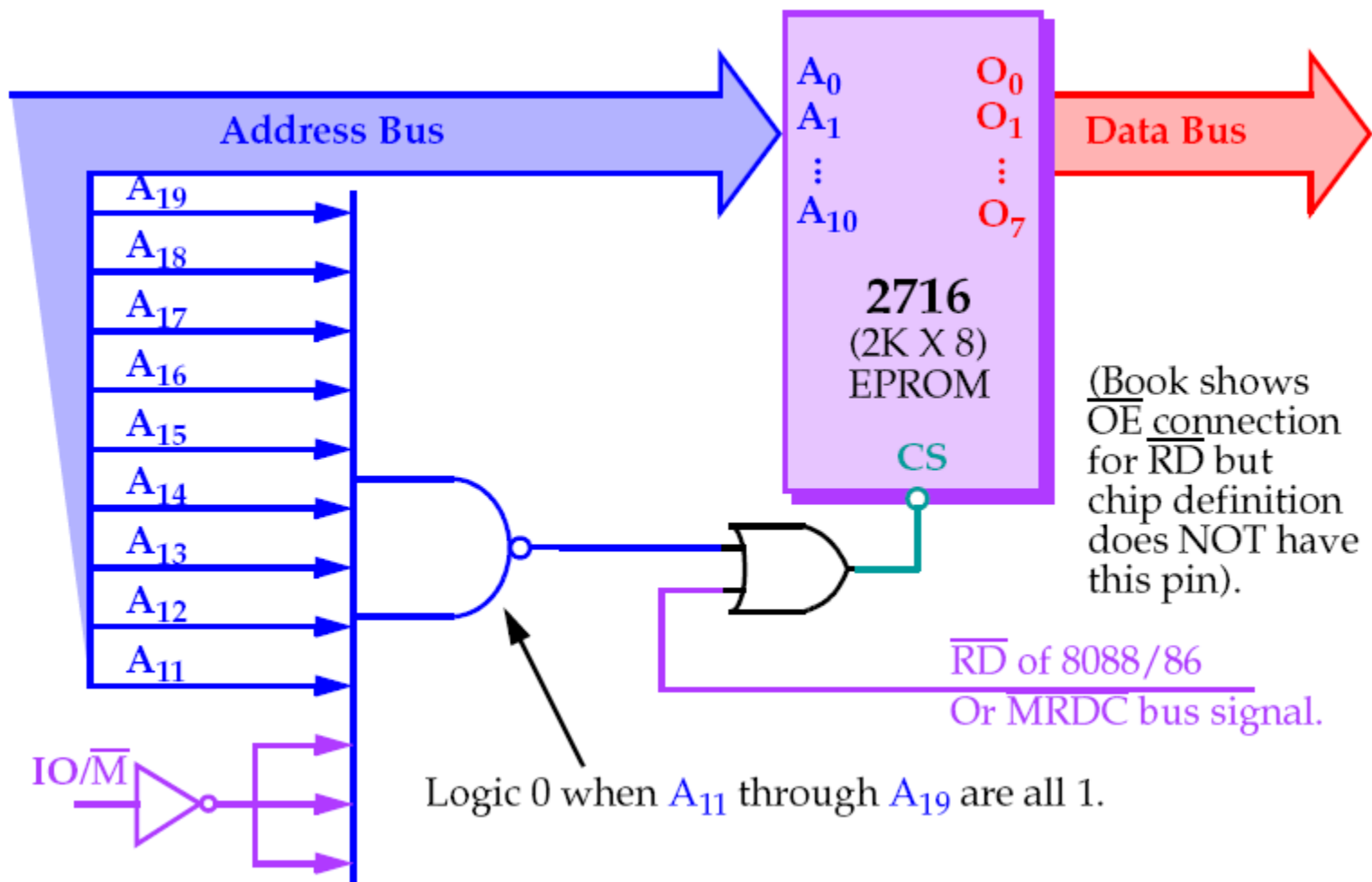
Sample of the data sheet for the 2716 A.C. Characteristics.

Symbol	Parameter	Limits			Unit	Test Condition
		Min	Typ.	Max		
t_{ACC1}	Addr. to Output Delay		250	450	ns	PD/PGM= $\overline{CS} = V_{IL}$
t_{OH}	Addr. to Output Hold	0			ns	PD/PGM= $CS = V_{IL}$
t_{DF}	Chip Deselect to Output Float	0		100	ns	PD/PGM= V_{IL}
...	...	...	...	...	...	...

This EPROM requires a wait state for use with the 8086 (460ns constraint).



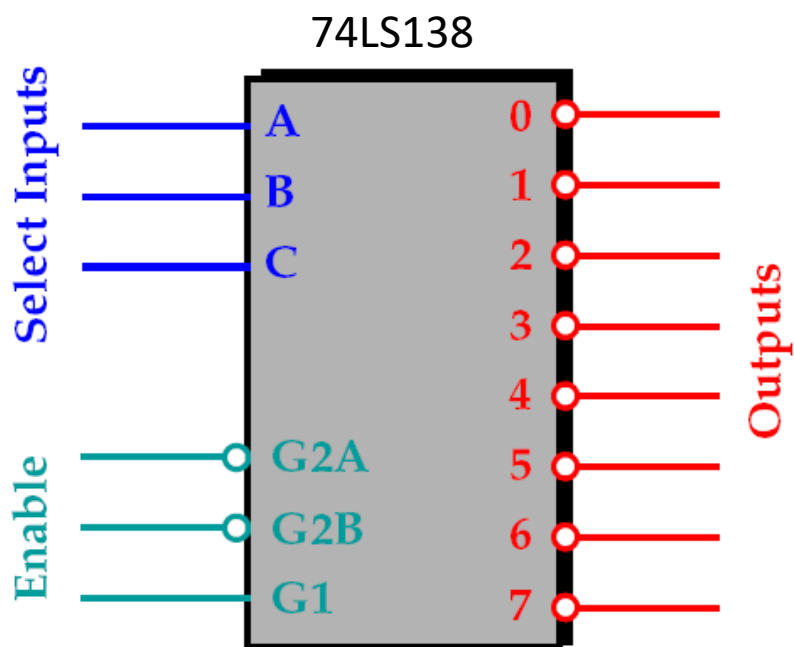
ROM / EPROM



Domeniul de adrese: FF800h - FFFFFh



Decodificarea adreselor



Inputs						Output							
Enable			Select										
G2A	G2B	G1	C	B	A	0	1	2	3	4	5	6	7
1	X	X	X	X	X	1	1	1	1	1	1	1	1
X	1	X	X	X	X	1	1	1	1	1	1	1	1
X	X	0	X	X	X	1	1	1	1	1	1	1	1
0	0	1	0	0	0	0	1	1	1	1	1	1	1
0	0	1	0	0	1	1	0	1	1	1	1	1	1
0	0	1	0	1	0	1	1	0	1	1	1	1	1
0	0	1	0	1	1	1	1	1	0	1	1	1	1
0	0	1	1	0	0	1	1	1	1	0	1	1	1
0	0	1	1	0	1	1	1	1	1	1	0	1	1
0	0	1	1	1	0	1	1	1	1	1	1	0	1
0	0	1	1	1	1	1	1	1	1	1	1	1	0

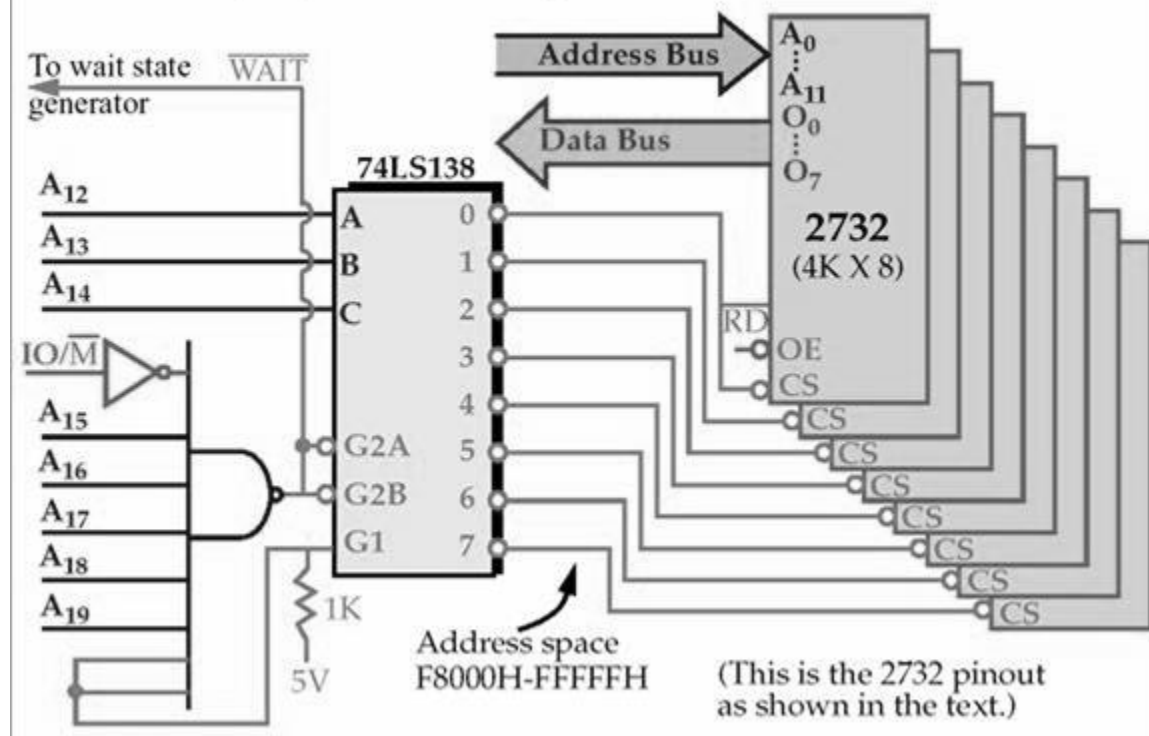
Note that all *three* Enables (G2A, G2B, and G1) must be active, e.g. low, low and high, respectively.



ROM / EPROM



8088 and 80188 (8-bit) EPROM Memory Interface



8088 – mod minim

EPROM $8 \times 2732 = 8 \times 4\text{kB} = 32 \text{ KB}$

EPROM	Density (bits)	Capacity (bytes)
2716	16K	2K × 8
2732	32K	4K × 8
27C64	64K	8K × 8
27C128	128K	16K × 8
27C256	256K	32K × 8
27C512	512K	64K × 8
27C010	1M	128K × 8
27C020	2M	256K × 8
27C040	4M	512K × 8

Organizarea adreselor:

0: F8000- F8FFF;

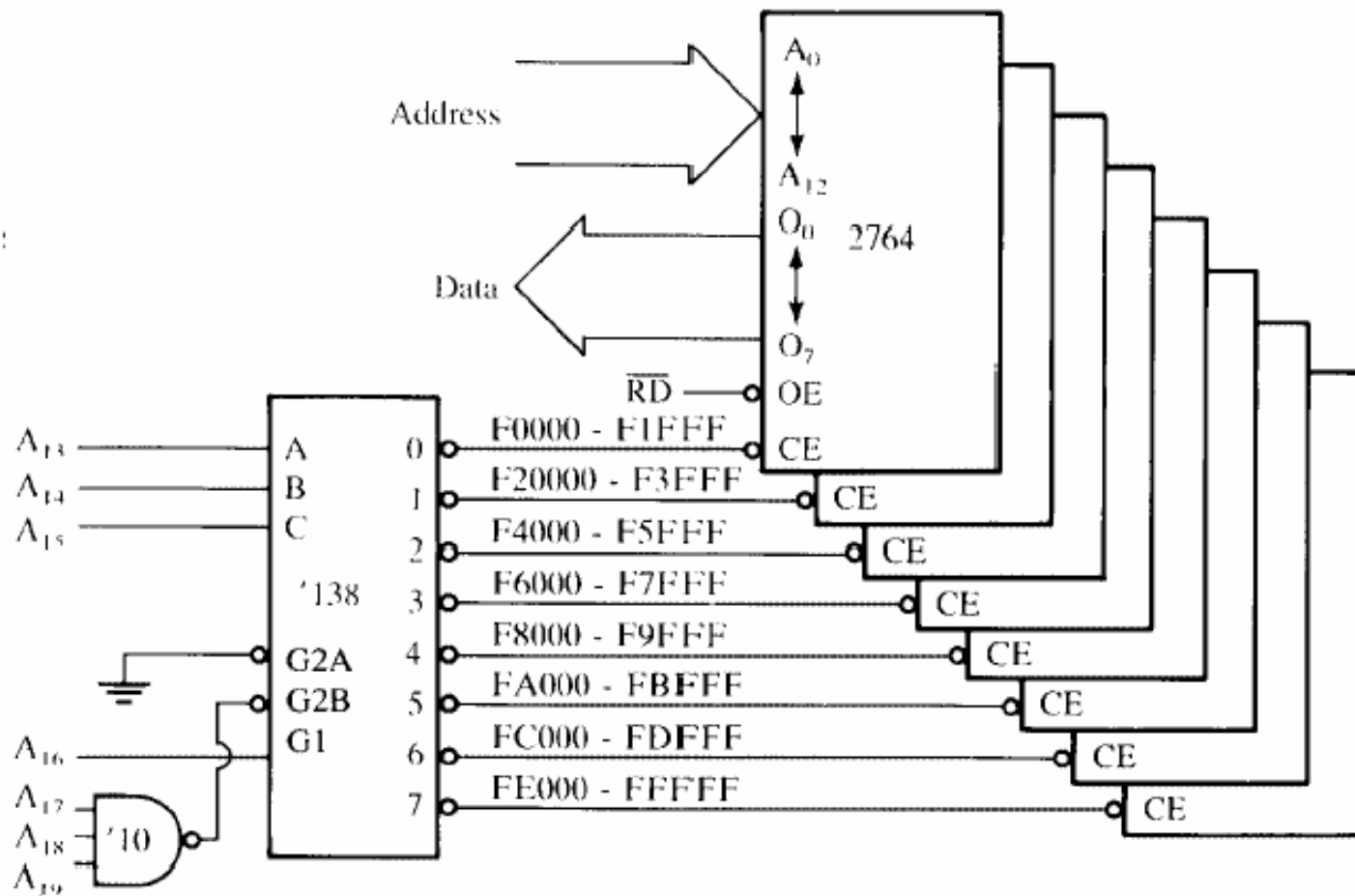
1: F9000- F9FFF;

...

7: FF000-FFFFF



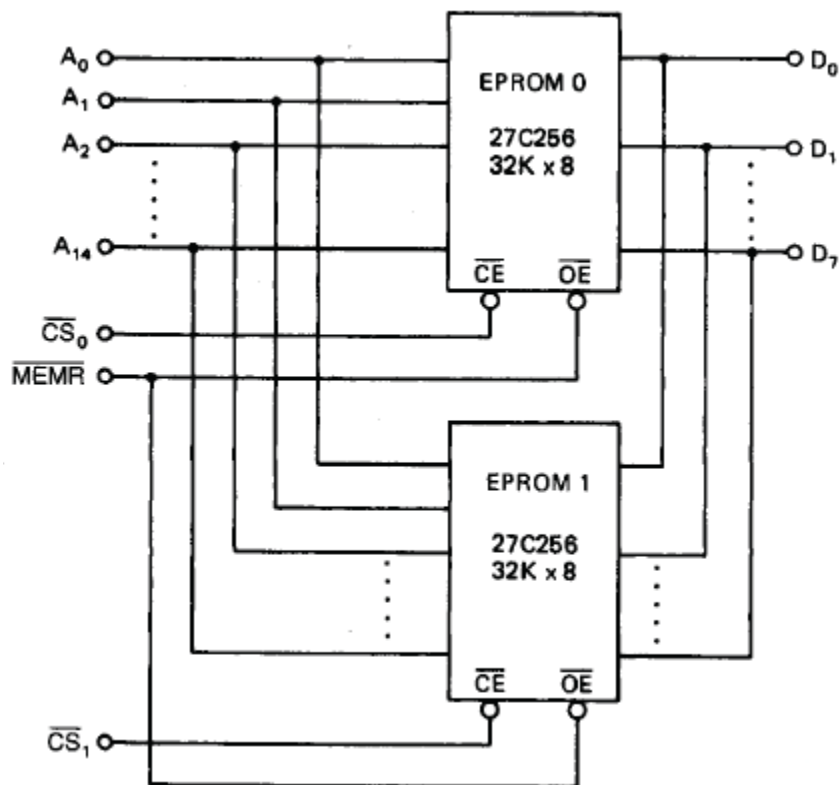
ROM / EPROM



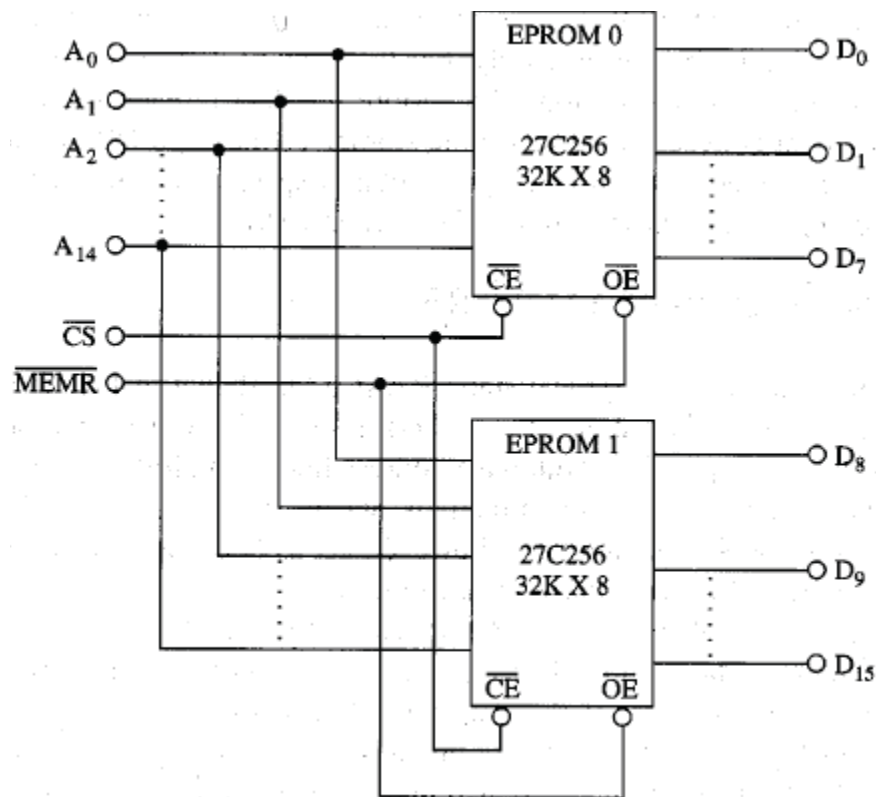
- 8088: $8 \times 2764 = 8 \times 8 \text{ KB} = 64 \text{ KB}$
- Spațiul de adrese: F0000 – FFFFF



Conectarea memoriilor



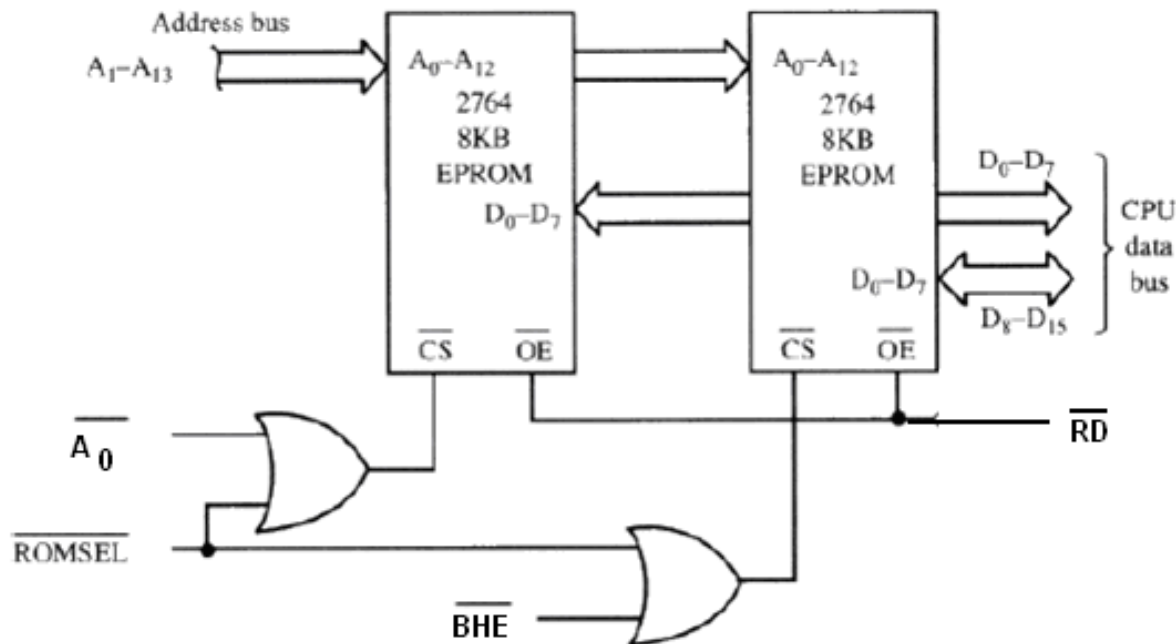
Conectarea paralelă: extinde adâncimea memoriei



Conectarea serială: extinde lățimea cuvântului



Conexiunea la o magistrală de 16 biți (8086)



#BHE	A0	Explicatie
0	0	Acces pe 16 biți (aliniat)
0	1	Byte superior, de la adresă impară
1	0	Byte inferior, de la adresă pară
1	1	Combinație nepermisă



Static RAM (SRAM)

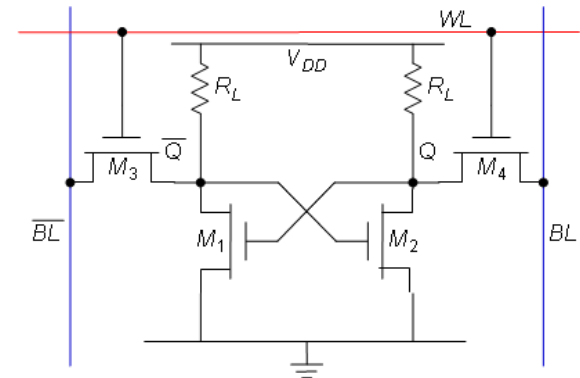
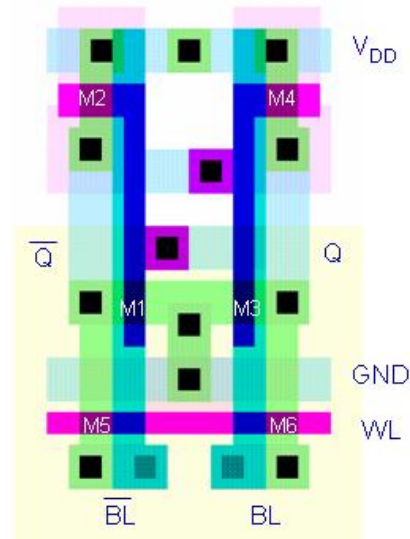
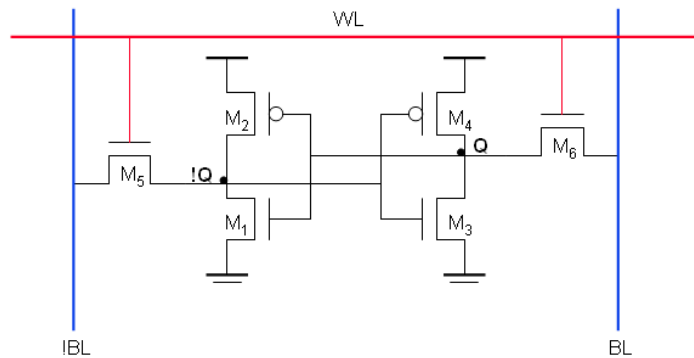
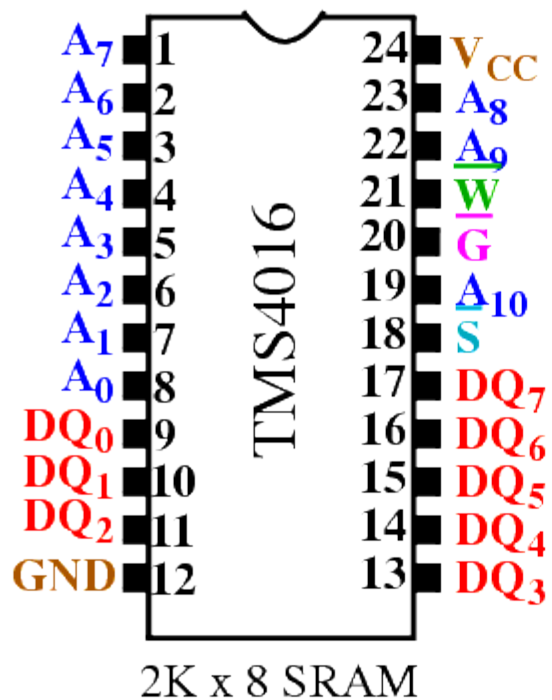


Table 12-2 Comparison of CMOS SRAM cells used in 1-Mbit memory (from [Takada91])

	Complementary CMOS	Resistive Load	TFT Cell
Number of transistors	6	4	4 (+2 TFT)
Cell size	58.2 μm^2 (0.7- μm rule)	40.8 μm^2 (0.7- μm rule)	41.1 μm^2 (0.8- μm rule)
Standby current (per cell)	10^{-15} A	10^{-12} A	10^{-13} A



Static RAM (SRAM)



Pin(s)	Function
A_0-A_{10}	Address
DQ_0-DQ_7	Data In/Data Out
$\overline{S}$ (CS)	Chip Select
$\overline{G}$ (OE)	Read Enable
$\overline{W}$ (WE)	Write Enable

[1]

Pini similari cu cei ai EPROM-ului, cu excepția semnalului de scriere

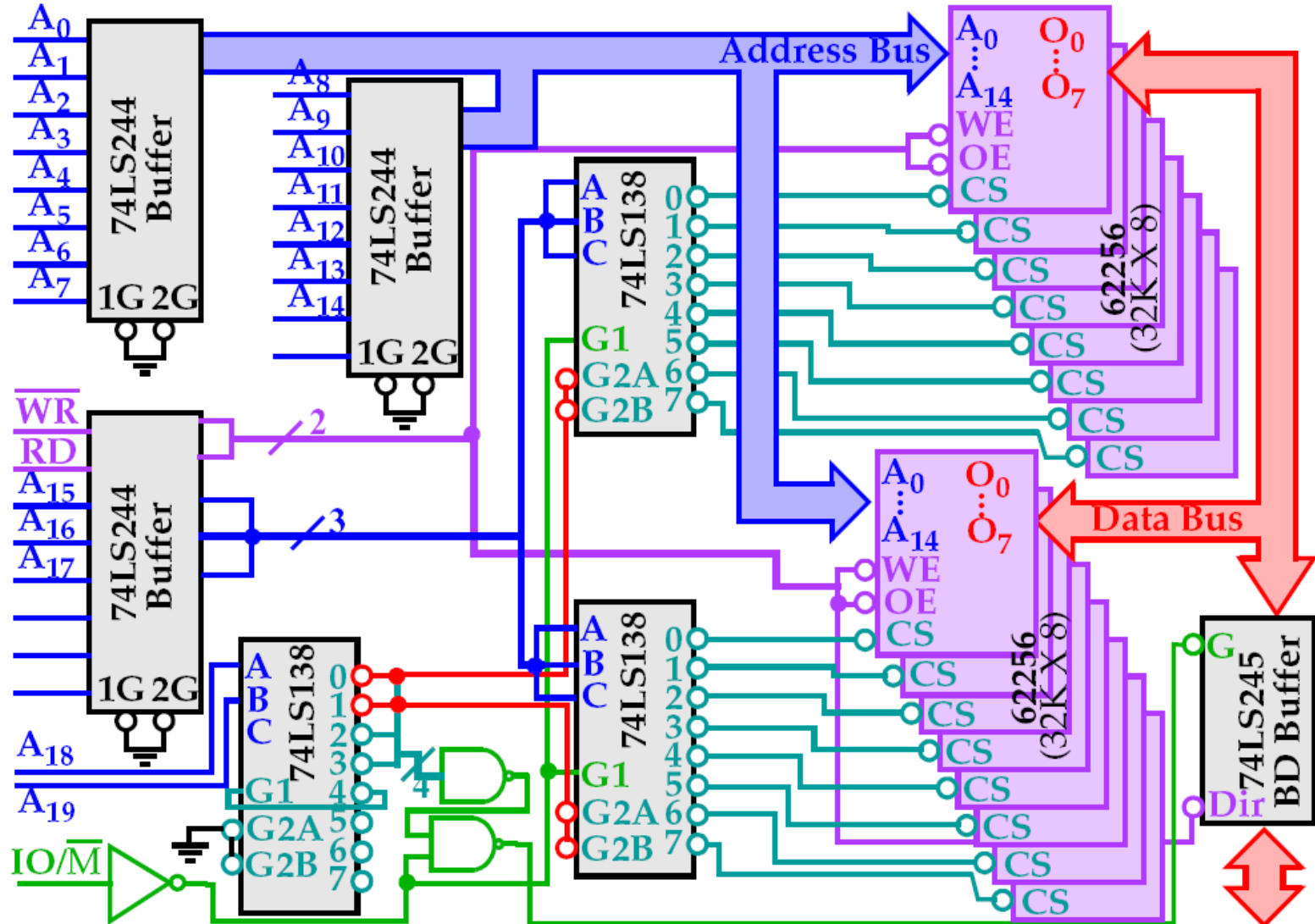
Timp de acces mai rapid

Folosit pentru memorii Cache



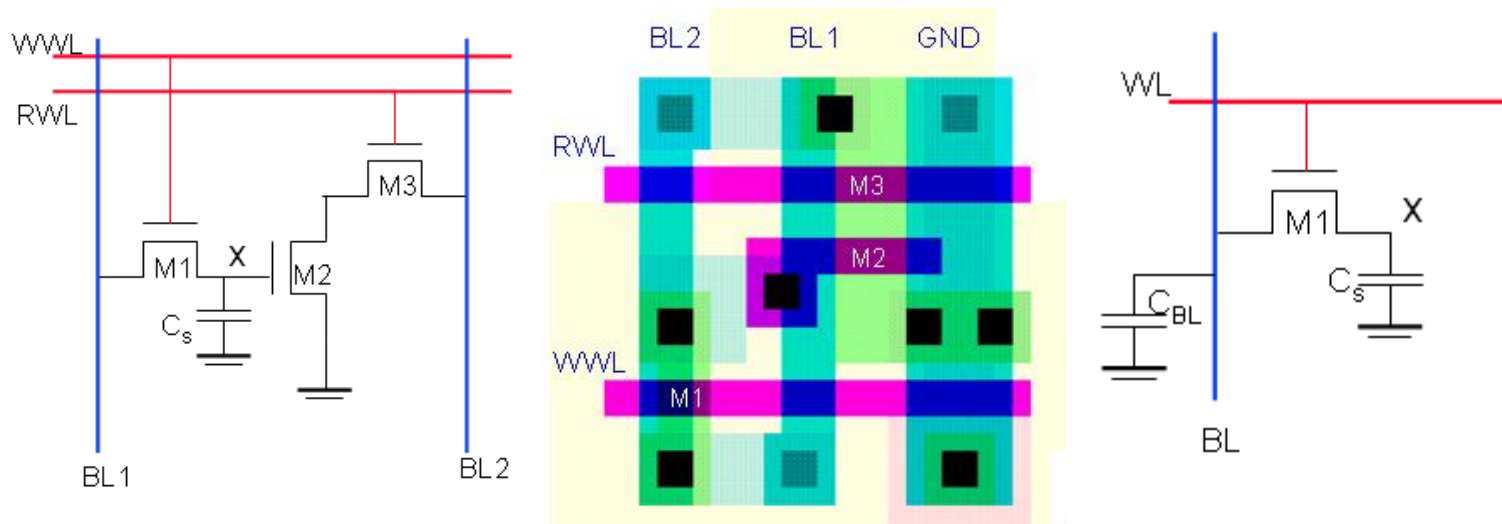
Static RAM (SRAM)

8088 and 80188 (8-bit) RAM Memory Interface





Dynamic RAM (DRAM)



1

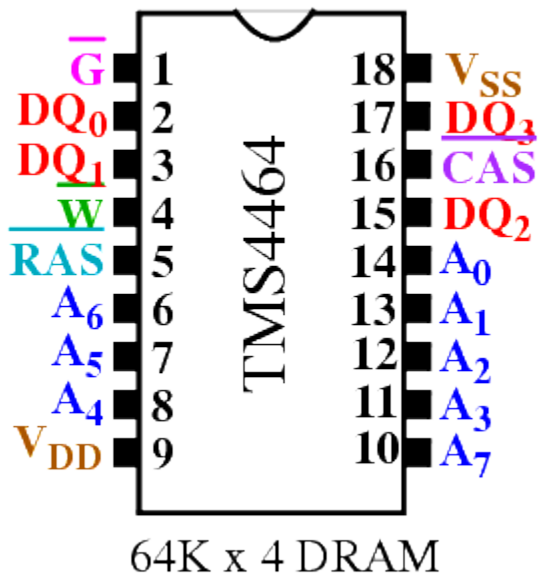
Dimensiune: $\frac{1}{2}$ din dimensiunea celulei SRAM $\Rightarrow$ capacitate sporită $\Rightarrow$ pinii de adresă sunt multiplexați

Refresh: 1 ... 4 ms $\Rightarrow$ circuit special $\Rightarrow$ cicluri de citire, scriere, refresh

Tipuri: SDR, DDR, Rambus



Dynamic RAM (DRAM)



Pin(s)	Function
A_0-A_7	Address
DQ_0-DQ_3	Data In/Data Out
$\overline{RAS}$	Row Address Strobe
$\overline{CAS}$	Column Address Strobe
$\overline{G}$	Output Enable
$\overline{W}$	Write Enable

[1]

64 K locații adresabile – necesită 16 linii de adresă, dar are numai 8

Adresa randului ($A_8:A_{15}$) este plasată pe pinii de adresă, și memorată în latch-uri interne, folosind RAS (row address strobe)

Adresa coloanei ($A_0:A_7$) este memorată ulterior, folosind CAS (column address strobe)

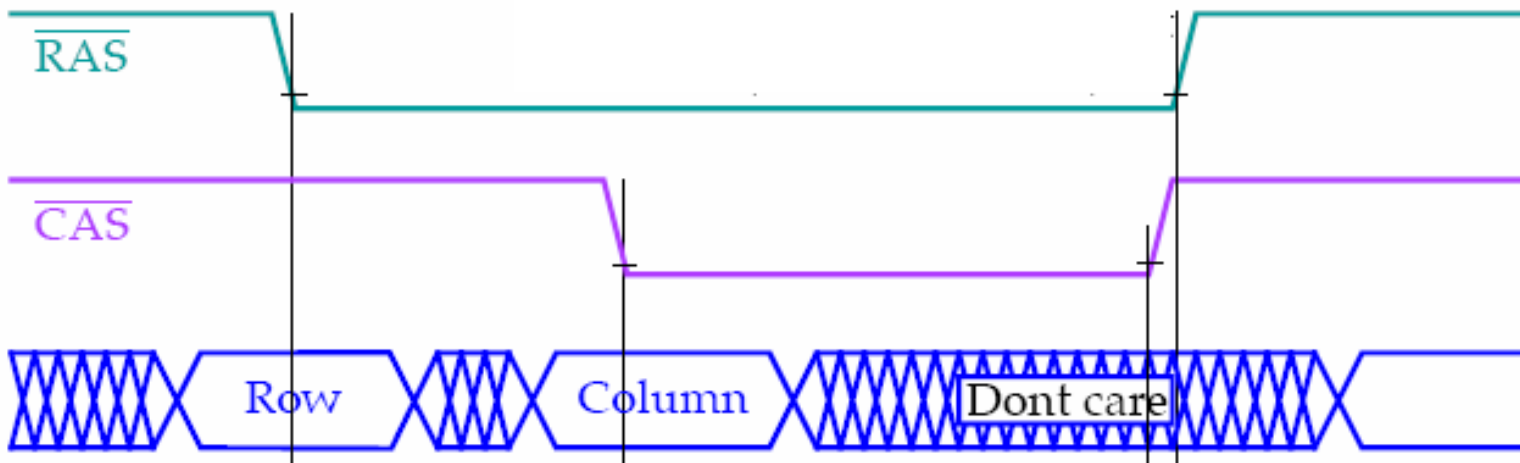


Dynamic RAM (DRAM)



DRAMs

TI TMS4464 DRAM (64K X 4) Timing Diagram:



CAS also performs the function of the chip select input.

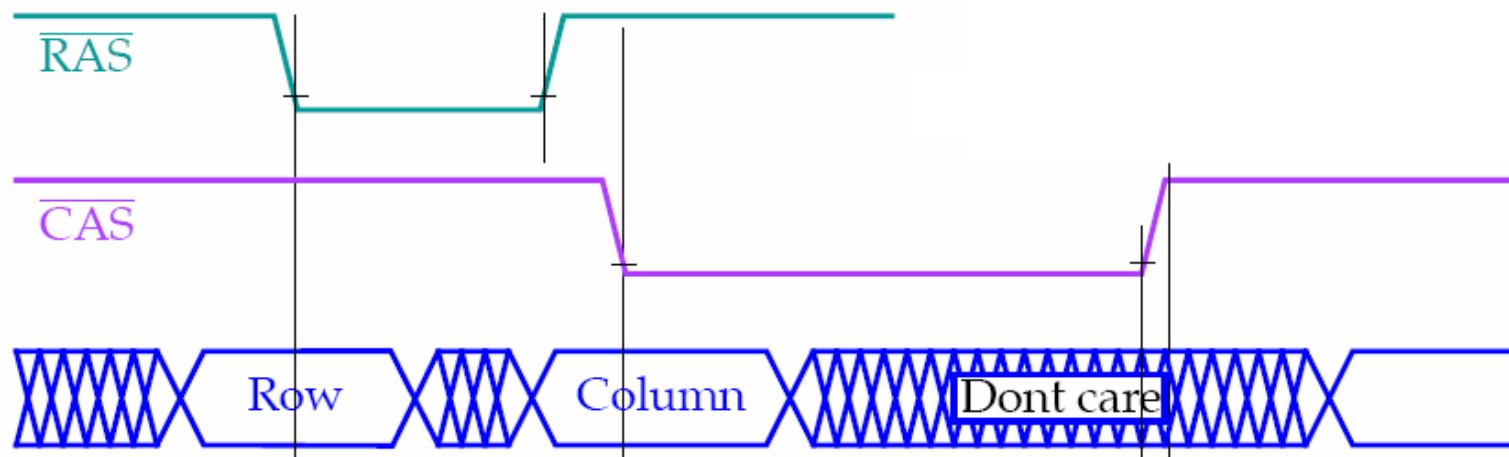
#RAS & #CAS trebuie transmise de un controller DRAM

Controllerul DRAM trebuie sa multiplexeze (în timp) liniile de adresa, ca:

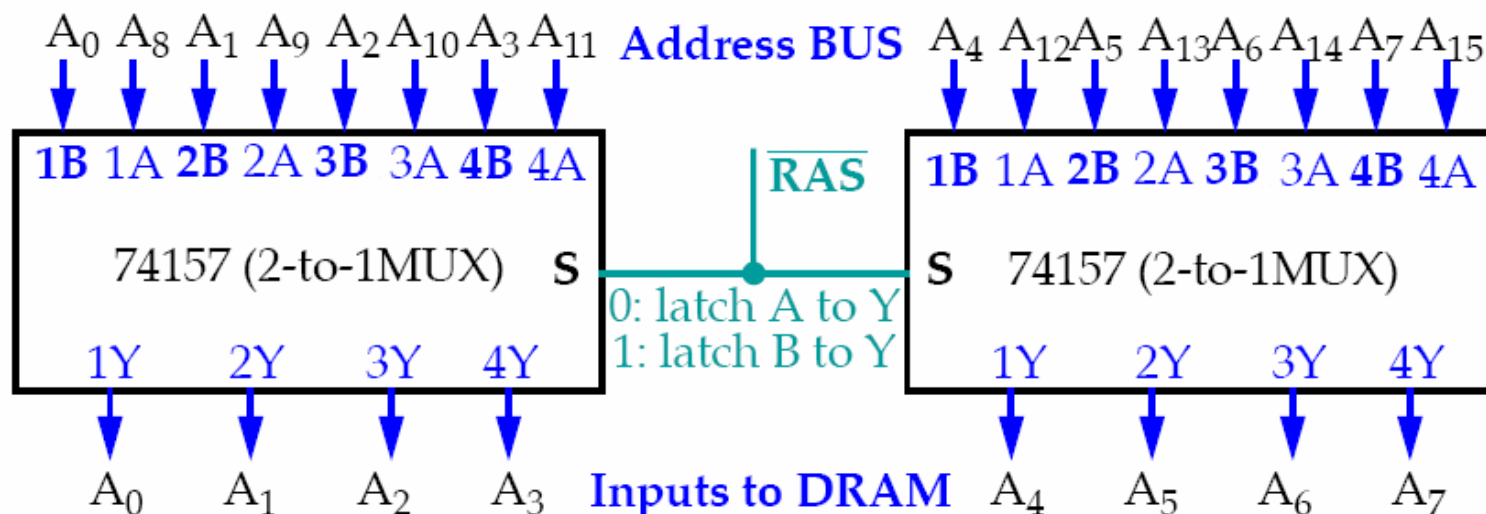
- Adresa randului (ex. A8-15)
- Adresa coloanei (ex. A0-7)



Dynamic RAM (DRAM)



CAS also performs the function of the chip select input.

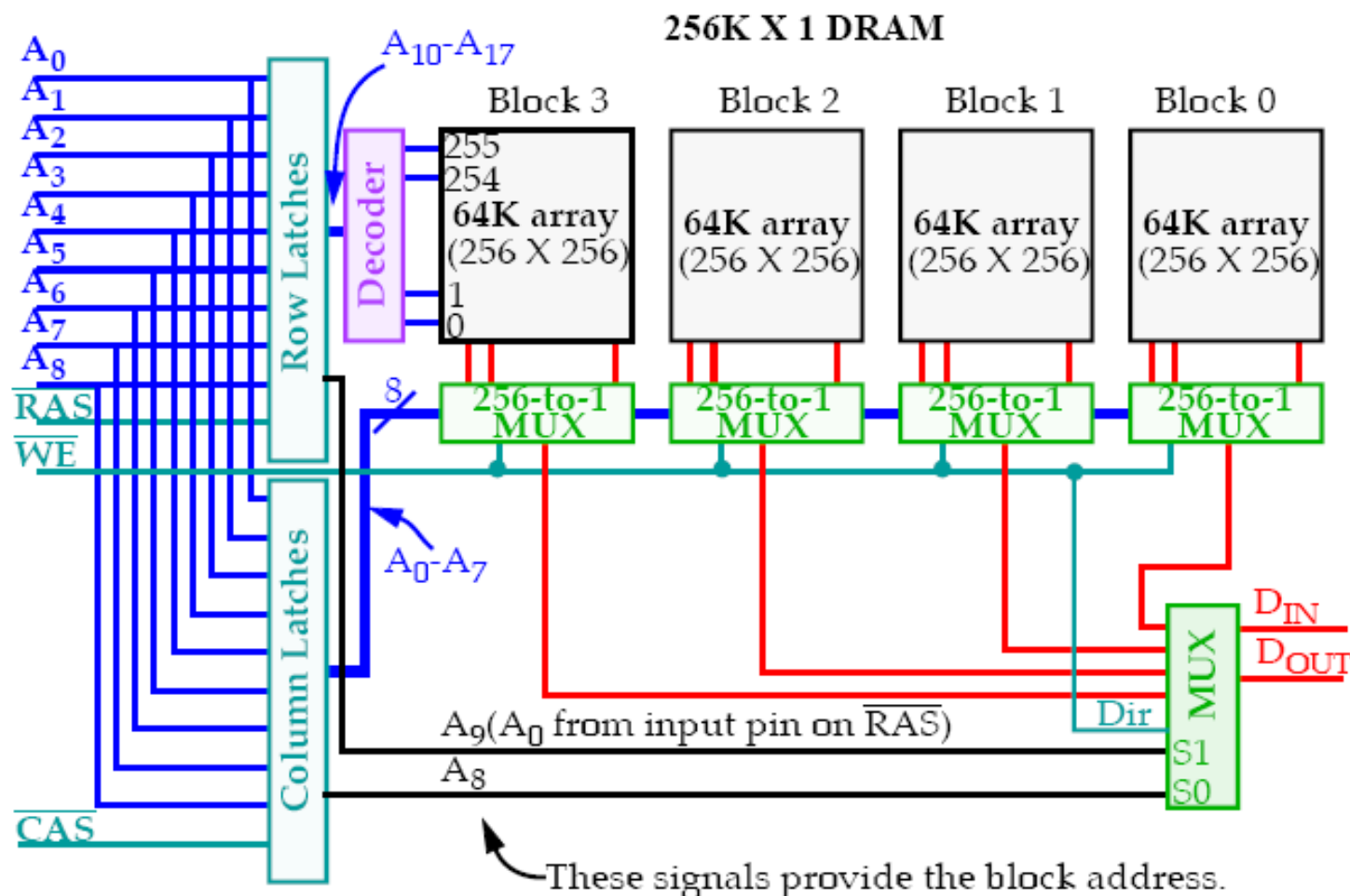




Dynamic RAM (DRAM)

256K X 1 DRAM – structura internă

Dynamic RAM





- **Ciclu special de refresh**

- Se petrece transparent cand sunt folosite alte componente – *transparent refresh sau cycle stealing*.
- Un ciclu ce folosește doar RAS incarcă o adresă a rândului în DRAM
- Condensatorii rândului selectat sunt reîncărcați prin citirea internă a biților și scrierea lor inapoi.

Exemplu:

- **256K X 1 DRAM** (256 randuri x 256 coloane x 4 blocuri)
 - $\Rightarrow$ refresh este necesar la fiecare 15.6 ms ($4\text{ ms} / 256$).
 - Pentru 8086, o citire sau o scriere se petrece la fiecare 800ns ($4 \times 200 = 4 \times T_{clk}$).
- $\Rightarrow$ **19** citiri/scrieri per refresh ($15.6\mu\text{s} / 0.8\mu\text{s} = 19.5$)
- $\Rightarrow$ Ciclurile de citire /scriere iau $\sim$ **5%** din timpul de refresh



Dynamic RAM (DRAM): Controller DRAM

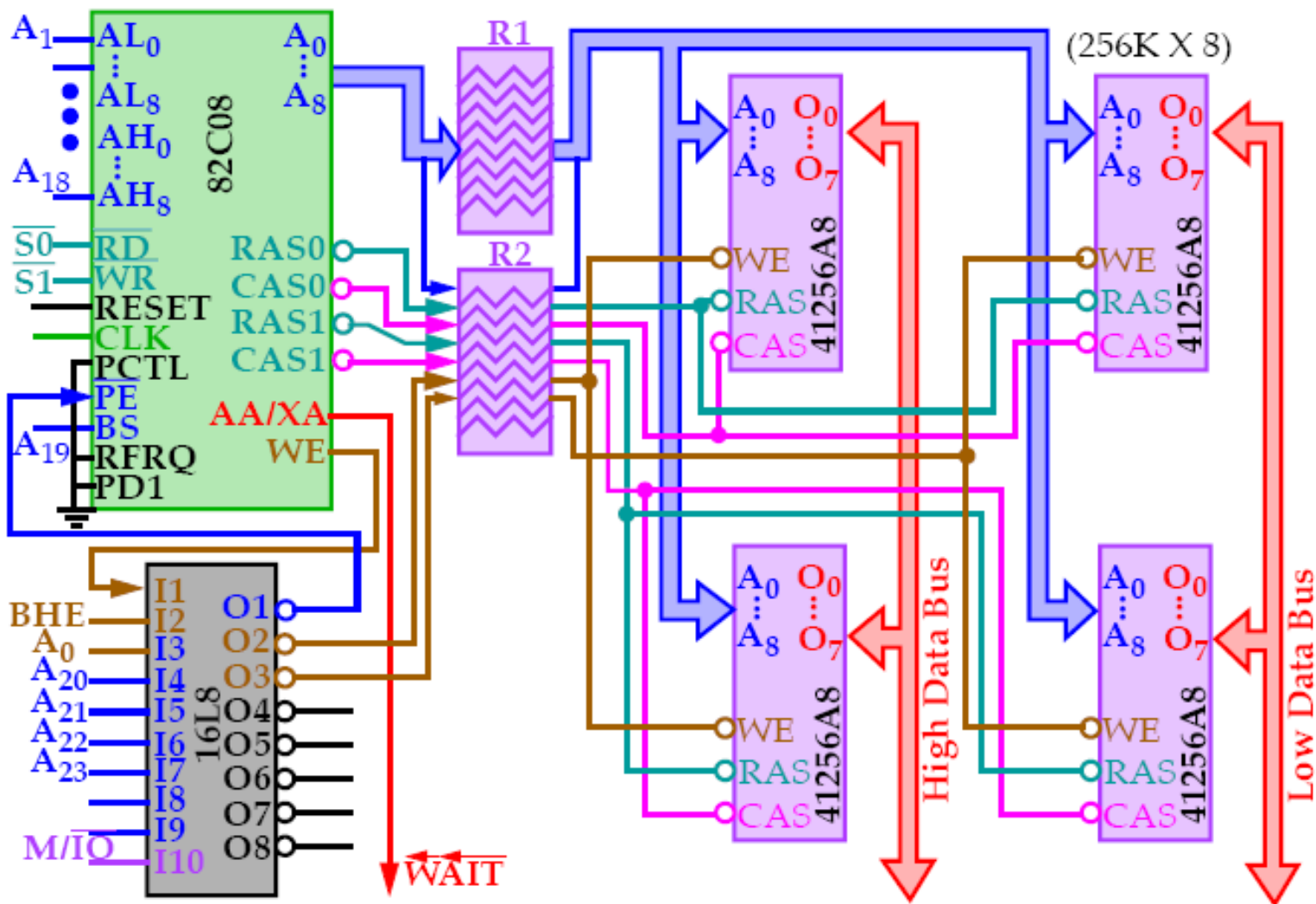


- Multiplexarea adreselor și generarea semnalelor de control pentru DRAM.
- **Intel 82C08**, poate controla doua bancuri de memorie DRAM 256 K X 16 , pentru un total de 1 MB.
- Bitii A1 - A18 (18 bits) sunt conectati la intrarile (AL – coloana) și (AH – linie) a lui 82C08.
- În funcție de adresă, se activează **RAS0/CAS0** sau **RAS1/CAS1**.
- **WE , BHE si A0** sunt folosiți pentru determinarea scrierii, și unde anume se va scrie



RAM – Dynamic (DRAM)

DRAM Controllers

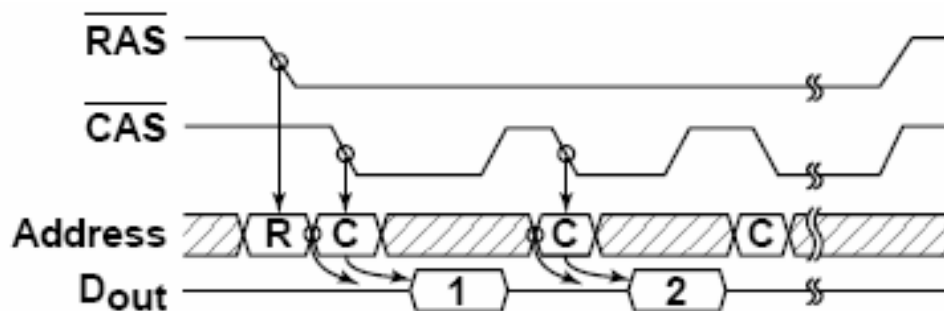




DRAM – Fast Page Mode (FPM)



- Permite citirea mai multor coloane pentru același rând
 - 1 RAS, multiple CAS
- Variații:
 - **Static column** – nu necesită pulsarea CAS pentru schimbarea coloanei, ci CAS este menținut activ, adresa coloanei se schimbă și ieșirea o urmează cu o anumită întârziere
 - **Nibble mode** – adresele consecutive primei coloane sunt generate de un contor intern, la fiecare puls CAS, nemaifiind preluate de pe liniile de adresa.

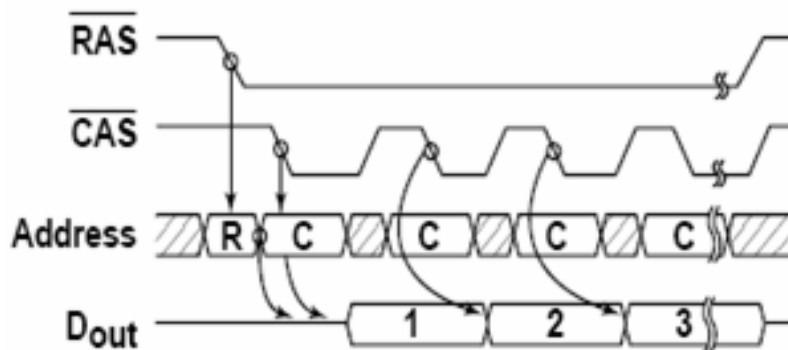




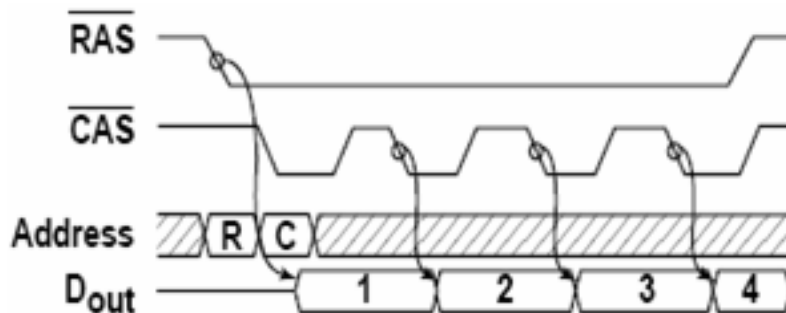
DRAM – Extended Data Out (EDO)



- Un ciclu de acces poate menține datele din ciclul anterior active



- Burst EDO (BEDO) – adresele coloanei sunt generate intern după primul CAS

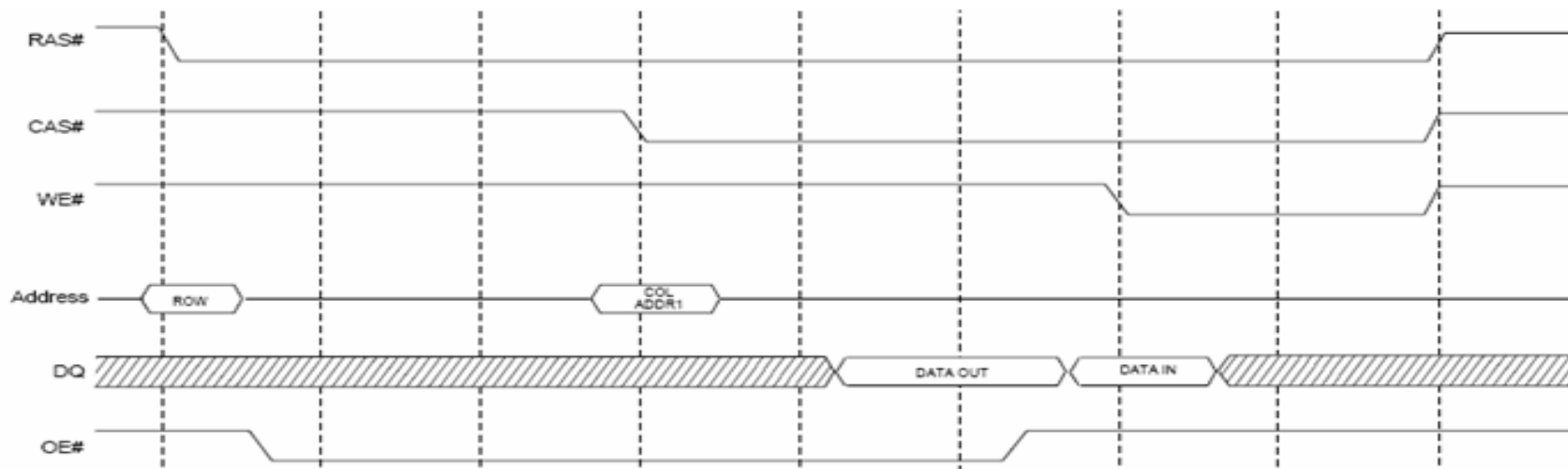




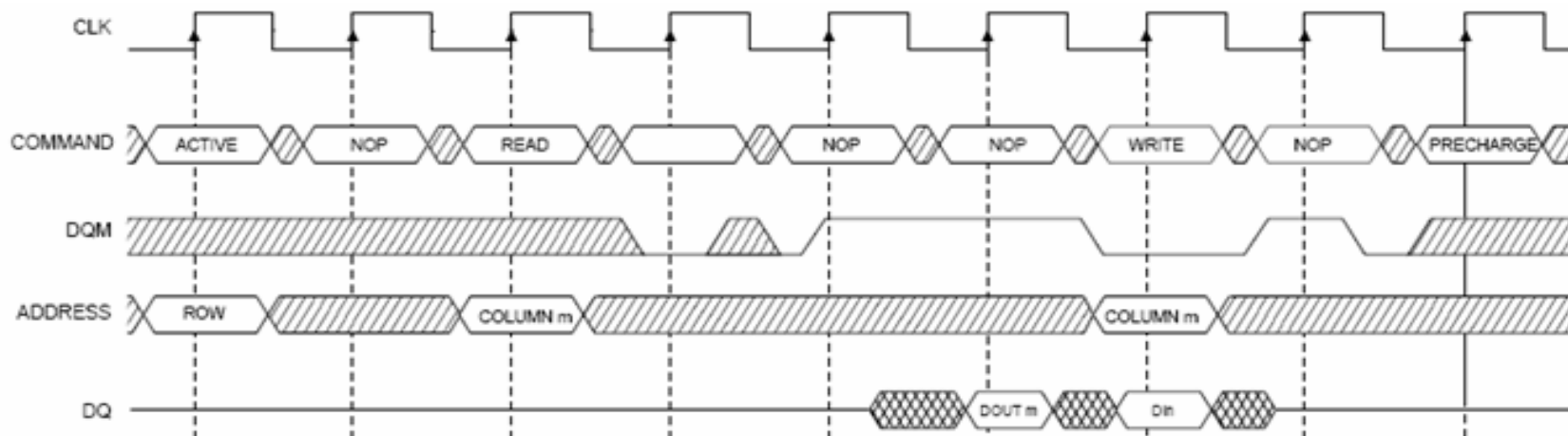
DRAM asincron / sincron



- Asincron

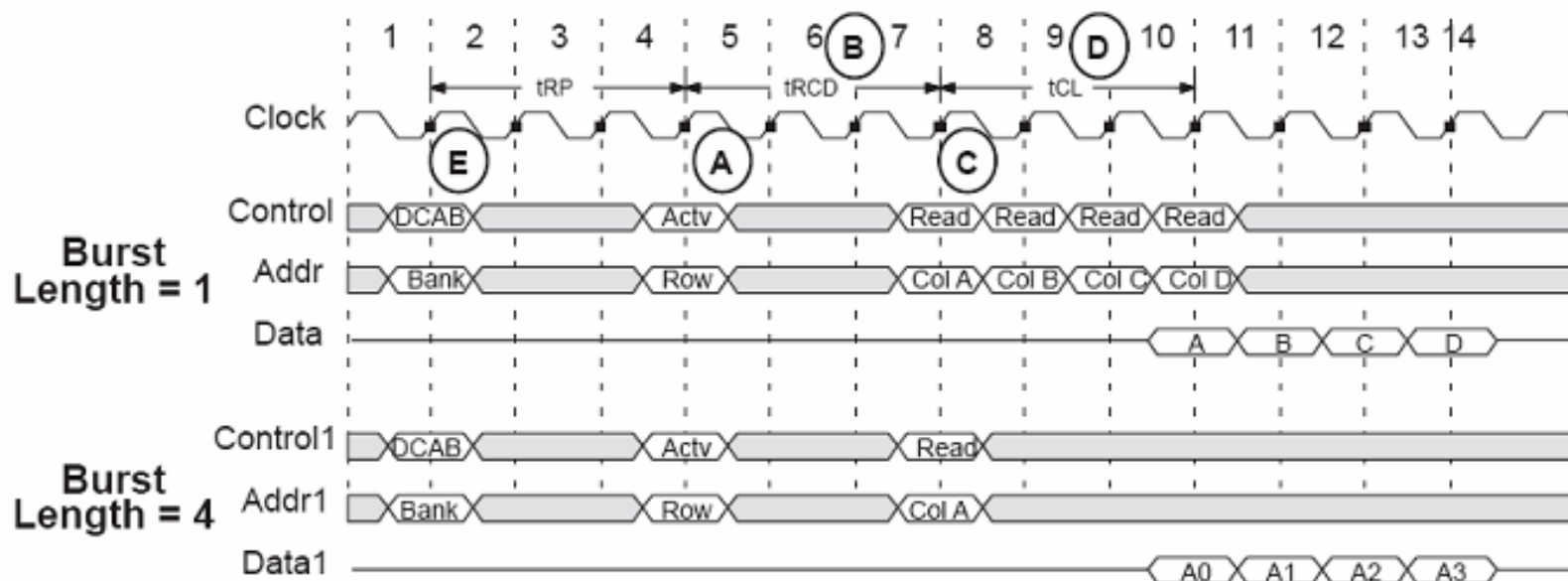


- Sincron





SDRAM – Synchronous DRAM



Semnale SDRAM:

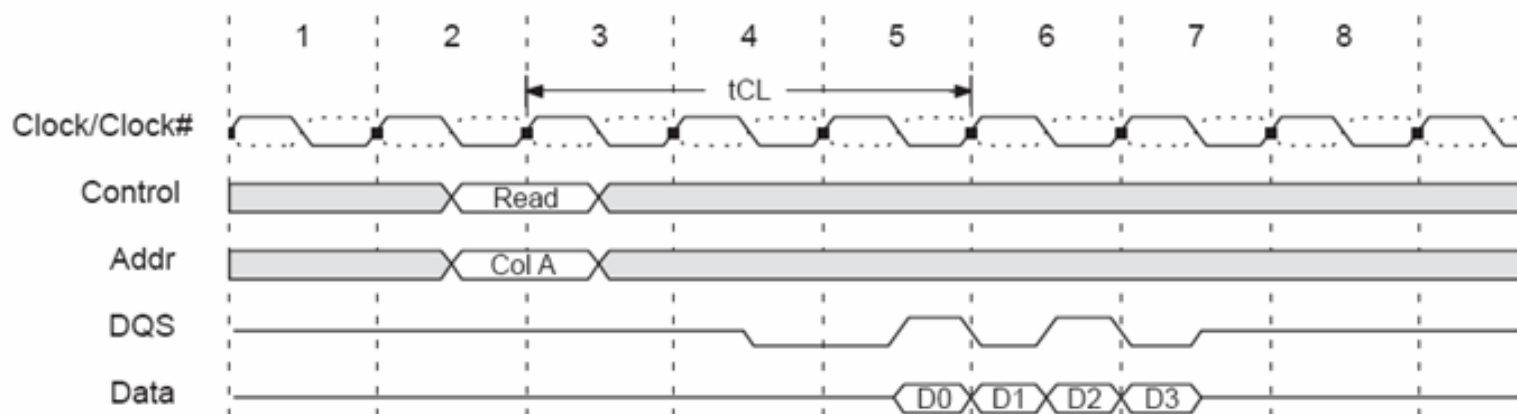
- **CKE Clock Enable.** Daca acest semnal este zero, memoria nu executa nici o operatie.
- **/CS Chip Select.** Când acest semnal este 1, memoria ignoră toate intrările.
- **DQM Data Mask.** Maska pentru octeții din cuvantul de date (1 linie de mască pentru fiecare octet).
- **/RAS Row Address Strobe.** Formează, împreună cu /CAS și /WE, comenzi pentru memorie.
- **/CAS Column Address Strobe.**
- **/WE Write enable.**



DDRAM – Double Data Rate SDRAM



- Transferul de date se efectuează pe ambele fronturi ale semnalului de ceas





Referințe



- https://www.csee.umbc.edu/courses/undergraduate/CMPE310/Fall07/cpatel2/slides/html_versions/chap10_lect04_memory.html
- https://www.csee.umbc.edu/courses/undergraduate/CMPE310/Fall07/cpatel2/slides/html_versions/chap10_lect06_memory3.html



Proiectarea cu Micro-Procesoare

Lector: Mihai Negru

An 3 – Calculatoare și Tehnologia Informației
Seria B

Curs 12: Direct Memory Access – DMA

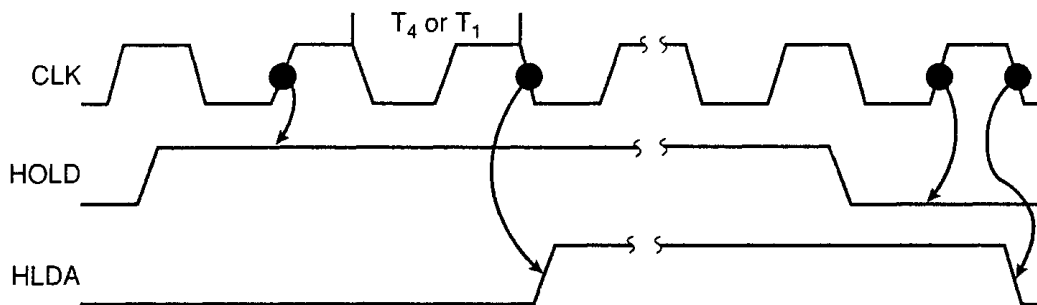
<http://users.utcluj.ro/~negrum/>



- **Direct memory access (DMA)** este un proces prin care un dispozitiv extern preia controlul magistralei, în locul procesorului.
- DMA se folosește la **transferul de mare viteză a datelor** la/de la dispozitive periferice.
- Ideea fundamentală a DMA este de a transfera blocuri de date **în mod direct între memorie și periferice**. Datele nu mai trec prin procesor, dar magistrala este ocupată.
- Un transfer “normal” al unui byte de date poate lua până la 29 de perioade de ceas. Un transfer prin DMA necesită doar 5 perioade.
- În calculatoarele moderne, DMA poate transfera date la viteză foarte mare. Rata de transfer este limitată de viteza memoriei, de viteza dispozitivelor periferice și de lățimea magistralei.

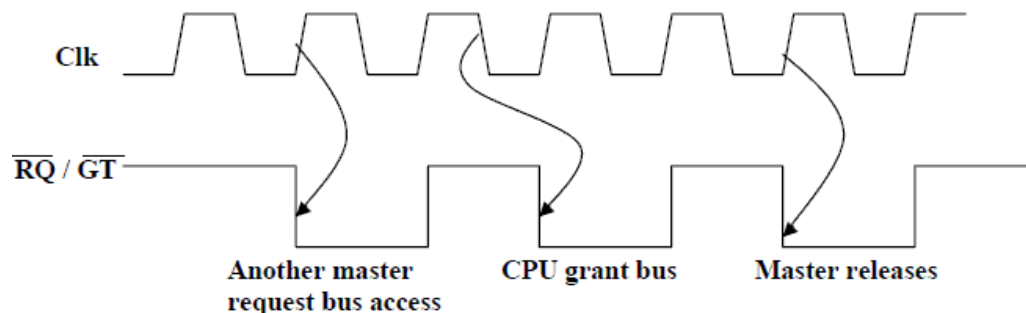


- Pinii **HOLD** și **HLDA** sunt folosiți pentru a primi și a confirma cererea pentru eliberarea magistralei.
- În mod normal, CPU are controlul absolut al magistralei. **În modul de operare DMA, componenta periferică preia controlul magistralei, temporar.**
- Pașii unui proces DMA tipic:
 - 1) Controllerul DMA anunță cererea, pe pinul HOLD.
 - 2) 8086 termină ciclul curent de magistrală, și intră în starea HOLD.
 - 3) 8086 cedează controlul magistralei prin activarea pinului HOLDA. Pinii de lucru cu magistrala de la 8086 (Addressa, Data, Control) se pun pe înaltă impedanță.
 - 4) Incepe transferul DMA.
 - 5) La terminarea transferului DMA, controllerul DMA dezactivează HOLD, pentru a elibera controlul magistralei.





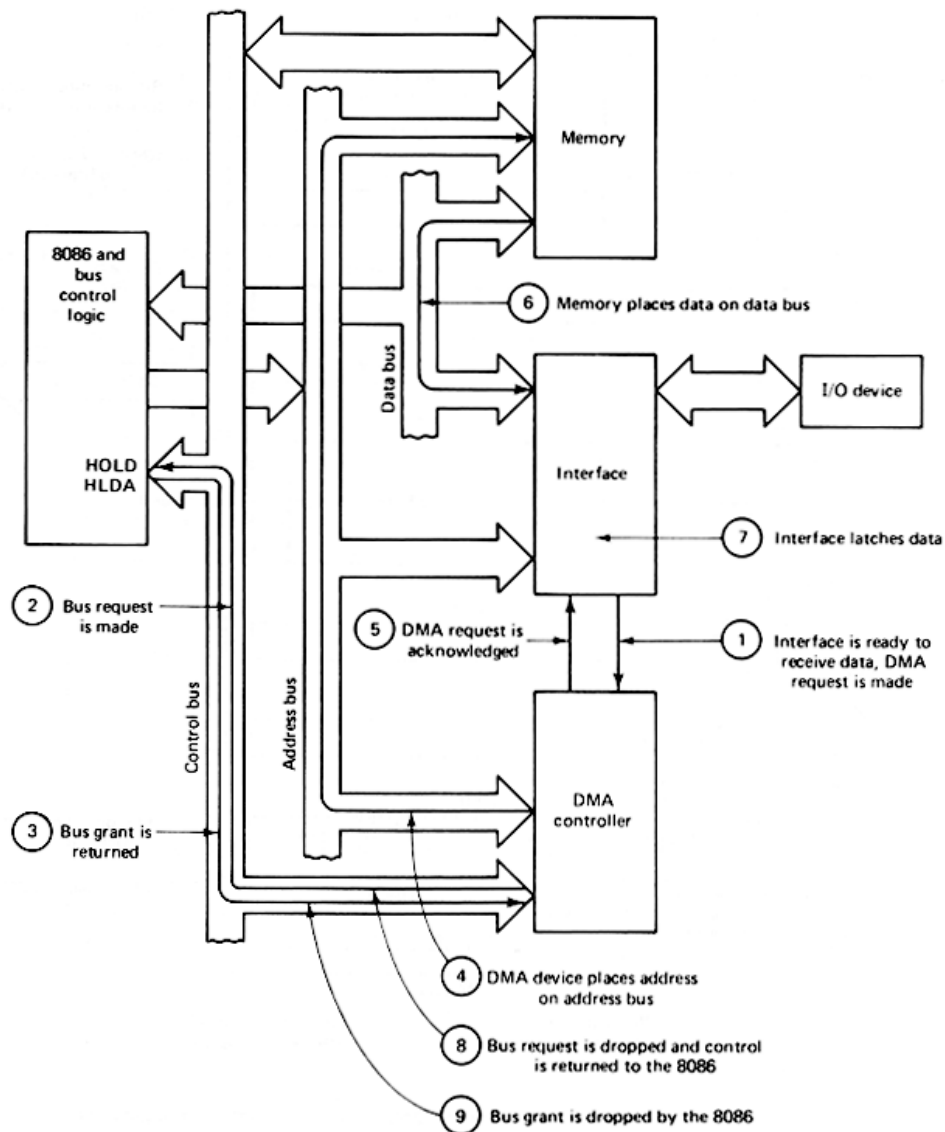
- Pinii RQ/GT1 și RQ/GT0 sunt folositi pentru a emite o cerere DMA și pentru a primi semnalele de confirmare.
- Secvența de evenimente pentru un proces DMA tipic
 - 1) Controllerul DMA activează unul din pinii de cerere, ex. RQ/GT1 sau RQ/GT0 (RQ/GT0 este prioritar)
 - 2) 8086 termina ciclul de magistrală curent, și intră în starea de eliberare a magistralei (stare HOLD)
 - 3) 8086 cedează controlul magistralei prin activarea unui semnal pe același pin pe care a fost primit semnalul de cerere.
 - 4) Incepe transferul DMA
 - 5) La terminarea operatiei, controllerul DMA activează pinul de cerere pentru a semnala eliberarea magistralei.



$\overline{RQ}/\overline{GT}$ Timings in Maximum Mode

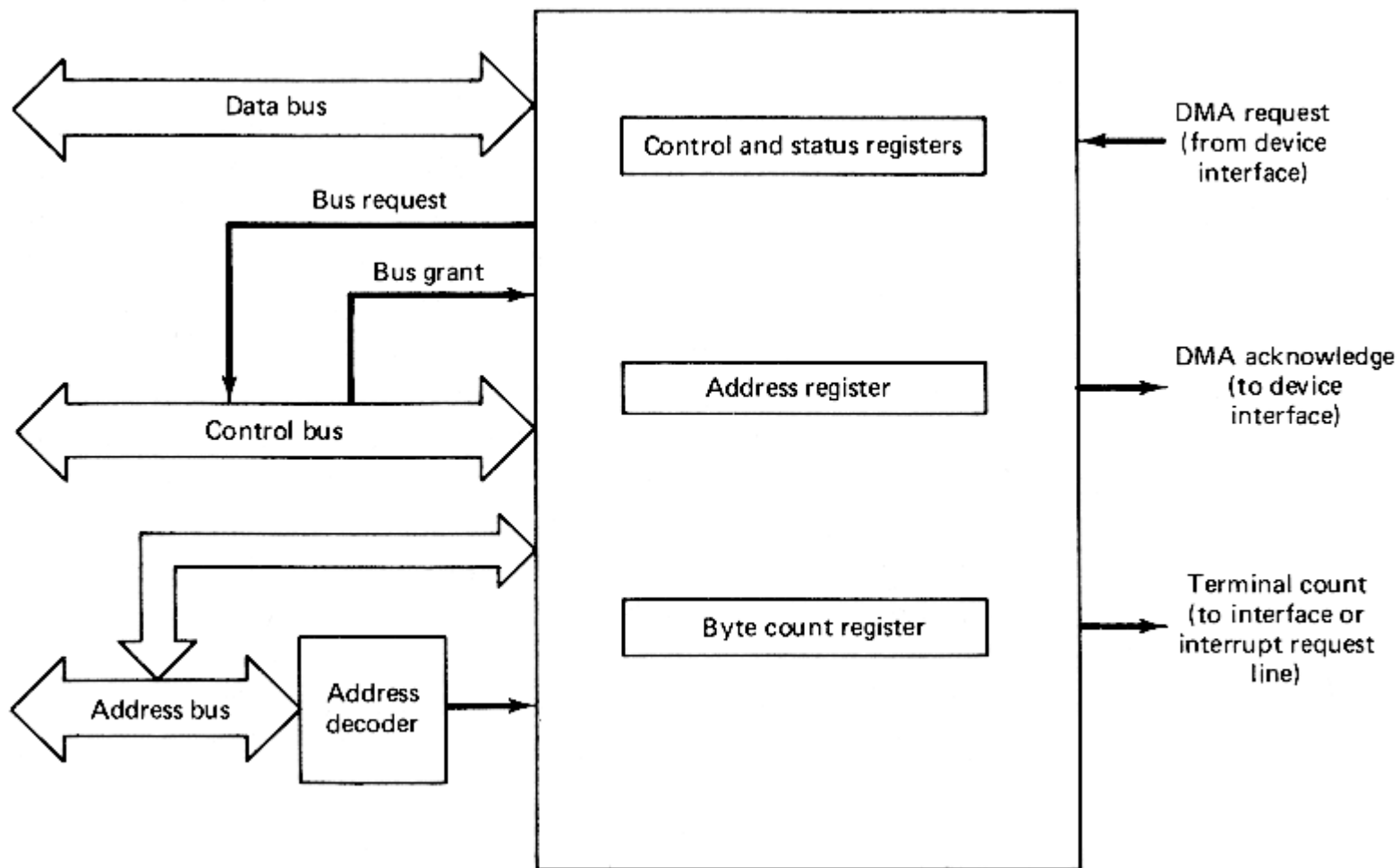


Accesul direct la memorie– DMA



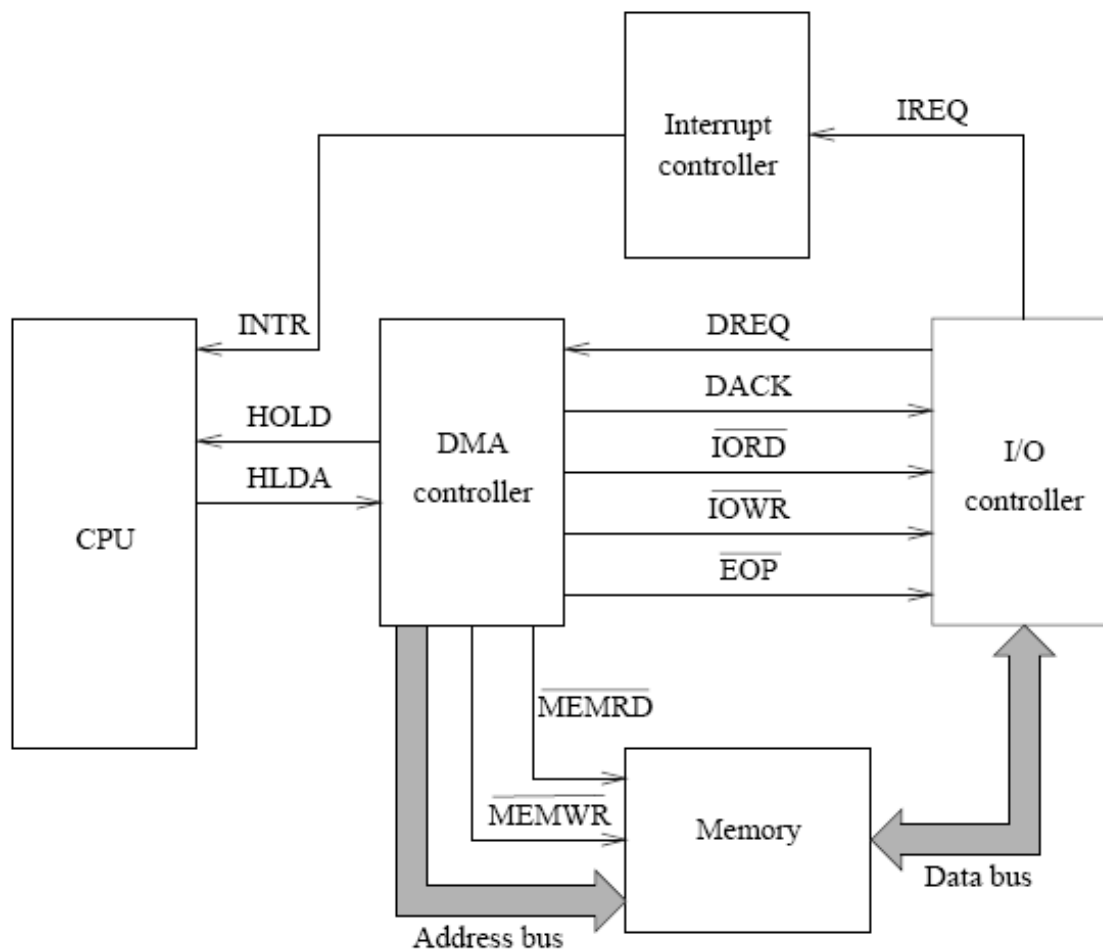


Organizarea generală a unui controller DMA





Semnalele pentru transferul DMA



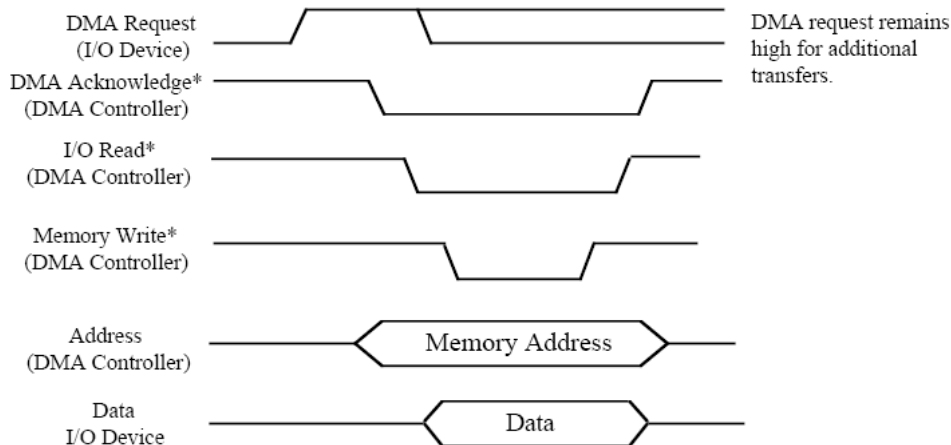


Tipuri de transfer DMA



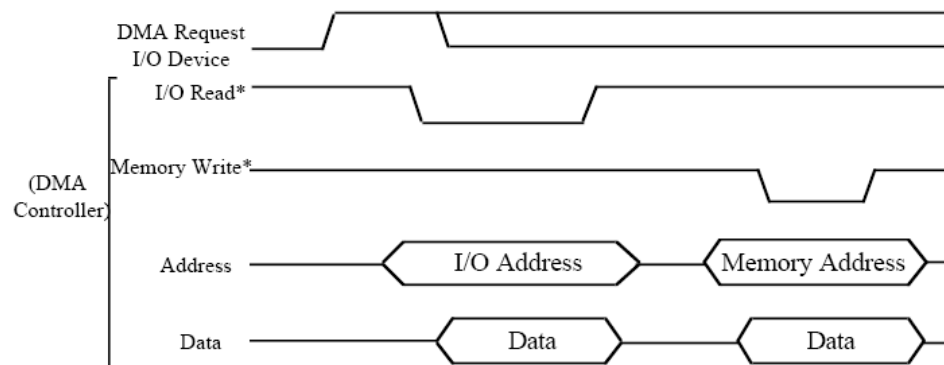
• Transfer “Fly-by”

- Datele nu trec prin controllerul DMA
- 1 ciclu de magistrală per transfer
- Mem $\leftrightarrow$ I/O
- Semnale de control simultane



• Transfer “Flow-through”

- Datele trec prin controller
- Transfer cu preluare și stocare:
 - 2 cicluri/transfer
- Mem – Mem, I/O – I/O

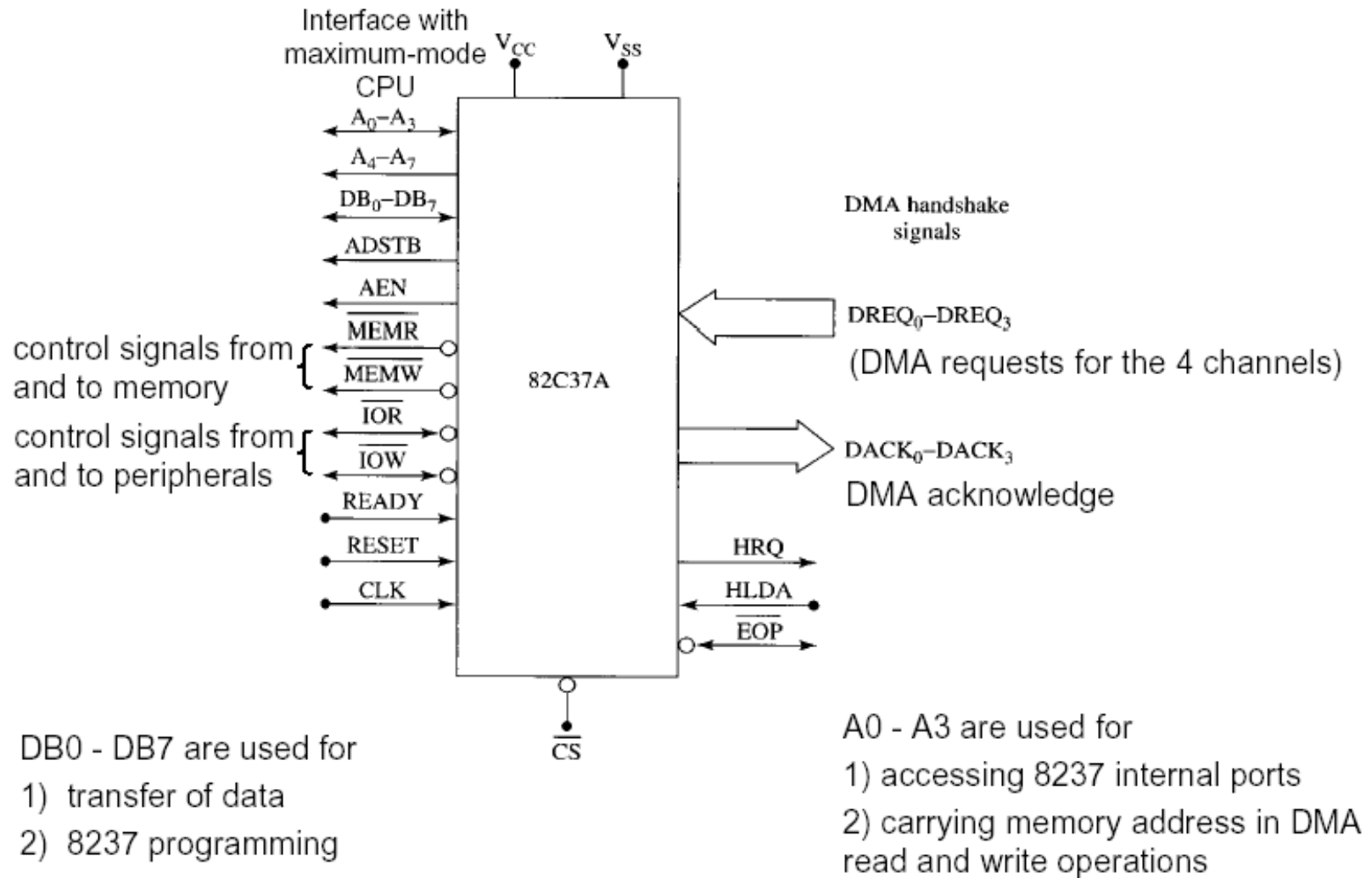




- Un controller DMA se interfațează cu mai multe periferice care pot cere transfer DMA.
- Controllerul decide prioritatea in cazul cererilor simultane, și oferă adresele de memorie pentru transferul datelor.
- Controllerul DMA folosit cu 8086/8088 este **dispozitivul programabil 8237.**
- 8237 este un dispozitiv cu 4 canale. Fiecare canal este dedicat unui dispozitiv periferic, și este capabil de a adresa o secțiune de memorie de 64 K Bytes.

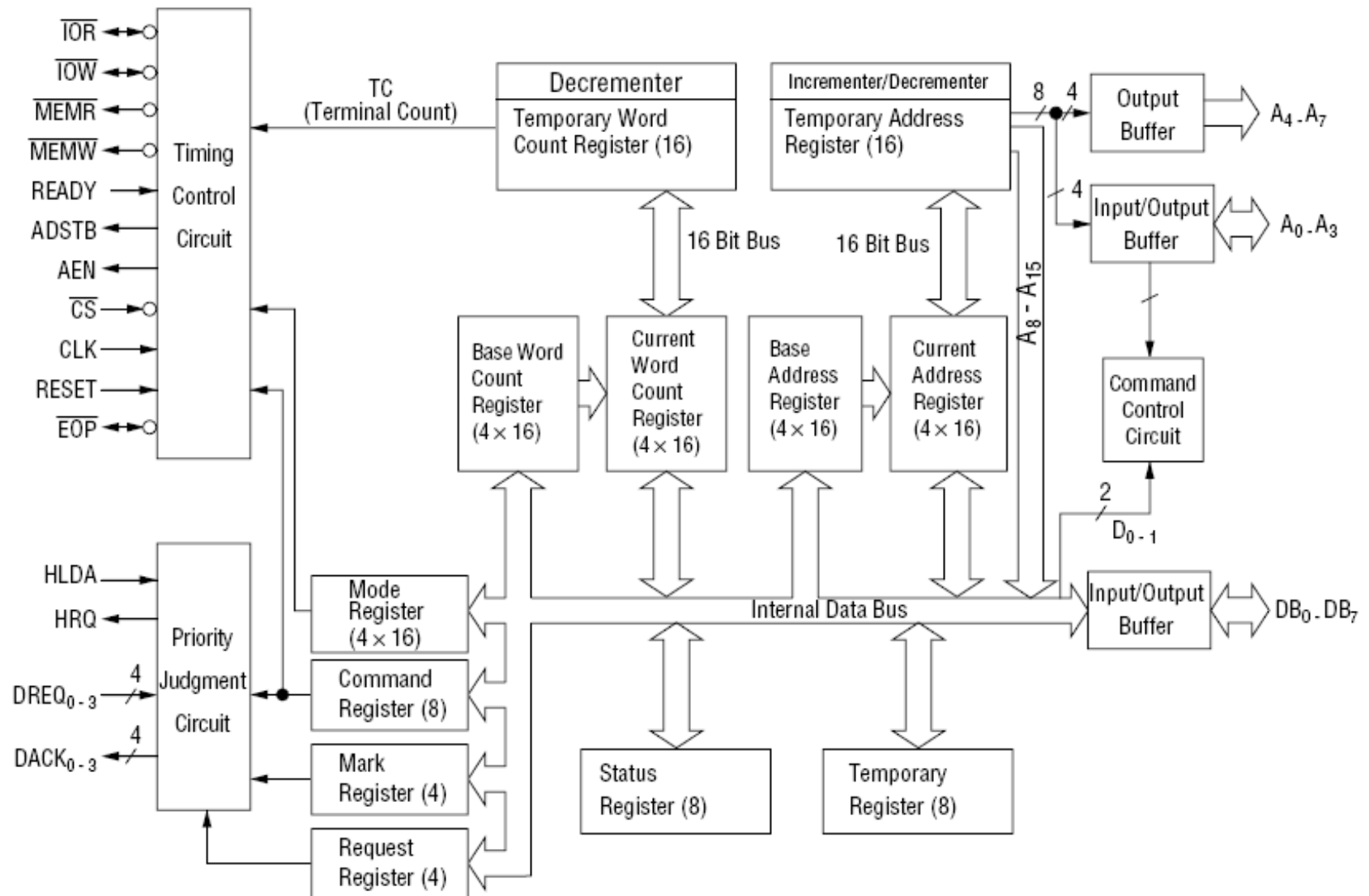


Controller DMA – 8237



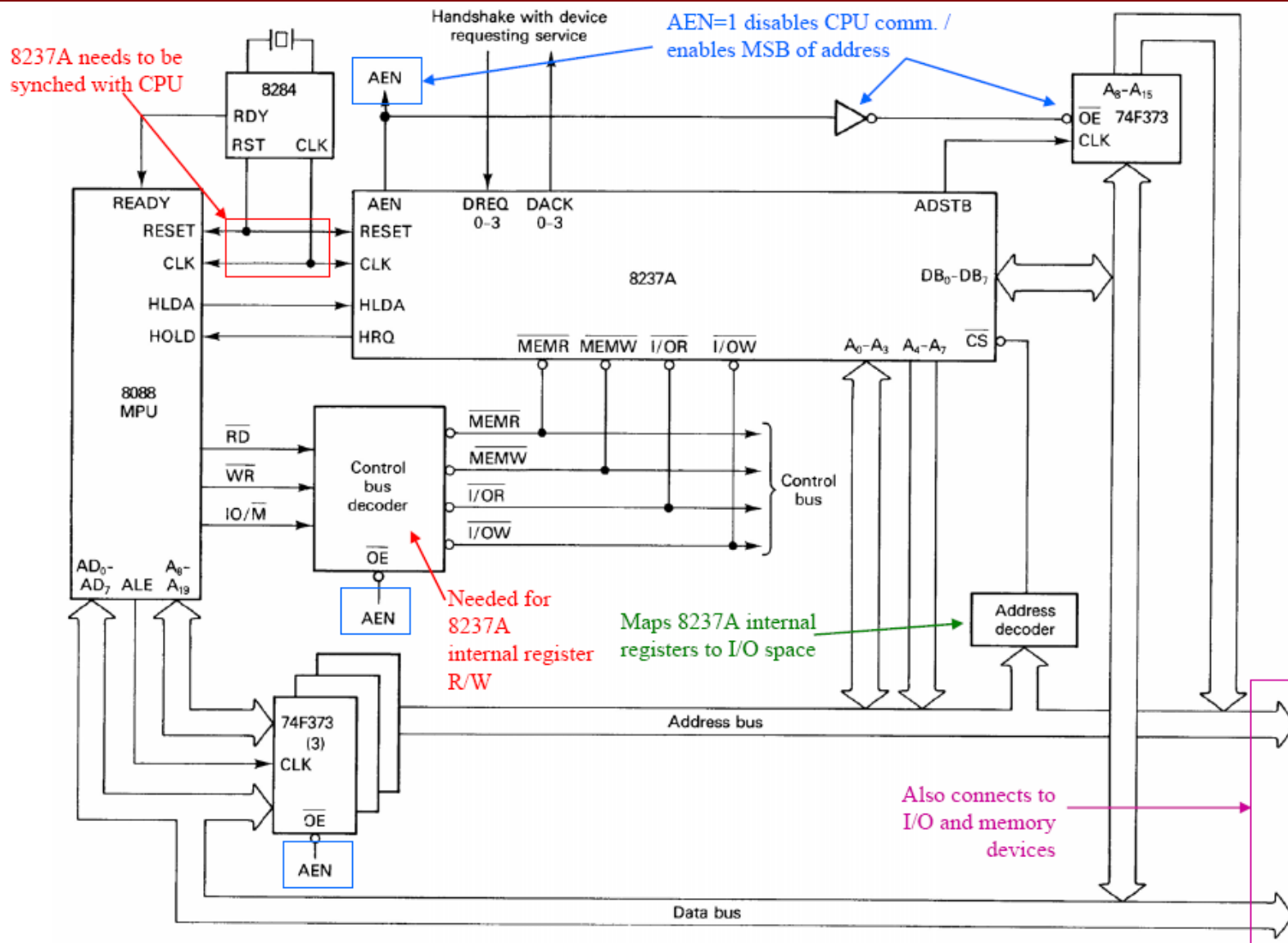


Controller DMA – 8237 (schema bloc)





8088 + i8237A





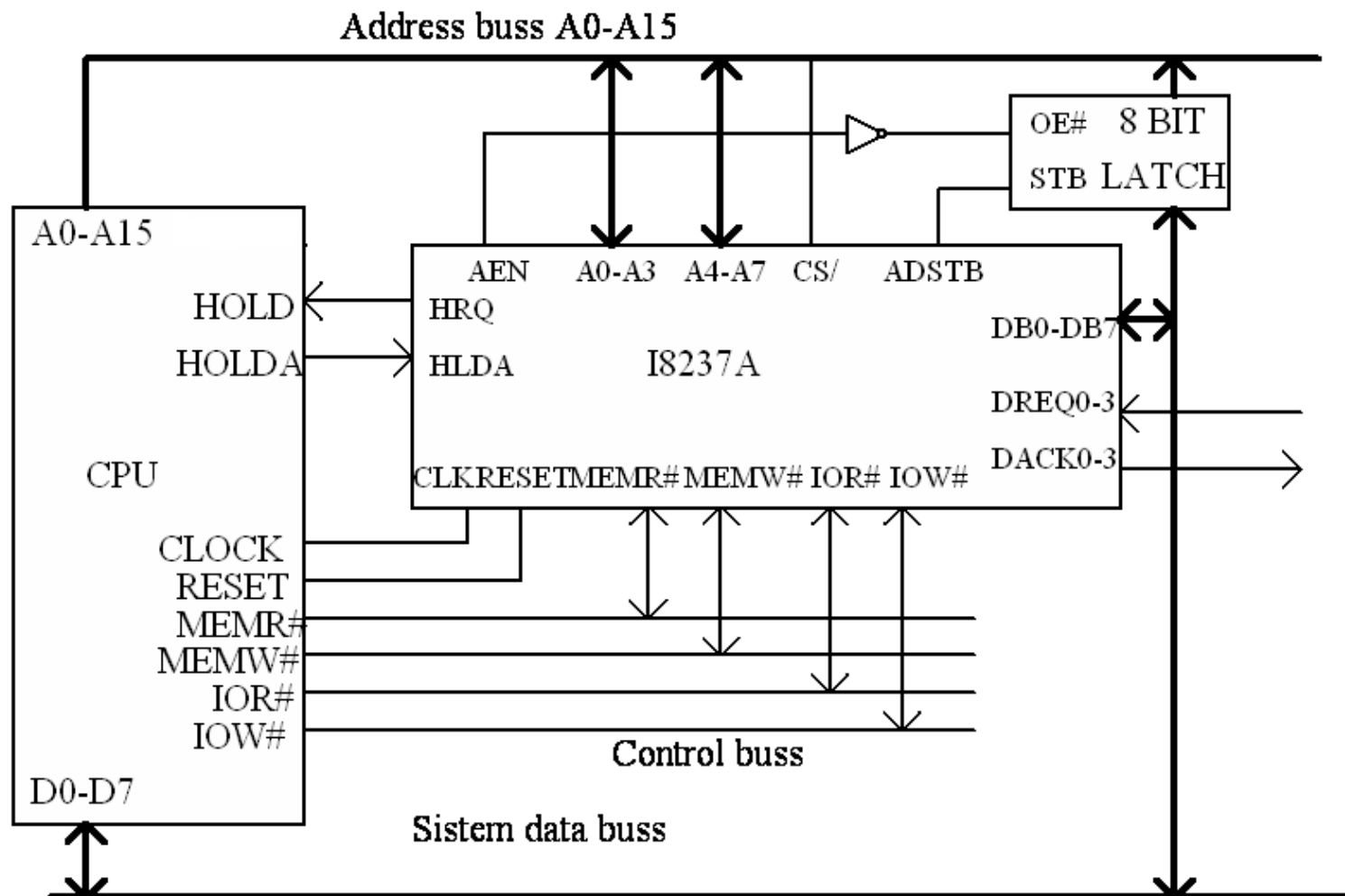
Tipuri de transfer



- **Scriere:** transfer de la dispozitiv I/O la memorie prin activarea MEMW și IOR.
- **Citire:** transfer de la memorie la dispozitiv I/O prin activarea MEMR și IOW.
- **Transfer de verificare:** pseudo-transferuri.
 - Dispozitivul 82C37A funcționează ca în modurile Citire sau Scriere, generând adrese și răspunzând la EOP, etc, dar semnalele pentru controlul memoriei și al I/O rămân inactive. Modul de verificare nu este permis pentru operații memorie-memorie.
- **Memorie la memorie**
 - Canal 0 – adresa și numărătorul pentru sursă
 - Canal 1 – adresa și numărătorul pentru destinație.
 - Octetul de date citit din memorie este stocat în registrul intern temporar din 82C37A.
 - Transferul este inițiat prin software, sau prin setarea hardware a lui DREQ pentru canalul 0. 82C37A cere un transfer DMA în manieră obisnuită.
- **Autoinițializare**
 - un canal poate fi configurat ca un canal de autoinițializare. În timpul autoinițializării, valorile originale ale adresei curente și ale numărătorului curent de cuvinte sunt automat restaurate din valoarea adresei de baza și a numărătorului de bază, în urma EOP. După autoinițializare, canalul este gata să efectueze o nouă operație DMA, fără intervenția CPU, dacă se detectează un nou DREQ, sau se face o cerere software.

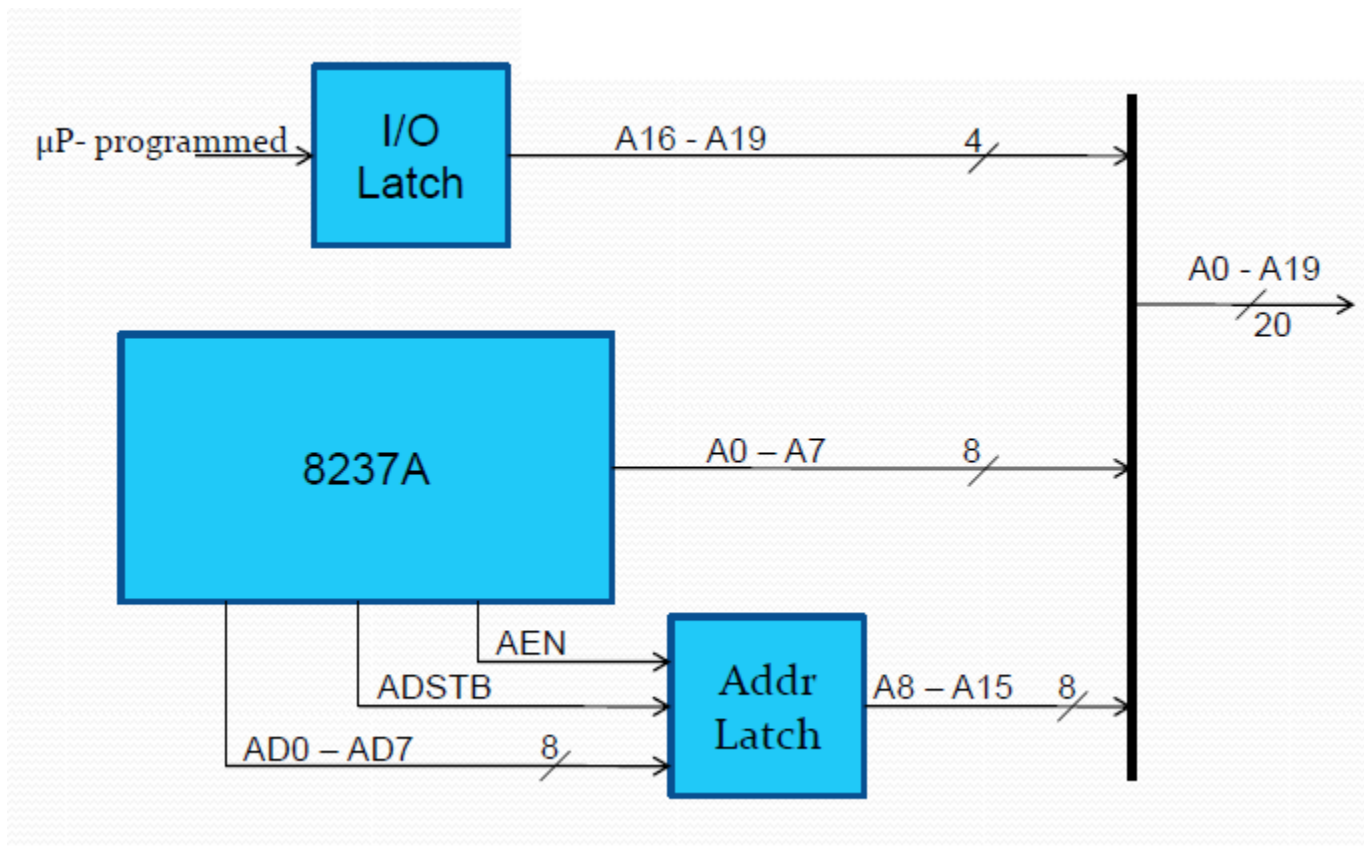


Generarea adreselor de 16 biți





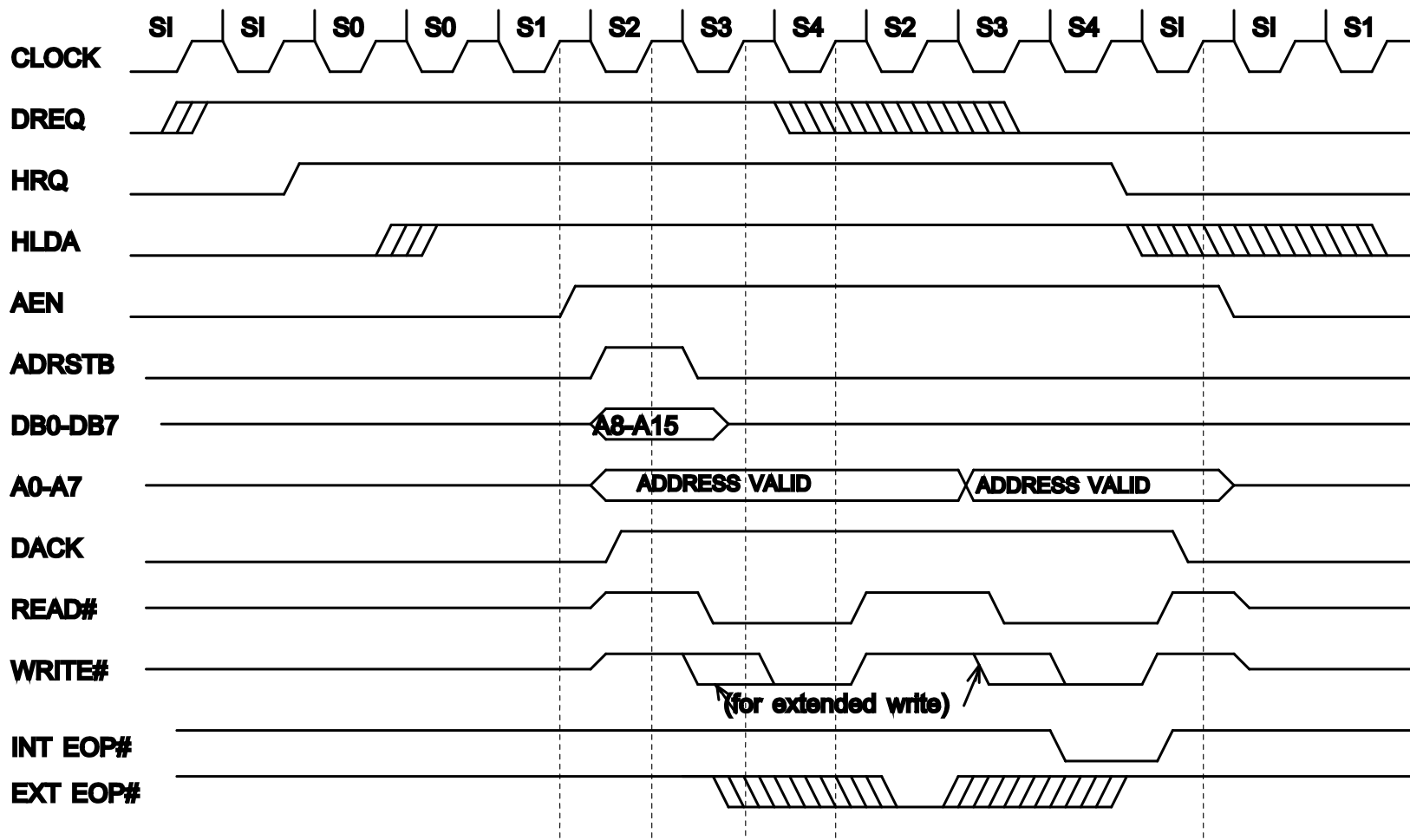
Generarea adreselor de 20 biți





Ciclu de transfer DMA

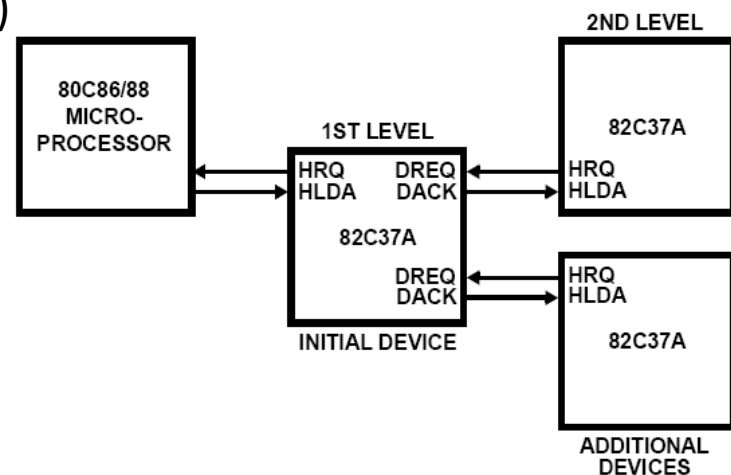
memorie la I/O sau I/O la memorie



Ciclu comprimat : S2 (schimbă adresa) + S4 (citește/scrie)



- Inactiv (slave)
 - programare (#CS=0, HOLDA=0)
 - DREQ se verifică pe frontul descrescător al clk
 - Dacă DREQ este activ pe un canal ne-mască, sau se face o solicitare software (transfer mem-to-mem) → Dispozitivul devine Activ
- Activ (master) – Transfer DMA
 - **Transfer individual** – se eliberează HOLD după fiecare byte transferat. Dacă DREQ este menținut activ, se cere HOLD din nou.
 - **Transfer bloc** – se transferă un bloc de date, de dimensiune specificată într-un registru de numărare (DREQ nu trebuie menținut)
 - **Transfer la cerere** – se transferă date în mod continuu până când se primește un #EOP extern, sau până când DREQ devine inactiv.
 - **Mod cascadat**





1. 82C37A datasheet